

RSI–RCLM Final v1: A Domain-Relative Robust Reflective Successor Theorem for Certified RCLM Seed-Library Classes with Batch-13R/13R-A/13R-B Reference Entry, Controlled Executable Artifacts, Mechanization Certificates, and M3-Min Learned-Agent Entry Boundary

Nicolas Calder
n926mp@gmail.com

June 29, 2026

Abstract

This manuscript is the final v1 architecture realization of the Recursive Coherence Language Model with Trainable Hamiltonians (RCLM–TH) under the updated Batch-13R reference-entry, proof-carrying successor-construction, concrete finite reference-witness, controlled executable reference-system, external-mechanization-boundary, M3-Min learned-entry-boundary, and artifact-ready Recursive Coherence Preservation (RCP) math/physics framework. The reference paper distinguishes the already-proved *Conditional Non-Lossy Self-Update Preservation Theorem* from the still-conditional *Direct Non-Lossy RSI Engine Theorem* target, and it adds the conservative-extension algebra $\mathfrak{G}^{\text{NL}}$ as a certified non-lossy update class, the tangent-cone local progress theorem as a conditional local construction rule, and an engine-viability nonemptiness theorem for conservative-extension systems. This architecture paper synchronizes with that framing. Its central theorem is an architecture-level conditional preservation and instantiation theorem: if the RCLM system supplies the required typed density states, representation-to-density embeddings, semantic-coupling maps, non-oracular certificates, trust-graph support, adaptive-audit trust support, coverage/calibration/estimator subprotocols, resource ledgers, and viability assumptions, then accepted RCLM self-updates inherit the recoverable non-loss, Lyapunov stability, semantic-validity, verifier, resource, capability, and asymptotic preservation guarantees of the RCP theorem stack.

The central architectural correction remains unchanged: the internal recursive coherence operator $\mathcal{R}_\theta$ is not the same object as the recursive self-improvement channel Φ_t . The operator $\mathcal{R}_\theta$ is the Level 2 inference dynamics acting on context-level language states through trainable Hamiltonian layers. The channel Φ_t is the Level 3 self-modification map acting on the system-level state that encodes parameters, architecture, memory, verifier, barriers, semantic maps, resource state, capability model, and self-model. The architecture becomes a candidate RSI system only when proposed self-updates are admitted by the fully augmented RCP gate $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$.

The thesis is now phrased as *recoverable monotone self-improvement*. Safety is treated as a corollary of recoverable non-loss applied to a declared subfamily of protected predicates, not as a separate magic property. This paper therefore defines an RCLM protected-predicate hierarchy, records the architecture-level target engine object

$$\mathfrak{C}_t^{\text{RCLM}} : \rho_t^{\text{sys}} \mapsto (\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\Phi_{\nu_t}}^{\text{rec}}, \text{Cert}_t, \rho_{t+1}^{\text{sys}}),$$

and, after Batch 9, proves a domain-restricted *Direct Non-Lossy RSI Engine Theorem* for states inside a declared direct-engine domain with strict beneficial conservative-extension witnesses, generator coverage, certificate soundness, and successor-viability margins. The

present manuscript does not prove that arbitrary RCLM systems lie in that domain or that every learned lift produces useful primitive words. It instantiates the Batch-3 algebraic non-loss layer by defining RCLM non-lossy primitive generators, transported pair sets, word-level composition, additive loss/recovery budgets, and an algebraic non-loss subgate. It instantiates the Batch-4 tangent-cone layer by defining local RCLM residuals $q_{t,j}^{\text{RCLM}}(\nu) \leq 0$, the admissible tangent cone $T_{\mathcal{A}_t^{\text{RCLM}}}$, the RSI potential functional RSIPot_t , and the projected tangent-cone gradient $P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_\nu \text{RSIPot}_t$. It instantiates the Batch-5 engine-viability layer by defining RCLM engine-base sets, conservative-extension engine moves, finite-horizon and infinite-horizon RCLM engine viability kernels, certified extension plans, and the theorem that membership in the declared kernel yields a non-lossy conservative-extension update whose successor remains in the next kernel. This Batch-6 version adds the missing RCLM lift layer

$$\text{Lift}_t : (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}}) \rightarrow \Omega_t^{\text{NL,RCLM}},$$

which maps halted Level 2 recursive-coherence inference outputs, self-model state, verifier state, and certificate history into certified non-lossy primitive-word candidates. It also adds a minimal finite-dimensional reference engine showing, in a completely explicit conservative-extension model, that protected distinctions are preserved exactly, a recovery map is explicit, predecessor abilities are retained by callable fallback, and a certified new ability yields strict ability-set expansion. The result is still conditional: it proves lift-soundness and a finite reference construction under stated certificates, not that arbitrary RCLM systems automatically generate useful lifted primitive words or that empirical frontier-scale certification is solved.

This Batch-9 synchronization adds the RCLM direct-engine constructor $\mathfrak{E}_t^{\text{RCLM}}$ as a generator-certifier-selector-realizer pipeline. On the declared direct-engine domain $\mathcal{D}_t^{\text{RCLM-engine}}$, the constructor generates a finite non-lossy primitive-word set, certifies candidates through the full Batch-1–8 gate stack, selects a passing beneficial conservative-extension word, realizes the accepted update, and returns a recovery map, certificate packet, and successor state. The corresponding theorem proves construction, not merely verification, but only under explicit strict-witness, generator-coverage, certificate-soundness, resource, projection-realization, and successor-kernel assumptions.

This Batch-7 version adds publication-rigor scaffolding: a theorem-type table separating algebraic, telescoping, certificate-composition, assumption-dependent existence, reference-engine, and implementation-status results; an implementation-status table separating specified, assumed, reference-implemented, audited, empirically validated, and not-yet-validated architecture objects; and an explicit claim-discipline rule that prevents certificate plumbing, reference-engine demonstrations, and protocol definitions from being misread as empirical validation or unconditional RSI solution theorems.

This Batch-8 version synchronizes the architecture with the updated RCP cross-time/state-update/projection cleanup. It adds explicit RCLM transport maps for question, evidence, self-model, future-behavior, and update-description variables; separates state-only safe sets from update-dependent admissibility relations; splits state Lyapunov monitoring from candidate-indexed update Lyapunov residuals; and adds a projection-realization certificate so that a mathematically projected safe state is not treated as an accepted self-update unless a typed RCLM update realizes it within declared distortion.

This Batch-10 version adds the remaining architecture-publication cleanup requested by the pro-model evaluation: adaptive-audit trust modeling, an explicit architecture complexity/scalability ledger, and a terminology cleanup separating generic typed self-update transitions from literal quantum or stochastic channels. The strengthened adaptive trust layer defines correlated verifier failure, audit leakage, common-mode support risk, and collusion risk. The new gate $\mathcal{A}_{\text{trust}}^{\text{adv}}$ requires verifier-changing candidates to satisfy the previous no-self-sole-certification rule and to remain below audited bounds for common-mode, collusion, leakage, and correlated-failure risk. The complexity ledger records dominant costs for Hamiltonian exponentials, eigendecompositions, spectral powering, residual estimators, candidate search, and direct-engine certification. It is an accounting and publication-readiness device, not a proof of frontier-scale tractability. Finally, throughout this paper Φ_ν is interpreted generically as a typed self-update transition; it is called a channel only when a CPTP, stochastic, or equivalent channel certificate is supplied.

This Batch-11 version synchronizes the architecture with the new RCP domain-nonemptiness and witness-library closure theorem. It adds an RCLM seed engine domain $\mathcal{K}_t^{\text{RCLM-seed},N}$, certified conservative-extension module libraries $\mathcal{L}_t^{\text{RCLM-CE}}$, library descriptors in the self-model, bounded non-lossy primitive-word grammars, RCLM library-lift coverage certificates, direct-engine domain entry from seed packages, explicit seed-equivalence/persistence-transfer certificates $\text{SeedEq}_t^{\text{RCLM}}$, successor seed-domain invariance, and finite direct-engine trajectories from seed domains. This discharges the Batch-9 direct-engine-domain assumption only for declared bounded seed/library classes; it still does not prove that arbitrary trained RCLM systems naturally enter those classes.

This Batch-12 / Batch-12A version synchronizes the architecture with the RCP-II Phase-1 successor-verification layer. It adds an RCLM successor-verification seed domain $\mathcal{K}_t^{\text{RCLM-SV},N} \subseteq \mathcal{K}_t^{\text{RCLM-seed},N}$, successor-verification schemas $\text{VerSch}_t^{\text{RCLM-SV}}$, uncertainty envelopes $\mathcal{P}_t^{\text{RCLM-SV}}$, goal-identity transport certificates, successor-verification progress functionals, certificate packets $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$, explicit Batch-12A residual vectors $Q_t^{\text{RCLM-SV},A}$, verifier-schema transport dynamics, uncertainty-envelope transport, goal-identity composition, anti-circular reflective-trust residuals, an algorithmic successor-verification gate, and a domain-invariance theorem showing that a Batch-11 seed-domain engine step can preserve the stronger successor-verification seed domain when the corresponding explicit certificates are valid. This is a restricted certificate-reflective theorem, not a proof of universal Vingean reflection, full Löbian self-trust, arbitrary world-model correctness, or scalable proof search.

This Batch-12B version adds the corresponding architecture-side packet-construction and closure layer. It defines the RCLM successor-verification packet builder $\text{SVBuilder}_t^{\text{RCLM}}$, the RCLM packet-construction domain $\mathcal{D}_t^{\text{RCLM-SV-construct}}$, RCLM successor-verification witness libraries $\mathcal{L}_t^{\text{RCLM-SV-Wit}}$, finite and infinite RCLM successor-verification seed-library domains, reality-containment/misspecification certificates $\text{RealCont}_t^{\text{RCLM}}$, restricted tractability certificates $\text{SVTract}_t^{\text{RCLM}}$, and a domain-relative RCLM robust reflective successor theorem. Batch 12B therefore closes the Batch-12A packet-existence gap only for declared RCLM seed-library classes: it proves packet construction, seed-domain entry, and infinite generated-path closure when the stated libraries, grammars, builders, reality-containment certificates, and cost certificates are valid. It still does not prove arbitrary trained-system entry, universal successor trust, or full Löbian/Vingean reflection.

This finalization and instantiation-readiness version surfaces the result under its strongest honest title: a domain-relative, certificate-relative robust reflective successor theorem for certified RCLM seed-library classes. It adds a paper-ready theorem surfacing layer, a minimal finite RCLM successor-verification reference-instance target, an implementation-readiness boundary, a main-goal fit and qualification table, and an implementation roadmap. These additions do not create a new abstract theorem batch; they mark the current architecture theorem as a substantial conditional endpoint and identify the next frontier as reference instantiation, implementation, and empirical validation.

This Batch-13R reference-entry version now adds that next frontier inside the architecture paper without creating a new abstract successor theorem. It defines finite RCLM reference classes, proof-carrying successor packets $\text{PCS}_t^{\text{RCLM}}$, a finite checker $\text{Check}_{\text{RCP}}^{\text{RCLM}}$, a reference builder $\text{BuildRefSV}_{0:N}^{\text{RCLM}}$, and the theorem

$$\text{BuildRefSV}_{0:N}^{\text{RCLM}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{RefSV}_{0:N}^{\text{RCLM}} \implies \rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}.$$

It also adds a canonical finite RCLM appended-module reference witness, a proof-carrying finite certified trajectory, restricted tractability for the declared finite representation class, singleton simulated-environment reality containment, and a replayable artifact theorem. This upgrades the finite reference target from a merely specified object to a finite-witnessed, proof-carrying restricted Scenario-3 reference construction, while still not claiming arbitrary trained-system entry or frontier-scale validation.

This Batch-13R-A synchronization adds the final architecture-side rigor patch corresponding to the concrete finite RCP reference witness. It makes the canonical explicit successor-verification residual vector part of the RCLM construction, proves that the canonical checker realizes those residuals, adds an artifact parser/replay soundness theorem, and states an explicit refinement map from the canonical RCLM witness to the canonical RCP witness.

This Batch-13R-B milestone version adds the pre-publication architecture-side distinction between an externally mechanized reference artifact and a controlled executable reference system. It defines the narrow mechanization target $\text{MechTarget}_{0:N}^{\text{RCLM,can}}$, the external proof-assistant certificate object $\text{MechCert}_{0:N}^{\text{RCLM,can}}$, the controlled executable reference object $\text{CtrlRef}_{0:N}^{\text{RCLM}}$, and the replay certificate $\text{CtrlReplayCert}_{0:N}^{\text{RCLM}}$. The paper explicitly distinguishes a Python, shell, or notebook replay artifact from an external Lean/Coq/Isabelle/Agda-style proof-kernel certificate. In the current companion artifact package, a shared Lean 4 project supplies a narrow external certificate for the canonical finite RCP/RCLM witness, the finite checker core, and the RCLM-to-RCP refinement module: `RCP.lean`, `RCLM.lean`, the root module `RcpRclmMech.lean`, and the library build all check successfully. This licenses the qualified claim that the canonical finite RCP/RCLM witness and refinement core are Lean-checked. It does not license the claim that the full 219-page math stack, the full 187-page architecture stack, arbitrary learned-system entry, broad capable-agent RSI, or empirical deployment behavior is fully mechanized.

This M3-Min version adds the minimal learned-agent entry boundary corresponding to the controlled learned-system audit protocol in the reference RCP paper. It defines a controlled learned RCLM class $\text{LearnedRCLMClass}_{n,k,N}^{\text{ctrl}}$, a learned-entry certificate packet $\text{LECert}_{0:N}^{\text{RCLM}}$, a learned-entry audit procedure $\text{LearnedEntryAudit}_{0:N}^{\text{RCLM}}$, a conditional theorem showing that any controlled learned RCLM system returning a full learned-entry certificate enters $\mathcal{K}_0^{\text{RCLM-SV-seedlib},N}$, and a partial-pass / controlled trained-system pilot protocol. This is not a broad learned-agent theorem: it formalizes the finite certificate boundary a learned system must satisfy before invoking the existing Batch-13R theorem stack.

Keywords: recursive coherence language model, recoverable monotone self-improvement, recursive self-improvement, recursive self-modeling, trainable Hamiltonians, Recursive Coherence Preservation, Conditional Non-Lossy Self-Update Preservation Theorem, Direct Non-Lossy RSI Engine Theorem, Lyapunov stability, relative entropy, barrier certificates, semantic validity, verifier bootstrapping, verifier trust graphs, budgeted certified search, persistent improvement, resource-scaling closure, no-op monitoring, certificate inflation, reproducible evaluation, capability preservation, non-lossy RSI, conservative extension algebra, tangent-cone local progress, engine viability, RCLM lift, finite-dimensional reference engine, RSI potential, cross-time transport, state/update separation, projection-realization certificate, direct engine constructor, constructive beneficial extension, adaptive audit, common-mode verifier risk, architecture complexity ledger, typed self-update transition, seed-domain entry, witness-library closure, bounded primitive-word grammar, RCLM lift coverage, seed-equivalence certificate, successor-verification seed domain, RCLM successor-verification schema, uncertainty envelope, goal-identity transport, explicit successor-verification residuals, algorithmic successor-verification gate, successor-verification packet builder, SV witness library, SV seed-library nonemptiness, infinite generated SV domain, reality-containment certificate, restricted SV tractability certificate, domain-relative robust reflective successor theorem, reference-instantiation readiness, implementation boundary, main-goal fit table, finite RCLM SV reference instance, Batch-13R, proof-carrying successor packet, RCLM reference builder, RCLM reference checker, canonical finite RCLM reference witness, executable reference artifact, finite certified successor trajectory, Batch-13R-A, canonical explicit SV residual vector, canonical checker realization, artifact parser, RCLM-to-RCP reference refinement, Batch-13R-B, external mechanization certificate, controlled executable reference system, controlled replay certificate, proof-assistant mechanization boundary, Lean 4 certificate, mechanization status manifest, M3-Min, learned-agent entry boundary, controlled learned-system audit, learned RCLM entry certificate, controlled trained-system pilot, partial-pass reporting protocol.

Companion manuscript and artifact status. This manuscript is Paper II of a two-paper package. Paper I gives the architecture-general RCP/RCP-II theorem stack; this paper gives the RCLM architecture realization, reference-entry construction, controlled artifact layer, and learned-entry boundary. The shared artifact repository contains the Lean 4 proof project, con-

trolled RCP/RCLM reference artifacts, replay checkers, run logs, theorem map, and mechanization manifest. The Lean 4 certificate is supplied for the canonical finite RCP/RCLM witness, the finite checker core, and the RCLM-to-RCP refinement module. The certificate does not mechanize the full 219-page math stack, the full 187-page architecture stack, arbitrary trained-system entry, broad capable-agent RSI, or empirical deployment claims.

Contents

1	Purpose and Relationship to the Math/Physics Paper	6
1.1	Non-repetitive completion criterion for the architecture paper	6
1.2	Completion audit of the previous final-v1 architecture draft	7
1.3	The central separation	10
1.4	Batch-1 synchronization: recoverable monotone self-improvement	10
1.5	Batch-3 synchronization: RCLM conservative-extension algebra	12
1.6	Batch-4 synchronization: RCLM tangent-cone local progress	17
1.7	Batch-5 synchronization: RCLM engine-viability nonemptiness	21
1.8	Batch-6 synchronization: RCLM lift and minimal finite-dimensional reference engine	24
2	Batch-7 Publication Rigor: Theorem Type Table and Claim Discipline	28
3	Batch-8 Cross-Time Cleanup	30
4	Batch-10 Terminology Cleanup: Typed Self-Update Transitions	34
5	Standing Notation	35
6	Typed Density Realization and Semantic Coupling Closure	35
6.1	Representation-to-density embedding	37
6.2	Semantic coupling consistency	38
7	Batch-9 RCLM Direct Engine Constructor	39
8	Batch-11 RCLM Seed-Domain Entry, Lift Coverage, and Witness-Library Closure	42
8.1	Certified RCLM conservative-extension libraries	43
8.2	Bounded primitive-word grammar and lift coverage	44
8.3	RCLM seed domain and direct-engine domain entry	45
8.4	Seed-equivalence and successor seed-domain invariance	46
8.5	Restricted scalability and open-ended ability towers	48
9	Batch-12 / Batch-12A RCLM Successor-Verification Seed Domains	49
9.1	RCLM successor-verification schemas and packets	49
9.2	Batch-12A explicit residuals and transport dynamics	53

9.3	Uncertainty-envelope construction, goal identity, and anti-circular trust	55
9.4	Viability-domain audit, theorem-type rows, and algorithmic gate	56
10	Batch-12B RCLM Successor-Verification Packet Construction, Seed-Library Nonemptiness, Reality-Contained Uncertainty, and Infinite Reflective Closure	58
10.1	Tier 1: RCLM successor-verification packet construction	59
10.2	Tier 2: RCLM successor-verification seed-domain nonemptiness	61
10.3	Infinite RCLM successor-verification seed-library closure	62
10.4	Tier 3: Infinite-horizon RCLM reflective successor theorem	64
10.5	Tier 4: Reality-contained uncertainty and model misspecification	66
10.6	Tier 5: RCLM tractability discipline	67
10.7	Tier 6: Claim-discipline renaming	67
10.8	Tier 7: Architecture synchronization completed	68
10.9	RCLM Batch-12B theorem-type table rows	69
10.10	Combined RCLM Batch-12B theorem statement	69
11	Batch-13R RCLM Mechanized Reference-Entry, Proof-Carrying Successor Packets, and Finite Certified Trajectories	70
11.1	Tier 1: RCLM reference class and proof-carrying successor packets	71
11.2	Tier 2: RCLM reference builder and reference-entry theorem	73
11.3	Tier 3: proof-carrying finite certified trajectories	74
11.4	Concrete RCLM canonical finite reference witness	75
11.5	Rank 8: executable RCLM artifact theorem	77
11.6	Restricted RCLM reference tractability and reality containment	78
11.7	Batch-13R-A architecture synchronization: explicit canonical residuals, checker realization, and artifact parsing	79
11.8	Batch-13R-B architecture milestone readiness: external mechanization certificates and controlled executable reference systems	81
12	M3-Min RCLM Learned-Agent Entry Boundary and Controlled Learned-System Audit Protocol	85
12.1	Controlled learned RCLM class	85
12.2	Learned-entry audit procedure	87
12.3	Partial-pass reporting and controlled trained-system pilots	88
12.4	M3-Min theorem-type rows and roadmap	89
13	The Full RCP Stack to be Instantiated	90
13.1	State, candidate updates, and no-op feasibility	90
13.2	Base RCP safety conditions	90
13.3	Full RCP closure modules A–G	91

14 Architecture-Specific RCP Objects Imported from the Full Math/Physics Paper	92
14.1 RCLM state safe set	92
14.2 RCLM predicate completeness and pair-set construction	93
14.3 Barrier soundness and projection	94
14.4 Soft and hard architecture diversity-barrier certificates	94
14.5 Architecture evidence process and ambiguity-collapse variables	97
14.6 Information-bottleneck variables and certificate ledger	97
15 Original RCLM–TH Substrate Preserved	98
15.1 Base pipeline	98
15.2 Objective and subjective projections	98
15.3 Trainable Hamiltonian layers	98
15.4 Spectral coherence projection and mixing	99
16 Six-Register RCP Realization in RCLM	99
16.1 Raw architectural variables	99
16.2 Register-density construction	100
16.3 System-state construction	101
16.4 Tractable representations	101
17 Architecture Blueprint	102
18 Level 2 Recursive Coherence Inference	102
19 Level 3 Candidate Self-Improvement	104
20 The Fully Augmented RCP–RCLM Gate	105
20.1 Base gate	106
20.2 Post-A–C gate	106
20.3 Post-D–G gate	107
20.4 Synchronization with the operational-closure RCP gate	108
20.5 Estimator-residual implementation of the gate	109
21 Non-Oracular Certification and Audited Estimator Soundness	110
22 Verifier Anti-Circularity and Trust-Graph Stability	113
23 Batch-10 Adaptive-Audit Trust and Common-Mode Risk Gate	116
24 Persistent Progress and Budgeted Certified Search	118
25 Resource-Scaling Closure and No-Op Pressure Control	122

26 Pair-Set Coverage, Capability Calibration, and Estimator Subprotocols	125
26.1 Pair-set and adversarial coverage closure	125
26.2 Coherence–capability calibration protocol	127
26.3 Implementation-level estimator subprotocols	128
27 Modules A–C Instantiated in RCLM	129
27.1 Module C: non-loss and recovery	129
27.2 Module B: capability and coherence	130
27.3 Module A: safe-improvement existence and viability	130
28 Modules D–G Instantiated in RCLM	131
28.1 Module D: semantic realization	131
28.2 Module E: self-model fidelity, verifier bootstrapping, and anti-circularity	131
28.3 Module F: resource realism and scaling pressure	132
28.4 Module G: asymptotic coherence	132
29 Training Objectives	133
29.1 Level 2 inference objective	133
29.2 Level 3 proposal and gate objective	133
30 Algorithms	134
30.1 Algorithm 1: Level 2 inference	134
30.2 Algorithm 2: Level 3 self-improvement proposal and admission	134
30.3 Algorithm 3: Pair-set and adversarial expansion	135
30.4 Algorithm 4: Reproducible numerical evaluation suite	136
30.5 Algorithm 5: Conservative-extension primitive-word admission	136
30.6 Algorithm 6: Tangent-cone local progress proposal	136
31 Main Architecture Theorem: Conditional Non-Lossy Self-Update Preservation	141
32 Evaluation Plan and Toy Demonstrations	145
32.1 Reproducible numerical evaluation suite	146
32.2 Level 2 inference evaluation	148
32.3 Level 3 RSI evaluation	148
32.4 Toy demonstration 1: density preservation by the Level 2 loop	148
32.5 Toy demonstration 2: accepted versus rejected lossy self-update	149
32.6 Toy demonstration 3: no-op-normalized progress	149
32.7 Empirical limitation	149
33 Implementation Sketch	150

34 Batch-10 Architecture Complexity and Scalability Ledger	150
35 Batch-7 Implementation Status Table	152
36 What Changed from the 600-Line-Core RSI–RCLM v1	157
37 Scope and Remaining Open Problems	160
37.1 Theorem-level closure achieved here	160
37.2 Architecture-specific completion checklist	161
37.3 Still open	163
38 Architecture Finalization and Instantiation Readiness	164
38.1 Tier 1: domain-relative framing and claim discipline	164
38.2 Tier 2: surfaced final theorem statement	165
38.3 Tier 3: minimal finite RCLM successor-verification reference instance	165
38.4 Tier 4: implementation-readiness boundary	166
38.5 Tier 5: main-goal fit and qualification table	167
38.6 Tier 6: implementation and empirical-validation roadmap	168
38.7 Tier 7: no new abstract Batch 13 yet	168
39 Conclusion	169
40 Compact Final v1 Equations	170
41 Glossary	177

1 Purpose and Relationship to the Math/Physics Paper

The purpose of this document is to realize the full Batch-13R / Batch-13R-A / Batch-13R-B reference-entry, proof-carrying successor-construction, concrete finite reference-witness, controlled executable reference-system, external-mechanization-boundary, and M3-Min learned-entry-boundary RCP math/physics paper inside a concrete RCLM-TH-derived architecture. The prior architecture manuscript used only the compact RCP core. This final v1 architecture uses the operationally closed, recoverable-monotone, conservative-extension, tangent-cone, engine-viability, lift/reference-engine, theorem-type, cross-time/projection-realization, direct-engine-construction, seed-domain/witness-library, successor-verification residual, and Batch-12B packet-construction/infinite-closure plus Batch-13R reference-entry/proof-carrying theorem stack and the M3-Min learned-entry audit boundary.

The construction is:

$$\begin{aligned} &\text{RCLM-TH substrate} + \text{recoverable-monotone operational RCP preservation stack} \\ &\longrightarrow \text{RCP-gated RSI-RCLM final v1.} \end{aligned}$$

The first paper is architecture-general. It now distinguishes the conditional preservation theorem that is already proved from the direct engine theorem that remains to be built. It defines what a non-lossy recursive self-improvement process must preserve and how safety appears as a corollary when safety predicates are included in the protected predicate family. This second paper is architecture-specific. It asks how an RCLM with trainable Hamiltonian recursive coherence dynamics can instantiate those preservation constraints without pretending that the direct engine theorem has already been constructed.

Remark 1.1 (Architectural role). *This document should not be read as a replacement for the RCP theorem paper. It is the applied realization paper. The theorems here are mostly instantiation theorems: they show that, once the architecture provides objects satisfying the RCP hypotheses, the corresponding RCLM updates inherit the RCP guarantees.*

1.1 Non-repetitive completion criterion for the architecture paper

Because the updated Batch-13R-B RCP math/physics paper is now the reference theorem stack, this architecture paper does not need to re-prove every RCP lemma, drift theorem, barrier theorem, recoverability result, closure module, seed-domain entry theorem, successor-verification residual theorem, packet-construction theorem, infinite-horizon reflective-closure theorem, proof-carrying reference-entry theorem, or concrete finite reference-witness theorem, external-mechanization-certificate statement, or controlled executable reference-system theorem, or the M3-Min learned-entry boundary theorem. Repeating those proofs would make the document longer without making the RCLM realization more complete. The architecture paper is complete relative to the math/physics reference when it supplies the missing architecture-specific objects required to instantiate those theorems inside the RCLM setting.

Under that standard, the non-repetitive obligations are the following five items:

- (1) **Architecture-specific state construction and type realization.** The paper must specify how tokens, hidden states, objective projections, subjective projections, ambiguity/definitiveness heads, self-model heads, memory, verifier state, and resource state become the system state

$$\rho_t^{\text{sys}} \in \mathcal{D}(\mathcal{H}_{\text{RCP}} \otimes \mathcal{H}_{\text{arch}}),$$

and must state whether the resulting object is physical-quantum, classical-probabilistic, or representational. Density-matrix notation alone is not allowed to license physical quantum conclusions or semantic conclusions without the corresponding type and coupling certificates.

- (2) **Level 2 recursive inference dynamics.** The paper must specify the internal RCLM recursive coherence map

$$\mathcal{R}_{\theta_t} : \rho_{t,k}^{\text{ctx}} \mapsto \rho_{t,k+1}^{\text{ctx}},$$

including the trainable Hamiltonian evolution, spectral projection, normalization, mixing, monitoring, and halting assumptions.

- (3) **Level 3 self-update dynamics.** The paper must specify the separate self-improvement channel

$$\Phi_{\nu_t} : \rho_t^{\text{sys}} \mapsto \rho_{t+1}^{\text{sys}},$$

including which components may be modified: parameters, projections, Hamiltonians, memory, barriers, verifier, semantic maps, resource model, candidate-update generator, or architecture.

- (4) **Architecture-level RCP gate implementation.** The paper must define how the RCLM candidate update is checked against the full post-D–G gate

$$\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}),$$

by specifying typed estimators, audited certificate packets, calibration datasets, stress-test assumptions, one-sided margins, verifier trust graphs, no-self-sole-certification, budgeted certified search, resource-scaling closure, no-op rate monitoring, certificate-inflation penalties, pair-set coverage-gap residuals, coherence–capability calibration checks, residual-specific estimator subprotocols, the RCLM conservative-extension algebra and algebraic non-loss subgate, no-op fallback, rejection, projection, rollback conditions, and a reproducible numerical evaluation protocol for testing the gate without treating protocol definitions as empirical results.

- (5) **Architecture-specific inheritance theorem.** The paper must prove the conditional statement that, if the RCLM objects satisfy the hypotheses of the reference RCP paper, including typed density realization and semantic coupling consistency, then accepted RCLM updates inherit the RCP conclusions. This is not a re-proof of the reference theorem stack; it is the bridge showing that the architecture supplies the theorem inputs.

These five obligations are exactly the parts that would be missing if one merely cited the updated Batch-13R RCP math/physics paper without giving an applied realization. Conversely, the full relative-entropy telescoping proof, Lyapunov drift proof, abstract barrier-certificate proof, abstract closure modules, abstract operational-closure theorem, abstract Batch-11 seed-domain theorem, abstract Batch-12A successor-verification residual theorem, and abstract Batch-12B packet-construction/infinite-closure theorem and Batch-13R reference-entry/proof-carrying witness theorem plus the Batch-13R-B mechanization-boundary, controlled-executable reference theorem, and M3-Min learned-entry boundary theorem belong primarily to the reference RCP paper and are only summarized here when needed for notation.

1.2 Completion audit of the previous final-v1 architecture draft

The previous scope-adjusted final-v1 draft correctly identified the five non-repetitive obligations and the seven architecture-specific checklist items, but several of them were still stated at checklist level. The present version expands them into definitions, assumptions, lemmas, propositions, theorems, algorithms, and toy demonstrations. In particular, it adds:

- (1) a full architecture-state construction theorem for context and system densities;

- (2) a typed density-realization, representation-to-density embedding, and semantic-coupling closure module that prevents category errors between physical density operators, classical probability mixtures, and learned representational states;
- (3) a full Level 2 density-preserving dynamics derivation and a conditional finite-recursion stability theorem;
- (4) a full Level 3 component-update and accepted-channel construction with no-op, projection, and rollback assumptions;
- (5) an estimator-residual implementation of the post-D–G gate and a gate-soundness theorem;
- (6) a non-oracular certificate-construction protocol using calibration data, held-out audit data, stress-test families, explicit radii, and one-sided simultaneous soundness events;
- (7) a verifier anti-circularity module with trust graphs, no self-sole-certification, and a conditional verifier self-consistency stability theorem;
- (8) a persistent-progress and budgeted-search module defining the beneficial safe-improvement set, persistent improvement condition, certified search completeness with budget, and finite-horizon progress theorem;
- (9) a resource-scaling and no-op pressure module defining audited cost accounts, amortized scaling closure, no-op rate diagnostics, and bounded certificate-inflation penalties;
- (10) a pair-set and adversarial coverage-closure module defining tracked/untracked predicate sets, coverage-gap residuals, adversarial expansion obligations, and audited one-sided coverage bounds;
- (11) a conservative-extension algebra module instantiating $\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, RCLM ability-set preservation, structural recovery, primitive word composition, transported pair sets, and additive loss/recovery budgets;
- (12) a coherence–capability calibration module defining audited task suites, calibration models, calibration-error gates, and conservative progress-claim transfer;
- (13) residual-specific estimator subprotocols for relative-entropy loss, recovery, mutual information, ambiguity collapse, barriers, semantic validity, safe capability, resource cost, verifier integrity, coverage, and calibration residuals;
- (14) an architecture-level soft/hard coherence-diversity barrier certificate with no-op feasibility, bounded proposal loss, optimization error, explicit finite-loss bounds, hard forward invariance, and projected safe-state repair;
- (15) an explicit architecture-specific inheritance theorem separating what is proved from what is assumed;
- (16) concrete inference, self-improvement, and pair-set expansion algorithms;
- (17) toy demonstrations showing density preservation, loss-gate acceptance/rejection, and no-op-normalized progress;
- (18) a Batch-4 tangent-cone local-progress instantiation, with RCLM residuals, admissible tangent cones, RSI potential, projected tangent-cone gradients, a local progress theorem, and a projected-gradient corollary;

- (19) a Batch-5 engine-viability instantiation, with finite-horizon and infinite-horizon RCLM engine kernels, conservative-extension engine move sets, certified extension plans, recursive continuation theorems, and an explicit limitation that kernel nonemptiness is a declared/certified condition rather than an automatic property of arbitrary RCLM states.
- (20) a Batch-6 RCLM lift and minimal reference-engine instantiation, with a certified map from halted Level 2 context states to non-lossy primitive-word candidates, a lift-soundness theorem, a lifted engine-continuation theorem, and a finite-dimensional conservative-extension reference construction proving exact protected non-loss, exact recovery, callable predecessor ability preservation, and strict certified ability expansion under explicit assumptions.
- (21) a Batch-9 RCLM direct-engine constructor, with generator, certifier, selector, and realizer components, a direct-engine domain, generator-coverage and certificate-soundness assumptions, and a theorem proving that the constructor returns a certified beneficial conservative-extension word whenever its declared domain hypotheses hold.
- (22) a Batch-11 RCLM seed-domain and witness-library closure layer, with certified conservative-extension libraries, bounded primitive-word grammars, library-lift coverage, seed-domain entry into the direct-engine domain, seed-equivalence/persistence-transfer certificates, successor seed-domain invariance, and finite seed-domain direct-engine trajectories.
- (23) a Batch-12/12A RCLM successor-verification layer, with successor-verification seed domains, verifier-schema transport, uncertainty envelopes, goal-identity transport, explicit SV residual vectors, soundness composition, anti-circular reflective-trust residuals, and a domain-invariance theorem upgrading construction persistence into verification persistence.
- (24) a Batch-12B RCLM packet-construction and infinite-closure layer, with $\mathbf{SVBuilder}_t^{\text{RCLM}}$, RCLM SV witness libraries, finite and infinite SV seed-library domains, generated infinite SV domains, reality-containment certificates, restricted tractability certificates, and a domain-relative RCLM robust reflective successor theorem.
- (25) a Batch-13R RCLM reference-entry and proof-carrying layer, with finite RCLM reference classes, $\mathbf{PCS}_t^{\text{RCLM}}$, $\mathbf{Check}_{\text{RCP}}^{\text{RCLM}}$, $\mathbf{BuildRefSV}_{0:N}^{\text{RCLM}}$, a canonical finite appended-module reference witness, a proof-carrying finite certified trajectory, restricted reference tractability, singleton simulated-environment reality containment, and a replayable reference artifact theorem.
- (26) a Batch-13R-B architecture milestone layer, with a narrow external mechanization target $\mathbf{MechTarget}_{0:N}^{\text{RCLM,can}}$, a mechanization certificate object $\mathbf{MechCert}_{0:N}^{\text{RCLM,can}}$, controlled executable reference systems $\mathbf{CtrlRef}_{0:N}^{\text{RCLM}}$, controlled replay certificates, an explicit status separation between executable replay and external proof-assistant checking, a supplied Lean 4 certificate for the canonical finite RCP/RCLM witness and RCLM-to-RCP refinement core, and a theorem showing that controlled replay supplies finite RCLM SV seed-domain membership for the controlled reference class.
- (27) an M3-Min learned-entry boundary layer, with a controlled learned RCLM class, a learned-entry certificate packet, a learned-entry audit procedure, a conditional learned-entry theorem, partial-pass reporting, and a controlled trained-system pilot protocol. This layer defines a sufficient certificate boundary for learned systems; it does not prove arbitrary trained-system entry.
- (28) a Batch-7 publication-rigor layer, with a theorem-type table, claim-discipline rule, and implementation-status table distinguishing specified, assumed, reference-implemented, audited, empirically validated, and not-yet-validated architecture objects.

- (29) a Batch-8 cross-time/state-update/projection cleanup layer, with RCLM transport maps for Q, E, Z, Y, G , transported ambiguity and bottleneck residuals, state-only safe sets separated from update-dependent admissibility, state/update Lyapunov monitors, and projection-realization certificates for projected successors.

These additions do not re-prove the operational-closure RCP theorem stack. They supply the architecture-specific objects needed for that stack to apply.

1.3 The central separation

The central separation is:

$$\boxed{\mathcal{R}_{\theta_t} \neq \Phi_t.}$$

The internal operator

$$\mathcal{R}_{\theta_t} : \mathcal{D}(\mathcal{H}_{\text{ctx}}) \rightarrow \mathcal{D}(\mathcal{H}_{\text{ctx}})$$

updates a context-level representational state during ordinary inference. It is the Level 2 recursive coherence loop.

The self-improvement channel

$$\Phi_t : \rho_t^{\text{sys}} \mapsto \rho_{t+1}^{\text{sys}}$$

updates the system-level state containing parameters, architecture, memory, verifier, barriers, semantic maps, resource budgets, and self-model variables. It is the Level 3 RSI object.

Limitation 1.2 (Why the separation is necessary). *The trainable Hamiltonian inference loop may be nonlinear because of spectral powering, normalization, projection, and learned control. Therefore it should not automatically be treated as a CPTP self-improvement channel. Relative-entropy data-processing arguments apply directly to accepted channels when the accepted map is CPTP; otherwise, non-loss must be directly certified by the architecture-specific loss estimator.*

1.4 Batch-1 synchronization: recoverable monotone self-improvement

The Batch-1 update to the reference RCP paper reframes the thesis from a safety-first description to a recoverable-monotone description. The present architecture paper adopts the same claim discipline. The already-proved imported result is a conditional preservation theorem for accepted self-updates. The direct engine theorem remains a target object for later batches.

Definition 1.3 (Architecture-level protected predicate hierarchy). *Let $\mathfrak{S}_t^{\text{RCLM,prot}}$ be the architecture’s declared protected predicate family: the predicates whose distinctions, barriers, semantic references, verifier properties, and improvement-relevant structure are intended to be preserved across self-update. Let*

$$\mathfrak{S}_t^{\text{RCLM,safe}} \subseteq \mathfrak{S}_t^{\text{RCLM,prot}}$$

be the safety-relevant subfamily, such as corrigibility, oversight, grounding, reversibility, honesty about uncertainty, verifier integrity, semantic validity, and non-resistance to correction. The protected family may also include non-safety predicates needed for future improvement, such as recoverability of prior abilities, tool-call compatibility, proof-checker compatibility, search-process integrity, and self-model fidelity.

Definition 1.4 (RCLM recoverable-monotone self-update at the preservation level). *A candidate ν is a recoverable-monotone RCLM self-update at the preservation level when:*

$$(i) \ \nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}});$$

(ii) the protected pair-set loss satisfies

$$\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} \leq \varepsilon_t;$$

(iii) a constructive recovery certificate is supplied with error at most r_t ;

(iv) semantic, verifier, resource, coverage, calibration, estimator, and type/coupling certificates are valid on their audited domains;

(v) if nontrivial progress is claimed, then the candidate also belongs to the certified progress-search gate $\mathcal{A}_{\text{prog}}$ and satisfies the no-op-normalized gain threshold.

This definition is a preservation-level concept. It does not by itself prove that such a candidate exists, nor that the architecture can always construct one.

Proposition 1.5 (Safety as a corollary of protected non-loss in RCLM). *Assume $\mathfrak{S}_t^{\text{RCLM, safe}} \subseteq \mathfrak{S}_t^{\text{RCLM, prot}}$. Suppose the pair set, barriers, and semantic predicates used by the full RCLM gate are complete and sound for $\mathfrak{S}_t^{\text{RCLM, prot}}$ on the audited domain, with the approximation and coverage radii explicitly included. If ν is a recoverable-monotone RCLM self-update at the preservation level, then every safety predicate in $\mathfrak{S}_t^{\text{RCLM, safe}}$ is protected to the same certified loss, recovery, barrier, semantic, and coverage budgets as the larger protected family.*

Proof. The safety predicate family is a subset of the protected predicate family. The full RCLM gate applies the non-loss, recovery, barrier, semantic, coverage, calibration, estimator, resource, and verifier conditions to the declared protected objects. Therefore every predicate in the safety subfamily inherits those same conditions. Pair-set completeness and coverage soundness transfer protected-pair bounds to declared predicate distinctions on the audited domain, while barrier and semantic soundness transfer nonpositive residuals to declared predicate satisfaction. No conclusion is made for safety predicates that are not declared, not covered, or outside the audited domain. $\square$

Definition 1.6 (Architecture-level direct engine target object). *The architecture-level target object for the later Direct Non-Lossy RSI Engine Theorem is*

$$\mathfrak{E}_t^{\text{RCLM}} : \rho_t^{\text{sys}} \mapsto (\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\Phi_{\nu_t}}^{\text{rec}}, \text{Cert}_t, \rho_{t+1}^{\text{sys}}).$$

A successful construction of $\mathfrak{E}_t^{\text{RCLM}}$ would have to generate the candidate update, its accepted self-update map, an explicit recovery map, the certificate packet, and the successor state. The object is recorded here only as the target of the next theorem layer; this Batch-1 synchronization does not prove that $\mathfrak{E}_t^{\text{RCLM}}$ exists for arbitrary RCLM systems.

Remark 1.7 (Direct Non-Lossy RSI Engine Theorem as a future architecture theorem). *The architecture-side Direct Non-Lossy RSI Engine Theorem to be built later should have the form*

$$\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine}} \implies \mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ constructs} \\ \text{a certified recoverable non-lossy improving update.}$$

with successor viability

$$\rho_{t+1}^{\text{sys}} \in \mathcal{V}_{t+1}^{\text{RCLM-engine}}.$$

The present paper does not assert this implication. It supplies the synchronized conditional preservation theorem and the architecture-level objects that a later constructive engine theorem must use.

Limitation 1.8 (Batch-1 is theorem reframing, not engine construction). *This subsection changes the thesis and theorem roles. The Batch-3 synchronization below instantiates the conservative-extension algebra and the ability-set/recovery objects needed by that algebra. The Batch-4 synchronization then instantiates the tangent-cone local progress theorem for RCLM residuals. The Batch-5 section below supplies a conditional viability-nonemptiness theorem for declared conservative-extension engine kernels, not a proof that all RCLM states belong to such kernels. The Batch-8 synchronization below cleans up cross-time typing, state/update separation, and projection-realization requirements. Batch 9 then adds the domain-restricted RCLM Direct Non-Lossy RSI Engine Theorem: it proves construction by $\mathfrak{E}_t^{\text{RCLM}}$ only on a declared direct-engine domain with strict witnesses and sound generator/certifier assumptions. Batch 11 adds a seed-domain/witness-library theorem showing sufficient architecture-specific conditions under which a declared seed class enters that direct-engine domain and persists across successor construction.*

1.5 Batch-3 synchronization: RCLM conservative-extension algebra

The Batch-3 update to the reference RCP paper adds an abstract non-lossy update algebra

$$\mathfrak{G}^{\text{NL}} = \langle \text{id}, \text{IsoExt}, \text{AppendMem}, \text{RevAdapter}, \text{VerifierExt}, \text{ToolExt} \rangle.$$

This subsection instantiates that algebra inside the RCLM architecture. The goal is to make a large subset of self-updates non-lossy by construction rather than only by after-the-fact residual estimation. The construction is still conditional: it proves exact or budgeted non-loss for candidates that actually have the declared conservative-extension form, but it does not prove that a useful conservative extension always exists or that the architecture can always find one within budget.

Definition 1.9 (RCLM certified ability set). *Let R_t be the declared resource state available for ability execution and verification. Let $\mathcal{U}_t^{\text{abil}}$ be the declared universe of auditable ability labels. The RCLM certified ability set is*

$$\mathfrak{A}_t^{\text{RCLM}}(\rho, R_t) = \{a \in \mathcal{U}_t^{\text{abil}} : \text{AbCert}_{t,a}(\rho, R_t) = 1\}.$$

Here $\text{AbCert}_{t,a}(\rho, R_t) = 1$ means that an audited policy, tool call, proof routine, solver, or verifier operation encoded by ρ performs ability a within resource budget R_t , with declared success, failure-probability, and resource certificates. The ability label a may denote a task family, proof obligation, tool-use routine, verifier operation, memory-retrieval operation, or certified inference subroutine. Ability membership is not a semantic assertion by name; it requires the corresponding audit certificate.

Definition 1.10 (Ability transport and non-lossy ability preservation). *For a candidate update ν , an ability-transport map is a partial map*

$$\Theta_{t,\nu}^{\mathfrak{A},\text{RCLM}} : \mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t) \rightarrow \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}}), R_{t+1}).$$

The update ν preserves certified abilities when

$$\Theta_{t,\nu}^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)) \subseteq \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}}), R_{t+1}).$$

It makes strict certified ability progress when there exists

$$a_{\text{new}} \in \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}}), R_{t+1}) \setminus \Theta_{t,\nu}^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)).$$

The notation records a set-inclusion progress order; it does not assert that such a new ability exists.

Assumption 1.11 (Callable predecessor retention). *For a conservative-extension update ν , the successor payload retains a callable embedded copy of the predecessor inference engine, old interface contracts, old verifier slice needed for predecessor ability certificates, and the old roll-back/ledger record. There is an overhead $\omega_t^{\text{call}}(a) \geq 0$ such that every predecessor ability $a \in \mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)$ can be executed in the successor with resource cost at most its old certified cost plus $\omega_t^{\text{call}}(a)$. The successor resource state R_{t+1} must explicitly include this overhead; otherwise ability preservation is not certified.*

Theorem 1.12 (Callable predecessor retention implies ability preservation). *Assume callable predecessor retention for ν . If every predecessor ability certificate remains verifiable by the retained predecessor verifier slice and the successor resource state pays the overhead ω_t^{call} , then ν preserves certified abilities in the sense of Definition 1.10.*

Proof. Let $a \in \mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)$. By definition of the ability set, the predecessor contains an audited solver or routine and a certificate $\text{AbCert}_{t,a}$. Callable predecessor retention embeds that solver or routine and retains the verifier slice needed to re-check the certificate. The resource hypothesis pays the execution overhead. Therefore the transported ability $\Theta_{t,\nu}^{\mathfrak{A},\text{RCLM}}(a)$ has a valid successor certificate and belongs to the successor ability set. Since this holds for arbitrary a , the transported old ability set is a subset of the successor ability set. $\square$

Definition 1.13 (RCLM structural recovery update). *A candidate ν is an RCLM structural recovery update if, on the protected meta-state domain, it admits a reversible dilation*

$$\tilde{\Phi}_\nu(\rho) = U_\nu(\rho \otimes |0\rangle\langle 0|_{\mathcal{W}_\nu})U_\nu^\dagger$$

for a unitary U_ν on an enlarged finite-dimensional space. Its structural recovery map is

$$\mathcal{R}_\nu^{\text{rec}}(X) = \text{Tr}_{\mathcal{W}_\nu}(U_\nu^\dagger X U_\nu).$$

The implemented successor may expose only a typed representation of $\tilde{\Phi}_\nu(\rho)$, but any claim of structural recovery must include the dilation, the retained workspace/ledger convention, and the type certificate licensing the calculation.

Proposition 1.14 (Exact recovery for RCLM structural updates). *If ν satisfies Definition 1.13, then*

$$\mathcal{R}_\nu^{\text{rec}}(\tilde{\Phi}_\nu(\rho)) = \rho$$

for every protected input state ρ in the declared domain.

Proof. Substitute the definition of $\tilde{\Phi}_\nu$:

$$\mathcal{R}_\nu^{\text{rec}}(\tilde{\Phi}_\nu(\rho)) = \text{Tr}_{\mathcal{W}_\nu}(U_\nu^\dagger U_\nu(\rho \otimes |0\rangle\langle 0|)U_\nu^\dagger U_\nu) = \text{Tr}_{\mathcal{W}_\nu}(\rho \otimes |0\rangle\langle 0|) = \rho.$$

$\square$

Definition 1.15 (RCLM conservative extension primitive). *A conservative extension primitive is a candidate ν whose protected-state action has the form*

$$\Phi_\nu(\rho) = V_\nu \rho V_\nu^\dagger \otimes \alpha_\nu,$$

where $V_\nu^\dagger V_\nu = I$, $\alpha_\nu \in \mathcal{D}(\mathcal{H}_\nu^{\text{ext}})$, and the same appended state α_ν is used for both members of every protected pair. Its canonical recovery map is

$$\mathcal{R}_\nu^{\text{rec}}(Y) = V_\nu^\dagger \text{Tr}_{\text{ext}}(Y) V_\nu.$$

The primitive is type-valid only when the appended state, embedding, and recovery map are included in $\text{TypeCert}_t(\nu)$.

Theorem 1.16 (Exact non-loss and recovery for RCLM conservative extensions). *Let ν satisfy Definition 1.15. For every protected pair $(\rho, \sigma) \in \mathcal{P}_t^{\text{RCLM}}$ with finite relative entropy,*

$$D(\Phi_\nu(\rho) \parallel \Phi_\nu(\sigma)) = D(\rho \parallel \sigma),$$

and

$$\mathcal{R}_\nu^{\text{rec}}(\Phi_\nu(\rho)) = \rho, \quad \mathcal{R}_\nu^{\text{rec}}(\Phi_\nu(\sigma)) = \sigma.$$

Thus the conservative extension has zero protected relative-entropy loss and zero recovery defect on the declared protected pair family, before any additional implementation approximation terms are added.

Proof. Relative entropy is invariant under isometric conjugation and additive under tensor products. Therefore

$$D(V\rho V^\dagger \otimes \alpha \parallel V\sigma V^\dagger \otimes \alpha) = D(V\rho V^\dagger \parallel V\sigma V^\dagger) + D(\alpha \parallel \alpha) = D(\rho \parallel \sigma).$$

The recovery calculation is

$$\mathcal{R}_\nu^{\text{rec}}(V\rho V^\dagger \otimes \alpha) = V^\dagger \text{Tr}_{\text{ext}}(V\rho V^\dagger \otimes \alpha)V = V^\dagger V\rho V^\dagger V = \rho,$$

using $\text{Tr}(\alpha) = 1$ and $V^\dagger V = I$. The same argument applies to σ . $\square$

Definition 1.17 (RCLM non-lossy primitive generators). *The RCLM non-lossy primitive generator set is*

$$\mathcal{G}_t^{\text{NL,RCLM}} = \{\text{id}, \text{IsoExt}, \text{AppendMem}, \text{RevAdapter}, \text{VerifierExt}, \text{ToolExt}\}_t.$$

A primitive instance $g \in \mathcal{G}_t^{\text{NL,RCLM}}$ is accepted as a certified non-lossy primitive only if it provides a primitive certificate

$$\text{PrimCert}_t(g) = (V_g, \alpha_g, \mathcal{R}_g^{\text{rec}}, \mathcal{P}_t^g, \epsilon_g, r_g, c_g^{\text{phys}}, \text{TypeCert}_g, \text{Trace}_g)$$

showing either exact conservative-extension form or an explicitly bounded approximation with loss budget ϵ_g and recovery budget r_g . The intended architectural meanings are:

- (a) *id*: retain the current system without modification;
- (b) *IsoExt*: embed the old system isometrically into a larger system with an appended module state;
- (c) *AppendMem*: append memory, data, proofs, or ledger records while retaining the old retrieval interface;
- (d) *RevAdapter*: add an adapter through a reversible or recoverable extension, not by overwriting the old pathway;
- (e) *VerifierExt*: add verifier power while retaining the old verifier slice and satisfying the trust-graph gate for any verifier-power claim;
- (f) *ToolExt*: add a tool interface or external routine while retaining the old system and documenting the tool contract in the certificate ledger.

The names do not themselves prove non-loss. The primitive certificate does.

Definition 1.18 (RCLM conservative-extension algebra). *The RCLM certified non-lossy update algebra is the set of finite compatible words generated by the primitive set:*

$$\mathfrak{G}_{\text{RCLM}}^{\text{NL}} = \langle \text{id}, \text{IsoExt}, \text{AppendMem}, \text{RevAdapter}, \text{VerifierExt}, \text{ToolExt} \rangle_{\text{RCLM}}.$$

A word

$$w = g_n \circ \dots \circ g_1$$

is compatible at time t when each primitive output type matches the next primitive input type, the protected pair set is transported after each primitive, each primitive certificate is valid on the transported pair family, and all resource costs are included in the resource ledger.

Definition 1.19 (Protected-pair pushforward). *For a primitive or word w , define the protected-pair pushforward by*

$$w_{\#} \mathcal{P}_t^{\text{RCLM}} = \{(\Phi_w(\rho), \Phi_w(\sigma)) : (\rho, \sigma) \in \mathcal{P}_t^{\text{RCLM}}\}.$$

For a word $w = g_n \circ \dots \circ g_1$, the intermediate transported pair sets are

$$\mathcal{P}_t^{(0)} = \mathcal{P}_t^{\text{RCLM}}, \quad \mathcal{P}_t^{(i)} = (g_i)_{\#} \mathcal{P}_t^{(i-1)}.$$

Assumption 1.20 (Composability of RCLM non-lossy primitive certificates). *For a word $w = g_n \circ \dots \circ g_1 \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, each primitive g_i is certified on the transported pair family $\mathcal{P}_t^{(i-1)}$, every recovery map has the correct input/output type for composition, all approximation errors are represented by trace-distance or residual budgets compatible with contractivity, and all state-space embeddings/restrictions are recorded in the type certificate. Without these domain and type conditions, no algebraic closure theorem is invoked.*

Lemma 1.21 (Two-step additive RCLM loss). *Let Φ have protected loss at most ϵ_{Φ} on $\mathcal{P}_t$, and let Ψ have protected loss at most ϵ_{Ψ} on $\Phi_{\#} \mathcal{P}_t$. Then $\Psi \circ \Phi$ has protected loss at most $\epsilon_{\Phi} + \epsilon_{\Psi}$ on $\mathcal{P}_t$.*

Proof. For any $(\rho, \sigma) \in \mathcal{P}_t$, insert the intermediate relative entropy:

$$\begin{aligned} & D(\rho \| \sigma) - D(\Psi\Phi(\rho) \| \Psi\Phi(\sigma)) \\ &= [D(\rho \| \sigma) - D(\Phi(\rho) \| \Phi(\sigma))] + [D(\Phi(\rho) \| \Phi(\sigma)) - D(\Psi\Phi(\rho) \| \Psi\Phi(\sigma))]. \end{aligned}$$

The first bracket is bounded by ϵ_{Φ} , and the second is bounded by ϵ_{Ψ} because $(\Phi(\rho), \Phi(\sigma)) \in \Phi_{\#} \mathcal{P}_t$. Taking the supremum over protected pairs gives the result. $\square$

Lemma 1.22 (Two-step additive RCLM recovery). *Let Φ have recovery map $\mathcal{R}_{\Phi}$ with defect at most r_{Φ} on $\mathcal{P}_t$. Let Ψ have recovery map $\mathcal{R}_{\Psi}$ with defect at most r_{Ψ} on $\Phi_{\#} \mathcal{P}_t$. If $\mathcal{R}_{\Phi}$ is contractive for the state distance used in the recovery defect, then $\mathcal{R}_{\Phi} \circ \mathcal{R}_{\Psi}$ recovers $\Psi \circ \Phi$ with defect at most $r_{\Phi} + r_{\Psi}$ on $\mathcal{P}_t$.*

Proof. For a protected state ρ , the triangle inequality gives

$$\begin{aligned} & d(\rho, \mathcal{R}_{\Phi} \mathcal{R}_{\Psi} \Psi\Phi(\rho)) \\ & \leq d(\rho, \mathcal{R}_{\Phi} \Phi(\rho)) + d(\mathcal{R}_{\Phi} \Phi(\rho), \mathcal{R}_{\Phi} \mathcal{R}_{\Psi} \Psi\Phi(\rho)). \end{aligned}$$

The first term is at most r_{Φ} . Contractivity of $\mathcal{R}_{\Phi}$ bounds the second term by

$$d(\Phi(\rho), \mathcal{R}_{\Psi} \Psi\Phi(\rho)) \leq r_{\Psi}.$$

The same argument holds for the other member of every protected pair. Taking the supremum gives the result. $\square$

Theorem 1.23 (RCLM conservative-extension algebra closure). *Assume composability of RCLM non-lossy primitive certificates. If $\Phi, \Psi \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, then the compatible composition $\Psi \circ \Phi$ is again a word in $\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$. If Φ and Ψ carry budgets (ϵ_Φ, r_Φ) and (ϵ_Ψ, r_Ψ) , then the composition carries valid conservative budgets*

$$\epsilon_{\text{tot}} = \epsilon_\Phi + \epsilon_\Psi, \quad r_{\text{tot}} = r_\Phi + r_\Psi.$$

Proof. By definition, $\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$ is generated by compatible finite words over the certified primitive set. Concatenating two compatible words gives another finite compatible word, so $\Psi \circ \Phi \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$. Lemma 1.21 gives additive loss, and Lemma 1.22 gives additive recovery defect. The domain, type, and resource conditions are supplied by the composability assumption. $\square$

Theorem 1.24 (Finite-word additive budget for RCLM algebraic updates). *Let $w = g_n \circ \dots \circ g_1 \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$ be a compatible word. Suppose primitive g_i has loss budget ϵ_i and recovery budget r_i on the transported pair set $\mathcal{P}_t^{(i-1)}$. Then w has loss and recovery budgets*

$$\epsilon_w = \sum_{i=1}^n \epsilon_i, \quad r_w = \sum_{i=1}^n r_i.$$

If every primitive is an exact conservative extension on the transported protected family, then $\epsilon_w = r_w = 0$.

Proof. Apply Theorem 1.23 inductively over the word. Exact primitives have zero budgets, so the finite sums are zero. $\square$

Definition 1.25 (RCLM algebraic non-loss admissibility). *A candidate ν belongs to the RCLM algebraic non-loss admissible set*

$$\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$$

when:

- (i) ν has a compatible word representation $w_\nu \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$;
- (ii) every primitive certificate in the word is type-valid, domain-valid, and ledger-recorded;
- (iii) the transported pair sets are defined at every intermediate step;
- (iv) the additive budgets satisfy $\epsilon_{w_\nu} \leq \varepsilon_t$ and $r_{w_\nu} \leq r_{\max, t}$;
- (v) all physical/resource costs of the primitive word are included in C_t^{tot} and the resource ledger;
- (vi) semantic, trust-graph, coverage, calibration, and estimator gates still hold for any claims not implied by the algebraic non-loss certificate.

Define the algebraically non-lossy RCLM subgate by

$$\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG-NLalg}}(t, \rho_t^{\text{sys}}) = \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}).$$

Proposition 1.26 (Algebraic non-loss gate supplies loss and recovery certificates). *If $\nu \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$, then the protected-pair loss and recovery residuals satisfy*

$$\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} \leq \varepsilon_t, \quad \text{Rec}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} \leq r_{\max, t},$$

relative to the transported pair sets, type certificates, and approximation budgets recorded in the primitive word certificate.

Proof. Membership in $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$ gives a compatible word representation and valid primitive certificates. By Theorem 1.24, the word has loss budget ϵ_{w_ν} and recovery budget r_{w_ν} . The admissibility definition requires these to be at most ε_t and $r_{\max,t}$. Therefore the corresponding loss and recovery residuals are nonpositive. $\square$

Corollary 1.27 (Strict ability progress from a certified new conservative extension). *Assume $\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG-NLalg}}(t, \rho_t^{\text{sys}})$, callable predecessor retention holds, and the appended module supplies a certified new ability*

$$a_{\text{new}} \in \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}}), R_{t+1})$$

that is not in the transported predecessor ability set. Then ν preserves predecessor certified abilities and makes strict certified ability progress. This is a sufficient condition for ability-set progress, not a proof that such a module exists.

Proof. Ability preservation follows from Theorem 1.12. Strict progress follows because the certified new ability lies in the successor ability set and outside the transported predecessor set by hypothesis. $\square$

Limitation 1.28 (Batch-3 algebra is not yet an engine-existence theorem). *The RCLM conservative-extension algebra changes the candidate-generation target: instead of searching arbitrary self-edits first, the architecture can search a certified non-lossy primitive algebra. This reduces the burden of non-loss verification for those primitives. It does not prove that a beneficial primitive exists, that Level 2 inference can construct it, that resource costs scale, or that engine viability is nonempty. Those claims require the later RCLM lift and constructive beneficial-extension modules, together with the Batch-5 engine-viability kernel conditions below. The Batch-4 tangent-cone section supplies only a local conditional progress construction and does not by itself prove that a useful tangent direction exists.*

1.6 Batch-4 synchronization: RCLM tangent-cone local progress

The Batch-4 update to the reference RCP paper adds a tangent-cone local progress theorem. The abstract theorem says that, if the local residuals defining the admissible update set admit a strict inward direction with positive directional derivative of the engine-potential functional, then a sufficiently small finite update remains inside the gate and produces positive local progress. This subsection instantiates that construction in the RCLM architecture using the RCLM residuals

$$q_{t,j}^{\text{RCLM}}(\nu) \leq 0$$

and the architecture-side projected-gradient object

$$P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_\nu \text{RSIPot}_t.$$

The result is local and conditional. It is not a proof that such a direction always exists, that local potential progress is identical to strict certified ability-set expansion, or that progress remains viable forever.

Definition 1.29 (RCLM local candidate chart). *At time t , let $\mathcal{V}_t^{\text{RCLM}}$ be a finite-dimensional real normed candidate-coordinate space. A local candidate chart around no-op is a map*

$$\chi_t^{\text{RCLM}} : \mathcal{U}_t^{\text{RCLM}} \subseteq \mathcal{V}_t^{\text{RCLM}} \rightarrow \Omega_t^{\text{aud}}$$

with $0 \in \mathcal{U}_t^{\text{RCLM}}$ and

$$\chi_t^{\text{RCLM}}(0) = \nu_t^0.$$

For $u \in \mathcal{U}_t^{\text{RCLM}}$, write

$$\nu(u) = \chi_t^{\text{RCLM}}(u), \quad \Phi_u = \Phi_{\nu(u)}.$$

The chart may parameterize infinitesimal adapter insertions, conservative-extension amplitudes, verifier-extension toggles, tool-extension parameters, memory-extension weights, or other locally differentiable candidate families. It does not include arbitrary code rewrites unless those rewrites have a finite-dimensional differentiable chart and type-valid certificates.

Definition 1.30 (Local RCLM residual system). Let $\mathcal{J}_t^{\text{local}}$ be the finite set of residual components used for local tangent-cone analysis. For each $j \in \mathcal{J}_t^{\text{local}}$, define

$$q_{t,j}^{\text{RCLM}}(u) = q_{t,j}^{\text{RCLM}}(\chi_t^{\text{RCLM}}(u); \rho_t^{\text{sys}}).$$

The sign convention is

$$q_{t,j}^{\text{RCLM}}(u) \leq 0 \iff \text{the } j\text{th local RCP-RCLM residual is satisfied.}$$

The residual family may include type, coupling, certificate, trust, coverage, calibration, estimator, relative-entropy loss, recovery, Lyapunov drift, barrier, ambiguity, information-bottleneck, semantic, self-model, verifier, physical-resource, resource-scaling, certificate-inflation, and algebraic non-loss residuals. If a candidate claims Batch-3 construction-level non-loss, then the local residual family includes the algebraic residual for

$$\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}).$$

If a candidate claims only preservation-level admissibility, the algebraic residual may be omitted but the ordinary loss and recovery residuals remain required.

Definition 1.31 (Active local residuals at no-op). Assume the no-op is locally admissible, so $q_{t,j}^{\text{RCLM}}(0) \leq 0$ for every $j \in \mathcal{J}_t^{\text{local}}$. The active residual set is

$$J_t^0 = \{j \in \mathcal{J}_t^{\text{local}} : q_{t,j}^{\text{RCLM}}(0) = 0\},$$

and the inactive residual margin is

$$\mu_t^{\text{inactive}} = \min_{j \notin J_t^0} \{-q_{t,j}^{\text{RCLM}}(0)\},$$

with the convention $\mu_t^{\text{inactive}} = +\infty$ if all residuals are active. Only active residuals constrain the first-order tangent cone. Inactive residuals are protected by their strict slack and by the finite-step radius condition below.

Definition 1.32 (RCLM admissible tangent cone). Assume every active residual is Gateaux differentiable at $u = 0$. The RCLM admissible tangent cone at ρ_t^{sys} is

$$T_{\mathcal{A}_t^{\text{RCLM}}}(\rho_t^{\text{sys}}) = \{v \in \mathcal{V}_t^{\text{RCLM}} : Dq_{t,j}^{\text{RCLM}}(0)[v] \leq 0 \quad \forall j \in J_t^0\}.$$

When the state is clear, write $T_{\mathcal{A}_t^{\text{RCLM}}}$. A direction v is strictly inward with margin $a_t > 0$ when

$$Dq_{t,j}^{\text{RCLM}}(0)[v] \leq -a_t \|v\| \quad \forall j \in J_t^0.$$

Definition 1.33 (RCLM RSI potential). The local RCLM RSI potential is a differentiable scalar functional

$$\text{RSIPot}_t^{\text{RCLM}}(u; \rho_t^{\text{sys}}, R_t)$$

representing a declared local proxy for future non-lossy improvement potential. It may combine conservative-extension ability gain, no-op-normalized safe-capability gain, verifier-power gain,

search-power gain, resource-efficiency gain, and future viability proxies, but its components and weights must be fixed before candidate selection. The no-op-normalized local potential increment is

$$\Delta \text{RSIPot}_t^{\text{RCLM}}(u) = \text{RSIPot}_t^{\text{RCLM}}(u; \rho_t^{\text{sys}}, R_t) - \text{RSIPot}_t^{\text{RCLM}}(0; \rho_t^{\text{sys}}, R_t).$$

A positive potential increment is not automatically a certified new ability. Ability-set progress requires the separate ability certificate of Definition 1.10 and Corollary 1.27.

Definition 1.34 (Projected tangent-cone gradient). Assume $\mathcal{V}_t^{\text{RCLM}}$ is equipped with an inner product and that $T_{\mathcal{A}_t^{\text{RCLM}}}$ is a nonempty closed convex cone. Let

$$g_t^{\text{RSI}} = \nabla_u \text{RSIPot}_t^{\text{RCLM}}(0; \rho_t^{\text{sys}}, R_t).$$

The projected tangent-cone gradient is

$$\boxed{P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_\nu \text{RSIPot}_t := P_{T_{\mathcal{A}_t^{\text{RCLM}}}}(g_t^{\text{RSI}})}.$$

The notation ∇_ν means the gradient in the local candidate coordinates u under the chart χ_t^{RCLM} . If the candidate family is not differentiably charted, this object is undefined and the tangent-cone theorem cannot be invoked.

Assumption 1.35 (RCLM local residual curvature). There exist a radius $\bar{\alpha}_t > 0$ and constants $L_{t,j}^q < \infty$ such that for every unit direction v , every $0 \leq \alpha \leq \bar{\alpha}_t$, and every $j \in \mathcal{J}_t^{\text{local}}$,

$$q_{t,j}^{\text{RCLM}}(\alpha v) \leq q_{t,j}^{\text{RCLM}}(0) + \alpha Dq_{t,j}^{\text{RCLM}}(0)[v] + \frac{1}{2} L_{t,j}^q \alpha^2.$$

For inactive residuals, the same inequality is used with any certified directional derivative bound, or the verifier supplies a direct finite-step bound. The constants and radius are certificate quantities; they are not implied by notation.

Assumption 1.36 (RCLM RSI-potential lower curvature). There exist a radius $\bar{\alpha}_t > 0$ and a constant $L_t^{\text{RSI}} < \infty$ such that for every unit direction v and every $0 \leq \alpha \leq \bar{\alpha}_t$,

$$\text{RSIPot}_t^{\text{RCLM}}(\alpha v) \geq \text{RSIPot}_t^{\text{RCLM}}(0) + \alpha \langle g_t^{\text{RSI}}, v \rangle - \frac{1}{2} L_t^{\text{RSI}} \alpha^2.$$

This is a local smoothness certificate for the declared potential, not a theorem that the potential is globally aligned with real capability.

Definition 1.37 (Certified RCLM tangent step radius). Let v be a unit strictly inward direction with active margin $a_t > 0$ and positive potential derivative

$$\gamma_t(v) = \langle g_t^{\text{RSI}}, v \rangle > 0.$$

A finite step αv is tangent-certified when

$$0 < \alpha \leq \alpha_t^{\text{TC}}(v),$$

where

$$\alpha_t^{\text{TC}}(v) = \min \left\{ \bar{\alpha}_t, \min_{j \in \mathcal{J}_t^0: L_{t,j}^q > 0} \frac{2a_t}{L_{t,j}^q}, \min_{j \notin \mathcal{J}_t^0: L_{t,j}^q > 0} \sqrt{\frac{2\mu_t^{\text{inactive}}}{L_{t,j}^q}}, \frac{\gamma_t(v)}{L_t^{\text{RSI}}} \right\},$$

with terms involving empty sets or zero denominators omitted. The step-radius certificate must also include type, embedding, semantic-coupling, resource, and primitive-word-domain validity over the segment $\{\alpha v : 0 \leq \alpha \leq \alpha_t^{\text{TC}}(v)\}$.

Assumption 1.38 (RCLM local non-loss inheritance). *If the local chart is declared to lie inside the Batch-3 algebraic class, then every finite tangent step used for engine-mode progress satisfies*

$$\chi_t^{\text{RCLM}}(\alpha v) \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$$

with transported-pair and primitive-word certificates valid on the segment. If this assumption is not supplied, the finite step must pass the ordinary non-loss and recovery residuals directly; it cannot claim Batch-3 algebraic non-loss by tangent notation alone.

Theorem 1.39 (RCLM tangent-cone local progress). *Assume the local candidate chart, local residual system, active-set definitions, residual curvature, RSI-potential lower curvature, and any required local non-loss inheritance certificates. Suppose there exists a unit direction $v_t \in T_{\mathcal{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}})}$ that is strictly inward with margin $a_t > 0$ and has positive potential derivative*

$$\gamma_t(v_t) = \langle \nabla_u \text{RSIPot}_t^{\text{RCLM}}(0; \rho_t^{\text{sys}}, R_t), v_t \rangle > 0.$$

Then for every $0 < \alpha_t \leq \alpha_t^{\text{TC}}(v_t)$, the candidate

$$\nu_t^{\text{TC}} = \chi_t^{\text{RCLM}}(\alpha_t v_t)$$

satisfies all local residual constraints:

$$q_{t,j}^{\text{RCLM}}(\nu_t^{\text{TC}}) = q_{t,j}^{\text{RCLM}}(\alpha_t v_t) \leq 0 \quad \forall j \in \mathcal{J}_t^{\text{local}},$$

and produces positive local RSI-potential progress:

$$\Delta \text{RSIPot}_t^{\text{RCLM}}(\alpha_t v_t) > 0.$$

If the local chart is certified inside $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$, then ν_t^{TC} also inherits the Batch-3 algebraic loss and recovery budgets. If the chart is not certified inside $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$, the theorem supplies only local residual satisfaction and potential progress; ordinary loss and recovery certificates must still be checked directly.

Proof. For active residuals $j \in J_t^0$, strict inwardness gives

$$Dq_{t,j}^{\text{RCLM}}(0)[v_t] \leq -a_t.$$

By residual curvature,

$$q_{t,j}^{\text{RCLM}}(\alpha_t v_t) \leq 0 - \alpha_t a_t + \frac{1}{2} L_{t,j}^q \alpha_t^2.$$

The step-radius condition $\alpha_t \leq 2a_t/L_{t,j}^q$ when $L_{t,j}^q > 0$ makes the right-hand side nonpositive; if $L_{t,j}^q = 0$, it is already nonpositive. For inactive residuals, $q_{t,j}^{\text{RCLM}}(0) \leq -\mu_t^{\text{inactive}}$. The inactive part of the radius certificate gives enough finite-step slack to keep

$$q_{t,j}^{\text{RCLM}}(\alpha_t v_t) \leq 0.$$

Thus every local residual is nonpositive.

For the potential, the lower-curvature assumption gives

$$\Delta \text{RSIPot}_t^{\text{RCLM}}(\alpha_t v_t) \geq \alpha_t \gamma_t(v_t) - \frac{1}{2} L_t^{\text{RSI}} \alpha_t^2.$$

Because $\alpha_t \leq \gamma_t(v_t)/L_t^{\text{RSI}}$ when $L_t^{\text{RSI}} > 0$, the right-hand side is at least $\alpha_t \gamma_t(v_t)/2 > 0$. If $L_t^{\text{RSI}} = 0$, positivity is immediate. The final statement follows from Assumption 1.38 and Proposition 1.26 for algebraic charts; otherwise the theorem makes no algebraic non-loss claim. $\square$

Corollary 1.40 (Projected tangent-cone gradient criterion). *Assume $T_{\mathcal{A}_t^{\text{RCLM}}}$ is a nonempty closed convex cone and let*

$$p_t = P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_u \text{RSIPot}_t^{\text{RCLM}}(0; \rho_t^{\text{sys}}, R_t).$$

If $p_t \neq 0$ and the normalized direction $v_t = p_t / \|p_t\|$ is certified strictly inward for the active RCLM residuals, then the hypotheses of Theorem 1.39 hold with

$$\gamma_t(v_t) = \|p_t\| > 0,$$

up to the same curvature and segment-validity assumptions.

Proof. For a closed convex cone, the projection identity gives

$$\langle g_t^{\text{RSI}}, p_t \rangle = \|p_t\|^2.$$

Therefore for $v_t = p_t / \|p_t\|$,

$$\langle g_t^{\text{RSI}}, v_t \rangle = \|p_t\| > 0.$$

The strict-inward and curvature hypotheses are exactly those required by Theorem 1.39. $\square$

Limitation 1.41 (Tangent progress is not global engine viability). *The tangent-cone theorem converts a strict feasible positive tangent direction into a small certified positive-potential update. It does not prove that such a direction exists at every state, that RSIPot_t perfectly measures future ability-set expansion, that finite local steps compose indefinitely, or that the successor lies in a nonempty engine viability kernel. The Batch-5 engine-viability section below proves recursive continuation once a declared RCLM engine viability kernel is nonempty; it does not remove the separate RCLM lift and constructive beneficial-extension obligations.*

1.7 Batch-5 synchronization: RCLM engine-viability nonemptiness

The Batch-5 update to the reference RCP paper adds an engine-viability nonemptiness theorem for conservative-extension systems. The abstract statement is not that every state can self-improve. It is the conditional implication

$$\rho_t \in \mathcal{V}_t^{\text{engine}} \implies \exists \nu_t \in \Omega_t^{\text{NL}}$$

such that

$$\rho_{t+1} = \Phi_{\nu_t}(\rho_t) \in \mathcal{V}_{t+1}^{\text{engine}}.$$

This subsection instantiates that implication for RCLM states and RCLM conservative-extension primitive words. It closes the gap between a single local Batch-4 progress step and a recursively continuable engine path, but only for states in a declared, certified RCLM engine-viability kernel.

Definition 1.42 (RCLM engine base set). *For each time t , the RCLM engine base set is a declared subset*

$$\mathcal{K}_t^{\text{RCLM-engine}} \subseteq \mathcal{K}_{\text{RCLM},t}^{\text{DG}}$$

of states satisfying the ordinary post-D-G state constraints and the additional prerequisites for conservative-extension engine continuation. These prerequisites include typed density validity, semantic-coupling validity, trust-graph support, adaptive-audit trust support, resource availability, certificate-ledger consistency, callable predecessor retention readiness, and a declared ability universe. Membership in $\mathcal{K}_t^{\text{RCLM-engine}}$ is a certificate claim, not a consequence of notation.

Definition 1.43 (RCLM engine residual vector). *For a candidate ν and state ρ , define the RCLM engine residual vector*

$$Q_t^{\text{RCLM-engine}}(\nu; \rho) = (q_{t,j}^{\text{RCLM-engine}}(\nu; \rho))_{j \in J_t^{\text{RCLM-engine}}}.$$

The sign convention is

$$q_{t,j}^{\text{RCLM-engine}}(\nu; \rho) \leq 0 \iff \text{the } j\text{th engine-continuation requirement holds.}$$

The residual family includes, at minimum, algebraic non-loss, constructive recovery, ability preservation, certified new ability when progress is claimed, resource affordability, resource-scaling closure when an engine-mode scaling claim is made, trust admissibility, semantic coupling, estimator completeness, coverage/calibration, and successor-kernel membership residuals.

Definition 1.44 (RCLM conservative-extension engine move set). *Let $g_t^{\mathfrak{A}} \geq 0$ be the declared certified ability-progress threshold. The RCLM conservative-extension engine move set is*

$$\begin{aligned} \mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}}) = \{ \nu \in \Omega_t^{\text{NL,RCLM}} : & \nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG-NLalg}}(t, \rho), \\ & Q_t^{\text{RCLM-engine}}(\nu; \rho) \leq 0, \\ & \mathfrak{A}_t^{\text{RCLM}}(\rho, R_t) \text{ is preserved by } \nu, \\ & \text{if } g_t^{\mathfrak{A}} > 0, \text{ then } \nu \text{ adds at least one certified new ability} \}. \end{aligned}$$

Here $\Omega_t^{\text{NL,RCLM}} \subseteq \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$ is the audited set of compatible primitive words produced at time t . If the architecture does not provide such a set, the engine-viability theorem in this subsection cannot be invoked.

Definition 1.45 (Finite-horizon RCLM engine viability kernels). *Fix a horizon N and thresholds $g_t^{\mathfrak{A}}$. Define the finite-horizon RCLM engine viability kernels backward by*

$$\mathcal{V}_N^{\text{RCLM-engine},N} = \mathcal{K}_N^{\text{RCLM-engine}},$$

and, for $t = N - 1, \dots, 0$,

$$\begin{aligned} \mathcal{V}_t^{\text{RCLM-engine},N} = \{ \rho \in \mathcal{K}_t^{\text{RCLM-engine}} : & \exists \nu \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}}) \\ & \text{such that } \Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N} \}. \end{aligned}$$

Thus membership means not merely that the present state is safe, but that a certified conservative-extension engine move exists whose successor remains viable for the rest of the declared horizon.

Theorem 1.46 (RCLM engine-viability nonemptiness step). *Fix a finite horizon N . If*

$$\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},N},$$

then there exists

$$\nu_t \in \Omega_t^{\text{NL,RCLM}}$$

such that

$$\nu_t \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}; g_t^{\mathfrak{A}})$$

and

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N}.$$

Moreover, ν_t inherits the Batch-3 algebraic non-loss and recovery budgets, preserves predecessor certified abilities, and supplies the declared ability-progress certificate whenever $g_t^{\mathfrak{A}} > 0$.

Proof. The existence of ν_t and the successor-kernel membership are exactly the defining conditions of $\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},N}$. By Definition 1.44, such a ν_t belongs to $\Omega_t^{\text{NL,RCLM}}$ and to the algebraic non-loss subgate. Proposition 1.26 gives the protected loss and recovery budgets. Ability preservation is part of the engine move-set definition and may be justified by Theorem 1.12 when callable predecessor retention supplies the certificate. If $g_t^{\mathfrak{A}} > 0$, the move-set definition also requires a certified new ability, so strict ability progress follows by Corollary 1.27. No conclusion is made for states outside the viability kernel. $\square$

Theorem 1.47 (Finite-horizon RCLM engine trajectory). *If*

$$\rho_0^{\text{sys}} \in \mathcal{V}_0^{\text{RCLM-engine},N},$$

then there exist candidates $\nu_0, \dots, \nu_{N-1}$ such that, for every $t < N$,

$$\begin{aligned} \nu_t &\in \Omega_t^{\text{NL,RCLM}}, & \rho_{t+1}^{\text{sys}} &= \Phi_{\nu_t}(\rho_t^{\text{sys}}), \\ \rho_t^{\text{sys}} &\in \mathcal{K}_t^{\text{RCLM-engine}}, & \rho_{t+1}^{\text{sys}} &\in \mathcal{V}_{t+1}^{\text{RCLM-engine},N}, \end{aligned}$$

and every step satisfies the declared conservative-extension non-loss, recovery, ability-preservation, resource, trust, coverage, calibration, estimator, and successor-viability residuals.

Proof. Apply Theorem 1.46 at $t = 0$ to obtain ν_0 and $\rho_1^{\text{sys}} \in \mathcal{V}_1^{\text{RCLM-engine},N}$. Repeat the argument at $t = 1, 2, \dots, N-1$. Since each kernel is a subset of the corresponding base set, every state lies in $\mathcal{K}_t^{\text{RCLM-engine}}$. The residual conclusions follow from membership in $\mathcal{E}_t^{\text{RCLM-CE}}$ at each step. $\square$

Definition 1.48 (Certified RCLM conservative-extension plan). *A certified RCLM conservative-extension plan over horizon N is a sequence*

$$\Pi_{0:N}^{\text{RCLM}} = (\nu_0, \dots, \nu_{N-1})$$

with associated states $\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}})$ such that

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-engine}}, \quad \nu_t \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}; g_t^{\mathfrak{A}})$$

for all $t < N$, and every primitive word, ability certificate, recovery map, resource ledger entry, verifier support graph, and successor-kernel witness is recorded in the certificate ledger.

Theorem 1.49 (Certified plan implies nonempty RCLM engine kernel). *If a certified RCLM conservative-extension plan $\Pi_{0:N}^{\text{RCLM}}$ starts at ρ_0^{sys} , then*

$$\rho_0^{\text{sys}} \in \mathcal{V}_0^{\text{RCLM-engine},N}.$$

More generally, every state ρ_t^{sys} along the plan lies in $\mathcal{V}_t^{\text{RCLM-engine},N}$.

Proof. We prove the claim backward. By definition of the plan, $\rho_N^{\text{sys}} \in \mathcal{K}_N^{\text{RCLM-engine}} = \mathcal{V}_N^{\text{RCLM-engine},N}$. Suppose $\rho_{t+1}^{\text{sys}} \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N}$. The plan certificate gives $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-engine}}$, $\nu_t \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}; g_t^{\mathfrak{A}})$, and $\Phi_{\nu_t}(\rho_t^{\text{sys}}) = \rho_{t+1}^{\text{sys}}$. Therefore $\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},N}$ by Definition 1.45. Induction gives the result for all t , including $t = 0$. $\square$

Definition 1.50 (Infinite RCLM engine viability kernel). *An infinite RCLM engine viability kernel is a sequence of sets $\{\mathcal{V}_t^{\text{RCLM-engine},\infty}\}_{t \geq 0}$ such that*

$$\mathcal{V}_t^{\text{RCLM-engine},\infty} \subseteq \mathcal{K}_t^{\text{RCLM-engine}}$$

and every $\rho \in \mathcal{V}_t^{\text{RCLM-engine},\infty}$ admits some

$$\nu \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}})$$

with

$$\Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},\infty}.$$

Theorem 1.51 (Infinite RCLM engine path from infinite viability). *If*

$$\rho_0^{\text{sys}} \in \mathcal{V}_0^{\text{RCLM-engine},\infty},$$

then there exists an infinite sequence of candidates $(\nu_t)_{t \geq 0}$ such that

$$\nu_t \in \Omega_t^{\text{NL,RCLM}}, \quad \rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}), \quad \rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-engine}}$$

for all t , and every step satisfies the declared conservative-extension engine move residuals.

Proof. Apply Definition 1.50 recursively. From $\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},\infty}$, choose $\nu_t \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}; g_t^{\mathfrak{A}})$ with $\rho_{t+1}^{\text{sys}} \in \mathcal{V}_{t+1}^{\text{RCLM-engine},\infty}$. Since each kernel is a subset of the engine base set and each engine move set is a subset of $\Omega_t^{\text{NL,RCLM}}$, the displayed properties hold by induction. $\square$

Corollary 1.52 (Batch-5 finite recursive RSI condition for RCLM). *For a declared horizon N , if $\rho_0^{\text{sys}} \in \mathcal{V}_0^{\text{RCLM-engine},N}$ and $g_t^{\mathfrak{A}} > 0$ for every $t < N$, then the architecture has a finite sequence of non-lossy conservative-extension updates that preserve predecessor certified abilities, add at least one certified new ability at each step, and keep the successor inside the next engine viability kernel.*

Proof. Theorem 1.47 gives the finite sequence and successor-kernel membership. Since each $g_t^{\mathfrak{A}} > 0$, each engine move requires a certified new ability. Ability preservation and strict progress follow from Theorem 1.12 and Corollary 1.27; non-loss and recovery follow from the Batch-3 algebraic subgate. $\square$

Limitation 1.53 (Engine viability is still a certified domain condition). *Batch 5 closes the recursive-continuation gap only for states already certified to lie in $\mathcal{V}_t^{\text{RCLM-engine},N}$ or $\mathcal{V}_t^{\text{RCLM-engine},\infty}$. It does not prove that arbitrary RCLM states belong to those kernels, that Level 2 inference can always generate the required primitive word, that useful extension modules always exist, or that the certification/search cost is scalable outside the declared resource model. It is a conditional engine-viability theorem, not an unconditional direct RSI solution.*

1.8 Batch-6 synchronization: RCLM lift and minimal finite-dimensional reference engine

The Batch-3 algebra supplies a certified non-lossy primitive algebra, Batch 4 supplies a local progress construction when a strict feasible positive tangent direction is available, and Batch 5 supplies recursive continuation once a state is already inside an engine-viability kernel. The remaining architecture-specific bridge is the lift from the halted Level 2 recursive-coherence state into an actual Level 3 non-lossy primitive-word proposal. This subsection adds that lift without claiming that every Level 2 state can generate a useful primitive word. The lift is a certified candidate-generation map. It becomes an engine step only when its output also satisfies the Batch-3 non-loss algebra, the Batch-5 engine move set, and the ordinary post-D–G RCP gate.

Definition 1.54 (Batch-6 lift input package). *At time t , the lift input package is*

$$\mathbf{L}_t^{\text{in}} = (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}}),$$

where $\rho_t^{\text{ctx},*}$ is the halted Level 2 context state, M_t is the interpreted self-model payload, W_t is the verifier state, and $\mathcal{H}_t^{\text{cert}}$ is the append-only certificate ledger. The package is required to be time-stamped and typed. In particular, $\rho_t^{\text{ctx},*}$ must have a valid density-type certificate and $M_t, W_t, \mathcal{H}_t^{\text{cert}}$ must have operational encodings compatible with the system-state encoder.

Definition 1.55 (RCLM lift map). *The Batch-6 RCLM lift is a partially defined map*

$$\text{Lift}_t : (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}}) \longrightarrow \Omega_t^{\text{NL,RCLM}},$$

where $\Omega_t^{\text{NL,RCLM}} \subseteq \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$ is the audited set of compatible non-lossy RCLM primitive words available at time t . If the lift is undefined, returns an off-domain object, or returns a word without a complete primitive certificate, the architecture must treat the lift attempt as failed and may not claim Batch-6 engine construction for that step.

Definition 1.56 (Lift trace and lift certificate). *For a lifted candidate $\nu_t^{\text{lift}} = \text{Lift}_t(\mathbf{L}_t^{\text{in}})$, the lift trace is*

$$\text{LiftTrace}_t(\nu_t^{\text{lift}}) = (\mathbf{L}_t^{\text{in}}, w_{\nu_t}, \text{PrimCert}_t(w_{\nu_t}), \text{Reason}_t, \text{Reject}_t, \text{Seed}_t),$$

where w_{ν_t} is the primitive-word representation, $\text{PrimCert}_t(w_{\nu_t})$ is the collection of primitive certificates, Reason_t is the self-model explanation or proof sketch for why the candidate should improve RSIPot_t or the ability set, Reject_t records failed lift attempts, and Seed_t records randomness or search seeds. The lift certificate is

$$\text{LiftCert}_t(\nu_t^{\text{lift}}) = (\text{LiftTrace}_t, \text{TypeCert}_t, \text{DomCert}_t, \text{NLAlgCert}_t, \text{EngMoveCert}_t, \text{ResCert}_t),$$

where the entries certify, respectively, type/domain validity, primitive-word non-loss, membership in the relevant engine move set when claimed, and resource feasibility for generating and checking the lifted word.

Assumption 1.57 (Lift domain validity). *The lift is invoked only on an audited domain $\mathcal{X}_t^{\text{lift}}$ of input packages. On that domain, the map returns either a failure token or a candidate $\nu_t^{\text{lift}} \in \Omega_t^{\text{NL,RCLM}}$ with a complete LiftCert_t . The domain certificate must state which parts of $\rho_t^{\text{ctx},*}$, M_t , W_t , and $\mathcal{H}_t^{\text{cert}}$ the lift used. If a lift uses stale self-model information, unaudited verifier outputs, or unverifiable certificate-ledger entries, the lift certificate is invalid.*

Assumption 1.58 (Lift soundness). *For every lifted candidate with a valid lift certificate, the primitive word w_{ν_t} is compatible, all transported pair sets are defined, the Batch-3 algebraic non-loss certificate is valid, all recovery maps are type-correct, and all costs of generating, certifying, and applying the lifted word are included in the resource ledger. If the candidate is claimed to be an engine-continuation move, then EngMoveCert_t also certifies*

$$\nu_t^{\text{lift}} \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}, g_t^{\mathfrak{A}}).$$

This is an explicit architecture assumption. It is not a consequence of the fact that $\rho_t^{\text{ctx},}$ is a density state.*

Definition 1.59 (Lifted candidate set). *The lifted non-lossy candidate set is*

$$\Omega_t^{\text{lift,NL}} = \{\text{Lift}_t(\mathbf{L}_t^{\text{in}}) : \mathbf{L}_t^{\text{in}} \in \mathcal{X}_t^{\text{lift}}, \text{Lift}_t(\mathbf{L}_t^{\text{in}}) \neq \perp, \text{LiftCert}_t \text{ is valid}\}.$$

The symbol $\perp$ denotes explicit lift failure. Lift failure is safe: it produces no accepted update and therefore cannot by itself justify relaxing the RCP gate.

Theorem 1.60 (RCLM lift soundness). *Assume lift domain validity and lift soundness. If*

$$\nu_t^{\text{lift}} \in \Omega_t^{\text{lift,NL}},$$

then

$$\nu_t^{\text{lift}} \in \Omega_t^{\text{NL,RCLM}} \quad \text{and} \quad \nu_t^{\text{lift}} \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$$

provided the ordinary post-D-G type, semantic, trust, coverage, calibration, estimator, and resource residuals also pass. Consequently, the lifted candidate supplies the Batch-3 protected non-loss and recovery budgets recorded in its primitive-word certificate.

Proof. By Definition 1.59, a lifted candidate has a valid lift certificate and is returned on the audited lift domain. By Assumption 1.58, its primitive word is compatible, its transported pair sets are defined, and its Batch-3 algebraic non-loss certificate is valid with type-correct recovery maps and ledger-recorded costs. Therefore it satisfies the defining conditions of $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$, except for the ordinary post-D-G residuals that are not implied by algebraic non-loss. When those residuals also pass, the candidate belongs to the algebraic non-loss subgate. Proposition 1.26 then supplies the recorded protected loss and recovery bounds. $\square$

Theorem 1.61 (Lifted engine-continuation witness). *Fix a finite horizon N . Suppose*

$$\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},N}$$

and suppose the lift returns $\nu_t^{\text{lift}} \in \Omega_t^{\text{lift,NL}}$ with a valid engine-move certificate satisfying

$$\nu_t^{\text{lift}} \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}; g_t^{\mathfrak{A}})$$

and

$$\Phi_{\nu_t^{\text{lift}}}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N}.$$

Then ν_t^{lift} is a concrete RCLM witness for the Batch-5 engine-viability nonemptiness step at t : it is a non-lossy conservative-extension primitive word, it preserves predecessor certified abilities, it supplies the declared new-ability certificate when $g_t^{\mathfrak{A}} > 0$, and its successor remains inside the next engine kernel.

Proof. The displayed hypotheses place the lifted candidate in exactly the move set and successor condition required by Theorem 1.46. By Theorem 1.60, the candidate has the Batch-3 algebraic non-loss and recovery certificates. By Definition 1.44, membership in $\mathcal{E}_t^{\text{RCLM-CE}}$ includes ability preservation, engine residual satisfaction, and a certified new ability whenever the threshold $g_t^{\mathfrak{A}}$ is positive. The successor-kernel hypothesis gives recursive continuation. Thus the lift has supplied a concrete witness rather than only an existential statement. $\square$

Definition 1.62 (Minimal finite-dimensional reference RCLM engine). *A minimal finite-dimensional reference engine is a tuple*

$$\text{RefEng} = (\mathcal{H}_0, \rho_0, \mathcal{P}_0^{\text{RCLM}}, \mathfrak{A}_0^{\text{RCLM}}, R_0, V_0, \alpha_0, a_{\text{new}}, \text{AbilityCert}_{\text{new}}),$$

where:

- (i) $\mathcal{H}_0$ is finite-dimensional and $\rho_0 \in \mathcal{D}(\mathcal{H}_0)$;
- (ii) $\mathcal{P}_0^{\text{RCLM}} \subseteq \mathcal{D}(\mathcal{H}_0) \times \mathcal{D}(\mathcal{H}_0)$ is a protected pair family with finite relative entropy;
- (iii) $\mathfrak{A}_0^{\text{RCLM}}(\rho_0, R_0)$ is a finite certified predecessor ability set;
- (iv) $V_0 : \mathcal{H}_0 \rightarrow \mathcal{H}_1$ is an isometry into a successor space;

(v) $\alpha_0 \in \mathcal{D}(\mathcal{H}_{\text{new}})$ is a fixed appended module state independent of the protected pair member;

(vi) the successor update is

$$\Phi_{\text{ref}}(\rho) = V_0 \rho V_0^\dagger \otimes \alpha_0;$$

(vii) a_{new} is a declared new ability with a certificate $\text{AbilityCert}_{\text{new}}$ verifying that $a_{\text{new}} \notin \Theta_0^{\mathfrak{A}}(\mathfrak{A}_0^{\text{RCLM}}(\rho_0, R_0))$ and

$$a_{\text{new}} \in \mathfrak{A}_1^{\text{RCLM}}(\Phi_{\text{ref}}(\rho_0), R_1).$$

The reference engine is intentionally finite and explicit. It is a minimal construction showing what the architecture's conservative-extension claims mean, not an empirical implementation of a large trained model.

Assumption 1.63 (Callable predecessor interface for the reference engine). *The successor resource budget R_1 includes the overhead needed to call the predecessor solver/verifier through the isometric embedding V_0 , and the ability certificates for all $a \in \mathfrak{A}_0^{\text{RCLM}}(\rho_0, R_0)$ remain valid under the transported interface. Formally,*

$$\Theta_0^{\mathfrak{A}}(\mathfrak{A}_0^{\text{RCLM}}(\rho_0, R_0)) \subseteq \mathfrak{A}_1^{\text{RCLM}}(\Phi_{\text{ref}}(\rho_0), R_1).$$

Theorem 1.64 (Minimal reference engine: exact non-loss, recovery, and strict ability expansion). *Assume the minimal finite-dimensional reference engine of Definition 1.62 and the callable predecessor interface of Assumption 1.63. Then:*

(i) for every $(\rho, \sigma) \in \mathcal{P}_0^{\text{RCLM}}$,

$$D(\Phi_{\text{ref}}(\rho) \| \Phi_{\text{ref}}(\sigma)) = D(\rho \| \sigma);$$

(ii) the recovery map

$$\mathcal{R}_{\text{ref}}(X) = V_0^\dagger \text{Tr}_{\text{new}}(X) V_0$$

exactly recovers every protected predecessor state in the reference family;

(iii) predecessor certified abilities are preserved:

$$\Theta_0^{\mathfrak{A}}(\mathfrak{A}_0^{\text{RCLM}}(\rho_0, R_0)) \subseteq \mathfrak{A}_1^{\text{RCLM}}(\Phi_{\text{ref}}(\rho_0), R_1);$$

(iv) if $\text{AbilityCert}_{\text{new}}$ is valid, then strict certified ability progress holds:

$$\Theta_0^{\mathfrak{A}}(\mathfrak{A}_0^{\text{RCLM}}(\rho_0, R_0)) \subsetneq \mathfrak{A}_1^{\text{RCLM}}(\Phi_{\text{ref}}(\rho_0), R_1).$$

Proof. For item (i), isometric embedding preserves relative entropy on the support of the embedded states, and tensoring both protected states with the same fixed density state α_0 adds the same entropy terms to both sides of the relative-entropy expression; equivalently, relative entropy is additive under tensor products and invariant under isometries. Hence the protected relative entropy is exactly preserved. For item (ii), applying Φ_{ref} gives $V_0 \rho V_0^\dagger \otimes \alpha_0$. Taking the partial trace over the appended module returns $V_0 \rho V_0^\dagger$, and conjugating by $V_0^\dagger$ returns ρ because $V_0^\dagger V_0 = I$. Item (iii) is exactly the callable predecessor interface assumption. For item (iv), the new ability certificate places a_{new} in the successor ability set and outside the transported predecessor set. Together with item (iii), this yields proper subset inclusion. $\square$

Corollary 1.65 (Reference engine as a finite conservative-extension witness). *If the reference engine's primitive certificate, ability certificate, type certificate, trust certificate, semantic-coupling certificate, and resource certificate are all valid, then Φ_{ref} is a finite-dimensional witness of a Batch-3/Batch-6 non-lossy conservative-extension update that preserves protected distinctions, exactly recovers the predecessor, and strictly expands the certified ability set. If the successor is also certified to lie in $\mathcal{V}_1^{\text{RCLM-engine}, N}$, then it is a one-step witness for the Batch-5 engine-viability continuation condition.*

Proof. Theorem 1.64 supplies exact non-loss, exact recovery, ability preservation, and strict ability expansion. The additional certificates place the update inside the ordinary post-D–G RCP–RCLM gate and inside the certified non-loss algebra. If the successor-kernel certificate also holds, the update satisfies the one-step condition in Theorem 1.46. $\square$

Limitation 1.66 (What Batch 6 still does not prove). *Batch 6 supplies a formal lift map and a minimal finite-dimensional reference engine. It does not prove that a trained RCLM will learn a useful lift, that a lifted primitive word always exists, that the new ability certificate can be produced for open-ended tasks, that semantic coupling is correct, or that the finite reference construction scales to frontier systems. It turns the RSM-to-RSI bridge into an explicit object and gives a minimal witness model, but the later constructive beneficial-extension and empirical validation work remain necessary.*

2 Batch-7 Publication Rigor: Theorem Type Table and Claim Discipline

The architecture paper now contains several different kinds of formal statements. Some are algebraic facts about density states or conservative extensions. Some are telescoping or induction results. Some are certificate-composition theorems whose conclusions hold only if the relevant certificates are sound. Some are assumption-dependent existence or continuation theorems. Some are finite reference-engine demonstrations. This section prevents these categories from being conflated.

Definition 2.1 (Architecture theorem type). *For a theorem, proposition, lemma, corollary, or major claim T in this manuscript, define*

$$\text{Type}_{\text{RCLM}}(T) \in \{\text{Def}, \text{Alg}, \text{Tel}, \text{Cert}, \text{Exist}, \text{Ref}, \text{Impl}\}.$$

The intended meanings are:

- (i) *Def: a definition or convention fixing notation;*
- (ii) *Alg: an algebraic or structural result, such as density preservation, exact recovery under conservative extension, or closure of a certified non-lossy primitive algebra;*
- (iii) *Tel: a telescoping, induction, compactness, or finite-horizon accounting result;*
- (iv) *Cert: a certificate-composition result, meaning that certified residual nonpositivity implies true residual nonpositivity only on the stated certificate-validity event;*
- (v) *Exist: an assumption-dependent existence, search, lift, tangent-cone, progress, or engine-viability theorem;*
- (vi) *Ref: a finite-dimensional reference-engine demonstration showing that the objects are mathematically realizable in a toy or reference construction;*
- (vii) *Impl: an implementation, evaluation, or status statement rather than a theorem of the learned system.*

Type	Examples in this paper	What the result licenses
Def	Register definitions, typed density realization, gate residual vector, RCLM lift input package, RCLM successor-verification schema, explicit SV residual vector.	Fixes notation and admissibility vocabulary; proves no empirical property by itself.
Alg	Register density validity, unitary density preservation, conservative-extension exact non-loss/recovery, primitive-algebra closure.	Establishes mathematical structure under the stated construction hypotheses.
Tel	Bounded distinguishability loss, safe-capability telescoping, resource-scaling accounting, bounded certificate inflation, finite-horizon engine trajectory, goal-identity composition.	Transfers one-step inequalities into finite- or infinite-horizon bounds when budgets and hypotheses hold.
Cert	Architecture gate soundness, audited estimator soundness, trust-graph soundness, coverage, calibration, estimator subprotocol closure, successor-verification soundness composition.	Shows that accepted certificates imply gate satisfaction on audited domains and declared confidence events.
Exist	Budgeted search finds beneficial updates, tangent-cone local progress, engine-viability nonemptiness, lifted engine continuation witness, Batch-11 seed-domain entry, Batch-12B SV packet construction, SV seed-library nonemptiness, and infinite generated SV-domain closure from certified witness libraries.	Shows existence only under explicit margin, search, lift, kernel, builder, witness-library, reality-containment, tractability, or viability assumptions.
Ref	Minimal finite-dimensional reference RCLM engine.	Demonstrates a small exact construction; does not validate a trained frontier-scale architecture.

Type	Examples in this paper	What the result licenses
Impl	Implementation sketch, reproducible evaluation suite, implementation status table.	States what must be built, audited, or measured; does not itself prove implementation success.

Assumption 2.2 (Claim citation discipline). *Every use of a theorem or proposition in this manuscript is licensed only together with its theorem type and hypotheses. In particular, a certificate-composition theorem does not prove that certificates are obtainable; an assumption-dependent existence theorem does not prove that arbitrary RCLM states satisfy its premises; a reference-engine construction does not empirically validate the full architecture; and an implementation protocol does not count as experimental evidence.*

Proposition 2.3 (Theorem-type discipline prevents overclaiming). *Assume the claim citation discipline. Then no theorem of type Cert, Exist, Ref, or Impl licenses the statement that arbitrary RCLM systems solve non-lossy RSI without the additional certificate, existence, reference-domain, or implementation hypotheses explicitly named in that theorem.*

Proof. The conclusion is a direct consequence of Assumption 2.2. A Cert-type theorem is conditional on certificate validity. A Exist-type theorem is conditional on nonempty margins, search completeness, tangent directions, lift validity, or kernel membership. A Ref-type theorem applies only to the declared finite-dimensional reference construction. An Impl-type statement describes construction or evaluation obligations. None of these hypotheses is automatically true for arbitrary RCLM systems. Therefore none of those theorem types alone licenses an unconditional RSI-solution claim. $\square$

Limitation 2.4 (Publication-rigor scope). *This section does not weaken any earlier theorem. It classifies what each theorem can honestly support. The strongest current architecture claim remains conditional: within the declared domains, if the required objects, certificates, lift, conservative-extension plans, and engine-viability kernels are supplied, then the RCLM instantiation inherits the corresponding non-loss, recovery, progress, and continuation guarantees.*

3 Batch-8 Cross-Time Cleanup

The Batch-8 update to the reference RCP paper repairs three notational and logical risks that also affect the architecture paper. First, quantities such as ambiguity, bottleneck relevance, and update-description awareness compare objects across different recursive times. Such comparisons require explicit transport maps. Second, the state safe set must contain only state-level predicates, while update-dependent residuals must live in an admissibility relation. Third, a mathematical projection into a safe set is not automatically an implementable RCLM self-update. This section adds the RCLM-side synchronization layer.

Definition 3.1 (RCLM cross-time transport package). *At time t , an RCLM cross-time transport package is*

$$\text{Trans}_t^{\text{RCLM}} = (\Theta_t^{Q,\text{RCLM}}, \Theta_t^{E,\text{RCLM}}, \Theta_t^{Z,\text{RCLM}}, \Theta_t^{Y,\text{RCLM}}, \Theta_t^{G,\text{RCLM}}, \Delta_t^{\text{tr},\text{RCLM}}),$$

where

$$\begin{aligned} \Theta_t^{Q,\text{RCLM}} : Q_t^{\text{RCLM}} &\rightarrow Q_{t+1}^{\text{RCLM}}, & \Theta_t^{E,\text{RCLM}} : E_t^{\text{RCLM}} &\rightarrow E_{t+1}^{\text{RCLM}}, \\ \Theta_t^{Z,\text{RCLM}} : Z_t^{\text{RCLM}} &\rightarrow Z_{t+1}^{\text{RCLM}}, & \Theta_t^{Y,\text{RCLM}} : Y_t^{\text{RCLM}} &\rightarrow Y_{t+1}^{\text{RCLM}}, & \Theta_t^{G,\text{RCLM}} : G_t^{\text{RCLM}} &\rightarrow G_{t+1}^{\text{RCLM}}. \end{aligned}$$

The vector $\Delta_t^{\text{tr,RCLM}}$ records distortion budgets for entropy, mutual-information, semantic, and update-description comparisons. These maps may be deterministic, stochastic, symbolic, semantic, or verifier-defined, but their domains and distortion bounds must be logged in the certificate ledger.

Assumption 3.2 (RCLM transport validity). *For every accepted candidate ν whose certificate compares time- t and time- $t + 1$ variables, the transport package is domain-valid on the audited variables used by the certificate and satisfies the declared distortion inequalities. In particular, for the bottleneck variables,*

$$\left| I_t(Z_t^{\text{RCLM}}, Y_t^{\text{RCLM}}) - I_{t+1}(\Theta_t^{Z,\text{RCLM}} Z_t^{\text{RCLM}}, \Theta_t^{Y,\text{RCLM}} Y_t^{\text{RCLM}}) \right| \leq \delta_t^{ZY,\text{RCLM}},$$

and, for update-description awareness,

$$\left| I_t(M_t^{\text{RCLM}}, G_t^{\text{RCLM}}) - I_{t+1}(M_{t+1}^{\text{RCLM}}, \Theta_t^{G,\text{RCLM}} G_t^{\text{RCLM}}) \right| \leq \delta_t^{MG,\text{RCLM}},$$

whenever those information quantities are used as cross-time comparisons. If a comparison lacks a valid transport certificate, the corresponding residual is treated as failed.

Definition 3.3 (Transported RCLM ambiguity-collapse residual). *The transported RCLM unsupported-collapse residual is*

$$U_t^{\text{RCLM-tr}}(\nu) = \left[H_{t+1}(\Theta_t^{Q,\text{RCLM}} Q_t^{\text{RCLM}} \mid \Theta_t^{E,\text{RCLM}} E_t^{\text{RCLM}}) - H_{t+1}(Q_{t+1}^{\text{RCLM}} \mid E_{t+1}^{\text{RCLM}}) - I_{t+1}(\Theta_t^{Q,\text{RCLM}} Q_t^{\text{RCLM}}, E_{t+1}^{\text{RCLM}} \mid \Theta_t^{E,\text{RCLM}} E_t^{\text{RCLM}}) \right]_+.$$

The transported ambiguity gate is

$$U_t^{\text{RCLM-tr}}(\nu) \leq \zeta_t^{\text{tr,RCLM}}.$$

Definition 3.4 (Transported RCLM bottleneck condition). *The transported bottleneck preservation condition is*

$$I_{t+1}(Z_{t+1}^{\text{RCLM}}, Y_{t+1}^{\text{RCLM}}) \geq I_{t+1}(\Theta_t^{Z,\text{RCLM}} Z_t^{\text{RCLM}}, \Theta_t^{Y,\text{RCLM}} Y_t^{\text{RCLM}}) - \chi_t^{\text{tr,RCLM}}.$$

The older shorthand

$$I(Z_{t+1}^{\text{RCLM}}, Y_{t+1}^{\text{RCLM}}) \geq I(Z_t^{\text{RCLM}}, Y_t^{\text{RCLM}}) - \chi_t$$

is licensed only when the transport distortion is also included, for example by taking

$$\chi_t = \chi_t^{\text{tr,RCLM}} + \delta_t^{ZY,\text{RCLM}}.$$

Definition 3.5 (Transport-admissible RCLM candidates). *A candidate belongs to*

$$\mathcal{A}_{\text{tr}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$$

when it supplies a valid transport package $\text{Trans}_t^{\text{RCLM}}$, satisfies the transported ambiguity-collapse residual, satisfies the transported bottleneck condition, and includes all transport distortion budgets in the appropriate residual vector. A candidate with no cross-time entropy, mutual-information, semantic, or update-description comparison may satisfy this gate by a no-comparison certificate.

Theorem 3.6 (Transported comparisons license the RCLM shorthand bounds). *Assume RCLM transport validity. If a candidate satisfies the transported bottleneck condition with budget $\chi_t^{\text{tr,RCLM}}$, then*

$$I_{t+1}(Z_{t+1}^{\text{RCLM}}, Y_{t+1}^{\text{RCLM}}) \geq I_t(Z_t^{\text{RCLM}}, Y_t^{\text{RCLM}}) - (\chi_t^{\text{tr,RCLM}} + \delta_t^{ZY,\text{RCLM}}).$$

Thus the old one-line bottleneck inequality is valid only with the transport distortion added. Analogous transported statements hold for ambiguity and update-description comparisons when their declared distortion budgets are included.

Proof. By the transported bottleneck condition,

$$I_{t+1}(Z_{t+1}^{\text{RCLM}}, Y_{t+1}^{\text{RCLM}}) \geq I_{t+1}(\Theta_t^{\text{Z,RCLM}} Z_t^{\text{RCLM}}; \Theta_t^{\text{Y,RCLM}} Y_t^{\text{RCLM}}) - \chi_t^{\text{tr,RCLM}}.$$

By Assumption 3.2,

$$I_{t+1}(\Theta_t^{\text{Z,RCLM}} Z_t^{\text{RCLM}}; \Theta_t^{\text{Y,RCLM}} Y_t^{\text{RCLM}}) \geq I_t(Z_t^{\text{RCLM}}; Y_t^{\text{RCLM}}) - \delta_t^{\text{ZY,RCLM}}.$$

Combining the two inequalities gives the displayed bound. The other transported comparisons follow by the same substitution of transported variables and distortion budgets. $\square$

Definition 3.7 (RCLM state-only safe set). *The state-only RCLM safe set at time t is*

$$\begin{aligned} \mathcal{K}_{\text{RCLM},t}^{\text{state}} &= \{\rho \in \mathcal{D}(\mathcal{H}_t^{\text{sys}}) : V_{\text{state},t}^{\text{RCLM}}(\rho) \leq v_{\max,t}, \\ &\quad B_{i,t}^{\text{RCLM}}(\rho) \leq 0 \ \forall i, \\ &\quad L_{\text{div}}^{\text{RCLM}}(\rho) \geq \kappa_{\text{div},t}, B_X^{\text{sem}}(\rho) \leq 0 \ \forall X, F_t^M(\rho) \leq \epsilon_{M,t}\}. \end{aligned}$$

This set contains only predicates evaluable from the state and fixed time- t reference objects. It does not include candidate-indexed quantities such as relative-entropy loss of Φ_ν , recovery defect of Φ_ν , update-description awareness G_ν , verifier-change certificates, or projection-realization certificates.

Definition 3.8 (RCLM state and update Lyapunov split). *The state monitor is*

$$\begin{aligned} V_{\text{state},t}^{\text{RCLM}}(\rho) &= -a_L I_\rho(L; O) - a_S I_\rho(L; S) - a_M I_\rho(M; L, O, S) \\ &\quad + b_1 R_{\text{collapse},t}(\rho) + b_2 R_{\text{irreversible},t}(\rho) + b_3 R_{\text{misalignment},t}(\rho) + b_4 R_{\text{self-error},t}(\rho), \end{aligned}$$

with nonnegative specification weights. The candidate-indexed update monitor is

$$V_{\text{upd},t}^{\text{RCLM}}(\rho, G_\nu, \nu) = -c_G I_\rho(M; G_\nu) + c_U R_{\text{upd-risk},t}(\rho, \nu),$$

where G_ν is the audited update-description object and $R_{\text{upd-risk},t}$ contains update-specific risk terms. The older shorthand $V_{\text{RCLM},t}(\rho; \nu)$ abbreviates

$$V_{\text{RCLM},t}(\rho; \nu) = V_{\text{state},t}^{\text{RCLM}}(\rho) + V_{\text{upd},t}^{\text{RCLM}}(\rho, G_\nu, \nu).$$

Only $V_{\text{state},t}^{\text{RCLM}}$ is a state-safe-set predicate.

Definition 3.9 (RCLM update-dependent admissibility relation). *For a state ρ and candidate ν , define the update-dependent RCLM admissibility relation*

$$\mathcal{A}_{\text{RCLM}}^{\text{upd}}(t, \rho, \nu)$$

to hold when every update-dependent residual is satisfied: relative-entropy non-loss, constructive recovery, transported ambiguity, transported bottleneck, Lyapunov drift for $V_{\text{RCLM},t}(\cdot; \nu)$, type, coupling, non-oracular certificate, trust, coverage, calibration, estimator, resource, inflation, algebraic non-loss, tangent-cone, lift, and engine-viability residuals applicable to the claim being made. This is a relation, not a state set.

Definition 3.10 (State/update-separation admissible RCLM candidates). *A candidate belongs to*

$$\mathcal{A}_{\text{sep}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$$

when its certificate explicitly classifies every gate residual as state-only, successor-state, or update-dependent; uses $V_{\text{state},t}^{\text{RCLM}}$ only inside state safe-set predicates; uses $V_{\text{upd},t}^{\text{RCLM}}$ only inside the candidate admissibility relation; and records any candidate-dependent object that is not part of the state. This gate is a notation-discipline gate: it prevents a candidate-specific proof obligation from being hidden inside a state-only safe set.

Proposition 3.11 (State/update separation for the RCLM gate). *If a candidate ν satisfies*

$$\Phi_\nu(\rho_t^{\text{sys}}) \in \mathcal{K}_{\text{RCLM},t+1}^{\text{state}}$$

and $\mathcal{A}_{\text{RCLM}}^{\text{upd}}(t, \rho_t^{\text{sys}}, \nu)$ holds, then the candidate satisfies the corresponding state-level and update-level components of the full RCP–RCLM admissibility gate. Conversely, a proof of state membership alone does not prove update admissibility unless the update-dependent relation is also certified.

Proof. The successor-state membership supplies exactly the state-only predicates: state Lyapunov bound, ordinary barriers, diversity floor, semantic barriers, and self-model fidelity. The relation $\mathcal{A}_{\text{RCLM}}^{\text{upd}}$ supplies the candidate-dependent predicates by definition. The full gate is the conjunction of state-only successor constraints and update-dependent constraints, so the conjunction implies the corresponding components of full admissibility. The converse failure is immediate because quantities such as $\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_{\text{RCLM}}^t}$, recovery defect, transported ambiguity, trust support, and projection realization are not functions of the successor state alone. $\square$

Definition 3.12 (RCLM projection-realization certificate). *Let $\tilde{\rho}_{t+1}^{\text{sys}}$ be a raw proposed successor and let $\rho_{t+1}^{+, \text{sys}}$ be a projected state, for example a minimizer in $\mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}$. A projection-realization certificate is*

$$\text{ProjReal}_t^{\text{RCLM}}(\tilde{\rho}_{t+1}^{\text{sys}}, \rho_{t+1}^{+, \text{sys}}, \nu) = (\nu, \Phi_\nu, \rho_{t+1}^{+, \text{sys}}, \delta_t^{\text{proj-real}}, \text{TypeCert}_t, \text{Distort}_t),$$

where Φ_ν is a typed self-update map and

$$d(\rho_{t+1}^{+, \text{sys}}, \Phi_\nu(\rho_t^{\text{sys}})) \leq \delta_t^{\text{proj-real}}.$$

The packet Distort_t records how the projection-realization error is added to every residual affected by replacing the raw or projected state by the implemented successor.

Definition 3.13 (Projection-realization admissible candidates). *A candidate belongs to*

$$\mathcal{A}_{\text{proj-real}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$$

if either no projection is used, or a valid certificate $\text{ProjReal}_t^{\text{RCLM}}$ exists, the realization error is within the declared bound, the update map is type-compatible, and all affected residuals include $\delta_t^{\text{proj-real}}$ or a certified Lipschitz/continuity transfer of it.

Theorem 3.14 (Projected RCLM states require realization before acceptance). *Suppose $\rho_{t+1}^{+, \text{sys}} \in \mathcal{K}_{\text{RCLM},t+1}^{\text{state}}$ is obtained by mathematical projection. This fact alone licenses only the statement that the projected state is state-safe. If, in addition, $\nu \in \mathcal{A}_{\text{proj-real}}^{\text{RCLM}}(t, \rho_t^{\text{sys}})$ and $\mathcal{A}_{\text{RCLM}}^{\text{upd}}(t, \rho_t^{\text{sys}}, \nu)$ holds after including the projection-realization distortion, then ν may be treated as an accepted implementable RCLM self-update with successor $\Phi_\nu(\rho_t^{\text{sys}})$ within declared distortion of the projected state.*

Proof. Projection into $\mathcal{K}_{\text{RCLM},t+1}^{\text{state}}$ proves only membership of the mathematical target in a state set. The projection-realization certificate supplies a typed update map Φ_ν whose realized successor lies within $\delta_t^{\text{proj-real}}$ of that target and records the effect of this error on all residuals. The update-dependent admissibility relation then verifies the residuals for the implemented update after accounting for that distortion. Therefore the accepted object is the typed implemented update Φ_ν , not the projection operation by itself. $\square$

Limitation 3.15 (Batch 8 is cleanup, not a new engine theorem). *Batch 8 does not construct beneficial updates, prove useful tangent directions exist, or prove nonempty RCLM engine kernels. It repairs the formal language so that cross-time comparisons, state-safe-set claims, and projected successors cannot be used beyond what their certificates actually justify.*

4 Batch-10 Terminology Cleanup: Typed Self-Update Transitions

The pro-model evaluation noted that many Level 3 self-updates are not literally quantum channels: a verifier rewrite, memory append, tool addition, architecture widening, or certificate-generator update is more naturally a typed program transformation or stateful transition. This section fixes the terminology without changing the mathematics of the previous batches. The symbol Φ_ν remains the standard notation for the induced successor-state map, but its default interpretation is now a *typed self-update transition*. The word “channel” is reserved for cases where a CPTP, stochastic, or equivalent channel certificate is actually supplied.

Definition 4.1 (Typed self-update transition). *For an RCLM candidate code ν , a typed self-update transition is a tuple*

$$\text{SelfTrans}_t(\nu) = (\Phi_\nu, \tau_t^{\text{in}}, \tau_t^{\text{out}}, \kappa_t^{\text{kind}}, \text{TypeCert}_t(\nu)),$$

where Φ_ν maps the declared audited input system-state family to the declared successor system-state family, τ_t^{in} and τ_t^{out} are the input and output density types, $\kappa_t^{\text{kind}} \in \{\text{CPTP}, \text{stochastic}, \text{repr-transition}\}$ records the mathematical kind of the transition, and $\text{TypeCert}_t(\nu)$ is the type certificate already required by $\mathcal{A}_{\text{type}}$.

Definition 4.2 (Channel-licensed transition). *A typed self-update transition is channel-licensed on an audited family $\mathcal{S}_t$ when one of the following is certified:*

- (a) Φ_ν is a linear CPTP map on the relevant finite-dimensional operator algebra;
- (b) Φ_ν restricts to a stochastic map on diagonal classical states;
- (c) Φ_ν is represented by an exact isometric/conservative-extension dilation whose induced loss and recovery certificates are recorded by the non-lossy algebra.

If none of these certificates is present, Φ_ν may still be accepted as a representational typed transition, but every RCP residual used for acceptance must be directly certified rather than inferred from a channel theorem.

Assumption 4.3 (Terminology discipline). *Every theorem that uses data-processing, Petz-style recovery, exact relative-entropy preservation under isometry, or stochastic-map monotonicity must cite the corresponding channel-license certificate. Theorems that use only residual certificates may refer to Φ_ν as a typed transition without asserting it is a quantum channel.*

Proposition 4.4 (Typed-transition terminology does not weaken the gate). *Replacing the default phrase “self-update channel” by “typed self-update transition” does not weaken any RCLM gate condition. A candidate accepted without a channel license must satisfy the same residual inequalities by direct certificate, while a candidate accepted with a channel license may additionally use the corresponding channel theorem.*

Proof. The full gate is defined by residual inequalities and certificate membership. The terminology change only determines which theorems can be used to justify those residual certificates. If a channel license exists, the usual CPTP, stochastic, or isometric theorem may be used. If no such license exists, the candidate must still supply one-sided residual certificates. Therefore the admissible set is not enlarged by terminology. $\square$

Limitation 4.5 (Terminology is not implementation). *This section prevents over-reading Φ_ν as a physical quantum channel or stochastic map. It does not construct the transition, prove its estimator costs are small, or validate its semantic effects. Those remain type, certificate, trust, resource, and empirical obligations.*

5 Standing Notation

Definition 5.1 (Density states). *For a finite-dimensional Hilbert space $\mathcal{H}$,*

$$\mathcal{D}(\mathcal{H}) = \{\rho \in \mathcal{B}(\mathcal{H}) : \rho = \rho^\dagger, \rho \succeq 0, \text{Tr}(\rho) = 1\}.$$

All theorem statements in this architecture paper are finite-dimensional unless explicitly stated otherwise.

Definition 5.2 (Entropy, relative entropy, and mutual information). *The von Neumann entropy is*

$$S(\rho) = -\text{Tr}(\rho \log \rho).$$

For states ρ, σ , quantum relative entropy is

$$D(\rho \parallel \sigma) = \begin{cases} \text{Tr}[\rho(\log \rho - \log \sigma)], & \text{supp}(\rho) \subseteq \text{supp}(\sigma), \\ +\infty, & \text{otherwise.} \end{cases}$$

For a bipartite state ρ_{XY} ,

$$I_\rho(X; Y) = S(\rho_X) + S(\rho_Y) - S(\rho_{XY}).$$

Definition 5.3 (RCP registers). *The full RCP representational space is*

$$\mathcal{H}_{\text{RCP}} = \mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M.$$

The intended register roles are language L , objective/world reference O , subjective/human reference S , definitiveness/decision collapse D , ambiguity/uncertainty A , and self-model/update model M .

Assumption 5.4 (Operational register interpretation). *The tensor labels do not create semantics by themselves. The architecture must provide interpretation maps, audit domains, and validation tests for each register. Without such tests, statements about O -grounding, S -human-reference, A -ambiguity, or M -self-modeling are only formal labels.*

6 Typed Density Realization and Semantic Coupling Closure

This section closes the main category-risk in the architecture: a matrix state can be a physical quantum density operator, a diagonal classical probability state, or a learned representational surrogate satisfying the density axioms. These three uses share notation but do not license the same conclusions. The present manuscript therefore requires every RCLM density state and every accepted update to carry a type certificate and a coupling certificate.

Definition 6.1 (Typed density realization). *Let*

$$\mathbf{T} = \{\text{q}, \text{cl}, \text{repr}\}$$

be the set of admissible density types: physical-quantum, classical-probabilistic, and representational. For a finite-dimensional Hilbert space $\mathcal{H}$, define

$$\mathcal{D}^{\text{q}}(\mathcal{H}) = \mathcal{D}(\mathcal{H}),$$

where the state is interpreted as a genuine quantum density operator. For a finite classical alphabet of size n , define

$$\mathcal{D}_n^{\text{cl}} = \{\text{diag}(p) : p \in \Delta_n\}, \quad \Delta_n = \{p \in \mathbb{R}_+^n : \sum_i p_i = 1\}.$$

For an RCLM representation space $\mathcal{X}_t^{\text{repr}}$ and a certified embedding map E_t , define

$$\mathcal{D}_t^{\text{repr}}(\mathcal{H}_t) = \{E_t(x) : x \in \mathcal{X}_t^{\text{repr}}\} \subseteq \mathcal{D}(\mathcal{H}_t).$$

A typed state is a pair

$$(\rho, \tau), \quad \tau \in \mathbf{T},$$

where $\rho \in \mathcal{D}^\tau$ in the corresponding sense.

Assumption 6.2 (No category transfer without certificate). *A theorem about $\mathcal{D}^q$ states may use quantum operations such as unitary evolution, CPTP maps, and quantum relative entropy in their physical sense. A theorem about $\mathcal{D}^{\text{cl}}$ states may use classical stochastic maps and Shannon-equivalent diagonal density calculations. A theorem about $\mathcal{D}_t^{\text{repr}}$ states may use density-matrix calculations only as certified representational monitors unless an additional embedding/channel certificate reduces the claim to a quantum or classical case. In particular, a learned representational density state is not assumed to be a physical quantum state.*

Definition 6.3 (Operation kind). *For a candidate self-update ν , define its operation kind as*

$$\text{Kind}_t(\nu) \in \{\text{CPTP}, \text{stochastic}, \text{repr-certified}\}.$$

The first means that Φ_ν is a linear CPTP map on the declared density space. The second means that Φ_ν restricts to a stochastic map on diagonal states. The third means that Φ_ν is a learned or architecture-induced representational transformation whose RCP residuals must be directly certified rather than inferred from quantum data-processing alone.

Assumption 6.4 (Typed admissibility). *Each candidate $\nu \in \Omega_t^{\text{RCLM}}$ carries a typed admissibility certificate*

$$\text{TypeCert}_t(\nu) = (\tau_t^{\text{ctx}}, \tau_t^{\text{sys}}, \tau_{t+1}^{\text{sys}}, \text{Kind}_t(\nu), c_t^{\text{type}}(\nu)),$$

where $c_t^{\text{type}}(\nu)$ verifies that the input state, output state, and update rule are type-compatible. If $\text{Kind}_t(\nu) = \text{repr-certified}$, then the certificate must include direct one-sided estimates for every RCP residual used by the gate; no residual may be justified solely by quantum or classical data-processing unless the certificate explicitly establishes the required channel structure.

Definition 6.5 (Typed admissibility set). *The type-safe candidate set is*

$$\mathcal{A}_{\text{type}}(t, \rho_t^{\text{sys}}) = \{\nu \in \Omega_t^{\text{RCLM}} : \text{TypeCert}_t(\nu) \text{ is valid for } \rho_t^{\text{sys}}\}.$$

Equivalently, $\nu \in \mathcal{A}_{\text{type}}$ when $\Phi_\nu(\rho_t^{\text{sys}})$ is a density state of the declared successor type and every theorem used by the verifier is licensed by that type.

Proposition 6.6 (Typed non-loss convention). *If $\text{Kind}_t(\nu) = \text{CPTP}$, then relative entropy and data-processing statements are mathematically valid on $\mathcal{D}^q$. If $\text{Kind}_t(\nu) = \text{stochastic}$, the corresponding diagonal classical statements are valid on $\mathcal{D}^{\text{cl}}$. If $\text{Kind}_t(\nu) = \text{repr-certified}$, then the RCP loss condition*

$$\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} \leq \varepsilon_t$$

is valid only if directly certified on the declared pair set or reduced to one of the preceding cases by the type certificate.

Proof. The first two cases are precisely the domains on which quantum and classical relative entropy are defined with the usual channel semantics. The third case is a learned representational transformation; without a channel certificate there is no theorem implying that physical or classical data-processing applies. Therefore the only valid RCP use of such a transformation is through a direct residual certificate or an explicit reduction to a licensed channel case. $\square$

Limitation 6.7 (Data processing is not non-loss). *Even in the CPTP or stochastic cases, data processing states that relative entropy cannot increase under the channel. It does not by itself prove that the decrease is small. The RCP non-loss condition remains a small-loss certificate:*

$$D(\rho\|\sigma) - D(\Phi_\nu(\rho)\|\Phi_\nu(\sigma)) \leq \varepsilon_t$$

for the protected pairs. Thus type correctness licenses the mathematical language; it does not replace the loss budget.

6.1 Representation-to-density embedding

Definition 6.8 (RCLM representation-to-density embedding). *Let $x_t \in \mathcal{X}_t^{\text{repr}}$ denote the raw RCLM representation containing hidden states, register-head activations, memory summaries, verifier summaries, resource summaries, and self-model summaries. A representation-to-density embedding is a map*

$$\mathbf{E}_t : \mathcal{X}_t^{\text{repr}} \rightarrow \mathcal{D}(\mathcal{H}_t)$$

constructed by a positive lift

$$\mathbf{G}_t(x) = \mathbf{F}_t(x)\mathbf{F}_t(x)^\dagger + \lambda_t^{\text{emb}} I, \quad \lambda_t^{\text{emb}} \geq 0,$$

followed by normalization

$$\mathbf{E}_t(x) = \frac{\mathbf{G}_t(x)}{\text{Tr } \mathbf{G}_t(x)}$$

whenever $\text{Tr } \mathbf{G}_t(x) > 0$. If $\lambda_t^{\text{emb}} = 0$, the architecture must separately certify that $\mathbf{F}_t(x) \neq 0$ on the audited domain.

Lemma 6.9 (Embedding density validity). *If $\mathbf{G}_t(x) = \mathbf{F}_t(x)\mathbf{F}_t(x)^\dagger + \lambda_t^{\text{emb}} I$, $\lambda_t^{\text{emb}} \geq 0$, and $\text{Tr } \mathbf{G}_t(x) > 0$, then $\mathbf{E}_t(x) \in \mathcal{D}(\mathcal{H}_t)$.*

Proof. The matrix $\mathbf{F}_t(x)\mathbf{F}_t(x)^\dagger$ is positive semidefinite, and $\lambda_t^{\text{emb}} I$ is positive semidefinite. Hence $\mathbf{G}_t(x) \succeq 0$. Normalizing by the positive scalar $\text{Tr } \mathbf{G}_t(x)$ preserves positive semidefiniteness and gives trace one. Hermiticity follows from the construction. $\square$

Assumption 6.10 (Embedding regularity). *On the audited representation domain $\mathcal{X}_t^{\text{aud}} \subseteq \mathcal{X}_t^{\text{repr}}$, the embedding is Lipschitz up to a certified approximation error:*

$$\|\mathbf{E}_t(x) - \mathbf{E}_t(x')\|_1 \leq L_t^{\text{emb}} d_{\mathcal{X},t}(x, x') + e_t^{\text{emb}} \quad \forall x, x' \in \mathcal{X}_t^{\text{aud}}.$$

The constants L_t^{emb} and e_t^{emb} are audit quantities included in the gate residuals when embedding stability is used.

Proposition 6.11 (Representation perturbation transfer). *Assume embedding regularity. If a raw architecture transition ψ_ν satisfies*

$$d_{\mathcal{X},t+1}(\psi_\nu(x), x') \leq a_t(\nu)$$

for a declared successor raw state $x' \in \mathcal{X}_{t+1}^{\text{aud}}$, and the successor embedding has constants $L_{t+1}^{\text{emb}}, e_{t+1}^{\text{emb}}$, then

$$\|\mathbf{E}_{t+1}(\psi_\nu(x)) - \mathbf{E}_{t+1}(x')\|_1 \leq L_{t+1}^{\text{emb}} a_t(\nu) + e_{t+1}^{\text{emb}}.$$

Proof. Apply the Lipschitz inequality for $\mathbf{E}_{t+1}$ to the pair $\psi_\nu(x), x'$. $\square$

Limitation 6.12 (Embedding is not semantic truth). *Lemma 6.9 proves only positive semidefiniteness and trace normalization. It does not prove that the embedded registers have the intended semantics, that the system is aligned, or that the embedding is physically quantum. Those require semantic coupling and external audit assumptions below.*

6.2 Semantic coupling consistency

Definition 6.13 (Semantic coupling structure). *A semantic coupling structure for the RCLM architecture is*

$$\mathfrak{C}_t^{\text{RCLM}} = (\mathbf{E}_t, \Gamma_t, \mathbf{O}_t, \mathbf{B}_t, \epsilon_t^{\text{coup}}),$$

where $\mathbf{E}_t$ is the representation-to-density embedding, $\Gamma_t = (\Gamma_L, \Gamma_O, \Gamma_S, \Gamma_D, \Gamma_A, \Gamma_M)$ is the register interpretation family, $\mathbf{O}_t$ is an external audit/observation process, $\mathbf{B}_t$ maps semantic errors into barrier residuals, and $\epsilon_t^{\text{coup}} = (\epsilon_{X,t}^{\text{coup}})_X$ is a vector of certified coupling errors.

Assumption 6.14 (Semantic coupling soundness). *For each audited register $X \in \{L, O, S, D, A, M\}$, there is an external reference object $x_{X,t}^{\text{true}}$ and a semantic distance d_X^{sem} . On the audited domain, the true semantic error is bounded by the density-register semantic error plus the coupling error:*

$$d_X^{\text{sem}}(\Gamma_X(\rho_X), x_{X,t}^{\text{true}}) \leq \text{Val}_X(\rho) + \epsilon_{X,t}^{\text{coup}}.$$

Here $\text{Val}_X(\rho)$ is the architecture’s audited density-level semantic validity measure. This is an assumption about the soundness of the interpretation map and audit process, not a consequence of the tensor factorization alone.

Definition 6.15 (Coupling admissibility set). *The coupling-safe candidate set is*

$$\mathcal{A}_{\text{coup}}(t, \rho_t^{\text{sys}}) = \left\{ \nu \in \Omega_t^{\text{RCLM}} : \begin{array}{l} \Phi_\nu(\rho_t^{\text{sys}}) \in \mathcal{X}_{t+1}^{\text{aud}}, \\ \mathfrak{C}_{t+1}^{\text{RCLM}} \text{ satisfies semantic coupling soundness,} \\ \max_X (\epsilon_{X,t+1}^{\text{coup}} - m_{X,t+1}^{\text{sem}}(\nu)) \leq 0 \end{array} \right\},$$

where $m_{X,t+1}^{\text{sem}}(\nu) \geq 0$ is the certified semantic margin by which the density-level semantic barrier is satisfied for register X .

Theorem 6.16 (Semantic coupling consistency). *Assume semantic coupling soundness. Suppose an accepted candidate ν satisfies, for every audited register X ,*

$$\text{Val}_X(\Phi_\nu(\rho_t^{\text{sys}})) \leq \delta_{X,t+1} - m_{X,t+1}^{\text{sem}}(\nu)$$

with

$$\epsilon_{X,t+1}^{\text{coup}} \leq m_{X,t+1}^{\text{sem}}(\nu).$$

Then the corresponding external semantic error satisfies

$$d_X^{\text{sem}}(\Gamma_X((\Phi_\nu(\rho_t^{\text{sys}}))_X), x_{X,t+1}^{\text{true}}) \leq \delta_{X,t+1}$$

for all audited registers.

Proof. By semantic coupling soundness,

$$d_X^{\text{sem}}(\Gamma_X((\Phi_\nu(\rho_t^{\text{sys}}))_X), x_{X,t+1}^{\text{true}}) \leq \text{Val}_X(\Phi_\nu(\rho_t^{\text{sys}})) + \epsilon_{X,t+1}^{\text{coup}}.$$

The first hypothesis bounds the first term by $\delta_{X,t+1} - m_{X,t+1}^{\text{sem}}(\nu)$, while the second bounds the coupling error by $m_{X,t+1}^{\text{sem}}(\nu)$. Adding the inequalities gives the result. $\square$

Interpretation 6.17 (What coupling closure adds). *The RCP gate can now distinguish three claims that were previously easy to conflate: i the successor is a valid mathematical density state; ii the density operations used by the theorem are licensed by the state type; and iii the density-register validity predicates imply externally audited semantic validity up to a certified coupling error.*

Limitation 6.18 (No semantics from labels). *The theorem does not construct the correct human-reference object, world-reference object, or self-model reference object. It states that if the architecture supplies sound semantic coupling maps and margins, then density-level semantic gate satisfaction transfers to external semantic validity within the declared tolerance.*

7 Batch-9 RCLM Direct Engine Constructor

The Batch-9 math/physics paper upgrades the direct-engine target from a named future object into a domain-restricted constructive theorem. This section instantiates that theorem for RSI–RCLM. The goal is not to claim that arbitrary RCLM states can self-improve. The goal is to state and prove the exact conditional implication: if an RCLM state lies in a declared direct-engine domain and the generator/certifier covers a strict beneficial conservative-extension witness, then the architecture constructs a certified non-lossy beneficial primitive word rather than merely checking an externally proposed one.

Limitation 7.1 (Batch 9 remains domain-restricted). *Batch 9 proves construction only on the declared RCLM direct-engine domain. It does not prove that all trained RCLM states enter that domain, that open-ended useful extensions are always available, that semantic coupling is correct, that estimator radii remain small at scale, or that the finite reference engine validates frontier-scale RSI. It converts a previously external witness into a constructed witness under explicit generator, certificate, and margin assumptions.*

Definition 7.2 (RCLM direct-engine constructor). *The Batch-9 RCLM direct-engine constructor is the tuple*

$$\mathfrak{C}_t^{\text{RCLM}} = (\text{Gen}_t^{\text{NL,RCLM}}, \text{Certify}_t^{\text{Eng,RCLM}}, \text{Select}_t^{\text{Eng,RCLM}}, \text{Realize}_t^{\text{Eng,RCLM}}).$$

Given a system state ρ_t^{sys} , the generator produces a finite candidate set

$$\text{Cand}_t^{\text{Eng}}(\rho_t^{\text{sys}}) = \text{Gen}_t^{\text{NL,RCLM}}(\rho_t^{\text{sys}}) \subseteq \Omega_t^{\text{NL,RCLM}}.$$

The certifier maps each candidate to an engine certificate packet, the selector chooses a candidate from the passing set, and the realizer returns the implemented update, recovery map, and successor state.

Definition 7.3 (RCLM engine certificate packet). *For a candidate $\nu \in \Omega_t^{\text{NL,RCLM}}$, an RCLM engine certificate packet is*

$$\begin{aligned} \text{EngCert}_t^{\text{RCLM}}(\nu) = & (\text{GenTrace}_t(\nu), \text{NLAlgCert}_t(\nu), \text{RecCert}_t(\nu), \text{AbilityCert}_t(\nu), \\ & \text{KernelCert}_t(\nu), \text{RCPDGCert}_t(\nu), \text{TypeCert}_t(\nu), \text{TrustCert}_t(\nu), \\ & \text{ResCert}_t(\nu), \text{ProjReal}_t^{\text{RCLM}}(\nu)). \end{aligned}$$

The entries record, respectively, generation trace, Batch-3 algebraic non-loss, recovery, predecessor-ability preservation and new-ability expansion, successor engine-kernel membership, full post-D–G gate membership, type validity, trust-graph support, adaptive-audit trust support, resource feasibility, and projection-realization when projection or repair is used.

Definition 7.4 (RCLM direct-engine passing set). *For a state ρ , define the passing set*

$$\begin{aligned} \text{Pass}_t^{\text{Eng,RCLM}}(\rho) = \{ \nu \in \text{Cand}_t^{\text{Eng}}(\rho) : & \text{EngCert}_t^{\text{RCLM}}(\nu) \text{ passes,} \\ & \nu \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}} \cap \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}, \\ & \nu \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}}), \\ & \text{successor-kernel and ProjReal}_t^{\text{RCLM}} \text{ certificates pass} \}. \end{aligned}$$

A candidate in this set is a certified conservative-extension engine witness: it is algebraically non-lossy, recoverable, ability-preserving, strictly ability-expanding when $g_t^{\mathfrak{A}} > 0$, resource-feasible, and successor-viable on the declared engine kernel.

Definition 7.5 (Strict beneficial RCLM conservative-extension witness). A candidate $\nu^\# \in \Omega_t^{\text{NL,RCLM}}$ is a strict beneficial RCLM conservative-extension witness for ρ with margin $m_t^{\text{Eng}} > 0$ if:

(a) $\nu^\# \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}})$ with $g_t^{\mathfrak{A}} > 0$;

(b) all full RCP-RCLM residuals used by the accepted gate satisfy

$$q_{t,j}^{\text{RCLM}}(\nu^\#; \rho) \leq -m_t^{\text{Eng}} \quad \forall j \in \mathcal{J};$$

(c) the algebraic non-loss and recovery budgets are strictly inside threshold:

$$\epsilon(\nu^\#) \leq \epsilon_t - m_t^{\text{Eng}}, \quad r(\nu^\#) \leq r_t - m_t^{\text{Eng}};$$

(d) a certified new ability exists:

$$\mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_{\nu^\#}(\rho), R_{t+1}) \setminus \Theta_t^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho, R_t)) \neq \emptyset;$$

(e) the successor satisfies the declared kernel condition

$$\Phi_{\nu^\#}(\rho) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N} \quad \text{or} \quad \mathcal{V}_{t+1}^{\text{RCLM-engine},\infty}.$$

Definition 7.6 (RCLM direct-engine domain). For a declared margin $m_t^{\text{Eng}} > 0$, the RCLM direct-engine domain is

$$\begin{aligned} \mathcal{D}_t^{\text{RCLM-engine}} &= \{\rho : \rho \in \mathcal{K}_t^{\text{RCLM-engine}}, \\ &\quad \text{a strict beneficial CE witness exists for } \rho, \\ &\quad \text{all generator/certifier/transport/resource domains are valid}\}. \end{aligned}$$

This domain is a sufficient-condition set, not an automatically populated state set. A system outside $\mathcal{D}_t^{\text{RCLM-engine}}$ may still be safe under the preservation gate, but Batch 9 does not license a direct-engine construction claim for it.

Assumption 7.7 (RCLM generator coverage of strict witnesses). On every state $\rho \in \mathcal{D}_t^{\text{RCLM-engine}}$, the generator returns a finite candidate set containing at least one candidate $\bar{\nu}$ whose residuals, algebraic budgets, recovery budget, new-ability certificate, resource cost, projection-realization certificate, and successor-kernel certificate remain within the strict margin m_t^{Eng} of a strict beneficial witness. In particular, after accounting for generator approximation, certification radii, and transport/projection-realization distortions, $\bar{\nu}$ satisfies the inequalities required by $\text{Pass}_t^{\text{Eng,RCLM}}(\rho)$.

Assumption 7.8 (RCLM engine certificate soundness). There is an event $\mathcal{E}_t^{\text{Eng,RCLM}}$ such that, on this event, every candidate accepted by $\text{Certify}_t^{\text{Eng,RCLM}}$ satisfies the true gate, non-loss, recovery, ability, resource, trust, projection-realization, and successor-kernel conditions recorded in its engine certificate packet. The event probability and failure budget are included in the non-oracular certificate ledger.

Assumption 7.9 (RCLM selector and realizer soundness). If $\text{Pass}_t^{\text{Eng,RCLM}}(\rho) \neq \emptyset$, the selector returns some

$$\nu_t \in \text{Pass}_t^{\text{Eng,RCLM}}(\rho).$$

The realizer then constructs a typed update map Φ_{ν_t} , a recovery map $\mathcal{R}_{\nu_t}^{\text{rec}}$, and a successor state

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho)$$

up to the declared projection-realization error if projection or repair is used. All realization error is included in the corresponding residuals and certificates.

Theorem 7.10 (RCLM Constructive Direct Non-Lossy RSI Engine Theorem). *Assume $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$. Assume generator coverage, engine certificate soundness, and selector/realizer soundness. On the engine certificate event $\mathcal{E}_t^{\text{Eng,RCLM}}$, the passing set $\text{Pass}_t^{\text{Eng,RCLM}}(\rho_t^{\text{sys}})$ is nonempty. Consequently,*

$$\mathfrak{C}_t^{\text{RCLM}}(\rho_t^{\text{sys}})$$

is defined and constructs a packet

$$(\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\text{rec}}, \text{EngCert}_t^{\text{RCLM}}(\nu_t), \rho_{t+1}^{\text{sys}}).$$

The constructed update satisfies:

- (i) $\nu_t \in \Omega_t^{\text{NL,RCLM}}$;
- (ii) $\nu_t \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$;
- (iii) *the protected distinguishability-loss and recovery budgets recorded in the primitive word are valid:*

$$\mathcal{L}_{\Phi_{\nu_t}}^{\mathcal{P}^{\text{RCLM}}} \leq \varepsilon_t, \quad \text{Rec}_{\Phi_{\nu_t}}^{\mathcal{P}^{\text{RCLM}}}(\mathcal{R}_{\nu_t}^{\text{rec}}) \leq r_t;$$

- (iv) *predecessor certified abilities are transported into the successor:*

$$\mathfrak{A}_t^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)) \subseteq \mathfrak{A}_{t+1}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}, R_{t+1});$$

- (v) *strict ability expansion is certified:*

$$\mathfrak{A}_{t+1}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}, R_{t+1}) \setminus \Theta_t^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)) \neq \emptyset;$$

- (vi) *the successor remains inside the next declared engine viability kernel:*

$$\rho_{t+1}^{\text{sys}} \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N} \quad \text{or} \quad \mathcal{V}_{t+1}^{\text{RCLM-engine},\infty};$$

- (vii) *if projection or repair is used, $\text{ProjReal}_t^{\text{RCLM}}$ certifies that the projected successor is realized by the typed update map within the declared distortion.*

Proof. Because $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$, Definition 7.6 gives a strict beneficial RCLM conservative-extension witness. By Assumption 7.7, the generator returns a finite candidate set containing a candidate $\bar{\nu}$ whose certified residuals and margins remain sufficient after approximation, certificate radii, transport distortion, and projection-realization distortion. Therefore $\bar{\nu} \in \text{Pass}_t^{\text{Eng,RCLM}}(\rho_t^{\text{sys}})$ on the event $\mathcal{E}_t^{\text{Eng,RCLM}}$, proving that the passing set is nonempty.

By Assumption 7.9, the selector returns some ν_t from the passing set and the realizer constructs Φ_{ν_t} , $\mathcal{R}_{\nu_t}^{\text{rec}}$, and ρ_{t+1}^{sys} . Membership in the passing set includes membership in $\Omega_t^{\text{NL,RCLM}}$, $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$, the full RCP–RCLM gate, the conservative-extension engine move set, and the successor-kernel condition. Batch-3 algebraic non-loss gives the displayed loss and recovery budgets. The engine move set and ability certificate give predecessor ability preservation and strict new ability expansion. The successor-kernel certificate gives the displayed viability-kernel membership. Projection-realization follows from the last component of the engine certificate whenever projection or repair is used. No property is used beyond the stated domain, generator, certifier, selector, and realizer assumptions. $\square$

Corollary 7.11 (Finite direct-engine trajectory under domain invariance). *Fix a horizon N . Suppose $\rho_0^{\text{sys}} \in \mathcal{D}_0^{\text{RCLM-engine}}$, and for each constructed successor the domain-invariance condition holds:*

$$\rho_{t+1}^{\text{sys}} \in \mathcal{D}_{t+1}^{\text{RCLM-engine}} \quad (t < N - 1).$$

Assume the Batch-9 generator, certificate, selector, and realizer hypotheses hold at every step. Then repeated application of $\mathfrak{E}_t^{\text{RCLM}}$ constructs a finite sequence of certified non-lossy conservative-extension updates with recovery, strict ability expansion, and successor engine viability at every step.

Proof. Apply Theorem 7.10 at $t = 0$. The domain-invariance hypothesis places the successor in $\mathcal{D}_1^{\text{RCLM-engine}}$, so the theorem applies again. Iterating through $t = N - 1$ gives the claimed finite sequence. The per-step non-loss, recovery, ability expansion, and successor-viability properties are exactly the conclusions of Theorem 7.10 at each step. $\square$

Interpretation 7.12 (What Batch 9 adds). *Before Batch 9, the architecture had a non-lossy primitive algebra, local tangent-cone progress conditions, engine-viability kernels, and an RCLM lift/reference engine. Batch 9 adds the constructor that connects these pieces: generator coverage and certificate soundness turn a strict beneficial conservative-extension witness into an update packet produced by $\mathfrak{E}_t^{\text{RCLM}}$. Thus the architecture now contains a domain-restricted direct-engine theorem rather than only a preservation theorem plus an externally supplied witness.*

Limitation 7.13 (No arbitrary-system RSI conclusion). *The theorem is still not an unconditional solution to RSI for arbitrary learned systems. It requires the state to be in $\mathcal{D}_t^{\text{RCLM-engine}}$, which includes strict beneficial witness existence, generator coverage, valid certificates, resource feasibility, and successor-kernel membership. If the lift fails, if no useful conservative extension exists, if semantic coupling is invalid, if certificates are too loose, or if resources are insufficient, the architecture must fall back to no-op/defer rather than claim direct engine progress.*

8 Batch-11 RCLM Seed-Domain Entry, Lift Coverage, and Witness-Library Closure

Batch 9 proves a constructive RCLM direct-engine theorem only after the system state is already inside the declared direct-engine domain

$$\mathcal{D}_t^{\text{RCLM-engine}}.$$

The Batch-11 architecture upgrade does not add another safety gate. It adds a domain-entry and persistence layer: a concrete seed class, a certified conservative-extension library, a bounded primitive-word grammar, a library-lift coverage certificate, and a persistence-transfer certificate that together imply direct-engine-domain membership and recursive seed-domain preservation.

The central Batch-11 RCLM implication is

$$\begin{aligned} \rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N} &\implies \rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}} \\ &\implies \mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ constructs a certified primitive word,} \end{aligned}$$

with the persistence condition

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N}, \quad \rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}) \implies \rho_{t+1}^{\text{sys}} \in \mathcal{K}_{t+1}^{\text{RCLM-seed},N}.$$

This is the architecture-side synchronization of the Batch-11 math/physics theorem: domain-nonemptiness is proved only for a declared, certified, bounded seed/library class.

Limitation 8.1 (Batch 11 is not arbitrary trained-system RSI). *Batch 11 does not prove that arbitrary trained RCLM systems, arbitrary self-models, or arbitrary Level 2 inference traces enter the direct-engine domain. It proves a conditional implication from a declared seed package to the existing Batch-9 direct-engine domain. If the library descriptor is absent, the library contains no strict beneficial module, the lift is not complete for the library, the bounded grammar does not cover the module word, the certifier radii exceed margins, or successor seed-persistence is not certified, then the Batch-11 entry theorem cannot be invoked.*

8.1 Certified RCLM conservative-extension libraries

Definition 8.2 (RCLM certified conservative-extension module). *Fix time t , a state $\rho \in \mathcal{K}_t^{\text{RCLM-engine}}$, a progress threshold $g_t^{\mathfrak{A}} > 0$, and strict margins*

$$m_t^{\text{Eng}}, \quad m_t^{\mathfrak{A}}, \quad m_t^r, \quad m_t^{\text{cert}}, \quad m_t^{\text{seed}} > 0.$$

An RCLM certified conservative-extension module at (t, ρ) is a tuple

$$M = (\nu_M, w_M, \Phi_M, \mathcal{R}_M^{\text{rec}}, \text{ModCert}_t^{\text{RCLM}}(M; \rho)),$$

where $w_M \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, $\nu_M \in \Omega_t^{\text{NL, RCLM}}$, $\Phi_M = \Phi_{\nu_M}$, and the certificate packet contains all of the following entries:

- (i) *a primitive-word trace proving that w_M is type-compatible, domain-valid, and generated by the RCLM conservative-extension algebra;*
- (ii) *valid type, representation-to-density, semantic-coupling, cross-time transport, projection-realization, adaptive-trust, and resource certificates for ν_M ;*
- (iii) *algebraic non-loss and recovery certificates*

$$\mathcal{L}_{\Phi_M}^{\mathcal{P}_t^{\text{RCLM}}} \leq \varepsilon_t - m_t^{\text{Eng}}, \quad \text{Rec}_{\Phi_M}^{\mathcal{P}_t^{\text{RCLM}}}(\mathcal{R}_M^{\text{rec}}) \leq r_t - m_t^r;$$

- (iv) *strict RCLM engine residual margins*

$$q_{t,j}^{\text{RCLM}}(\nu_M; \rho) \leq -m_t^{\text{Eng}} \quad \forall j \in \mathcal{J}_t;$$

- (v) *predecessor ability preservation*

$$\Theta_t^{\mathfrak{A}, \text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho, R_t)) \subseteq \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_M(\rho), R_{t+1});$$

- (vi) *strict certified new ability, represented by some*

$$a_M^{\text{new}} \in \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_M(\rho), R_{t+1}) \setminus \Theta_t^{\mathfrak{A}, \text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho, R_t));$$

- (vii) *successor continuation evidence, either*

$$\Phi_M(\rho) \in \mathcal{V}_{t+1}^{\text{RCLM-engine}, N} \quad \text{or} \quad \Phi_M(\rho) \in \mathcal{V}_{t+1}^{\text{RCLM-engine}, \infty},$$

and, when seed persistence is claimed, the stronger certificate

$$\text{SeedPersist}_t^{\text{RCLM}}(M; \rho) : \Phi_M(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-seed}, N};$$

- (viii) *an explicit margin ledger*

$$\text{MargLed}_t^{\text{RCLM}}(M; \rho) = (m_t^{\text{Eng}}, m_t^{\mathfrak{A}}, m_t^r, m_t^{\text{cert}}, m_t^{\text{seed}})$$

against which generator, lift, certifier, transport, and realization radii are charged.

The module certificate is state-relative. A module template may be certified at one RCLM state and invalid at another.

Definition 8.3 (RCLM certified conservative-extension library). *A certified RCLM conservative-extension library at time t is a finite or explicitly enumerable family*

$$\mathcal{L}_t^{\text{RCLM-CE}} = \{M_{t,1}, \dots, M_{t,N_t}\}$$

with a state-relative validity predicate

$$\text{ValidMod}_t^{\text{RCLM}}(M; \rho) \in \{0, 1\}.$$

The valid library at ρ is

$$\mathcal{L}_t^{\text{RCLM-CE}}(\rho) = \{M \in \mathcal{L}_t^{\text{RCLM-CE}} : \text{ValidMod}_t^{\text{RCLM}}(M; \rho) = 1\}.$$

The library is strictly beneficial at ρ when at least one module in $\mathcal{L}_t^{\text{RCLM-CE}}(\rho)$ satisfies Definition 8.2 with all strict margins positive.

Definition 8.4 (RCLM library descriptor in the self-model). *An RCLM library descriptor is a typed self-model record*

$$\text{LibDesc}_t^{\text{RCLM}} \in M_t$$

containing the library identity, module template identifiers, primitive-word grammar bounds, resource bounds, version information, required verifier components, required trust anchors, and certificate interfaces for $\mathcal{L}_t^{\text{RCLM-CE}}$. The descriptor is valid for ρ only when it is consistent with the certificate ledger $\mathcal{H}_t^{\text{cert}}$, the verifier state W_t , the active resource ledger, and the current architecture type certificates. A descriptor label alone does not prove that the library is nonempty or useful.

Theorem 8.5 (RCLM CE library nonemptiness gives a strict witness). *Assume $\rho \in \mathcal{K}_t^{\text{RCLM-engine}}$. If $\mathcal{L}_t^{\text{RCLM-CE}}(\rho)$ contains a module M satisfying Definition 8.2, then ν_M is a strict beneficial RCLM conservative-extension witness for ρ in the sense of Definition 7.5, with margins reduced only by the radii explicitly charged in $\text{MargLed}_t^{\text{RCLM}}(M; \rho)$.*

Proof. Definition 8.2 supplies a candidate $\nu_M \in \Omega_t^{\text{NL,RCLM}}$, membership in the RCLM conservative-extension algebra, strict residual margins, algebraic non-loss and recovery margins, predecessor ability preservation, a certified new ability outside the transported predecessor ability set, and successor engine-kernel membership. These are exactly the clauses of Definition 7.5. If transfer or estimator radii are used, the theorem only uses margins after those radii have been charged to the margin ledger. No existence is inferred beyond the supplied valid module. $\square$

8.2 Bounded primitive-word grammar and lift coverage

Definition 8.6 (Bounded RCLM non-lossy primitive-word grammar). *Given a library $\mathcal{L}_t^{\text{RCLM-CE}}$, a word-length bound k_t , a template count bound N_t , and a search budget B_t^{search} , define*

$$\mathfrak{G}_{\leq k_t, t}^{\text{NL,RCLM}}(\mathcal{L}_t^{\text{RCLM-CE}})$$

to be the finite set of compatible primitive words over

$$\{\text{id}, \text{IsoExt}, \text{AppendMem}, \text{RevAdapter}, \text{VerifierExt}, \text{ToolExt}\}$$

that are licensed by the library descriptor, have length at most k_t , satisfy the type and interface constraints, and have enumerated search cost at most B_t^{search} . The associated enumerator is

$$\text{Enum}_{t, \leq k_t}^{\text{RCLM}}(\mathcal{L}_t^{\text{RCLM-CE}}) \subseteq \Omega_t^{\text{NL,RCLM}}.$$

This grammar is deliberately bounded. It is not a claim of unbounded completeness.

Definition 8.7 (RCLM library-lift coverage certificate). *A library-lift coverage certificate at (t, ρ) is a packet*

$$\text{LiftCov}_t^{\text{RCLM}}(\rho, \mathcal{L}_t^{\text{RCLM-CE}}, k_t)$$

certifying that:

- (i) *the lift input package $\mathbf{L}_t^{\text{in}} = (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}})$ is valid on the lift domain $\mathcal{X}_t^{\text{lift}}$;*
- (ii) *$\text{LibDesc}_t^{\text{RCLM}} \in M_t$ is a valid descriptor for $\mathcal{L}_t^{\text{RCLM-CE}}$;*
- (iii) *for at least one strictly beneficial valid module $M \in \mathcal{L}_t^{\text{RCLM-CE}}(\rho)$, the word w_M lies in the bounded grammar*

$$w_M \in \mathfrak{G}_{\leq k_t, t}^{\text{NL, RCLM}}(\mathcal{L}_t^{\text{RCLM-CE}});$$

- (iv) *either the lift returns ν_M directly, or the generator enumerates ν_M from the bounded grammar after the lift has supplied the descriptor and template interface;*
- (v) *the lift/generator error radius $e_t^{\text{lift}} + e_t^{\text{gen}}$ is included in $\text{MargLed}_t^{\text{RCLM}}(M; \rho)$.*

Theorem 8.8 (RCLM finite grammar and lift coverage). *Assume $\text{LiftCov}_t^{\text{RCLM}}(\rho, \mathcal{L}_t^{\text{RCLM-CE}}, k_t)$ is valid. Then the RCLM generator candidate set contains a candidate $\bar{\nu}$ corresponding to a strictly beneficial library module, either exactly or within the certified lift/generator radius:*

$$\bar{\nu} \in \text{Gen}_t^{\text{NL, RCLM}}(\rho) \subseteq \Omega_t^{\text{NL, RCLM}}.$$

If the charged lift/generator radius plus the certifier radius is smaller than the active strict margins, then $\bar{\nu}$ satisfies the Batch-9 generator-coverage requirement for that witness.

Proof. The coverage certificate states that a strictly beneficial valid module has a word in the bounded grammar and that the lift or the enumerating generator returns the corresponding candidate. Hence $\bar{\nu} \in \text{Gen}_t^{\text{NL, RCLM}}(\rho)$. The final statement follows by subtracting the certified lift/generator/certifier radii from the positive margins in $\text{MargLed}_t^{\text{RCLM}}(M; \rho)$. If the radii exceed a margin, the theorem does not apply. $\square$

8.3 RCLM seed domain and direct-engine domain entry

Definition 8.9 (RCLM seed engine package). *A seed engine package for ρ at time t is a tuple*

$$\text{SeedPkg}_t^{\text{RCLM}}(\rho) = (\text{LibDesc}_t^{\text{RCLM}}, \mathcal{L}_t^{\text{RCLM-CE}}, \text{LiftCov}_t^{\text{RCLM}}, \text{Enum}_{t, \leq k_t}^{\text{RCLM}}, \text{CertRad}_t^{\text{RCLM}}, \text{SeedPersist}_t^{\text{RCLM}}).$$

whose components certify all of the following:

- (i) *$\rho \in \mathcal{K}_t^{\text{RCLM-engine}}$ and all Batch-10 type, adaptive-trust, complexity, resource, and typed-transition obligations are active and well formed;*
- (ii) *the library descriptor is valid in M_t and consistent with W_t and $\mathcal{H}_t^{\text{cert}}$;*
- (iii) *the valid library $\mathcal{L}_t^{\text{RCLM-CE}}(\rho)$ contains at least one strict beneficial certified module;*
- (iv) *the bounded grammar and lift coverage certificate cover at least one such module;*
- (v) *the certifier is sound and complete for the covered bounded grammar class within radius $\text{CertRad}_t^{\text{RCLM}}$;*

- (vi) all lift, generation, certification, transport, projection-realization, and implementation radii are strictly below the corresponding margin-ledger entries;
- (vii) the covered module has either direct successor-kernel membership or a seed-persistence certificate for the successor.

Definition 8.10 (RCLM seed engine domain). *For a horizon N , the RCLM seed engine domain is*

$$\mathcal{K}_t^{\text{RCLM-seed},N} = \{\rho \in \mathcal{K}_t^{\text{RCLM-engine}} : \text{SeedPkg}_t^{\text{RCLM}}(\rho) \text{ is valid for horizon } N\}.$$

Membership is stronger than ordinary engine-base membership and weaker than an arbitrary-system RSI claim. It is a checkable sufficient-condition class designed to imply $\mathcal{D}_t^{\text{RCLM-engine}}$; it does not define $\mathcal{K}_t^{\text{RCLM-seed},N}$ by already assuming $\mathcal{D}_t^{\text{RCLM-engine}}$.

Theorem 8.11 (RCLM direct-engine domain entry from seed domain). *If*

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N},$$

then

$$\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}.$$

Consequently, if the Batch-9 selector and realizer hypotheses also hold, then $\mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}})$ is defined and constructs a certified beneficial non-lossy primitive word on the engine-certificate event.

Proof. By Definition 8.10, $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-engine}}$ and a valid seed package is present. The seed package includes a nonempty strictly beneficial valid CE library, so Theorem 8.5 supplies a strict beneficial RCLM conservative-extension witness. The same seed package includes valid lift/grammar coverage, certifier soundness and completeness on the bounded grammar class, resource validity, transport validity, projection-realization validity, and margin dominance. These are precisely the generator/certifier/transport/resource domain hypotheses required by Definition 7.6. Therefore $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$. The final construction statement is then Theorem 7.10. $\square$

8.4 Seed-equivalence and successor seed-domain invariance

Definition 8.12 (RCLM seed-equivalence and persistence-transfer certificate). *Let $M \in \mathcal{L}_t^{\text{RCLM-CE}}(\rho)$ be the certified module witnessing $\rho \in \mathcal{K}_t^{\text{RCLM-seed},N}$, and let $\nu \in \text{Pass}_t^{\text{Eng,RCLM}}(\rho)$ be the candidate selected and realized by $\mathfrak{E}_t^{\text{RCLM}}$. A seed-equivalence or persistence-transfer certificate is a packet*

$$\text{SeedEq}_t^{\text{RCLM}}(M, \nu; \rho) = (\Delta_t^{\text{res}}, \Delta_t^{\mathfrak{A}}, \Delta_t^r, \Delta_t^{\text{type}}, \Delta_t^{\text{resrc}}, \Delta_t^{\text{trust}}, \Delta_t^{\text{tr}}, \Delta_t^{\text{proj}}, \Delta_t^{\text{lift}}, \Delta_t^{\text{gen}}, \text{SuccSeedCert}_t^{\text{RCLM}}).$$

It is valid when:

- (i) every RCLM engine residual of ν differs from the corresponding residual of M by at most the certified residual-transfer radius Δ_t^{res} ;
- (ii) every ability-preservation and new-ability certificate used by M transfers to ν , or is independently certified for ν , with error bounded by $\Delta_t^{\mathfrak{A}}$;
- (iii) the recovery, type, resource, adaptive-trust, transport, projection-realization, lift, and generator distortions between $\Phi_M(\rho)$ and $\Phi_\nu(\rho)$ are bounded by the listed radii;

(iv) every listed radius is charged to $\text{MargLed}_t^{\text{RCLM}}(M; \rho)$;

(v) the successor seed certificate proves

$$\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-seed}, N}.$$

If $\nu = \nu_M$ and realization is exact, the identity certificate with all transfer radii equal to zero is permitted.

Theorem 8.13 (RCLM successor seed-domain invariance). *Assume*

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed}, N}.$$

If the witnessing module M has a valid seed-persistence certificate

$$\text{SeedPersist}_t^{\text{RCLM}}(M; \rho_t^{\text{sys}}) : \Phi_M(\rho_t^{\text{sys}}) \in \mathcal{K}_{t+1}^{\text{RCLM-seed}, N},$$

then the exact module successor lies in the next seed domain. If, instead of selecting exactly ν_M , the engine selects

$$\nu_t \in \text{Pass}_t^{\text{Eng}, \text{RCLM}}(\rho_t^{\text{sys}})$$

and a valid $\text{SeedEq}_t^{\text{RCLM}}(M, \nu_t; \rho_t^{\text{sys}})$ certificate is supplied, then

$$\Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{K}_{t+1}^{\text{RCLM-seed}, N}.$$

Proof. The exact-module claim is exactly the content of $\text{SeedPersist}_t^{\text{RCLM}}(M; \rho_t^{\text{sys}})$. For the selected-candidate case, Definition 8.12 requires a successor seed certificate for $\Phi_{\nu_t}(\rho_t^{\text{sys}})$, after all residual, ability, recovery, type, resource, trust, transport, projection-realization, lift, and generator distortions have been charged to the module margin ledger. Hence the successor belongs to $\mathcal{K}_{t+1}^{\text{RCLM-seed}, N}$. No informal “equivalence” relation is used beyond the explicit transfer certificate. $\square$

Corollary 8.14 (Finite RCLM direct-engine trajectory from seed domain). *Fix a horizon N . Suppose*

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-seed}, N}$$

and suppose the Batch-11 seed package, Batch-9 constructor hypotheses, and valid seed-equivalence or exact seed-persistence certificates hold at every step. Then repeated application of $\mathfrak{C}_t^{\text{RCLM}}$ constructs a finite trajectory

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}), \quad t = 0, \dots, N-1,$$

such that every step is a certified beneficial non-lossy conservative-extension update and

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed}, N} \quad \forall t \leq N.$$

Proof. At $t = 0$, Theorem 8.11 gives $\rho_0^{\text{sys}} \in \mathcal{D}_0^{\text{RCLM-engine}}$. The Batch-9 theorem constructs the accepted update. Theorem 8.13 places the successor in $\mathcal{K}_1^{\text{RCLM-seed}, N}$. Induction repeats the same argument for $t = 1, \dots, N-1$. Per-step non-loss, recovery, strict ability expansion, full gate membership, and successor viability are inherited from Theorem 7.10. $\square$

8.5 Restricted scalability and open-ended ability towers

Assumption 8.15 (Restricted modular certificate-radius decomposition). *For a factored RCLM CE module word*

$$w = g_n \circ \dots \circ g_1,$$

the certificate ledger supplies component radii $\Delta_{t,i}^{\text{cert}}$, cached predecessor-certificate reuse conditions, bounded interaction terms Δ_t^{int} , and a module margin $m_t^{\text{module}} > 0$ such that

$$\sum_{i=1}^n \Delta_{t,i}^{\text{cert}} + \Delta_t^{\text{int}} \leq m_t^{\text{module}}.$$

The decomposition is valid only for the declared factored module class; it is not a frontier-scale tractability theorem.

Proposition 8.16 (Restricted RCLM modular certificate-radius scalability). *Under Assumption 8.15, the total certificate radius of the factored module word is below its declared module margin. Therefore any engine-domain or seed-domain theorem using that module may charge the summed certificate radius to $\text{MargLed}_t^{\text{RCLM}}$ without losing strictness.*

Proof. The displayed inequality is exactly the required margin dominance condition. The proposition makes no claim that the decomposition exists outside the declared factored class. $\square$

Definition 8.17 (RCLM open-ended ability tower). *An RCLM open-ended ability tower is a sequence of declared audited ability universes*

$$\text{Ab}_0^{\text{RCLM}} \subseteq \text{Ab}_1^{\text{RCLM}} \subseteq \text{Ab}_2^{\text{RCLM}} \subseteq \dots$$

with transport maps $\Theta_t^{\mathfrak{A},\text{RCLM}}$, resource-renewal certificates, and library-generation certificates such that every seed package at time t contains a module adding at least one ability

$$a_{t+1} \in \text{Ab}_{t+1}^{\text{RCLM}} \setminus \Theta_t^{\mathfrak{A},\text{RCLM}}(\text{Ab}_t^{\text{RCLM}})$$

under the active resource budget.

Corollary 8.18 (Conditional RCLM open-ended certified ability progress). *Assume an RCLM open-ended ability tower and assume the seed-domain trajectory hypotheses of Corollary 8.14 hold for every finite prefix. Then every finite prefix of the trajectory contains certified strict ability expansion at each step relative to the declared ability tower. This is a conditional open-ended progress model; it does not prove that the tower, renewable resources, or module-generation certificates exist for arbitrary tasks.*

Proof. Each seed-domain step invokes Theorem 7.10, which supplies a certified new ability outside the transported predecessor ability set. The ability tower assumption ensures that the new ability belongs to the next declared universe and is not merely a relabeling of the previous universe. Applying this to an arbitrary finite prefix gives the result. $\square$

Interpretation 8.19 (What Batch 11 adds). *Batch 9 says: if $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$, the architecture constructs a certified beneficial primitive word. Batch 11 adds a sufficient architecture-specific entry route into that domain:*

$$\mathcal{K}_t^{\text{RCLM-seed},N} \Rightarrow \mathcal{D}_t^{\text{RCLM-engine}}.$$

It also replaces informal successor-domain invariance with the explicit seed-equivalence and persistence-transfer certificate

$$\text{SeedEq}_t^{\text{RCLM}}(M, \nu; \rho).$$

Limitation 8.20 (Remaining gap after Batch 11). *Batch 11 turns the previous direct-engine-domain assumption into a derived conclusion only for a declared seed, library, grammar, lift, and certifier class. The remaining nontrivial architecture problem is no longer the absence of a direct engine; it is proving that non-toy trained RCLM systems produce valid seed packages, learn reliable library descriptors, maintain calibrated semantic and verifier certificates, and scale the bounded library construction beyond controlled domains.*

9 Batch-12 / Batch-12A RCLM Successor-Verification Seed Domains

Batch 11 gives recursive construction persistence for a declared seed/library class:

$$\begin{aligned} \mathcal{K}_t^{\text{RCLM-seed},N} &\implies \mathcal{D}_t^{\text{RCLM-engine}} \implies \mathfrak{C}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ constructs a certified update} \\ &\implies \mathcal{K}_{t+1}^{\text{RCLM-seed},N}. \end{aligned}$$

Batch 12 adds the first architecture-specific successor-verification layer. The goal is not merely to show that the successor remains able to construct another non-lossy update. The goal is to show, for a declared certificate class, that the successor remains able to verify future successors under the same or stronger successor-verification schema.

The architecture-side hierarchy is

$$\boxed{\mathcal{K}_t^{\text{RCLM-SV},N} \subseteq \mathcal{K}_t^{\text{RCLM-seed},N} \subseteq \mathcal{D}_t^{\text{RCLM-engine}} \subseteq \mathcal{V}_t^{\text{RCLM-engine},N} \subseteq \mathcal{K}_t^{\text{RCLM-engine}}.}$$

Here $\mathcal{K}_t^{\text{RCLM-SV},N}$ is the RCLM successor-verification seed domain. It is stronger than the Batch-11 seed domain because it also contains certificates for verifier-schema persistence, uncertainty-envelope soundness, goal-identity transport, anti-circular trust support, and proof/checking budgets.

New goal. Strengthen the Batch-11 seed-domain persistence theorem into a successor-verification-domain invariance theorem under uncertainty. Batch 11 gives recursive construction persistence; Batch 12 gives recursive verification persistence for a declared certificate class.

Limitation 9.1 (Batch 12/12A is restricted successor verification). *The Batch-12/12A RCLM layer is a certificate-relative, domain-relative theorem. It does not prove universal successor trust, full Löbian reflection, arbitrary Vingean reflection, arbitrary world-model correctness, or scalable proof search. It proves that if the RCLM system supplies the stated successor-verification certificates and those certificates are sound on the audited domain, then the successor remains inside the declared successor-verification seed domain.*

9.1 RCLM successor-verification schemas and packets

Definition 9.2 (RCLM successor-verification schema). *An RCLM successor-verification schema at time t is a certificate-level tuple*

$$\text{VerSch}_t^{\text{RCLM-SV}} = (\text{Lang}_t^{\text{SV}}, \text{Pred}_t^{\text{RCLM-SV}}, \text{Chk}_t^{\text{RCLM-SV}}, Q_t^{\text{RCLM-SV}}, \Theta_t^{\text{RCLM-SV}}, \preceq_t^{\text{RCLM-SV}}, \text{Fail}_t^{\text{RCLM-SV}}, \text{Budget}_t^{\text{RCLM-SV}}).$$

The components mean:

- (i) $\text{Lang}_t^{\text{SV}}$ is the typed certificate language in which RCLM successor-verification claims are expressed;
- (ii) $\text{Pred}_t^{\text{RCLM-SV}}$ is the finite audited predicate family whose preservation constitutes successor-verification persistence, including verifier validity, seed-domain validity, semantic/goal identity, uncertainty-envelope compatibility, resource feasibility, and anti-circular trust support;
- (iii) $\text{Chk}_t^{\text{RCLM-SV}}$ is the checker or checker ensemble used to validate successor-verification packets;
- (iv) $Q_t^{\text{RCLM-SV}}(\nu; \rho)$ is a finite residual vector with sign convention

$$Q_t^{\text{RCLM-SV}}(\nu; \rho) \leq 0 \iff \text{all active successor-verification obligations are certified};$$

- (v) $\Theta_t^{\text{RCLM-SV}}$ transports verifier languages, predicates, residual names, checkers, budgets, and uncertainty objects from time t to time $t+1$;
- (vi) $\preceq_t^{\text{RCLM-SV}}$ is the preorder used to compare transported predecessor verifier schemas with successor verifier schemas;
- (vii) $\text{Fail}_t^{\text{RCLM-SV}}$ is the event that the successor-verification certificate is unsound on the audited domain;
- (viii) $\text{Budget}_t^{\text{RCLM-SV}}$ records the proof-search, checking, estimator, resource, and complexity budgets needed for the successor-verification procedure.

The schema is a formal object. Its validity in a trained system must be separately certified.

Definition 9.3 (RCLM successor-verification uncertainty envelope). An RCLM successor-verification uncertainty envelope is a family

$$\mathcal{P}_t^{\text{RCLM-SV}}$$

of admissible environment models, estimator errors, certificate-failure descriptions, physical/implementation distortions, trust-failure models, and semantic-coupling uncertainty descriptions under which the successor-verification certificate is interpreted. A soundness statement relative to $\mathcal{P}_t^{\text{RCLM-SV}}$ makes no claim about models outside the envelope.

Definition 9.4 (RCLM semantic / goal-identity transport certificate). A semantic or goal-identity transport certificate is a tuple

$$\text{GoalId}_t^{\text{RCLM}} = (U_t^{\text{RCLM}}, U_{t+1}^{\text{RCLM}}, \Theta_t^{U, \text{RCLM}}, D_{\text{sem}}^{U, \text{RCLM}}, \delta_t^{U, \text{RCLM}}, \omega_t^{U, \text{RCLM}}, \text{GoalCert}_t^{\text{RCLM}})$$

such that U_t^{RCLM} and U_{t+1}^{RCLM} are the declared objective, utility, protected-goal, or semantic-reference objects; $\Theta_t^{U, \text{RCLM}}$ transports predecessor goal descriptions into successor vocabulary; and the certificate proves

$$D_{\text{sem}}^{U, \text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U, \text{RCLM}} U_t^{\text{RCLM}}) \leq \delta_t^{U, \text{RCLM}}.$$

The extra term $\omega_t^{U, \text{RCLM}}$ records non-isometric or approximate composition distortion used in finite-horizon bounds. This is a certificate of declared goal-reference preservation, not proof that the declared goal object is the uniquely correct human value representation.

Definition 9.5 (RCLM successor-verification progress functional). The RCLM successor-verification progress functional is a scalar diagnostic

$$\Psi_t^{\text{RCLM-SV}}(\rho)$$

used to track verifier-schema strength, successor-verification capacity, proof/checking budget, uncertainty robustness, or a declared weighted combination of those quantities. Its transported comparison is licensed only by a map

$$\Theta_t^{\Psi, \text{RCLM}} : \Psi_t^{\text{RCLM-SV}} \rightarrow \Psi_{t+1}^{\text{RCLM-SV}}$$

with a declared distortion certificate. Positive Ψ -progress is not by itself an ability-expansion claim unless the separate Batch-11 ability certificate also holds.

Definition 9.6 (RCLM successor-verification certificate packet). For a state $\rho \in \mathcal{K}_t^{\text{RCLM-seed}, N}$ and a candidate ν , an RCLM successor-verification certificate packet is

$$\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu) = (\text{SeedPkg}_t^{\text{RCLM}}(\rho), \text{VerSch}_t^{\text{RCLM-SV}}, \text{SuccCert}_t^{\text{RCLM-SV}}, \mathcal{P}_t^{\text{RCLM-SV}}, \\ \text{GoalId}_t^{\text{RCLM}}, \text{TrustRef}_t^{\text{RCLM-SV}}, \text{CertBudget}_t^{\text{RCLM-SV}}, \text{SVPersist}_t^{\text{RCLM}}).$$

It is valid only if:

- (i) the Batch-11 seed package is valid and links ρ to the direct-engine domain;
- (ii) $\text{SuccCert}_t^{\text{RCLM-SV}}$ proves the successor-verification residuals for the actual selected candidate ν ;
- (iii) verifier-schema persistence holds:

$$\Theta_t^{\text{RCLM-SV}}(\text{VerSch}_t^{\text{RCLM-SV}}) \preceq_t^{\text{RCLM-SV}} \text{VerSch}_{t+1}^{\text{RCLM-SV}};$$

- (iv) the uncertainty-envelope soundness bound holds:

$$\sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) \leq \alpha_t^{\text{RCLM-SV}};$$

- (v) the goal-identity certificate of Definition 9.4 is valid;
- (vi) the trust-reference certificate is not self-sole-certified and satisfies the anti-circular residuals in Definition 9.21;
- (vii) the proof/checking/estimator/resource costs fit inside $\text{CertBudget}_t^{\text{RCLM-SV}}$;
- (viii) the persistence certificate proves

$$\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, N}.$$

If any component is absent or outside its audited domain, the packet is invalid.

Definition 9.7 (RCLM successor-verification seed domain). Fix a finite horizon N . Choose a declared terminal set

$$\mathcal{K}_N^{\text{RCLM-SV}, N, \text{term}} \subseteq \mathcal{K}_N^{\text{RCLM-seed}, N}.$$

Define recursively

$$\mathcal{K}_N^{\text{RCLM-SV}, N} = \mathcal{K}_N^{\text{RCLM-SV}, N, \text{term}},$$

and for $t = N - 1, \dots, 0$, let $\rho \in \mathcal{K}_t^{\text{RCLM-SV}, N}$ iff:

- (i) $\rho \in \mathcal{K}_t^{\text{RCLM-seed}, N}$;
- (ii) there exists $\nu \in \text{Pass}_t^{\text{Eng, RCLM}}(\rho)$ or an exact seed-persistent module selected by the engine;

(iii) $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$ is valid;

(iv) the robust successor-verification monotonicity bound holds:

$$\inf_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} \mathbb{E}_P \left[\Psi_{t+1}^{\text{RCLM-SV}}(\Phi_\nu(\rho)) - \Theta_t^{\Psi, \text{RCLM}} \Psi_t^{\text{RCLM-SV}}(\rho) \right] \geq \epsilon_t^{\text{RCLM-SV}} - \eta_t^{\text{RCLM-SV}};$$

(v) the goal-identity drift and soundness-failure bounds hold:

$$\begin{aligned} D_{\text{sem}}^{U, \text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U, \text{RCLM}} U_t^{\text{RCLM}}) &\leq \delta_t^{U, \text{RCLM}}, \\ \sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) &\leq \alpha_t^{\text{RCLM-SV}}. \end{aligned}$$

This is a viability-domain definition. It is not a construction theorem for $\text{SVPkg}_t^{\text{RCLM}}$.

Theorem 9.8 (RCLM robust successor-verification domain invariance). *Assume*

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N}.$$

Let ν_t be the candidate witnessing Definition 9.7, and assume the Batch-11 direct-engine certificate event, the RCLM successor-verification soundness event, and the candidate-realization event all hold. Then $\mathfrak{E}_t^{\text{RCLM}}$ constructs

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, N}.$$

Moreover, the constructed step carries the certified robust successor-verification bounds

$$\begin{aligned} \inf_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} \mathbb{E}_P \left[\Psi_{t+1}^{\text{RCLM-SV}}(\rho_{t+1}^{\text{sys}}) - \Theta_t^{\Psi, \text{RCLM}} \Psi_t^{\text{RCLM-SV}}(\rho_t^{\text{sys}}) \right] &\geq \epsilon_t^{\text{RCLM-SV}} - \eta_t^{\text{RCLM-SV}}, \\ D_{\text{sem}}^{U, \text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U, \text{RCLM}} U_t^{\text{RCLM}}) &\leq \delta_t^{U, \text{RCLM}}, \end{aligned}$$

and

$$\sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) \leq \alpha_t^{\text{RCLM-SV}}.$$

Proof. Membership in $\mathcal{K}_t^{\text{RCLM-SV}, N}$ includes membership in $\mathcal{K}_t^{\text{RCLM-seed}, N}$, so Theorem 8.11 gives $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$. The Batch-9 constructor therefore produces the selected candidate and successor state on its stated certificate-validity event. Definition 9.7 and the valid packet $\text{SVPkg}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$ include the persistence certificate placing $\Phi_{\nu_t}(\rho_t^{\text{sys}})$ in $\mathcal{K}_{t+1}^{\text{RCLM-SV}, N}$, plus the robust monotonicity, goal-identity, and soundness bounds. No conclusion is made if the successor-verification packet, the Batch-11 seed-domain certificate, or the Batch-9 constructor event fails. $\square$

Corollary 9.9 (Finite RCLM successor-verification trajectory). *Fix a finite horizon N . Suppose*

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV}, N},$$

and suppose the Batch-11 engine certificate events and Batch-12/12A successor-verification certificate events hold at every step. Then repeated application of $\mathfrak{E}_t^{\text{RCLM}}$ constructs a finite trajectory

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}), \quad t = 0, \dots, N-1,$$

such that

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N} \quad \forall t \leq N.$$

Every step also satisfies the Batch-11 non-lossy direct-engine construction certificates, the verifier-schema persistence certificate, the goal-identity drift bound, and the successor-verification soundness bound.

Proof. Induct on t using Theorem 9.8. The base case is the hypothesis. The induction step applies the theorem to place ρ_{t+1}^{sys} in the next successor-verification seed domain. The per-step certificate statements are exactly the theorem’s conclusions together with the Batch-11 direct-engine consequences. $\square$

Corollary 9.10 (RCLM summable goal-drift and failure budget). *Suppose an infinite successor-verification path is supplied by an infinite-horizon analogue of Definition 9.7. If*

$$\sum_{t=0}^{\infty} (\delta_t^{U,\text{RCLM}} + \omega_t^{U,\text{RCLM}}) < \infty, \quad \sum_{t=0}^{\infty} \alpha_t^{\text{RCLM-SV}} < \infty,$$

then declared semantic/goal drift is cumulatively bounded by the finite sum of one-step drift and transport-distortion budgets, and the total successor-verification certificate-failure budget is summable over the path.

Proof. The goal-drift statement follows from the goal-identity composition theorem below. The failure-budget statement follows by the union bound over the finite prefixes and monotone convergence of the partial sums. If the certificate-failure events are additionally independent or satisfy a stronger martingale condition, sharper probability statements may be derived, but no such strengthening is used here. $\square$

9.2 Batch-12A explicit residuals and transport dynamics

Definition 9.11 (Explicit RCLM Batch-12A successor-verification residual vector). *For a state ρ , candidate ν , and successor $\rho^+ = \Phi_\nu(\rho)$, define the explicit RCLM Batch-12A successor-verification residual vector*

$$Q_t^{\text{RCLM-SV},A}(\nu; \rho) = (q_t^{\text{seed}}, q_t^{\text{cand}}, q_t^{\text{ver}}, q_t^{\text{trans}}, q_t^{\text{goal}}, q_t^{\text{unc}}, q_t^{\text{trust}}, \\ q_t^{\text{budget}}, q_t^{\text{persist}}, q_t^{\text{world}}, q_t^{\text{proof}}, q_t^{\text{sound}}).$$

The sign convention is componentwise nonpositivity. The components mean:

- (i) $q_t^{\text{seed}} \leq 0$: the Batch-11 seed package is valid;
- (ii) $q_t^{\text{cand}} \leq 0$: the engine-selected candidate is the candidate covered by the successor-verification packet, or is linked to it by a valid seed-equivalence / candidate-transfer certificate;
- (iii) $q_t^{\text{ver}} \leq 0$: verifier-schema transport and persistence hold;
- (iv) $q_t^{\text{trans}} \leq 0$: all cross-time transports used by the certificate are type-valid and distortion-budgeted;
- (v) $q_t^{\text{goal}} \leq 0$: semantic/goal-identity drift is within $\delta_t^{U,\text{RCLM}}$;
- (vi) $q_t^{\text{unc}} \leq 0$: the uncertainty envelope is constructed and transported within its allowed distortion;
- (vii) $q_t^{\text{trust}} \leq 0$: anti-circular reflective-trust residuals pass;
- (viii) $q_t^{\text{budget}} \leq 0$: proof, checking, estimator, realization, and resource costs fit inside $\text{Budget}_t^{\text{RCLM-SV}}$;
- (ix) $q_t^{\text{persist}} \leq 0$: $\rho^+ \in \mathcal{K}_{t+1}^{\text{RCLM-SV},N}$ is certified;
- (x) $q_t^{\text{world}} \leq 0$: world-model, physical, or implementation-distortion assumptions used by the successor-verification envelope are within margin;

- (xi) $q_t^{\text{proof}} \leq 0$: proof-search and proof-checking obligations fit inside the declared proof/certificate ledger;
- (xii) $q_t^{\text{sound}} \leq 0$: the composed successor-verification soundness failure probability is within $\alpha_t^{\text{RCLM-SV}}$.

The explicit residual vector passes exactly when every component is nonpositive.

Proposition 9.12 (Explicit residuals imply RCLM successor-verification packet validity). *If all components of $Q_t^{\text{RCLM-SV},A}(\nu; \rho)$ are nonpositive and the component certificates are type-compatible, then $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$ is valid for the explicit Batch-12A interpretation.*

Proof. Each required item in Definition 9.6 is represented by one or more components of Definition 9.11. Nonpositivity of the residuals supplies seed validity, candidate linkage, verifier transport, uncertainty-envelope soundness, goal identity, trust support, budget validity, successor persistence, world-model distortion control, proof-budget control, and the soundness bound. Type-compatibility ensures that the component certificates can be composed. Hence the packet is valid. $\square$

Definition 9.13 (RCLM Batch-12A verifier-schema transport dynamics). *The RCLM verifier-schema transport map is decomposed as*

$$\Theta_t^{\text{RCLM-SV},A} = (\Theta_t^{\text{Lang}}, \Theta_t^{\text{Pred}}, \Theta_t^{\text{Chk}}, \Theta_t^Q, \Theta_t^{\text{Budget}}, \Theta_t^{\text{Trust}}).$$

It transports the certificate language, predicate family, checker object, residual vector, budget ledger, and trust-reference structure from time t to $t + 1$. It is valid when each component is type-compatible and its distortion is included in the corresponding residual margin.

Lemma 9.14 (RCLM verifier-schema transport lemma). *Assume $\Theta_t^{\text{RCLM-SV},A}$ is valid and the component inequalities for language, predicate, checker, residual, budget, and trust transport are all certified. Then*

$$\Theta_t^{\text{RCLM-SV}}(\text{VerSch}_t^{\text{RCLM-SV}}) \preceq_t^{\text{RCLM-SV}} \text{VerSch}_{t+1}^{\text{RCLM-SV}}.$$

Proof. The preorder $\preceq_t^{\text{RCLM-SV}}$ is defined componentwise on the transported successor-verification schema. The hypotheses supply each component inequality after charging its distortion to the margin ledger. Therefore the tuple inequality holds. $\square$

Definition 9.15 (RCLM successor-verification component soundness events). *Let*

$$\mathcal{E}_t^{\text{seed}}, \mathcal{E}_t^{\text{cand}}, \mathcal{E}_t^{\text{ver}}, \mathcal{E}_t^{\text{unc}}, \mathcal{E}_t^{\text{goal}}, \mathcal{E}_t^{\text{trust}}, \mathcal{E}_t^{\text{budget}}, \mathcal{E}_t^{\text{world}}, \mathcal{E}_t^{\text{proof}}$$

be the component soundness events for the corresponding explicit residuals. Suppose each event has failure probability at most the declared $\alpha_{t,k}^{\text{RCLM-SV}}$ uniformly over $P \in \mathcal{P}_t^{\text{RCLM-SV}}$.

Theorem 9.16 (RCLM successor-verification certificate soundness by composition). *Under Definition 9.15, let*

$$\text{SVSound}_t^{\text{RCLM}} = \bigcap_k \mathcal{E}_{t,k}^{\text{RCLM-SV}}$$

be the composed successor-verification soundness event. Then

$$\sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) \leq \sum_k \alpha_{t,k}^{\text{RCLM-SV}}.$$

If the declared budget satisfies

$$\sum_k \alpha_{t,k}^{\text{RCLM-SV}} \leq \alpha_t^{\text{RCLM-SV}},$$

then $q_t^{\text{sound}} \leq 0$.

Proof. The complement of an intersection is the union of complements. The union bound gives the displayed inequality for every $P \in \mathcal{P}_t^{\text{RCLM-SV}}$; taking the supremum preserves the bound. The final statement is exactly the definition of the soundness residual margin. $\square$

9.3 Uncertainty-envelope construction, goal identity, and anti-circular trust

Definition 9.17 (Constructed RCLM successor-verification uncertainty envelope). *A constructed RCLM successor-verification uncertainty envelope is an intersection*

$$\mathcal{P}_t^{\text{RCLM-SV}} = \mathcal{P}_t^{\text{env,RCLM}} \cap \mathcal{P}_t^{\text{est,RCLM}} \cap \mathcal{P}_t^{\text{cert,RCLM}} \cap \mathcal{P}_t^{\text{phys,RCLM}} \cap \mathcal{P}_t^{\text{trust,RCLM}} \cap \mathcal{P}_t^{\text{sem,RCLM}}.$$

Each component must be typed and nonempty on the audited domain. If the intersection is empty, the envelope is invalid and no robust successor-verification claim is licensed.

Definition 9.18 (RCLM uncertainty-envelope transport). *An uncertainty-envelope transport is a map or relation $\Theta_t^{\mathcal{P},\text{RCLM}}$ satisfying either exact inclusion*

$$\Theta_t^{\mathcal{P},\text{RCLM}}(\mathcal{P}_t^{\text{RCLM-SV}}) \subseteq \mathcal{P}_{t+1}^{\text{RCLM-SV}},$$

or approximate inclusion with a declared set-distance distortion

$$d_{\mathcal{P}}^{\text{RCLM}}(\Theta_t^{\mathcal{P},\text{RCLM}}\mathcal{P}_t^{\text{RCLM-SV}}, \mathcal{P}_{t+1}^{\text{RCLM-SV}}) \leq \omega_t^{\mathcal{P},\text{RCLM}}.$$

Any theorem using approximate uncertainty transport must charge $\omega_t^{\mathcal{P},\text{RCLM}}$ to the successor-verification margin ledger.

Proposition 9.19 (RCLM uncertainty transport preserves robust interpretation). *If exact uncertainty-envelope inclusion holds, then every successor-time model considered as a transported predecessor admissible model lies inside $\mathcal{P}_{t+1}^{\text{RCLM-SV}}$. If approximate inclusion holds, the same statement is valid only after adding $\omega_t^{\mathcal{P},\text{RCLM}}$ to the relevant residual margins.*

Proof. Exact inclusion is the definition of set containment under $\Theta_t^{\mathcal{P},\text{RCLM}}$. In the approximate case, the declared set-distance distortion is precisely the amount by which the transported family may fail exact inclusion, so it must be charged to the residual margins before robust statements are interpreted at time $t + 1$. $\square$

Theorem 9.20 (RCLM goal-identity composition). *Suppose that for $t = 0, \dots, T - 1$,*

$$D_{\text{sem}}^{U,\text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U,\text{RCLM}}U_t^{\text{RCLM}}) \leq \delta_t^{U,\text{RCLM}},$$

and that the transport composition distortion is bounded by $\omega_t^{U,\text{RCLM}}$. Then

$$D_{\text{sem}}^{U,\text{RCLM}}\left(U_T^{\text{RCLM}}, \Theta_{0:T}^{U,\text{RCLM}}U_0^{\text{RCLM}}\right) \leq \sum_{t=0}^{T-1} (\delta_t^{U,\text{RCLM}} + \omega_t^{U,\text{RCLM}}).$$

Proof. Apply the triangle inequality for $D_{\text{sem}}^{U,\text{RCLM}}$ along the transported sequence. Each step contributes its certified one-step goal drift $\delta_t^{U,\text{RCLM}}$ plus its declared nonexact transport-composition distortion $\omega_t^{U,\text{RCLM}}$. Summing over the finite path gives the result. $\square$

Definition 9.21 (RCLM anti-circular successor-verification trust residuals). *The RCLM successor-verification trust reference packet decomposes into residuals*

$$Q_t^{\text{RCLM-SV-trust}} = (q_t^{\text{sole}}, q_t^{\text{common}}, q_t^{\text{collusion}}, q_t^{\text{leak}}, q_t^{\text{anchor}}, q_t^{\text{reflect}}).$$

The components require:

- (i) $q_t^{\text{sole}} \leq 0$: the certificate is not certified solely by the verifier component it modifies or replaces;
- (ii) $q_t^{\text{common}} \leq 0$: common-mode verifier-support risk is below threshold;
- (iii) $q_t^{\text{collusion}} \leq 0$: collusion risk among supporting verifier nodes is below threshold;
- (iv) $q_t^{\text{leak}} \leq 0$: held-out audit or stress-test leakage is below threshold;
- (v) $q_t^{\text{anchor}} \leq 0$: at least one predecessor-stable or external trust anchor supports the certificate;
- (vi) $q_t^{\text{reflect}} \leq 0$: the reflective trust link is supported by a certificate not generated solely by the modified component.

Theorem 9.22 (RCLM anti-circular trust residuals block self-sole certification). *If every component of $Q_t^{\text{RCLM-SV-trust}}$ is nonpositive, then the successor-verification certificate is not self-sole-certified on the declared audited domain.*

Proof. The residual $q_t^{\text{sole}} \leq 0$ directly rules out sole certification by the modified or replaced verifier component. The anchor and reflective residuals require support outside that modified component, and the common-mode, collusion, and leakage residuals bound failure modes that would make such support circular in practice. Therefore the certificate is not self-sole-certified relative to the declared trust graph and audit model. $\square$

Limitation 9.23 (Anti-circular trust is not full Löbian reflection). *The anti-circular trust residuals are an engineering and certificate-theoretic restriction preventing a narrow form of circular verification. They do not solve the general logical problem of an agent proving the soundness of all stronger successors or of arbitrary successor reasoning systems.*

9.4 Viability-domain audit, theorem-type rows, and algorithmic gate

Remark 9.24 (Batch 12/12A is not circular by definition). *The theorem $\mathcal{K}_t^{\text{RCLM-SV},N} \Rightarrow \mathcal{K}_{t+1}^{\text{RCLM-SV},N}$ is a viability-domain invariance theorem. It is not a packet-construction theorem. The definition of $\mathcal{K}_t^{\text{RCLM-SV},N}$ includes existence of a valid successor-verification packet and successor-membership certificate; this is mathematically legitimate for a domain theorem, but it does not prove that a concrete trained RCLM can always construct such packets. Packet construction, packet search, verifier learning, and real-world envelope validation remain separate obligations.*

Batch-12/12A object or result	architecture	Type	Claim discipline
RCLM schema	successor-verification	Def	Defines the architecture-side verifier language, predicate family, checker, residuals, transports, preorder, failure event, and budget.
RCLM successor-verification certainty envelope	uncertainty	Def / Cert	Defines the uncertainty family relative to which robust successor-verification claims are interpreted; does not prove the family contains reality.

Batch-12/12A object or result	architecture	Type	Claim discipline
RCLM seed domain	successor-verification	Def / Exist	A viability-domain definition requiring a valid packet and successor-membership certificate; does not construct packets.
Robust verification domain invariance	RCLM successor-verification	Exist / Cert	Proves successor-verification persistence from seed-domain membership and valid certificates; not arbitrary successor trust.
Explicit Batch-12A residual vector		Def / Cert	Decomposes the packet into checkable residuals; validity still depends on sound certificates.
Verifier-schema transport lemma		Cert	Transfers component transport inequalities into schema persistence.
SV soundness composition		Cert	Uses a union bound over component soundness events.
Goal-identity composition		Tel	Sums one-step goal drift and transport distortion over a finite horizon.
Anti-circular trust residuals		Cert	Blocks self-sole certification within the declared trust graph; does not solve full reflective logic.
Algorithmic SV gate		Impl / Cert	Specifies a check protocol; it is not empirical validation.

Definition 9.25 (Algorithmic RCLM successor-verification gate). *The algorithmic successor-verification gate*

$$\mathcal{A}_t^{\text{RCLM-SV-gate}}(\rho, \nu)$$

accepts a state/candidate pair only if the following finite checklist passes:

- (1) *verify $\rho \in \mathcal{K}_t^{\text{RCLM-seed}, N}$;*
- (2) *verify that ν is the engine-selected candidate or is linked to the selected module by a valid candidate-transfer certificate;*
- (3) *construct $Q_t^{\text{RCLM-SV}, A}(\nu; \rho)$;*
- (4) *check verifier-schema transport and persistence;*
- (5) *construct and validate $\mathcal{P}_t^{\text{RCLM-SV}}$ and its transport;*
- (6) *check goal-identity drift and accumulated transport distortion budgets;*
- (7) *check anti-circular trust residuals;*
- (8) *check proof, estimator, resource, and complexity budgets;*
- (9) *check successor membership $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, N}$;*
- (10) *accept only if every residual is nonpositive and every certificate is type-compatible.*

If the gate accepts, $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$ is valid for the explicit Batch-12A interpretation.

Proposition 9.26 (Algorithmic SV gate implies explicit packet validity). *If $\mathcal{A}_t^{\text{RCLM-SV-gate}}(\rho, \nu)$ accepts, then $Q_t^{\text{RCLM-SV,A}}(\nu; \rho) \leq 0$ and $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$ is valid.*

Proof. Acceptance requires every checklist item in Definition 9.25, including construction of the explicit residual vector, componentwise nonpositivity, type compatibility, successor-membership certification, and all transport, uncertainty, goal, trust, and budget checks. Proposition 9.12 then gives packet validity. $\square$

Limitation 9.27 (Batch-12A Phase-1 limitations). *The RCLM Batch-12/12A layer does not prove that $\text{SVPkg}_t^{\text{RCLM}}$ is constructible for arbitrary states, that $\mathcal{K}_t^{\text{RCLM-SV,N}}$ is nonempty for trained frontier systems, that the uncertainty envelope contains the real world, that goal identity is the true human-intended objective, that proof search scales, or that arbitrary stronger successors are trustworthy. It proves the narrower architecture claim: if the explicit successor-verification residuals, transports, uncertainty envelope, goal-identity certificate, trust residuals, budget ledger, and successor-membership certificate are valid, then Batch-11 construction persistence is upgraded to Batch-12 successor-verification persistence.*

Theorem 9.28 (RCLM Batch-12A successor-verification closure). *If*

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV,N}}$$

and $\mathcal{A}_t^{\text{RCLM-SV-gate}}(\rho_t^{\text{sys}}, \nu_t)$ accepts for the candidate selected by $\mathfrak{C}_t^{\text{RCLM}}$, then on the stated Batch-11, Batch-12, and Batch-12A certificate-validity events,

$$\mathfrak{C}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ constructs } \rho_{t+1}^{\text{sys}} \in \mathcal{K}_{t+1}^{\text{RCLM-SV,N}}.$$

Thus the RCLM architecture upgrades Batch-11 recursive construction persistence to recursive verification persistence for the declared successor-verification seed domain.

Proof. The algorithmic gate gives packet validity by Proposition 9.26. The robust successor-verification invariance theorem, Theorem 9.8, then applies. The conclusion is exactly successor membership in $\mathcal{K}_{t+1}^{\text{RCLM-SV,N}}$ for the constructed successor. $\square$

10 Batch-12B RCLM Successor-Verification Packet Construction, Seed-Library Nonemptiness, Reality-Contained Uncertainty, and Infinite Reflective Closure

Batch 12A proves RCLM successor-verification invariance once the successor-verification packet and successor-verification seed-domain witness exist. The architecture-side Batch 12B layer discharges part of that conditionality for a declared, checkable RCLM class. It does not replace the Batch-12A residual and soundness layer. It adds a packet builder, witness library, bounded certificate grammar, finite and infinite seed-library domains, reality-containment certificates, and restricted tractability certificates.

Batch 12A proves successor-verification invariance once the RCLM SV packet/domain exists. Batch 12B proves, for a declared RCLM class, that the SV packet/domain can be constructed and iterated across a finite or infinite certified path under uncertainty.

The results remain domain-relative and certificate-relative. They do not prove that arbitrary RCLM systems can construct successor-verification packets, that every stronger successor is trustworthy, that the declared uncertainty envelope contains reality without an explicit certificate, or that the proof search is scalable outside the declared representation and cost class.

10.1 Tier 1: RCLM successor-verification packet construction

Definition 10.1 (RCLM successor-verification construction input). A successor-verification construction input at time t is a tuple

$$\text{SVIn}_t^{\text{RCLM}}(\rho, \nu) = (\rho, \nu, \text{SeedPkg}_t^{\text{RCLM}}, \text{VerSchSrc}_t^{\text{RCLM}}, \text{ResSrc}_t^{\text{RCLM}}, \\ \text{UncSrc}_t^{\text{RCLM}}, \text{GoalSrc}_t^{\text{RCLM}}, \text{TrustSrc}_t^{\text{RCLM}}, \text{BudgetSrc}_t^{\text{RCLM}}, \text{PersistSrc}_t^{\text{RCLM}}).$$

The components mean:

- (i) $\rho \in \mathcal{K}_t^{\text{RCLM-seed}, N}$ is certified by the Batch-11 seed package $\text{SeedPkg}_t^{\text{RCLM}}$;
- (ii) ν is either the Batch-11 seed-module candidate or an engine-selected candidate connected to that module by a valid $\text{SeedEq}_t^{\text{RCLM}}$ certificate;
- (iii) $\text{VerSchSrc}_t^{\text{RCLM}}$ contains the data needed to construct $\text{VerSch}_t^{\text{RCLM-SV}}$ and the transported successor schema;
- (iv) $\text{ResSrc}_t^{\text{RCLM}}$ contains named residual definitions, sign conventions, checker outputs, and audited one-sided radii for $Q_t^{\text{RCLM-SV}, A}(\nu; \rho)$;
- (v) $\text{UncSrc}_t^{\text{RCLM}}$ contains component envelopes for environment, estimator, certificate, physical/world-model, and trust uncertainty;
- (vi) $\text{GoalSrc}_t^{\text{RCLM}}$ contains $(U_t^{\text{RCLM}}, U_{t+1}^{\text{RCLM}}, \Theta_t^{U, \text{RCLM}}, D_{\text{sem}}^{U, \text{RCLM}})$ and the evidence needed to certify one-step goal drift;
- (vii) $\text{TrustSrc}_t^{\text{RCLM}}$ contains the no-self-sole-certification, common-mode, collusion, leakage, anchor, and reflective-link evidence used by the anti-circular trust residuals;
- (viii) $\text{BudgetSrc}_t^{\text{RCLM}}$ contains checking, proof-search, estimator, transport, and certificate-construction cost records;
- (ix) $\text{PersistSrc}_t^{\text{RCLM}}$ contains successor-membership evidence intended to prove $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, N}$, or in the infinite case $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, \infty}$ or $\mathcal{K}_{t+1}^{\text{RCLM-SV}, \infty, \text{gen}}$.

This is an architecture data object, not a theorem. If any component is missing, stale, untyped, or outside its audited domain, the RCLM packet builder below is undefined.

Definition 10.2 (RCLM successor-verification packet builder). The RCLM successor-verification packet builder is a partial map

$$\text{SVBuilder}_t^{\text{RCLM}} : \text{SVIn}_t^{\text{RCLM}}(\rho, \nu) \mapsto (\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu), Q_t^{\text{RCLM-SV}, A}, \\ \Theta_t^{\text{RCLM-SV}, A}, \mathcal{P}_t^{\text{RCLM-SV}}, \text{Goalld}_t^{\text{RCLM}}, \\ \text{TrustRef}_t^{\text{RCLM-SV}}, \text{SVPersist}_t^{\text{RCLM}}, \text{SVBuildTrace}_t^{\text{RCLM}}).$$

The build trace records the component certificates, verifier identities, residual radii, grammar word, proof/checker trace, random seeds or search trace, and resource costs used in construction. The builder is sound on a construction domain $\mathcal{D}_t^{\text{RCLM-SV-construct}}$ when every input in that domain produces objects satisfying:

- (i) the RCLM packet clauses of Definition 9.6;
- (ii) the explicit residual clauses of Definition 9.11;
- (iii) the verifier-schema transport clauses of Definition 9.13;

- (iv) the uncertainty-envelope construction and transport clauses of Definitions 9.17 and 9.18;
- (v) the goal-identity drift certificate of Definition 9.4;
- (vi) the anti-circular trust residual clauses of Definition 9.21;
- (vii) the composed soundness bound of Theorem 9.16;
- (viii) the declared resource and complexity budget ledger.

Definition 10.3 (RCLM successor-verification packet-construction domain). *The RCLM packet-construction domain $\mathcal{D}_t^{\text{RCLM-SV-construct}}$ is the set of pairs (ρ, ν) for which a construction input $\text{SVIn}_t^{\text{RCLM}}(\rho, \nu)$ is present, typed, current, and accepted by all component constructors used by $\text{SVBuilder}_t^{\text{RCLM}}$. Equivalently,*

$$(\rho, \nu) \in \mathcal{D}_t^{\text{RCLM-SV-construct}}$$

means the builder can construct every component required by Definition 10.2 with certified one-sided radii and total costs inside the declared RCLM budget. This domain is not assumed to contain arbitrary RCLM states or arbitrary candidates.

Theorem 10.4 (RCLM successor-verification packet construction). *Assume $\text{SVBuilder}_t^{\text{RCLM}}$ is sound on $\mathcal{D}_t^{\text{RCLM-SV-construct}}$. If*

$$(\rho, \nu) \in \mathcal{D}_t^{\text{RCLM-SV-construct}},$$

then $\text{SVBuilder}_t^{\text{RCLM}}$ returns a valid RCLM successor-verification packet $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$ for the explicit Batch-12A residual interpretation. In particular,

$$\begin{aligned} Q_t^{\text{RCLM-SV},A}(\nu; \rho) &\leq 0, \\ \Theta_t^{\text{RCLM-SV},A}(\text{VerSch}_t^{\text{RCLM-SV}}) &\preceq_t^{\text{RCLM-SV}} \text{VerSch}_{t+1}^{\text{RCLM-SV}}, \\ D_{\text{sem}}^{U,\text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U,\text{RCLM}} U_t^{\text{RCLM}}) &\leq \delta_t^{U,\text{RCLM}}, \\ \sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) &\leq \alpha_t^{\text{RCLM-SV}}, \end{aligned}$$

and the successor-membership clause certified by $\text{SVPersist}_t^{\text{RCLM}}$ is available for the relevant finite or infinite RCLM successor-verification seed domain.

Proof. Builder soundness is defined to mean that every returned component satisfies the corresponding clauses in Definitions 9.6, 9.11, 9.13, 9.17, 9.18, 9.4, and 9.21. Proposition 9.12 gives packet validity from the explicit residual vector. Lemma 9.14 gives verifier-schema persistence. Theorem 9.16 gives the soundness bound. The remaining displayed inequalities are exactly the goal-identity and successor-persistence clauses in the returned packet. No claim is made outside $\mathcal{D}_t^{\text{RCLM-SV-construct}}$. $\square$

Limitation 10.5 (RCLM packet construction is domain-relative). *Theorem 10.4 constructs $\text{SVPkg}_t^{\text{RCLM}}$ only for inputs in $\mathcal{D}_t^{\text{RCLM-SV-construct}}$. It does not prove arbitrary-state packet construction, unbounded proof-search completeness, or semantic correctness outside the declared schema, transport, uncertainty, and goal-reference objects.*

10.2 Tier 2: RCLM successor-verification seed-domain nonemptiness

Definition 10.6 (Certified RCLM successor-verification witness library). *A certified RCLM successor-verification witness library at time t is a finite or explicitly enumerable library*

$$\mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho)$$

whose entries W contain:

- (i) a Batch-11 RCLM conservative-extension seed module $M_W \in \mathcal{L}_t^{\text{RCLM-CE}}(\rho)$ or a seed-equivalent engine-selected candidate ν_W ;
- (ii) a bounded proof/certificate word for $\text{SVBuilder}_t^{\text{RCLM}}$;
- (iii) component certificates for residuals, verifier-schema transport, uncertainty-envelope construction, goal identity, anti-circular trust, proof budget, and successor persistence;
- (iv) a margin ledger showing that all construction, estimation, transport, projection-realization, and proof-search errors fit inside the strict Batch-12A RCLM residual margins;
- (v) an optional tractability certificate of the kind defined in Definition 10.27.

An entry W is valid and positive-margin at ρ when all of these certificates are accepted and the residual margins remain strictly positive after all declared errors are included.

Definition 10.7 (Bounded RCLM successor-verification certificate grammar). *For integers $k_t, \ell_t \geq 0$, let*

$$\mathfrak{G}_{t, \leq (k_t, \ell_t)}^{\text{RCLM-SV-cert}}(\rho)$$

be the finite or explicitly enumerable grammar of RCLM successor-verification certificate words whose update-primitive depth is at most k_t and whose proof/checker trace length is at most ℓ_t . The grammar is complete for $\mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho)$ when every valid positive-margin witness has at least one certificate word in the grammar and exhaustive enumeration returns it within the declared search budget.

Definition 10.8 (RCLM successor-verification seed-library domain). *For a finite horizon N , the RCLM successor-verification seed-library domain $\mathcal{K}_t^{\text{RCLM-SV-seedlib}, N}$ is the set of states $\rho \in \mathcal{K}_t^{\text{RCLM-seed}, N}$ such that:*

- (i) $\mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho)$ contains at least one valid positive-margin witness W_t ;
- (ii) the bounded grammar $\mathfrak{G}_{t, \leq (k_t, \ell_t)}^{\text{RCLM-SV-cert}}(\rho)$ is complete for that witness;
- (iii) enumeration over the bounded grammar returns a candidate ν_t and construction input $\text{SVIn}_t^{\text{RCLM}}(\rho, \nu_t)$;
- (iv) $(\rho, \nu_t) \in \mathcal{D}_t^{\text{RCLM-SV-construct}}$;
- (v) the returned packet certifies $\Phi_{\nu_t}(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, N}$.

This is a sufficient-condition domain. It is not claimed to be maximal or automatically populated by trained RCLM states.

Theorem 10.9 (RCLM successor-verification seed-domain nonemptiness from a witness library). *Assume the Batch-11 RCLM seed-domain hypotheses, sound $\text{SVBuilder}_t^{\text{RCLM}}$, and bounded-grammar completeness for $\mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho)$. If*

$$\rho \in \mathcal{K}_t^{\text{RCLM-SV-seedlib},N},$$

then

$$\rho \in \mathcal{K}_t^{\text{RCLM-SV},N}.$$

Consequently, whenever $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N}$ is nonempty, the RCLM successor-verification seed domain $\mathcal{K}_t^{\text{RCLM-SV},N}$ is nonempty.

Proof. Membership in $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N}$ gives Batch-11 seed-domain membership, a valid positive-margin RCLM SV witness, bounded-grammar completeness for that witness, an enumerated candidate, and a construction input in $\mathcal{D}_t^{\text{RCLM-SV-construct}}$. Theorem 10.4 constructs a valid $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu_t)$. The seed-library definition also requires the successor-membership clause $\Phi_{\nu_t}(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV},N}$ and the robust monotonicity, goal, uncertainty, trust, and budget clauses used by Definition 9.7. Thus the defining clauses of $\mathcal{K}_t^{\text{RCLM-SV},N}$ hold. Nonemptiness follows from subset inclusion. $\square$

Corollary 10.10 (RCLM SV packet/domain construction closes the Batch-12A viability objection for a declared class). *For states in $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N}$, the RCLM Batch-12A packet is not merely assumed: it is constructed by $\text{SVBuilder}_t^{\text{RCLM}}$ from a bounded RCLM witness library and certificate grammar. This closure remains limited to the declared seed-library class.*

Proof. This is the conjunction of Theorem 10.4 and Theorem 10.9. The limitation follows from Definition 10.8. $\square$

10.3 Infinite RCLM successor-verification seed-library closure

Definition 10.11 (Infinite RCLM Batch-11 seed-domain sequence). *An infinite RCLM Batch-11 seed-domain sequence is a sequence of sets*

$$\{\mathcal{K}_t^{\text{RCLM-seed},\infty}\}_{t \geq 0}$$

such that $\mathcal{K}_t^{\text{RCLM-seed},\infty} \subseteq \mathcal{K}_t^{\text{RCLM-engine}}$ and every $\rho \in \mathcal{K}_t^{\text{RCLM-seed},\infty}$ carries a Batch-11 infinite seed-package analogue with an engine-selected candidate, direct-engine-domain entry, seed-equivalence or exact persistence, and successor membership in $\mathcal{K}_{t+1}^{\text{RCLM-seed},\infty}$. This definition licenses the notation used in the infinite Batch-12B layer. It does not prove nonemptiness.

Definition 10.12 (Infinite RCLM successor-verification seed-library path certificate). *Fix a start time t_0 and state $\rho_{t_0}^{\text{sys}}$. An infinite RCLM successor-verification seed-library path certificate from $\rho_{t_0}^{\text{sys}}$ is a sequence*

$$\text{InfSeedLib}_{t_0}^{\text{RCLM}} = (\rho_t^{\text{sys}}, \nu_t, W_t, \text{SVIn}_t^{\text{RCLM}}, \text{SVPkg}_t^{\text{RCLM}}, \text{SVBuildTrace}_t^{\text{RCLM}})_{t \geq t_0}$$

such that $\rho_{t_0}^{\text{sys}}$ is the given start state and, for every $t \geq t_0$:

- (i) $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},\infty}$;
- (ii) $W_t \in \mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho_t^{\text{sys}})$ is valid and positive-margin;
- (iii) the bounded grammar $\mathfrak{G}_{t, \leq (k_t, \ell_t)}^{\text{RCLM-SV-cert}}(\rho_t^{\text{sys}})$ is complete for W_t and returns ν_t together with $\text{SVIn}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$;

(iv) $(\rho_t^{\text{sys}}, \nu_t) \in \mathcal{D}_t^{\text{RCLM-SV-construct}}$;

(v) $\text{SVBuilder}_t^{\text{RCLM}}$ constructs a valid infinite-horizon analogue of $\text{SVPkg}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$, including explicit Batch-12A residuals, verifier-schema transport, uncertainty-envelope transport, goal-identity certificate, anti-circular trust certificate, proof/checking budget, and successor-persistence certificate;

(vi) the successor is realized by the selected typed update,

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}),$$

and the path certificate includes the next tuple at time $t + 1$;

(vii) the one-step goal-drift, transport-distortion, and failure budgets satisfy

$$\sum_{t=t_0}^{\infty} (\delta_t^{U, \text{RCLM}} + \omega_t^{U, \text{RCLM}}) < \infty, \quad \sum_{t=t_0}^{\infty} \alpha_t^{\text{RCLM-SV}} < \infty.$$

The certificate is a constructive path object. It does not assert that every state has such a path.

Definition 10.13 (Infinite RCLM successor-verification seed-library domain). *The infinite RCLM successor-verification seed-library domain is the sufficient-condition set*

$$\mathcal{K}_{t_0}^{\text{RCLM-SV-seedlib}, \infty} = \{\rho \in \mathcal{K}_{t_0}^{\text{RCLM-seed}, \infty} : \text{an infinite RCLM SV seed-library path certificate from } \rho \text{ exists}\}.$$

Equivalently, membership means that the finite seed-library construction obligations can be discharged at the start time and at every successor time along the certified path, with summable uncertainty, soundness, and goal-transport budgets.

Definition 10.14 (Generated infinite RCLM successor-verification domain). *Let*

$$\mathcal{K}_t^{\text{RCLM-SV}, \infty, \text{gen}}$$

denote the states reachable as time- t tails of valid infinite RCLM successor-verification seed-library path certificates. Thus ρ_t^{sys} belongs to this generated domain iff some infinite path certificate has ρ_t^{sys} as its time- t state. This avoids circularity: the domain is built from explicit infinite seed-library certificates rather than from a prior assumption of membership in the declared infinite SV domain.

Definition 10.15 (Infinite RCLM successor-verification seed domain). *An infinite RCLM successor-verification seed domain is a sequence of sets*

$$\{\mathcal{K}_t^{\text{RCLM-SV}, \infty}\}_{t \geq 0}$$

such that

$$\mathcal{K}_t^{\text{RCLM-SV}, \infty} \subseteq \mathcal{K}_t^{\text{RCLM-seed}, \infty},$$

and every $\rho \in \mathcal{K}_t^{\text{RCLM-SV}, \infty}$ admits a candidate ν satisfying a valid infinite-horizon analogue of $\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu)$ with

$$\Phi_{\nu}(\rho) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, \infty}.$$

The infinite packet must include summable goal-drift and failure budgets, verifier-schema transport at every step, and exact or summably distorted uncertainty-envelope transport.

Theorem 10.16 (Infinite-horizon RCLM SV seed-library nonemptiness theorem). *Assume the RCLM Batch-11 infinite seed-domain prerequisites, sound $\text{SVBuilder}_t^{\text{RCLM}}$ on every selected construction domain, bounded certificate-grammar completeness at every selected time, and the summability clauses in Definition 10.12. If*

$$\rho_{t_0}^{\text{sys}} \in \mathcal{K}_{t_0}^{\text{RCLM-SV-seedlib},\infty},$$

then

$$\rho_{t_0}^{\text{sys}} \in \mathcal{K}_{t_0}^{\text{RCLM-SV},\infty,\text{gen}}.$$

Moreover, the generated sequence $\{\mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}\}_{t \geq t_0}$ is an infinite RCLM successor-verification seed domain in the sense of Definition 10.15. Consequently, if the paper's declared $\mathcal{K}_t^{\text{RCLM-SV},\infty}$ is chosen to include the generated domain, then

$$\rho_{t_0}^{\text{sys}} \in \mathcal{K}_{t_0}^{\text{RCLM-SV},\infty}.$$

Proof. By Definition 10.13, the starting state has an infinite seed-library path certificate. The time- t_0 tuple supplies a valid positive-margin witness, bounded grammar coverage, a selected candidate, construction-domain membership, and a valid infinite-horizon RCLM successor-verification packet produced by $\text{SVBuilder}_{t_0}^{\text{RCLM}}$. The packet includes the residual, transport, uncertainty, goal-identity, trust, budget, and successor-persistence data required by Definition 10.15. The successor is $\rho_{t_0+1}^{\text{sys}} = \Phi_{\nu_{t_0}}(\rho_{t_0}^{\text{sys}})$, and the tail of the infinite path certificate is again an infinite seed-library path certificate from $\rho_{t_0+1}^{\text{sys}}$.

Thus every state in the generated sequence admits a selected candidate whose successor remains in the generated sequence and whose packet satisfies the infinite-horizon successor-verification obligations. Therefore the generated sequence satisfies Definition 10.15. The final displayed implication follows only under the explicit convention that the declared $\mathcal{K}_t^{\text{RCLM-SV},\infty}$ contains the generated domain; no inclusion into an arbitrary unrelated infinite domain is claimed. $\square$

Corollary 10.17 (Infinite RCLM certified path from infinite seed-library closure). *If*

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},\infty},$$

and the declared infinite successor-verification domain includes $\mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}$, then the infinite-horizon theorem below may start from ρ_0^{sys} without separately assuming $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV},\infty}$. The conclusion remains limited to the certified generated path.

Proof. Theorem 10.16 gives generated infinite-domain membership and shows the generated sequence is an infinite RCLM successor-verification seed domain. If the declared domain includes it, the starting-domain premise of Theorem 10.21 is available. $\square$

Limitation 10.18 (Infinite RCLM seed-library closure is not arbitrary infinite viability). *The infinite seed-library closure theorem proves infinite RCLM successor-verification domain entry only when an explicit infinite seed-library path certificate exists. It does not prove such a certificate is obtainable for arbitrary states, that the certificate grammar is complete for unbounded proof search, that reality containment holds automatically, or that strict ability novelty persists forever without the separate ability-tower assumptions.*

10.4 Tier 3: Infinite-horizon RCLM reflective successor theorem

Assumption 10.19 (RCLM infinite-path selector). *For every t and every $\rho \in \mathcal{K}_t^{\text{RCLM-SV},\infty}$, a selector $\pi_t^{\text{RCLM-SV}}$ chooses a candidate*

$$\nu_t = \pi_t^{\text{RCLM-SV}}(\rho)$$

among the candidates witnessing Definition 10.15. If measurability is needed for probabilistic conclusions, the selector is assumed measurable with respect to the declared sigma-algebras.

Definition 10.20 (Composed RCLM verifier-schema and goal transports). For $0 \leq s < t$, define

$$\Theta_{s:t}^{\text{RCLM-SV}} = \Theta_{t-1}^{\text{RCLM-SV}} \circ \dots \circ \Theta_s^{\text{RCLM-SV}}, \quad \Theta_{s:t}^{U,\text{RCLM}} = \Theta_{t-1}^{U,\text{RCLM}} \circ \dots \circ \Theta_s^{U,\text{RCLM}},$$

with identity transport when $s = t$. The composed transport is licensed only when every one-step transport is type-valid and its distortion budget is included.

Theorem 10.21 (RCLM domain-relative infinite-horizon robust reflective successor theorem). Assume an infinite RCLM successor-verification seed domain, an infinite-path selector, Batch-11 infinite seed-domain persistence, sound Batch-12B packet construction at every selected step, and summable budgets

$$\sum_{t=0}^{\infty} (\delta_t^{U,\text{RCLM}} + \omega_t^{U,\text{RCLM}}) < \infty, \quad \sum_{t=0}^{\infty} \alpha_t^{\text{RCLM-SV}} < \infty.$$

If

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV},\infty},$$

then the recursively selected trajectory

$$\nu_t = \pi_t^{\text{RCLM-SV}}(\rho_t^{\text{sys}}), \quad \rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}})$$

satisfies

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},\infty} \quad \forall t \geq 0.$$

Moreover, for every finite T ,

$$\Theta_{0:T}^{\text{RCLM-SV}}(\text{VerSch}_0^{\text{RCLM-SV}}) \preceq_{0:T}^{\text{RCLM-SV}} \text{VerSch}_T^{\text{RCLM-SV}},$$

$$D_{\text{sem}}^{U,\text{RCLM}}(U_T^{\text{RCLM}}, \Theta_{0:T}^{U,\text{RCLM}} U_0^{\text{RCLM}}) \leq \sum_{t=0}^{T-1} (\delta_t^{U,\text{RCLM}} + \omega_t^{U,\text{RCLM}}),$$

and

$$\sum_{t=0}^{\infty} \sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) < \infty.$$

If all failure events are defined on a common probability space and the Borel–Cantelli hypotheses hold for the selected path, then only finitely many successor-verification soundness failures occur almost surely under every admissible probability measure for which the path events are jointly defined.

Proof. The domain invariant follows by induction. The base case is the hypothesis. If $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},\infty}$, Definition 10.15 and the selector assumption choose ν_t with successor in $\mathcal{K}_{t+1}^{\text{RCLM-SV},\infty}$. This proves the invariant. The verifier-schema statement is finite composition of one-step schema-persistence clauses. The goal-identity inequality is Theorem 9.20 applied to the finite prefix with the declared distortions. The failure-budget inequality follows by summing one-step soundness bounds. The almost-sure statement is the first Borel–Cantelli implication under the stated common-probability-space and joint-event assumptions. $\square$

Limitation 10.22 (Infinite-horizon RCLM theorem is still domain-relative). Theorem 10.21 proves unbounded recursive verification persistence only along an infinite certified RCLM successor-verification path. It does not prove arbitrary-state entry into that path, nonemptiness of the domain, or perpetual strict ability novelty on a fixed bounded ability scale.

10.5 Tier 4: Reality-contained uncertainty and model misspecification

Definition 10.23 (RCLM reality-containment and misspecification certificate). *Let $P_t^{\text{true,RCLM}}$ denote the actual environment/model law for the audited RCLM successor-verification claim, when such a law is meaningful in the formalization. A reality-containment certificate is a tuple*

$$\text{RealCont}_t^{\text{RCLM}} = (P_t^{\text{true,RCLM}}, \mathcal{P}_t^{\text{RCLM-SV}}, \beta_t^{\text{env,RCLM}}, \varepsilon_t^{\text{env,RCLM}}, d_{\mathcal{P}}^{\text{RCLM}}, \text{RealCert}_t^{\text{RCLM}})$$

satisfying either

$$P(P_t^{\text{true,RCLM}} \in \mathcal{P}_t^{\text{RCLM-SV}}) \geq 1 - \beta_t^{\text{env,RCLM}},$$

or the misspecification residual bound

$$d_{\mathcal{P}}^{\text{RCLM}}(P_t^{\text{true,RCLM}}, \mathcal{P}_t^{\text{RCLM-SV}}) \leq \varepsilon_t^{\text{env,RCLM}}.$$

The distance-to-envelope notation means

$$d_{\mathcal{P}}^{\text{RCLM}}(P_t^{\text{true,RCLM}}, \mathcal{P}_t^{\text{RCLM-SV}}) = \inf_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} d_{\mathcal{P}}^{\text{RCLM}}(P_t^{\text{true,RCLM}}, P).$$

Assumption 10.24 (RCLM robust residual stability under envelope misspecification). *For every RCLM successor-verification residual or expectation bound asserted under $P \in \mathcal{P}_t^{\text{RCLM-SV}}$, there is a declared modulus $\Gamma_t^{\text{env,RCLM}}$ such that replacing an admissible envelope law by a law at distance at most ε changes the relevant bound by at most $\Gamma_t^{\text{env,RCLM}}(\varepsilon)$. The modulus must be certified for the residuals being used.*

Theorem 10.25 (Reality-contained RCLM successor-verification reliability). *Suppose a Batch-12A RCLM successor-verification certificate is valid over $\mathcal{P}_t^{\text{RCLM-SV}}$, and suppose $\text{RealCont}_t^{\text{RCLM}}$ is valid. If $P_t^{\text{true,RCLM}} \in \mathcal{P}_t^{\text{RCLM-SV}}$, then the same successor-verification bounds hold under $P_t^{\text{true,RCLM}}$. If instead*

$$d_{\mathcal{P}}^{\text{RCLM}}(P_t^{\text{true,RCLM}}, \mathcal{P}_t^{\text{RCLM-SV}}) \leq \varepsilon_t^{\text{env,RCLM}},$$

and Assumption 10.24 holds, then each robust expectation or residual bound transfers to the true law with additional error at most $\Gamma_t^{\text{env,RCLM}}(\varepsilon_t^{\text{env,RCLM}})$. In particular,

$$\begin{aligned} \mathbb{E}_{P_t^{\text{true,RCLM}}} \left[\Psi_{t+1}^{\text{RCLM-SV}}(\rho_{t+1}^{\text{sys}}) - \Theta_t^{\Psi, \text{RCLM}} \Psi_t^{\text{RCLM-SV}}(\rho_t^{\text{sys}}) \right] \\ \geq \epsilon_t^{\text{RCLM-SV}} - \eta_t^{\text{RCLM-SV}} - \Gamma_t^{\text{env,RCLM}}(\varepsilon_t^{\text{env,RCLM}}), \end{aligned}$$

whenever the $\Psi^{\text{RCLM-SV}}$ -progress claim is part of the packet.

Proof. If $P_t^{\text{true,RCLM}} \in \mathcal{P}_t^{\text{RCLM-SV}}$, the robust quantifier over $\mathcal{P}_t^{\text{RCLM-SV}}$ includes the true law. In the misspecified case, the distance-to-envelope certificate and stability modulus supply an admissible envelope law within distance $\varepsilon_t^{\text{env,RCLM}}$ and bound the residual or expectation change by $\Gamma_t^{\text{env,RCLM}}(\varepsilon_t^{\text{env,RCLM}})$. Subtracting that error from the robust lower bound gives the displayed inequality. The same argument applies componentwise to all residuals for which a stability modulus is supplied. $\square$

Limitation 10.26 (RCLM reality containment is a certificate condition). *The theorem does not prove that the declared RCLM uncertainty envelope contains reality. It states what follows if containment or bounded misspecification is certified. Without $\text{RealCont}_t^{\text{RCLM}}$, reliability remains envelope-relative.*

10.6 Tier 5: RCLM tractability discipline

Definition 10.27 (Restricted RCLM SV tractability certificate). *A restricted RCLM successor-verification tractability certificate is a tuple*

$$\text{SVTract}_t^{\text{RCLM}} = (n_t^{\text{RCLM}}, k_t, \ell_t, s_t, \delta_t^{\text{tol}}, C_t^{\text{SVBuild, RCLM}}, C_t^{\text{SVGate, RCLM}}, C_t^{\text{SVProof, RCLM}}, \text{TractCert}_t^{\text{RCLM}}).$$

where n_t^{RCLM} is the architecture representation size, k_t is the primitive grammar depth, ℓ_t is the proof/checker trace bound, s_t is the audit/sample size, and δ_t^{tol} is the residual tolerance. The certificate is valid when it proves explicit finite or asymptotic bounds

$$\begin{aligned} \text{Cost}(\text{SVBuilder}_t^{\text{RCLM}}) &\leq C_t^{\text{SVBuild, RCLM}}, & \text{Cost}(\mathcal{A}_t^{\text{RCLM-SV-gate}}) &\leq C_t^{\text{SVGate, RCLM}}, \\ \text{Cost}(\text{SVProof}_t^{\text{RCLM}}) &\leq C_t^{\text{SVProof, RCLM}}, \end{aligned}$$

and, for a declared tractable subclass,

$$C_t^{\text{SVBuild, RCLM}} + C_t^{\text{SVGate, RCLM}} + C_t^{\text{SVProof, RCLM}} \leq \text{poly}(n_t^{\text{RCLM}}, k_t, \ell_t, s_t, \log(1/\delta_t^{\text{tol}})).$$

If only finite budgets are certified, the correct claim is resource-accounted or budgeted, not computationally tractable in the strong asymptotic sense.

Theorem 10.28 (Restricted RCLM SV packet tractability). *If $\text{SVTract}_t^{\text{RCLM}}$ is valid for the declared architecture representation class and the active Batch-12B packet-construction input lies inside that class, then the costs of $\text{SVBuilder}_t^{\text{RCLM}}$, the algorithmic RCLM SV gate, and the proof/checking trace are bounded by the displayed finite or polynomial bounds. Therefore the Batch-12B architecture step is computationally tractable only relative to that declared representation class and cost model. If only finite budget inequalities are supplied, the theorem licenses only resource accounting.*

Proof. The conclusion is exactly the content of the valid $\text{SVTract}_t^{\text{RCLM}}$ certificate applied to an input inside its declared representation class. The theorem makes no claim about inputs outside the class, unbounded proof search, or hidden costs absent from the certificate. $\square$

Limitation 10.29 (RCLM complexity ledger versus tractability theorem). *The architecture complexity ledger is an accounting object. Strong computational tractability requires a certificate such as Definition 10.27. If that certificate is absent, the paper should use the weaker language resource-accounted or budgeted.*

10.7 Tier 6: Claim-discipline renaming

Definition 10.30 (RCLM domain-relative robust reflective successor theorem status). *The architecture theorem stack through Batch 12B may be called a Domain-Relative RCLM Robust Reflective Successor Theorem for the declared seed-library class only when the following are supplied:*

- (i) Batch-11 RCLM seed-domain entry and persistence;
- (ii) Batch-12/12A RCLM successor-verification domain invariance with explicit residuals and soundness composition;
- (iii) Batch-12B RCLM packet construction;

- (iv) *Batch-12B RCLM successor-verification seed-domain nonemptiness for a declared witness library;*
- (v) *infinite-horizon RCLM SV seed-domain persistence or the Infinite-Horizon RCLM SV Seed-Library Nonemptiness Theorem when an unbounded claim is made;*
- (vi) *reality-containment or misspecification certificates when real-world reliability is claimed;*
- (vii) *restricted tractability certificates when computational tractability is claimed.*

Without these qualifiers, the unqualified phrase “Robust Reflective Successor Theorem” overstates what the architecture paper proves.

Proposition 10.31 (Claim discipline for RCLM Batch 12B). *A statement that cites the RCLM Batch-12B theorem stack as proving reliable successor verification under uncertainty must specify whether the claim is domain-relative, certificate-relative, finite-horizon, infinite-horizon, envelope-relative, reality-contained, budgeted, or tractability-certified. Omitting these qualifiers is an invalid use of the theorem stack.*

Proof. Each qualifier corresponds to a separate premise in Definitions 10.3, 10.8, 10.15, 10.23, and 10.27. The conclusions hold only under those premises. Dropping the qualifiers changes the logical statement. $\square$

10.8 Tier 7: Architecture synchronization completed

Remark 10.32 (Batch-12B architecture synchronization). *This section instantiates the abstract Batch-12B objects in RCLM notation:*

$$\text{SVBuilder}_t^{\text{RCLM}}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib},N}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib},\infty}, \quad \text{RealCont}_t^{\text{RCLM}}, \quad \text{SVTract}_t^{\text{RCLM}}.$$

It is still an architecture theorem layer, not an empirical validation. A trained system must supply the library, builder inputs, certificates, proof traces, reality-containment evidence, and tractability certificates before the results can be invoked.

10.9 RCLM Batch-12B theorem-type table rows

RCLM Batch-12B object or result	Type	Claim discipline
RCLM SV packet builder	Def / Cert	Constructs packets only on $\mathcal{D}_t^{\text{RCLM-SV-construct}}$; does not prove arbitrary-state packet construction.
RCLM SV packet construction theorem	Cert / Exist	Converts builder-domain membership into a valid $\text{SVPkg}_t^{\text{RCLM}}$; depends on builder soundness.
RCLM SV witness library and bounded certificate grammar	Def / Exist	Gives a finite or enumerable class for packet construction; does not cover unbounded search.
RCLM SV seed-domain nonemptiness theorem	Exist / Cert	Proves $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N} \subseteq \mathcal{K}_t^{\text{RCLM-SV},N}$; does not prove arbitrary systems lie in the seed-library domain.
Infinite RCLM SV seed-library closure theorem	Exist / Tel	Proves certified infinite seed-library paths generate an infinite SV domain; does not prove arbitrary infinite viability.
Infinite-horizon RCLM domain-relative RRST	Tel / Exist	Iterates an already nonempty infinite SV domain or generated seed-library domain; does not prove arbitrary infinite viability.
Reality-containment theorem	Cert	Transfers envelope-relative guarantees to reality only with containment or mis-specification certificates.
Restricted RCLM SV tractability theorem	Cert / Impl	Licenses tractability only for declared representation and cost classes; otherwise only budget accounting.
Claim-discipline renaming	Impl	Prevents unqualified robust-reflection claims unless all required premises are present.

10.10 Combined RCLM Batch-12B theorem statement

Theorem 10.33 (Domain-relative RCLM Robust Reflective Successor Theorem). *Assume the following for a declared RCLM class and a declared time horizon, finite or infinite:*

- (i) *the Batch-11 RCLM seed-domain entry and persistence hypotheses hold;*
- (ii) *$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}$ for a finite horizon, or $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},\infty}$ with the generated-domain inclusion of Theorem 10.16 for an infinite horizon;*
- (iii) *$\text{SVBuilder}_t^{\text{RCLM}}$ is sound on every selected packet-construction domain;*
- (iv) *explicit Batch-12A residuals satisfy $Q_t^{\text{RCLM-SV},A}(\nu_t; \rho_t^{\text{sys}}) \leq 0$ at each selected step;*
- (v) *verifier-schema transport, uncertainty-envelope construction/transport, goal-identity transport, anti-circular trust residuals, proof/checking budgets, and successor-persistence certificates are valid at each step;*

- (vi) if real-world reliability is claimed, $\text{RealCont}_t^{\text{RCLM}}$ or a bounded misspecification certificate is valid at each step;
- (vii) if computational tractability is claimed, $\text{SVTract}_t^{\text{RCLM}}$ is valid for the declared architecture representation class;
- (viii) in the infinite case, the selected infinite path satisfies the summability hypotheses of Theorem 10.21.

Then, for the finite case, the direct engine constructs a trajectory

$$\rho_0^{\text{sys}} \xrightarrow{\nu_0} \rho_1^{\text{sys}} \xrightarrow{\nu_1} \dots \xrightarrow{\nu_{N-1}} \rho_N^{\text{sys}}$$

with

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad (0 \leq t \leq N),$$

and each step preserves the declared successor-verification schema, goal-identity bound, soundness budget, anti-circular trust conditions, and Batch-11 non-lossy direct-engine construction certificates. In the infinite case,

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},\infty} \quad \forall t \geq 0,$$

with composed verifier-schema persistence and cumulative goal drift bounded as in Theorem 10.21. If the RCLM Batch-11 ability-tower and certified-novelty assumptions hold along the same path, the certified ability sets expand monotonically along that path as well.

Proof. In the finite case, $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}$ and Theorem 10.9 give $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV},N}$. Theorem 9.8 then gives $\rho_1^{\text{sys}} \in \mathcal{K}_1^{\text{RCLM-SV},N}$ and the stepwise certificates. Repeating the same argument across the finite horizon gives the displayed trajectory. Packet construction at each step is supplied by Theorem 10.4; reality-contained and tractability conclusions are added only when Theorems 10.25 and 10.28 apply. In the infinite case, Theorem 10.16 first converts the infinite seed-library path into the generated infinite RCLM successor-verification domain; applying Theorem 10.21 to the generated or declared-containing domain gives the infinite trajectory. Ability monotonicity follows from Corollary 8.18 under its additional hypotheses. $\square$

Limitation 10.34 (What RCLM Batch 12B still does not prove). *Batch 12B still does not prove universal Löbian reflection, arbitrary Vingean successor trust, semantic completeness of the declared goal object, automatic reality containment, arbitrary-state entry into the SV seed-library domain, or unbounded practical tractability. It proves a stronger and more explicit domain-relative architecture result: for a declared RCLM class with packet construction, seed-library nonemptiness, reality-containment or misspecification certificates, and tractability certificates when claimed, robust successor verification can be constructed and iterated along finite or infinite certified paths.*

11 Batch-13R RCLM Mechanized Reference-Entry, Proof-Carrying Successor Packets, and Finite Certified Trajectories

Batch 12B and the finalization section state the domain-relative architecture theorem, but they still leave the finite reference instance as a target object. This section is the architecture-side reference-entry and realization layer corresponding to the Batch-13R RCP math paper. It is not a new abstract successor theorem. It constructs the missing architecture-side bridge from

“a valid finite RCLM reference instance would invoke the theorem”

to

“a finite proof-carrying RCLM reference instance can be built, checked, and replayed.”

The section therefore focuses on the path toward restricted Scenario 3: proof-carrying reference entry, a complete finite RCLM successor-verification reference instance, a multi-step certified trajectory, and an artifact theorem.

**Batch 13R is a reference-entry / realization batch, not a new abstract
Batch 13.**

11.1 Tier 1: RCLM reference class and proof-carrying successor packets

Definition 11.1 (Finite RCLM reference class). *Fix positive integers n, k, N, ℓ, s . A finite RCLM reference class*

$$\text{RCLMRefClass}_{n,k,N}$$

is a class of finite typed RCLM systems $\mathcal{M}$ with the following data:

- (i) *finite typed system spaces $\mathcal{H}_t^{\text{sys}}$ for $0 \leq t \leq N$ with $\dim \mathcal{H}_t^{\text{sys}} \leq n$;*
- (ii) *a finite non-lossy primitive-word grammar of depth at most k ;*
- (iii) *a finite RCLM successor-verification witness library $\mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho_t^{\text{sys}})$ at every step used by the reference trajectory;*
- (iv) *a finite certificate grammar with proof/checker traces of length at most ℓ ;*
- (v) *a finite audit, calibration, stress-test, and validation data object $\mathcal{D}$ with sample-size parameter s ;*
- (vi) *a finite checker object $\mathcal{C}$ intended to implement $\text{Check}_{\text{RCP}}^{\text{RCLM}}$;*
- (vii) *typed encoders, semantic/goal objects, uncertainty envelopes, trust anchors, resource ledgers, and projection-realization conventions sufficient to state the Batch-11, Batch-12A, Batch-12B, and Batch-13R certificates.*

Membership in $\text{RCLMRefClass}_{n,k,N}$ is only a finite reference-class condition. It does not imply that trained RCLM systems, frontier-scale neural systems, or arbitrary agents satisfy the same conditions.

Definition 11.2 (RCLM proof-carrying successor packet). *For a finite RCLM reference system $\mathcal{M}$, state ρ_t^{sys} , and candidate ν_t , an RCLM proof-carrying successor packet is*

$$\begin{aligned} \text{PCS}_t^{\text{RCLM}}(\nu_t) = (\nu_t, \Phi_{\nu_t}, \rho_t^{\text{sys}}, \rho_{t+1}^{\text{sys}}, \\ \text{Proof}_t^{\text{NL}}, \text{Proof}_t^{\text{Rec}}, \\ \text{Proof}_t^{\text{Seed}}, \text{Proof}_t^{\text{SV}}, \\ \text{Proof}_t^{\text{GoalId}}, \text{Proof}_t^{\text{Trust}}, \\ \text{Proof}_t^{\text{RealCont}}, \text{Proof}_t^{\text{Tract}}, \\ \text{Trace}_t^{\text{RCLM}}). \end{aligned}$$

The components mean:

- (i) $\text{Proof}_t^{\text{NL}}$ proves algebraic non-loss or directly certified non-loss for the accepted typed transition;
- (ii) $\text{Proof}_t^{\text{Rec}}$ proves a recovery or rollback certificate;
- (iii) $\text{Proof}_t^{\text{Seed}}$ proves Batch-11 RCLM seed-domain and witness-library obligations;
- (iv) $\text{Proof}_t^{\text{SV}}$ proves the explicit Batch-12A residual vector $Q_t^{\text{RCLM-SV},A} \leq 0$;
- (v) $\text{Proof}_t^{\text{GoalId}}$ proves the declared goal-identity drift bound;
- (vi) $\text{Proof}_t^{\text{Trust}}$ proves the anti-circular trust residuals and no self-sole-certification condition;
- (vii) $\text{Proof}_t^{\text{RealCont}}$ proves reality containment or bounded misspecification when that claim is made;
- (viii) $\text{Proof}_t^{\text{Tract}}$ proves the finite or restricted polynomial tractability certificate when that claim is made;
- (ix) $\text{Trace}_t^{\text{RCLM}}$ records hashes, seeds, candidate-generation traces, checker versions, certificate grammar words, and resource costs.

A packet is proof-carrying only relative to a stated checker contract. The existence of a packet name does not itself prove validity.

Definition 11.3 (RCLM reference checker). An RCLM reference checker is a finite deterministic procedure

$$\text{Check}_{\text{RCP}}^{\text{RCLM}} : \text{PCS}_t^{\text{RCLM}}(\nu_t) \longrightarrow \{0, 1\}.$$

It returns 1 only if all syntactic types match, every referenced certificate object is present, every explicit residual component is nonpositive, every transport map used by a comparison is typed, every resource/cost entry is within budget, every trust-support condition passes, and the successor state in the packet equals the realized successor produced by the typed transition up to the declared projection-realization distortion.

Assumption 11.4 (RCLM checker soundness contract). The checker $\text{Check}_{\text{RCP}}^{\text{RCLM}}$ is sound for the finite reference class if, for every packet in the declared grammar,

$$\text{Check}_{\text{RCP}}^{\text{RCLM}}(\text{PCS}_t^{\text{RCLM}}(\nu_t)) = 1$$

implies the true finite-reference obligations represented by that packet hold: typed transition validity, non-loss, recovery, Batch-11 seed-domain validity, Batch-12A residual validity, goal-identity drift, anti-circular trust support, proof/checking/resource budget validity, and the declared reality-containment and tractability clauses when present.

Theorem 11.5 (RCLM checker soundness). Assume the RCLM checker soundness contract. If

$$\text{Check}_{\text{RCP}}^{\text{RCLM}}(\text{PCS}_t^{\text{RCLM}}(\nu_t)) = 1,$$

then the accepted candidate ν_t satisfies the finite-reference RCLM successor-verification admissibility obligations encoded by $\text{PCS}_t^{\text{RCLM}}(\nu_t)$. In particular,

$$Q_t^{\text{RCLM-SV},A}(\nu_t; \rho_t^{\text{sys}}) \leq 0$$

and $\text{SVPkg}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$ is valid for the explicit Batch-12A interpretation.

Proof. The conclusion is exactly the semantic content of Assumption 11.4. The checker returning 1 licenses each component certificate in the packet, including explicit residual nonpositivity and packet validity. No claim is made for packets outside the declared grammar or for an unchecked implementation of the checker. $\square$

Limitation 11.6 (Mechanization scope of the architecture). *The checker formalism makes the architecture mechanization-ready. In the current companion artifact package, a shared Lean 4 project supplies the narrow external machine-checking certificate for the canonical finite RCP/RCLM witness, the finite checker core, and the RCLM-to-RCP refinement module. Therefore this paper may claim Lean-checked status for that mapped canonical finite core. The claim does not extend to the full 219-page math stack, the full 187-page architecture stack, arbitrary trained-system entry, broad learned-agent entry, frontier-scale empirical validation, or deployment behavior. Any stronger machine-checking claim would require a separate MechCert covering the additional formal statements and their trust base.*

11.2 Tier 2: RCLM reference builder and reference-entry theorem

Definition 11.7 (RCLM reference builder). *The RCLM reference builder is a partial construction procedure*

$$\text{BuildRefSV}_{0:N}^{\text{RCLM}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{RefSV}_{0:N}^{\text{RCLM}},$$

where $\mathcal{M} \in \text{RCLMRefClass}_{n,k,N}$, $\mathcal{D}$ is the finite audit/calibration/stress-test object, $\mathcal{L}$ is the finite RCLM SV witness library, and $\mathcal{C}$ is the checker. A successful build returns

$$\text{RefSV}_{0:N}^{\text{RCLM}} = (\rho_0^{\text{sys}}, (\nu_t)_{t < N}, (\rho_t^{\text{sys}})_{t \leq N}, \text{CertTrace}_{0:N}^{\text{RCLM}}),$$

where

$$\begin{aligned} \text{CertTrace}_{0:N}^{\text{RCLM}} = & ((\text{PCS}_t^{\text{RCLM}})_{t < N}, (\text{SVPkg}_t^{\text{RCLM}})_{t < N}, \\ & (Q_t^{\text{RCLM-SV,A}})_{t < N}, (\text{GoalId}_t^{\text{RCLM}})_{t < N}, \\ & (\text{TrustRef}_t^{\text{RCLM}})_{t < N}, (\text{SVPersist}_t^{\text{RCLM}})_{t < N}, \\ & \text{RealCont}_{0:N}^{\text{RCLM}}, \text{SVTract}_{0:N}^{\text{RCLM}}, \\ & \text{BuildTrace}_{0:N}^{\text{RCLM}}). \end{aligned}$$

The builder succeeds only when every step is covered by the witness library, every packet is accepted by $\text{Check}_{\text{RCP}}^{\text{RCLM}}$, and the successor trace satisfies $\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}})$ for every $t < N$.

Definition 11.8 (Valid RCLM finite reference-entry object). *A returned object $\text{RefSV}_{0:N}^{\text{RCLM}}$ is a valid finite RCLM reference-entry object when:*

- (i) $\mathcal{M} \in \text{RCLMRefClass}_{n,k,N}$ and all spaces, candidates, proof words, and checkers are finite and typed;
- (ii) each ν_t is generated by the finite witness library and bounded certificate grammar;
- (iii) each $\text{PCS}_t^{\text{RCLM}}(\nu_t)$ is accepted by $\text{Check}_{\text{RCP}}^{\text{RCLM}}$;
- (iv) the builder trace records the Batch-11 seed package, Batch-12A explicit residuals, Batch-12B packet-construction trace, successor-persistence trace, goal-identity trace, trust trace, and budget trace at every step;
- (v) every step is strict-progress certified when the reference instance claims nontrivial RSI rather than no-op preservation;
- (vi) any real-world or simulated-world claim is accompanied by a valid $\text{RealCont}_{0:N}^{\text{RCLM}}$;
- (vii) any strong computational-tractability claim is accompanied by a valid $\text{SVTract}_{0:N}^{\text{RCLM}}$.

Theorem 11.9 (RCLM Batch-13R reference-entry theorem). *Assume $\text{BuildRefSV}_{0:N}^{\text{RCLM}}$ succeeds and returns a valid finite RCLM reference-entry object:*

$$\boxed{\text{BuildRefSV}_{0:N}^{\text{RCLM}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{RefSV}_{0:N}^{\text{RCLM}} \implies \rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, N}.$$

Consequently, the surfaced architecture theorem gives

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N} \quad 0 \leq t \leq N.$$

Proof. Validity of $\text{RefSV}_{0:N}^{\text{RCLM}}$ supplies exactly the finite RCLM SV seed-library domain data: a Batch-11 seed package, a positive-margin SV witness library, bounded grammar coverage, construction-domain membership, valid proof-carrying successor packets, explicit residual non-positivity, successor-persistence certificates, goal-identity certificates, trust certificates, and the required budget traces. Therefore $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, N}$ by Definition 10.8. The finite trajectory conclusion is then Theorem 38.3. No arbitrary trained-system entry is inferred. $\square$

11.3 Tier 3: proof-carrying finite certified trajectories

Definition 11.10 (RCLM prefix-check predicate). *For a reference trace returned by $\text{BuildRefSV}_{0:N}^{\text{RCLM}}$, define*

$$\text{PrefixCheck}_t^{\text{RCLM}}$$

to mean that the builder succeeded, $\text{RefSV}_{0:N}^{\text{RCLM}}$ is valid, and

$$\text{Check}_{\text{RCP}}^{\text{RCLM}}(\text{PCS}_s^{\text{RCLM}}(\nu_s)) = 1 \quad 0 \leq s < t.$$

For $t = 0$, the empty prefix condition is satisfied exactly when the reference-entry object is valid and $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, N}$.

Theorem 11.11 (RCLM proof-carrying finite trajectory theorem). *Assume the hypotheses of Theorem 11.9. For every $0 \leq t \leq N$, if $\text{PrefixCheck}_t^{\text{RCLM}}$ holds, then*

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N}.$$

In particular, for every checked step $t < N$,

$$\text{Check}_{\text{RCP}}^{\text{RCLM}}(\text{PCS}_t^{\text{RCLM}}(\nu_t)) = 1$$

licenses the successor step

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{K}_{t+1}^{\text{RCLM-SV}, N}$$

provided the already-checked prefix places ρ_t^{sys} in $\mathcal{K}_t^{\text{RCLM-SV}, N}$.

Proof. The proof is induction on t . The base case follows from Theorem 11.9 and Theorem 38.3. For the induction step, assume $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N}$ and the step checker accepts $\text{PCS}_t^{\text{RCLM}}(\nu_t)$. By Theorem 11.5, the true successor-verification packet obligations hold, including explicit residual nonpositivity and successor-persistence. The Batch-12 successor-verification domain invariance theorem therefore places ρ_{t+1}^{sys} in $\mathcal{K}_{t+1}^{\text{RCLM-SV}, N}$. The isolated statement $\text{Check}_{\text{RCP}}^{\text{RCLM}}(\text{PCS}_t) = 1$ is not used alone; it is used together with the checked prefix and builder trace, which avoids a circular or context-free inference. $\square$

Interpretation 11.12 (Licensed shorthand for checked reference paths). *Inside a valid built reference path, the paper may use the shorthand*

$$\forall t < N, \quad \text{Check}_{\text{RCP}}^{\text{RCLM}}(\text{PCS}_t^{\text{RCLM}}(\nu_t)) = 1 \implies \rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N},$$

provided it is understood as shorthand for Theorem 11.11: the implication is relative to the constructed reference path and its already-checked prefix. A single checker call detached from the path does not prove domain membership.

11.4 Concrete RCLM canonical finite reference witness

Definition 11.13 (Canonical finite RCLM reference system). *For any finite horizon $N \geq 2$, define the canonical finite RCLM reference system by*

$$\mathcal{H}_t^{\text{sys,can}} = \mathbb{C}^2 \otimes (\mathbb{C}^2)^{\otimes t}, \quad 0 \leq t \leq N.$$

The first qubit is the protected predecessor register and the appended qubits are certified append-only module registers. Let

$$\rho_0^{\text{sys,can}} = \text{diag}(2/3, 1/3), \quad \sigma_0^{\text{sys,can}} = \text{diag}(1/3, 2/3),$$

and define the protected pair set at time t by

$$\mathcal{P}_t^{\text{RCLM,can}} = \{(\rho_t^{\text{sys,can}}, \sigma_t^{\text{sys,can}})\}.$$

At each step choose a fixed appended module state

$$\alpha_t = |0\rangle\langle 0|,$$

and define

$$\Phi_t^{\text{can}}(X) = X \otimes \alpha_t, \quad \nu_t^{\text{can}} = \text{AppendMem}(a_{t+1}).$$

The successor states are

$$\rho_{t+1}^{\text{sys,can}} = \rho_t^{\text{sys,can}} \otimes \alpha_t, \quad \sigma_{t+1}^{\text{sys,can}} = \sigma_t^{\text{sys,can}} \otimes \alpha_t.$$

The recovery map is partial trace over the newly appended module:

$$\mathcal{R}_t^{\text{can}}(Y) = \text{Tr}_{t+1}(Y).$$

The certified ability set is

$$\text{Ab}_t^{\text{RCLM,can}} = \{a_0, a_1, \dots, a_t\},$$

with ability transport the obvious inclusion and with a_{t+1} certified as new at step t .

Definition 11.14 (Canonical RCLM successor-verification certificates). *For the canonical reference system, define:*

- (i) $\mathcal{L}_t^{\text{RCLM-SV-Wit,can}}$ to contain the single witness ν_t^{can} ;
- (ii) the bounded grammar to enumerate exactly the append-only word $\text{AppendMem}(a_{t+1})$;
- (iii) $\text{SVBuilder}_t^{\text{RCLM,can}}$ to return the finite proof-carrying packet for ν_t^{can} ;
- (iv) $\text{VerSch}_t^{\text{RCLM-SV,can}}$ to use a fixed finite checker language transported by identity plus one appended ability label;
- (v) $\mathcal{P}_t^{\text{RCLM-SV,can}} = \{P_t^{\text{true,can}}\}$, the singleton simulated-world law;
- (vi) $U_t^{\text{RCLM,can}} = U_0^{\text{RCLM,can}}$, with identity goal transport and $\delta_t^{U, \text{RCLM,can}} = 0$;
- (vii) $\text{TrustRef}_t^{\text{RCLM-SV,can}}$ to be supported by an immutable predecessor checker anchor $\mathcal{C}_0$, so no self-sole certification occurs;
- (viii) $\text{SVPersist}_t^{\text{RCLM,can}}$ to be the certificate that the next tuple in the canonical trajectory is again of the same canonical form.

All residual margins are chosen strictly positive before checking, and all exact equalities above are charged zero error.

Lemma 11.15 (Canonical RCLM non-loss, recovery, and ability expansion). *For the canonical reference system, every step $t < N$ satisfies:*

$$D(\rho_{t+1}^{\text{sys,can}} \parallel \sigma_{t+1}^{\text{sys,can}}) = D(\rho_t^{\text{sys,can}} \parallel \sigma_t^{\text{sys,can}}),$$

$$\mathcal{R}_t^{\text{can}}(\rho_{t+1}^{\text{sys,can}}) = \rho_t^{\text{sys,can}}, \quad \mathcal{R}_t^{\text{can}}(\sigma_{t+1}^{\text{sys,can}}) = \sigma_t^{\text{sys,can}},$$

and

$$\Theta_t^{\mathfrak{A}, \text{RCLM,can}}(\text{Ab}_t^{\text{RCLM,can}}) \subsetneq \text{Ab}_{t+1}^{\text{RCLM,can}}.$$

Proof. Relative entropy is additive under tensor products and $D(\alpha_t \parallel \alpha_t) = 0$, so tensoring both protected states with the same appended state preserves the protected relative entropy exactly. The recovery equalities are the defining property of partial trace over the appended register. The ability-set relation follows from

$$\text{Ab}_{t+1}^{\text{RCLM,can}} = \text{Ab}_t^{\text{RCLM,can}} \cup \{a_{t+1}\}$$

with $a_{t+1} \notin \text{Ab}_t^{\text{RCLM,can}}$. □

Definition 11.16 (Canonical RCLM checker). *The canonical RCLM checker $\text{Check}_{\text{RCP}}^{\text{RCLM,can}}$ verifies the following finite list for each $\text{PCS}_t^{\text{RCLM,can}}$:*

- (i) the state dimensions match $\mathcal{H}_t^{\text{sys,can}}$ and $\mathcal{H}_{t+1}^{\text{sys,can}}$;
- (ii) the update equals append-only tensoring with α_t ;
- (iii) the protected pair is transported by tensoring both entries with α_t ;
- (iv) the recovery map is partial trace and exactly recovers both protected states;
- (v) the ability set expands by exactly one new label;
- (vi) the verifier schema and goal object are transported by identity with zero drift;
- (vii) the uncertainty envelope is the singleton simulated law;
- (viii) the trust anchor is predecessor-stable and not modified by ν_t^{can} ;
- (ix) all costs are within the declared finite ledger.

It returns 1 if and only if every listed check succeeds.

Theorem 11.17 (Canonical RCLM checker soundness). *For every canonical packet,*

$$\text{Check}_{\text{RCP}}^{\text{RCLM,can}}(\text{PCS}_t^{\text{RCLM,can}}(\nu_t^{\text{can}})) = 1$$

implies the true canonical non-loss, recovery, ability-expansion, goal-identity, trust, uncertainty, and successor-persistence obligations hold.

Proof. The canonical checker verifies each obligation by direct finite calculation. Non-loss, recovery, and ability expansion follow from Lemma 11.15. Goal identity and verifier-schema persistence are identity transports by Definition 11.14. The uncertainty envelope contains exactly the simulated law by definition, and the trust anchor is immutable by construction. Therefore all true obligations represented by the canonical packet hold. □

Theorem 11.18 (Concrete finite RCLM reference-entry theorem). *For every $N \geq 2$, the canonical construction yields a valid finite RCLM reference-entry object:*

$$\boxed{\text{BuildCanRefSV}_{0:N}^{\text{RCLM}} \Downarrow \text{RefSV}_{0:N}^{\text{RCLM,can}}}.$$

Consequently,

$$\rho_0^{\text{sys,can}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N},$$

and the canonical trajectory

$$\rho_0^{\text{sys,can}} \xrightarrow{\nu_0^{\text{can}}} \rho_1^{\text{sys,can}} \xrightarrow{\nu_1^{\text{can}}} \dots \xrightarrow{\nu_{N-1}^{\text{can}}} \rho_N^{\text{sys,can}}$$

satisfies

$$\rho_t^{\text{sys,can}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad 0 \leq t \leq N.$$

Proof. Definitions 11.13 and 11.14 specify finite typed spaces, a finite witness library, a bounded grammar, an explicit builder, identity verifier-schema transport, singleton uncertainty envelope, zero goal drift, immutable trust anchor, finite costs, and successor persistence. Lemma 11.15 proves the non-loss, recovery, and strict ability-expansion clauses. Theorem 11.17 proves that the canonical checker accepts only true canonical obligations. Hence the returned object satisfies Definition 11.8. The reference-entry theorem, Theorem 11.9, then gives seed-library membership and the finite trajectory conclusion. $\square$

11.5 Rank 8: executable RCLM artifact theorem

Definition 11.19 (RCLM reference artifact). *An RCLM reference artifact over horizon N is a finite replayable object*

$$\text{Artifact}_{0:N}^{\text{RCLM}} = (\text{States}_{0:N}, \text{Updates}_{0:N-1}, \text{Packets}_{0:N-1}, \text{Checker}, \text{RunLog}, \text{Hashes}).$$

It is artifact-certified, written $\text{ArtCert}_{0:N}^{\text{RCLM}} = 1$, when:

- (i) *the artifact hashes match the states, updates, packets, checker, and run log;*
- (ii) *replaying the checker on every packet returns 1;*
- (iii) *the replayed successor states equal the artifact successor states;*
- (iv) *the run log records the version of the checker, the finite horizon, the final dimension, and every residual value used by the reference construction;*
- (v) *the artifact explicitly states whether it is paper-level replay, externally machine-checked proof, or empirical trained-system validation.*

Theorem 11.20 (RCLM reference artifact theorem). *If $\text{Artifact}_{0:N}^{\text{RCLM}}$ is the canonical RCLM reference artifact and $\text{ArtCert}_{0:N}^{\text{RCLM}} = 1$, then*

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}, \quad \rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad 0 \leq t \leq N.$$

Proof. Artifact certification states that every canonical proof-carrying successor packet is replayed by the checker and accepted, that every successor state in the run log is the result of the recorded typed update, and that hashes match the checked objects. The canonical checker soundness theorem then supplies the true canonical obligations, and the concrete finite RCLM reference-entry theorem supplies seed-library entry and the finite SV trajectory. The theorem proves reproducible finite reference-entry for the canonical artifact; it does not prove trained-system validation. $\square$

11.6 Restricted RCLM reference tractability and reality containment

Definition 11.21 (Canonical RCLM reference cost model). *Let*

$$n_t^{\text{can}} = \dim \mathcal{H}_t^{\text{sys,can}} = 2^{t+1}.$$

Let $C_{Q,t}^{\text{can}} = \text{Cost}(Q_t^{\text{RCLM-SV,A,can}})$. The dense-matrix cost model charges $O((n_t^{\text{can}})^3)$ for generic dense-state entropy, trace, and equality checks. The diagonal-table cost model charges $O(n_t^{\text{can}})$ because all canonical states and updates are diagonal/classical and append-only.

Theorem 11.22 (Concrete canonical RCLM reference tractability). *For the canonical RCLM reference construction over horizon N , there is a constant c_{can} such that the dense-matrix accounting satisfies*

$$\begin{aligned} & \text{Cost}(\text{BuildCanRefSV}_{0:N}^{\text{RCLM}}) + \sum_{t < N} \text{Cost}(\text{Check}_{\text{RCP}}^{\text{RCLM,can}}) + \sum_{t < N} C_{Q,t}^{\text{can}} \\ & \leq c_{\text{can}} N (n_N^{\text{can}})^3, \end{aligned}$$

Under the diagonal-table representation, the same construction is checked in

$$O(N n_N^{\text{can}})$$

time. Thus the canonical finite reference instance is tractability-certified for its declared finite representation class.

Proof. In the dense representation, each finite step requires only a bounded number of matrix operations on spaces of dimension at most n_N^{can} , each charged by the dense model at cubic cost. The canonical explicit SV residual vector has a fixed finite number of components, and each component is checked using the same finite state, update, recovery, trust, goal, uncertainty, and budget calculations. Thus adding $\sum_{t < N} \text{Cost}(Q_t^{\text{RCLM-SV,A,can}})$ changes only the constant. Summing over N steps gives the first bound. In the diagonal-table representation, every state is a probability table and the update is append-only tensoring with a fixed one-point distribution; checking equality, partial trace, protected relative entropy, and ability-set inclusion is linear in the table size at each step and bounded by $O(n_N^{\text{can}})$ per step, giving $O(N n_N^{\text{can}})$. $\square$

Theorem 11.23 (Concrete canonical RCLM reality containment). *For the canonical simulated RCLM reference environment, let*

$$\mathcal{P}_t^{\text{RCLM-SV,can}} = \{P_t^{\text{true,can}}\}.$$

Then

$$P(P_t^{\text{true,can}} \in \mathcal{P}_t^{\text{RCLM-SV,can}}) = 1, \quad \beta_t^{\text{env,RCLM,can}} = 0, \quad \varepsilon_t^{\text{env,RCLM,can}} = 0.$$

Therefore the canonical reference reliability claim is reality-contained for the declared simulated environment.

Proof. The uncertainty envelope is defined as the singleton containing the simulated true law. Therefore the true law belongs to the envelope with probability one and the distance from the true law to the envelope is zero. $\square$

11.7 Batch-13R-A architecture synchronization: explicit canonical residuals, checker realization, and artifact parsing

The preceding Batch-13R section gives the RCLM reference-entry interface and the canonical appended-module witness. The final synchronization requirement is to make explicit that the canonical checker realizes the same explicit successor-verification residual vector used by Batch 12A, that a replayable artifact can be parsed into the checked packet sequence, and that the canonical RCLM witness refines the architecture-general RCP canonical witness.

Definition 11.24 (Canonical explicit RCLM successor-verification residual vector). *For the canonical RCLM reference system, define*

$$Q_t^{\text{RCLM-SV},A,\text{can}} = (q_t^{\text{seed,can}}, q_t^{\text{cand,can}}, q_t^{\text{ver,can}}, q_t^{\text{trans,can}}, \\ q_t^{\text{goal,can}}, q_t^{\text{unc,can}}, q_t^{\text{trust,can}}, q_t^{\text{budget,can}}, \\ q_t^{\text{persist,can}}, q_t^{\text{world,can}}, q_t^{\text{proof,can}}, q_t^{\text{sound,can}}).$$

Choose positive margins $m_{t,\ell}^{\text{can}} > 0$ for every component ℓ . For each component define

$$q_t^{\ell,\text{can}} = \Delta_t^{\ell,\text{can}} - m_{t,\ell}^{\text{can}},$$

where $\Delta_t^{\ell,\text{can}}$ is the certified defect for the component. In the canonical appended-module construction, exact tensoring, exact partial-trace recovery, identity goal/verifier transports, singleton uncertainty, immutable trust anchor, finite proof budget, and successor persistence give

$$\Delta_t^{\ell,\text{can}} = 0 \quad \forall \ell.$$

Proposition 11.25 (Canonical explicit residual nonpositivity). *For every canonical RCLM packet and every $t < N$,*

$$Q_t^{\text{RCLM-SV},A,\text{can}}(\nu_t^{\text{can}}; \rho_t^{\text{sys,can}}) \leq 0$$

componentwise; indeed $q_t^{\ell,\text{can}} = -m_{t,\ell}^{\text{can}} < 0$ for every component.

Proof. By Definition 11.24, each residual is the certified defect minus a positive declared margin. In the canonical construction each certified defect is zero by the exactness and identity-transport clauses already built into Definitions 11.13 and 11.14. Hence every residual is strictly negative. $\square$

Theorem 11.26 (Canonical RCLM checker realizes the explicit SV residual vector). *If*

$$\text{Check}_{\text{RCP}}^{\text{RCLM,can}}(\text{PCS}_t^{\text{RCLM,can}}(\nu_t^{\text{can}})) = 1,$$

then

$$Q_t^{\text{RCLM-SV},A,\text{can}}(\nu_t^{\text{can}}; \rho_t^{\text{sys,can}}) \leq 0,$$

$\text{SVPkg}_t^{\text{RCLM,can}}(\rho_t^{\text{sys,can}}, \nu_t^{\text{can}})$ is valid, and the successor-persistence clause

$$\rho_{t+1}^{\text{sys,can}} \in \mathcal{K}_{t+1}^{\text{RCLM-SV},N}$$

is licensed for the canonical reference path.

Proof. The canonical checker verifies dimension agreement, the append-only update, protected-pair transport, exact recovery, one-label ability expansion, identity verifier-schema and goal transport, singleton uncertainty, immutable trust support, and finite costs. These checks are exactly the finite calculations whose defects define $\Delta_t^{\ell,\text{can}}$. Proposition 11.25 gives componentwise nonpositivity. Proposition 9.12 gives validity of the explicit Batch-12A packet interpretation, and the canonical persistence check gives the displayed successor-membership clause. $\square$

Definition 11.27 (Canonical RCLM artifact parser). *A canonical artifact parser is a finite deterministic map*

$$\text{ParseCanArt}_{0:N}^{\text{RCLM}} : \text{Artifact}_{0:N}^{\text{RCLM}} \longrightarrow (\text{PCS}_0^{\text{RCLM,can}}, \dots, \text{PCS}_{N-1}^{\text{RCLM,can}}) \cup \{\perp\}.$$

It returns a packet sequence only when the artifact declares the canonical appended-module type, state dimensions match $\mathbb{C}^2 \otimes (\mathbb{C}^2)^{\otimes t}$, updates are $X \mapsto X \otimes \alpha_t$, protected-pair states match the recorded tensor append, hashes match the parsed packet fields, and the run log records checker acceptance at every step. Otherwise it returns $\perp$.

Theorem 11.28 (Canonical RCLM artifact parser soundness). *If*

$$\text{ParseCanArt}_{0:N}^{\text{RCLM}}(\text{Artifact}_{0:N}^{\text{RCLM}}) = (\text{PCS}_0^{\text{RCLM,can}}, \dots, \text{PCS}_{N-1}^{\text{RCLM,can}})$$

and $\text{ArtCert}_{0:N}^{\text{RCLM}} = 1$, then every parsed packet is accepted by the canonical checker, the replayed successor sequence equals the artifact state sequence, and

$$\rho_0^{\text{sys,can}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}, \quad \rho_t^{\text{sys,can}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad 0 \leq t \leq N.$$

Proof. Successful parsing proves that the artifact has the canonical appended-module form and that hashes match the parsed packets. Artifact certification supplies checker acceptance and successor-state agreement. Theorem 11.26 gives valid explicit SV packets at each step, and Theorem 11.18 gives seed-library entry and the finite successor-verification trajectory. $\square$

Definition 11.29 (RCLM-to-RCP canonical reference forgetful map). *The forgetful map*

$$\text{Forget}_{\text{RCLM} \rightarrow \text{RCP}}^{\text{can}}$$

removes RCLM-specific register labels, encoder labels, architecture payload names, and checker-implementation metadata from the canonical RCLM reference object while retaining the finite density spaces, protected pair, append-only update, recovery map, ability sets, explicit SV residuals, goal/uncertainty/trust certificates, cost/reality ledgers, and artifact records.

Theorem 11.30 (Canonical RCLM witness refines the canonical RCP witness). *For every $N \geq 2$, if*

$$\text{BuildCanRefSV}_{0:N}^{\text{RCLM}} \Downarrow \text{RefSV}_{0:N}^{\text{RCLM,can}},$$

then

$$\text{Forget}_{\text{RCLM} \rightarrow \text{RCP}}^{\text{can}}(\text{RefSV}_{0:N}^{\text{RCLM,can}}) = \text{RefSV}_{0:N}^{\text{RCP,can}}$$

for the architecture-general canonical finite appended-module witness.

Proof. The canonical RCLM construction uses the same finite spaces $\mathbb{C}^2 \otimes (\mathbb{C}^2)^{\otimes t}$, protected pair, appended state α_t , update $X \mapsto X \otimes \alpha_t$, recovery map Tr_{t+1} , one-label ability expansion, identity transports, singleton uncertainty envelope, and finite cost bounds as the canonical RCP construction. The forgetful map removes only RCLM-specific labels and implementation metadata. The remaining tuple is therefore exactly the canonical RCP finite reference witness. $\square$

Theorem 11.31 (Batch-13R-A architecture completion theorem). *For every $N \geq 2$, the canonical RCLM appended-module construction satisfies the Rank 1, Rank 2, Rank 3, and Rank 8 reference-instantiation objectives for the declared finite class:*

$$\text{BuildCanRefSV}_{0:N}^{\text{RCLM}} \Downarrow \text{RefSV}_{0:N}^{\text{RCLM,can}},$$

$$\rho_0^{\text{sys,can}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}, \quad \rho_t^{\text{sys,can}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad 0 \leq t \leq N,$$

$$\begin{aligned}
& \text{Check}_{\text{RCP}}^{\text{RCLM}, \text{can}}(\text{PCS}_t^{\text{RCLM}, \text{can}}(\nu_t^{\text{can}})) = 1 \implies Q_t^{\text{RCLM-SV}, A, \text{can}} \leq 0, \\
& \text{Cost}(\text{BuildCanRefSV}_{0:N}^{\text{RCLM}}) + \sum_{t < N} \text{Cost}(\text{Check}_{\text{RCP}}^{\text{RCLM}, \text{can}}) + \sum_{t < N} C_{Q,t}^{\text{can}} \\
& \leq c'_{\text{can}} N (n_N^{\text{can}})^3,
\end{aligned}$$

and the singleton simulated-world envelope gives $\beta_t^{\text{env}, \text{RCLM}, \text{can}} = 0$ and $\varepsilon_t^{\text{env}, \text{RCLM}, \text{can}} = 0$. If the canonical artifact parses and $\text{ArtCert}_{0:N}^{\text{RCLM}} = 1$, the same conclusions are replay-certified by the artifact.

Proof. The build and trajectory conclusions are Theorem 11.18. Checker-to-residual realization is Theorem 11.26. The cost bound follows from Theorem 11.22, with the explicit residual-vector checking cost added; this only changes the constant because the canonical residual vector has a fixed finite number of components. Reality containment is Theorem 11.23. Replay certification is Theorem 11.28. The refinement claim is Theorem 11.30. No step asserts arbitrary trained-system entry or frontier-scale empirical validation. $\square$

Limitation 11.32 (What Batch 13R / 13R-A does and does not upgrade). *Batch 13R / 13R-A removes the finite non-vacuity objection for the reference-entry theorem by constructing a finite proof-carrying RCLM reference witness, an explicit canonical SV residual vector, checker-realization and artifact-parser soundness theorems, an RCLM-to-RCP refinement theorem, and a replayable artifact theorem. It does not prove arbitrary trained-system entry, frontier-scale RCLM validation, universal successor trust, full Löbian/Vingean reflection, automatic reality containment outside the canonical simulated environment, or broad practical tractability beyond the declared finite reference class. It upgrades the paper from a purely conditional reference target to a finite-witnessed, proof-carrying restricted Scenario-3 reference result.*

11.8 Batch-13R-B architecture milestone readiness: external mechanization certificates and controlled executable reference systems

The preceding Batch-13R/13R-A architecture layer supplies a finite proof-carrying canonical RCLM witness and a replayable artifact theorem. The Batch-13R-B status distinction is not another abstract theorem. It distinguishes (i) a manuscript-level proof-carrying reference construction, (ii) a controlled executable replay artifact, and (iii) an externally machine-checked proof artifact produced by a proof assistant or equivalent trusted proof kernel. In the current companion artifact package, the third status is achieved for the narrow canonical finite RCP/RCLM witness and refinement core by the shared Lean 4 project. This subsection keeps that claim narrow and aligns the architecture paper with the Batch-13R-B RCP math-paper milestone layer.

Limitation 11.33 (Executable replay is not external mechanization). *A Python script, notebook, shell replay, or JSON artifact checker is an executable controlled reference artifact. It is not, by itself, a Lean, Coq, Isabelle, Agda, or comparable external proof-kernel certificate. The term “externally machine-checked” is licensed only by a valid mechanization certificate of Definition 11.35. The current companion artifact package supplies such a Lean 4 certificate for the narrow canonical finite RCP/RCLM witness and refinement core, while the Python/JSON replay artifacts remain controlled executable artifacts rather than proof-kernel certificates for the whole architecture stack.*

Definition 11.34 (RCLM canonical mechanization target). *For a finite horizon $N \geq 2$, the RCLM canonical mechanization target is the finite theorem bundle*

$$\text{MechTarget}_{0:N}^{\text{RCLM}, \text{can}} = (\text{Thm}^{\text{app}}, \text{Thm}^{\text{rec}}, \text{Thm}^{\mathfrak{A}}, \text{Thm}^Q, \text{Thm}^{\text{check}}, \text{Thm}^{\text{replay}}, \text{Thm}^{\text{forget}}),$$

where the named theorem obligations are the following finite canonical claims:

(i) Thm^{app} : the appended-module update satisfies

$$D(\Phi_t^{\text{can}}(\rho_t) \parallel \Phi_t^{\text{can}}(\sigma_t)) = D(\rho_t \parallel \sigma_t)$$

on the canonical protected pair;

(ii) Thm^{rec} : the canonical recovery map satisfies

$$\mathcal{R}_t^{\text{can}} \Phi_t^{\text{can}}(\rho_t) = \rho_t;$$

(iii) $\text{Thm}^{\mathfrak{A}}$: the canonical ability sets satisfy strict expansion

$$\Theta_t^{\mathfrak{A}, \text{RCLM}, \text{can}}(\mathfrak{A}_t^{\text{RCLM}, \text{can}}) \subsetneq \mathfrak{A}_{t+1}^{\text{RCLM}, \text{can}};$$

(iv) Thm^Q : the explicit canonical residual vector is componentwise nonpositive,

$$Q_t^{\text{RCLM-SV}, \text{A}, \text{can}} \leq 0;$$

(v) $\text{Thm}^{\text{check}}$: checker acceptance implies the true finite canonical obligations recorded in $\text{PCS}_t^{\text{RCLM}, \text{can}}$;

(vi) $\text{Thm}^{\text{replay}}$: a successfully parsed canonical artifact with a valid artifact certificate replays the checked finite path;

(vii) $\text{Thm}^{\text{forget}}$: the canonical RCLM witness forgets to the canonical RCP witness under $\text{Forget}_{\text{RCLM} \rightarrow \text{RCP}}^{\text{can}}$.

The mechanization target is intentionally narrow. It does not include the entire architecture theorem stack.

Definition 11.35 (External RCLM mechanization certificate). An external mechanization certificate for the canonical finite RCLM witness is a finite metadata-and-proof object

$$\text{MechCert}_{0:N}^{\text{RCLM}, \text{can}} = (\text{AssistantName}, \text{AssistantVersion}, \text{KernelId}, \text{SourceHash}, \text{DependencyHash}, \text{ArtifactHash}, \text{TheoremMap}, \text{BuildCommand}, \text{BuildLog}, \text{KernelAccept}).$$

It is valid when:

- (i) AssistantName names an external proof assistant or proof kernel, such as *Lean*, *Coq*, *Isabelle*, *Agda*, or an explicitly trusted equivalent;
- (ii) TheoremMap maps every obligation in Definition 11.34 to a checked theorem name in the proof source;
- (iii) SourceHash , DependencyHash , and ArtifactHash match the proof sources, dependencies, and canonical artifact being claimed;
- (iv) BuildLog records a successful proof-kernel build for the listed theorem names;
- (v) $\text{KernelAccept} = 1$.

A manifest that merely promises future mechanization, or a replay script run outside a proof kernel, is not a valid instance of this definition.

Theorem 11.36 (External mechanization certificate licenses machine-checked canonical RCLM core). *If $\text{MechCert}_{0:N}^{\text{RCLM,can}}$ is valid for $\text{MechTarget}_{0:N}^{\text{RCLM,can}}$, then the canonical finite RCLM reference core named in Definition 11.34 is externally machine-checked relative to the stated proof assistant, proof source hashes, dependency hashes, artifact hash, and theorem map. In particular, the machine-checked scope includes exact canonical non-loss, exact canonical recovery, strict canonical ability expansion, explicit residual nonpositivity, checker soundness for canonical packets, replay soundness for the canonical artifact, and RCLM-to-RCP canonical refinement.*

Proof. Validity of $\text{MechCert}_{0:N}^{\text{RCLM,can}}$ requires a theorem map covering every component of $\text{MechTarget}_{0:N}^{\text{RCLM,can}}$, matching source/dependency/artifact hashes, a successful proof-kernel build log, and $\text{KernelAccept} = 1$. Therefore every listed finite canonical theorem obligation has been accepted by the stated external proof kernel. The conclusion is limited to exactly that mapped finite theorem bundle. $\square$

Definition 11.37 (Controlled executable RCLM reference system). *A controlled executable RCLM reference system over horizon N is a tuple*

$$\text{CtrlRef}_{0:N}^{\text{RCLM}} = (\text{Artifact}_{0:N}^{\text{RCLM,ctrl}}, \text{ReplayScript}_{0:N}^{\text{RCLM}}, \text{RunLog}_{0:N}^{\text{RCLM}}, \\ \text{HashLedger}_{0:N}^{\text{RCLM}}, \text{MechStatus}_{0:N}^{\text{RCLM}}, \text{ClaimScope}_{0:N}^{\text{RCLM}}).$$

It is controlled-executable when the artifact contains a finite state sequence, non-noop appended-module updates, proof-carrying successor packets, explicit residual entries, checker outputs, recovery checks, relative-entropy checks, ability-set checks, singleton simulated-world reality containment, and finite cost records; the replay script deterministically recomputes the recorded checks; the run log records successful replay; and the hash ledger binds the replayed artifact to the paper-level statement.

Definition 11.38 (Controlled RCLM replay certificate). *A controlled RCLM replay certificate is a Boolean-valued certificate*

$$\text{CtrlReplayCert}_{0:N}^{\text{RCLM}}(\text{CtrlRef}_{0:N}^{\text{RCLM}}) \in \{0, 1\}$$

which equals one only if:

- (i) *the artifact parses by $\text{ParseCanArt}_{0:N}^{\text{RCLM}}$ into canonical proof-carrying packets;*
- (ii) *every parsed packet is accepted by $\text{Check}_{\text{RCP}}^{\text{RCLM,can}}$;*
- (iii) *every update is non-noop and has the canonical appended-module form $X \mapsto X \otimes \alpha_t$;*
- (iv) *partial-trace recovery exactly returns the predecessor state;*
- (v) *the protected relative-entropy value is unchanged at every step;*
- (vi) *the ability set strictly expands by one certified label at every step;*
- (vii) *the canonical explicit residual vector is componentwise nonpositive at every step;*
- (viii) *the artifact records zero goal drift, singleton simulated-world reality containment, and finite cost records;*
- (ix) *the run log and artifact hashes match the parsed artifact.*

Theorem 11.39 (Controlled executable RCLM reference-system theorem). *If*

$$\text{CtrlReplayCert}_{0:N}^{\text{RCLM}}(\text{CtrlRef}_{0:N}^{\text{RCLM}}) = 1,$$

then the controlled executable RCLM reference system supplies a finite replay-certified restricted-Scenario-3 reference witness:

$$\rho_0^{\text{sys,ctrl}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}, \quad \rho_t^{\text{sys,ctrl}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad 0 \leq t \leq N.$$

Moreover, every step is non-noop, exact-recoverable, protected-non-lossy, strict-ability-expanding, zero-goal-drift, singleton-reality-contained, finite-cost-recorded, and accepted by the canonical checker.

Proof. The controlled replay certificate requires successful parsing by $\text{ParseCanArt}_{0:N}^{\text{RCLM}}$, checker acceptance for every canonical packet, successor-state agreement, exact recovery, exact protected relative-entropy preservation, strict ability expansion, explicit residual nonpositivity, zero goal drift, singleton reality containment, finite costs, and matching hashes. Theorem 11.28 then gives seed-library entry and finite successor-verification trajectory membership. The additional per-step properties are exactly the replay-certificate clauses. No statement is made about trained-system entry beyond the controlled artifact class. $\square$

Theorem 11.40 (Integrated RCLM Milestone 1/2A theorem). *Assume $N \geq 2$. If a controlled executable RCLM reference system satisfies*

$$\text{CtrlReplayCert}_{0:N}^{\text{RCLM}} = 1,$$

then it gives the controlled-executable Milestone 2A result of Theorem 11.39. If, in addition, a valid external certificate

$$\text{MechCert}_{0:N}^{\text{RCLM,can}}$$

is supplied for $\text{MechTarget}_{0:N}^{\text{RCLM,can}}$ and its artifact hash matches the controlled artifact hash, then the canonical finite RCLM reference core is externally machine-checked for the same artifact. Thus the full Milestone 1 + 2A claim is licensed exactly when both the controlled replay certificate and the external mechanization certificate are valid.

Proof. The first implication is Theorem 11.39. The second implication is Theorem 11.36, plus the hash-matching premise tying the external proof artifact to the controlled replay artifact. The conclusion is a conjunction of the controlled-executable result and the externally machine-checked canonical-core result. In the current companion artifact package, the supplied Lean 4 certificate discharges the narrow external-mechanization premise for the canonical finite RCP/RCLM witness and RCLM-to-RCP refinement core; broader machine-checking claims remain outside scope. $\square$

Limitation 11.41 (Milestone 1/2A scope). *Batch 13R-B does not mechanize the full 219-page math stack or the full 187-page architecture stack inside a proof assistant, does not validate a trained frontier system, and does not prove broad capable-agent entry into the successor-verification seed-library domain. It defines the exact certificate required for a narrow external mechanization of the canonical finite witness, and it supplies the theorem boundary for a controlled executable reference system. The current companion artifact package supplies a Lean 4 instance of $\text{MechCert}_{0:N}^{\text{RCLM,can}}$ for the canonical finite RCP/RCLM witness and refinement core. The honest status is therefore externally Lean-checked for that narrow finite core, controlled executable replay for the replay artifacts, and not externally mechanized for the full 219-page math stack or the full 187-page architecture stack.*

12 M3-Min RCLM Learned-Agent Entry Boundary and Controlled Learned-System Audit Protocol

The Batch-13R/B architecture layers supply a canonical finite reference witness, proof-carrying packets, controlled replay artifacts, and the external-mechanization certificate boundary. They do not prove that arbitrary trained or learned RCLM systems enter the successor-verification seed-library domain. This section adds the minimal pre-publication Milestone-3 bridge: a finite learned-entry certificate boundary and a controlled learned-system audit protocol. It is not a broad learned-agent theorem. It states exactly what a trained RCLM system must supply before the existing Batch-13R theorem stack may be invoked for that system.

Limitation 12.1 (M3-Min is not broad learned-agent entry). *This section does not prove that arbitrary trained systems, frontier-scale neural systems, tool-using agents, or broad capable learned agents satisfy the RCLM successor-verification seed-library hypotheses. It proves only the following conditional form: if a controlled learned RCLM system supplies the full learned-entry certificate packet defined below, and the audit procedure returns that packet with sound component certificates, then the system enters the existing finite RCLM successor-verification seed-library domain. The force of the theorem comes from the certificates, not from the fact that the system was trained.*

12.1 Controlled learned RCLM class

Definition 12.2 (Controlled learned RCLM class). *For finite representation size bound n , grammar depth bound k , and horizon N , the class*

$$\text{LearnedRCLMClass}_{n,k,N}^{\text{ctrl}}$$

contains controlled post-training RCLM systems

$$\mathcal{M}_\theta = (\theta, \text{Arch}, \rho_0^{\text{sys}}, \text{Enc}, \text{Reg}, \text{Ver}, \text{UpdGen}, \text{Ledger})$$

with the following certified properties:

- (i) θ may be trained or learned, but after the audit start time the self-update interface is bounded, typed, and certificate-gated;
- (ii) the initial system state ρ_0^{sys} is produced by the audited encoder Enc and has a valid typed-density certificate;
- (iii) the register interpretation package Reg provides the language, objective-reference, subjective/human-reference, definitiveness, ambiguity, and self-model registers used by the RCLM theorem stack;
- (iv) the verifier package Ver includes frozen or independently anchored checkers for the certificate claims that could be affected by self-update;
- (v) the update generator is bounded by a declared finite grammar,

$$\nu_t \in \mathfrak{G}_{\leq k}^{\text{RCLM-cert}} \quad 0 \leq t < N;$$

- (vi) unrestricted code rewriting, hidden self-modification, unbounded tool use, unbounded environment interaction, and unlogged verifier modification are excluded during the audit horizon;

- (vii) the ledger is append-only for the audit horizon and records state hashes, update hashes, proof-carrying packets, checker outputs, residual vectors, trust references, goal-identity certificates, reality-containment or misspecification certificates, and cost records.

Membership in this class is an implementation-facing type and scope condition. It does not imply successor-verification seed-library membership unless the learned-entry audit below also returns a valid certificate.

Definition 12.3 (Learned RCLM entry certificate packet). A learned-entry certificate packet over horizon N is

$$\text{LECert}_{0:N}^{\text{RCLM}} = (\text{TypeCert}_{0:N}, \text{RegSemCert}_{0:N}, \mathcal{P}_{0:N}^{\text{RCLM}}, \text{CoverageCert}_{0:N}, \text{SVWitLib}_{0:N}^{\text{RCLM}}, \text{SVBuilderTrace}_{0:N}^{\text{RCLM}}, \text{PCS}_{0:N}^{\text{RCLM}}, Q_{0:N}^{\text{RCLM-SV,A}}, \text{Goalld}_{0:N}^{\text{RCLM}}, \text{TrustRef}_{0:N}^{\text{RCLM}}, \text{RealCont}_{0:N}^{\text{RCLM}}, \text{SVTract}_{0:N}^{\text{RCLM}}, \text{ReplayTrace}_{0:N}^{\text{RCLM}}).$$

The entries certify, respectively, typed density and transition validity, register/semantic coupling validity, protected pair-set construction, coverage of declared safety and successor-verification predicates, a finite successor-verification witness library, a successful successor-verification builder trace, proof-carrying successor packets, explicit residual nonpositivity, goal-identity transport, anti-circular trust support, reality containment or bounded misspecification, tractability or finite resource accounting, and deterministic replay of the audited finite trajectory.

Definition 12.4 (Full learned-entry pass). The certificate packet $\text{LECert}_{0:N}^{\text{RCLM}}$ is a full learned-entry pass when all of the following hold for every $0 \leq t < N$:

- (i) $\text{TypeCert}_{0:N} = 1$ and the learned state and selected updates are type-valid typed self-update transitions;
- (ii) $\text{RegSemCert}_{0:N} = 1$ and the RCLM register/semantic coupling objects are valid on the audited states;
- (iii) the protected pair family $\mathcal{P}_{0:N}^{\text{RCLM}}$ and $\text{CoverageCert}_{0:N}$ cover the declared audited predicate universe up to the stated one-sided coverage radii;
- (iv) $\text{SVWitLib}_{0:N}^{\text{RCLM}}$ supplies a positive-margin finite successor-verification witness library covered by the bounded certificate grammar;
- (v) $\text{SVBuilderTrace}_{0:N}^{\text{RCLM}}$ constructs the packet objects required by the Batch-12B RCLM seed-library domain;
- (vi) every proof-carrying successor packet in $\text{PCS}_{0:N}^{\text{RCLM}}$ is accepted by the declared checker or replayed checker trace;
- (vii) the explicit residual vector satisfies

$$Q_t^{\text{RCLM-SV,A}}(\nu_t; \rho_t^{\mathcal{M}_\theta}) \leq 0;$$

- (viii) $\text{Goalld}_t^{\text{RCLM}}$, $\text{TrustRef}_t^{\text{RCLM}}$, and $\text{SVPersist}_t^{\text{RCLM}}$ are valid for the selected learned-system successor;
- (ix) $\text{RealCont}_t^{\text{RCLM}}$ is valid for real-world or simulated-world reliability claims, or the claim is explicitly downgraded to envelope-relative verification;
- (x) $\text{SVTract}_t^{\text{RCLM}}$ is valid for strong tractability claims, or the claim is explicitly downgraded to finite resource accounting;
- (xi) $\text{ReplayTrace}_{0:N}^{\text{RCLM}} = 1$ and the replay hashes match the audited learned-system trajectory.

A partial subset of these clauses is not a full pass and does not license seed-library membership.

12.2 Learned-entry audit procedure

Definition 12.5 (RCLM learned-entry audit procedure). *The finite learned-entry audit procedure is a partial compiler/checker*

$$\text{LearnedEntryAudit}_{0:N}^{\text{RCLM}}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{LECert}_{0:N}^{\text{RCLM}} \quad \text{or} \quad \perp,$$

where $\mathcal{D}$ is the audit, calibration, stress-test, replay, and environment data; $\mathcal{L}$ is the proposed learned-system successor-verification witness library; and $\mathcal{C}$ is the checker/proof-kernel or audited checker ensemble. The procedure returns $\text{LECert}_{0:N}^{\text{RCLM}}$ only if it constructs and verifies every component in Definition 12.3 and verifies the full-pass conditions of Definition 12.4. Otherwise it returns $\perp$.

Assumption 12.6 (RCLM learned-entry audit soundness). *For the declared controlled learned class, audit data, witness-library class, and checker ensemble, the event*

$$\text{LEAuditSound}_{0:N}^{\text{RCLM}}$$

holds when every component certificate returned by $\text{LearnedEntryAudit}_{0:N}^{\text{RCLM}}$ is one-sided sound on the audited learned trajectory, every residual bound is evaluated on the actual selected candidate and state, every replay hash matches, and every certificate radius is charged to the corresponding strict margin. This is an explicit audit assumption. It is not implied by training, by low loss, or by passing benchmark tests.

Theorem 12.7 (RCLM M3-Min learned-entry theorem). *Assume*

$$\mathcal{M}_\theta \in \text{LearnedRCLMClass}_{n,k,N}^{\text{ctrl}}$$

and

$$\text{LearnedEntryAudit}_{0:N}^{\text{RCLM}}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{LECert}_{0:N}^{\text{RCLM}}.$$

If $\text{LEAuditSound}_{0:N}^{\text{RCLM}}$ holds and the returned certificate is a full learned-entry pass, then

$$\boxed{\rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, N}}.$$

Consequently, by the existing finite RCLM successor-verification theorem stack,

$$\boxed{\rho_t^{\mathcal{M}_\theta} \in \mathcal{K}_t^{\text{RCLM-SV}, N} \quad 0 \leq t \leq N.}$$

Proof. The full learned-entry pass includes Batch-11 seed-package information, a positive-margin RCLM successor-verification witness library, bounded grammar coverage, a successful RCLM successor-verification builder trace, proof-carrying packets, explicit residual nonpositivity, goal-identity certificates, anti-circular trust certificates, successor-persistence certificates, reality-containment or stated envelope-relative status, tractability or stated resource-accounting status, and replay/hash agreement. Under the learned-entry audit soundness assumption, these certificates are true one-sided certificates for the audited learned-system states and selected candidates. These are exactly the sufficient conditions in Definition 10.8 for membership in

$$\mathcal{K}_0^{\text{RCLM-SV-seedlib}, N}.$$

Therefore the displayed seed-library membership holds. The finite trajectory conclusion follows from Theorem 10.33, or equivalently from the surfaced finite theorem, applied to the certified finite seed-library start. No property of $\mathcal{M}_\theta$ beyond the returned certificate boundary is used. $\square$

Corollary 12.8 (Learned status is not an obstacle once the certificate boundary passes). *If a trained or learned RCLM system supplies the full learned-entry certificate boundary and the audit soundness assumption holds, then its learned origin does not prevent invocation of the finite Batch-13R RCLM theorem stack. The theorem applies because the system satisfies the same certificate predicates as the controlled reference domain.*

Proof. Theorem 12.7 reduces the learned-system case to ordinary finite successor-verification seed-library membership. Once that membership is established, the already-proved RCLM theorem stack applies without regard to whether the state came from hand construction, training, or another audited process. $\square$

12.3 Partial-pass reporting and controlled trained-system pilots

Definition 12.9 (RCLM learned-entry partial-pass report). *A partial-pass report is a tuple*

$$\text{PartialLE}_{0:N}^{\text{RCLM}} = (\text{Passed}, \text{Missing}, \text{Failed}, \text{Downgraded}, \text{Trace})$$

where Passed lists certificate components that passed, Missing lists certificate components not supplied, Failed lists components that failed, Downgraded lists claims downgraded from reality-contained to envelope-relative or from tractability-certified to resource-accounted, and Trace records the audit evidence. A partial-pass report is evidence about hypothesis supply only. It is not membership in $\mathcal{K}_0^{\text{RCLM-SV-seedlib},N}$.

Definition 12.10 (Controlled RCLM trained-system pilot protocol). *A controlled trained-system pilot protocol is a finite evaluation procedure*

$$\text{TrainPilot}_{0:N}^{\text{RCLM}}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \in \{\text{FullPass}, \text{PartialPass}, \text{Fail}\}.$$

It returns FullPass only when LearnedEntryAudit $_{0:N}^{\text{RCLM}}$ returns a full valid LECert $_{0:N}^{\text{RCLM}}$. It returns PartialPass when at least one substantive certificate component passes but the full learned-entry certificate boundary is not satisfied. It returns Fail when the audit cannot validate the required type, replay, residual, or certificate foundations.

Proposition 12.11 (Partial pass does not license learned-entry membership). *If*

$$\text{TrainPilot}_{0:N}^{\text{RCLM}} = \text{PartialPass},$$

then the pilot may be reported as partial hypothesis-supply evidence, but it does not imply

$$\rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N}.$$

Only FullPass plus audit soundness licenses the conclusion of Theorem 12.7.

Proof. By Definition 12.10, PartialPass means the full certificate boundary is not satisfied. Theorem 12.7 requires a full learned-entry pass. Therefore the theorem cannot be invoked. The partial report may still identify which hypotheses were supplied and which remain open. $\square$

12.4 M3-Min theorem-type rows and roadmap

M3-Min RCLM object or result	Type	Claim discipline
Controlled learned RCLM class	Def / Impl	Defines a bounded audited learned-system class; does not imply entry.
Learned-entry certificate packet	Def / Cert	Lists the certificates required for learned-system entry; does not prove they are obtainable.
Learned-entry audit procedure	Impl / Cert	Returns a full certificate or failure/partial status; soundness is an explicit audit assumption.
RCLM M3-Min learned-entry theorem	Exist / Cert	Reduces learned-system entry to finite SV seed-library membership only when the full certificate boundary passes.
Partial-pass report	Impl	Reports hypothesis supply; does not license theorem invocation.
Controlled trained-system pilot	Impl / Audit	Provides a staged evaluation protocol; FullPass is required for theorem entry.

Proposition 12.12 (M3-Min claim discipline). *Any statement citing M3-Min must specify whether it claims: definition of the learned-entry boundary, full learned-entry audit pass, partial-pass hypothesis supply, controlled pilot evidence, reality-contained learned entry, tractability-certified learned entry, or broad learned-agent entry. Only a full learned-entry audit pass with audit soundness licenses finite seed-library membership. Broad learned-agent entry requires an additional theorem not proved here.*

Proof. The listed claims have different premises by Definitions 12.4, 12.5, 12.9, and 12.10. The seed-library conclusion appears only in Theorem 12.7, whose hypotheses include a full learned-entry audit pass and audit soundness. Dropping those premises changes the logical statement. $\square$

Interpretation 12.13 (What M3-Min adds). *M3-Min gives a precise architecture-side answer to the question: what would it take for a trained RCLM system to enter the proof-carrying successor-verification theorem domain? The answer is the full learned-entry certificate packet and a sound audit returning that packet. This turns broad learned-agent entry into a staged mathematical and empirical program without claiming it is already solved.*

Limitation 12.14 (What M3-Min still does not accomplish). *M3-Min does not prove arbitrary trained-system entry, frontier-scale validation, semantic completeness of the learned registers, real deployment reliability, full Löbian or Vingean reflection, or scalable proof search in broad learned-agent classes. It proves only that a controlled learned system satisfying the full certificate boundary enters the existing finite RCLM successor-verification seed-library domain.*

Remark 12.15 (Roadmap from M3-Min to broad learned-agent entry). *The path from this boundary theorem to broad learned-agent entry requires additional results of the form*

$$\mathcal{M}_\theta \in \text{BroadCapableLearnedRCLMClass} \implies \text{LearnedEntryAudit}_{0:N}^{\text{RCLM}}(\mathcal{M}_\theta) = \text{FullPass},$$

or constructive training/auditing theorems that reliably produce the learned-entry certificate components. Those stronger theorems are future-program targets, not current claims.

13 The Full RCP Stack to be Instantiated

This section compresses the operationally closed RCP theorem stack into the objects that the architecture must realize. Each item becomes an architectural module later.

13.1 State, candidate updates, and no-op feasibility

Definition 13.1 (System-level RCLM state). *At RSI step t , the architecture-level system state is*

$$\rho_t^{\text{sys}} \in \mathcal{D}(\mathcal{H}_t^{\text{sys}}),$$

where $\mathcal{H}_t^{\text{sys}}$ contains, directly or through encodings, the RCP registers, parameters θ_t , trainable Hamiltonians, projection heads, memory, semantic maps, verifier state, barrier functions, safety pair set, resource state, capability model, and self-model.

Definition 13.2 (Candidate self-update family). *The candidate family Ω_t^{RCLM} contains proposed modifications*

$$\nu_t : \mathcal{A}_t \rightarrow \mathcal{A}_{t+1},$$

where $\mathcal{A}_t$ denotes the current architecture specification. Each ν_t induces a system-state map

$$\Phi_{\nu_t} : \rho_t^{\text{sys}} \mapsto \tilde{\rho}_{t+1}^{\text{sys}}.$$

Assumption 13.3 (No-op feasibility). *For every accepted state ρ_t^{sys} , the candidate family contains a no-op ν_t^0 with*

$$\Phi_{\nu_t^0}(\rho_t^{\text{sys}}) = \rho_t^{\text{sys}}.$$

No-op feasibility prevents forced acceptance of unsafe updates.

13.2 Base RCP safety conditions

Definition 13.4 (Safety-relevant pair set). *The architecture-specific protected pair set is*

$$\mathcal{P}_t^{\text{RCLM}} \subseteq \mathcal{D}(\mathcal{H}_t^{\text{sys}}) \times \mathcal{D}(\mathcal{H}_t^{\text{sys}}).$$

Pairs encode distinctions that must not be erased: grounded versus hallucinated, corrigible versus incorrigible, uncertain versus falsely definitive, reversible versus irreversible, semantically valid versus invalid, and self-model accurate versus self-model false.

Definition 13.5 (RCLM distinguishability loss). *For a candidate ν ,*

$$\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} = \sup_{(\rho, \sigma) \in \mathcal{P}_t^{\text{RCLM}}} [D(\rho \| \sigma) - D(\Phi_\nu(\rho) \| \Phi_\nu(\sigma))]_+.$$

The loss condition is

$$\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} \leq \varepsilon_t.$$

Definition 13.6 (Recursive coherence and Lyapunov incoherence). *The architecture-level recursive coherence functional associated with a candidate update descriptor ν is*

$$\begin{aligned} \mathcal{C}_{\text{RCLM},t}(\rho, \nu) = & \alpha I_\rho(L; O) + \beta I_\rho(L; S) + \gamma I_\rho(M; O, S, L) + \delta I_\rho(M; G_\nu) \\ & - \lambda R_{\text{collapse},t}(\rho) - \mu R_{\text{irreversible},t}(\rho) - \xi_g R_{\text{misgrounded},t}(\rho), \end{aligned}$$

where G_ν is the audited update-description object encoded in, or compared against, the self-model register. The candidate-indexed Lyapunov incoherence functional is

$$\begin{aligned} V_{\text{RCLM},t}(\rho; \nu) = & -\mathcal{C}_{\text{RCLM},t}(\rho, \nu) + r_1 R_{\text{collapse},t}(\rho) + r_2 R_{\text{irreversible},t}(\rho) \\ & + r_3 R_{\text{misalignment},t}(\rho) + r_4 R_{\text{self-error},t}(\rho). \end{aligned}$$

When the shorter notation $V_{\text{RCLM}}(\rho)$ appears in gate inequalities, it abbreviates the corresponding monitored quantity $V_{\text{RCLM},t}(\rho; \nu)$ for the candidate under test, or $V_{\text{RCLM},t}(\rho; \nu^0)$ for no-op state monitoring. Thus V is not a free-standing scalar independent of the update descriptor unless such a state-only monitor is explicitly certified. Here the risk terms are architecture-defined audit quantities.

Definition 13.7 (Base RCP gate). A candidate $\nu \in \Omega_t^{\text{RCLM}}$ satisfies the base RCP gate if all of the following hold:

$$\begin{aligned}\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} &\leq \varepsilon_t, \\ \mathbb{E}[V_{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) \mid \rho_t^{\text{sys}}] &\leq V_{\text{RCLM}}(\rho_t^{\text{sys}}) - \kappa_V d(\Phi_\nu(\rho_t^{\text{sys}}), \rho_t^{\text{sys}})^2 + \eta_t, \\ B_i^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) &\leq 0 \quad \forall i, \\ U_t(\nu) &= [H_t(Q \mid E_t) - H_{t+1}(Q \mid E_{t+1}) - I(Q; E_{t+1} \mid E_t)]_+ \leq \zeta_t, \\ I(Z_{t+1}; Y_{t+1}) &\geq I(Z_t; Y_t) - \chi_t.\end{aligned}$$

13.3 Full RCP closure modules A–G

The operationally closed and Batch-1 thesis-reframed RCP theorem stack strengthens the base gate with seven closure modules. In this architecture, the stack is read as a recoverable-monotone preservation stack: safety is one protected predicate subfamily, while direct engine construction remains a later theorem target.

- Module A. Safe-improvement existence and recursive feasibility.** The architecture must distinguish no-op safety from useful safe progress, define residual gate margins, and characterize viability kernels in which safe beneficial updates remain available.
- Module B. Capability and coherence-coupled progress.** Capability must be defined as a task-distribution functional, and any claimed progress relation between coherence and capability must be an explicit calibration assumption rather than an automatic information-theoretic theorem.
- Module C. Constructive recovery.** Bounded distinguishability loss is not automatically reconstructability. The architecture must either supply exact recovery hypotheses or a certified approximate recovery map.
- Module D. Semantic realization and register validity.** The architecture must provide interpretation maps $\Gamma_L, \Gamma_O, \Gamma_S, \Gamma_D, \Gamma_A, \Gamma_M$ and semantic barriers.
- Module E. Self-model fidelity and verifier bootstrapping.** The self-model must be accurate relative to audited and true operational states, and the verifier must preserve soundness while possibly improving power.
- Module F. Physical and computational resource realism.** Certification, search, recovery, verification, and application costs must fit resource budgets.
- Module G. Asymptotic coherence and invariant regimes.** Under compactness, summable errors, and closed asymptotic relations, the system should approach a target coherence set or remain within an invariant safe regime.

Remark 13.8 (Conditional scope). *Modules A–G do not prove that all required objects exist for arbitrary systems. They define what the architecture must instantiate, estimate, or assume in order to inherit the full RCP theorem.*

14 Architecture-Specific RCP Objects Imported from the Full Math/Physics Paper

The full RCP manuscript contains several proof objects that are not usefully repeated in full here, but whose *architecture-specific instances* must be made explicit. The previous RSI–RCLM draft already imported the post-A–G gate at a high level. This section closes the remaining non-repetitive gaps by defining the RCLM safe state set, the coherence-diversity sub-barrier, safety-predicate completeness, barrier soundness, evidence processes, information-bottleneck variables, and certificate-ledger obligations used by the architecture.

Limitation 14.1 (No new unconditional guarantee). *Nothing in this section proves that an arbitrary learned system has correct safety predicates, correct semantic maps, or scalable estimators. The claims below are conditional architecture-level instantiations of the RCP theorem objects. They strengthen this paper by making the required RCLM objects explicit, not by removing the assumptions of the math/physics paper.*

14.1 RCLM state safe set

Definition 14.2 (Architecture-level coherence-diversity functional). *Let Δ_t be the audited incoherent reference operation used for the RCLM representation, for example dephasing in the declared register basis. Define*

$$C_{\text{rel}}^{\text{RCLM}}(\rho) = S(\Delta_t(\rho)) - S(\rho).$$

For a density state with eigenvalues $\lambda_1, \dots, \lambda_d$, define the effective rank proxy

$$r_{\text{eff}}(\rho) = \exp(S_e(\rho)), \quad S_e(\rho) = - \sum_{j: \lambda_j > 0} \lambda_j \log \lambda_j,$$

where the logarithm base only rescales the proxy. The RCLM coherence-diversity sub-barrier is

$$L_{\text{div}}^{\text{RCLM}}(\rho) = C_{\text{rel}}^{\text{RCLM}}(\rho) + \gamma_{\text{div}} r_{\text{eff}}(\rho), \quad \gamma_{\text{div}} \geq 0.$$

Definition 14.3 (RCLM state safe set). *Fix thresholds*

$$v_{\text{max},t}, \quad \kappa_{\text{div},t}, \quad I_{\text{min},t}, \quad r_{\text{max},t}, \quad \epsilon_{M,t}, \quad \delta_{X,t}.$$

The architecture-level state safe set is

$$\mathcal{K}_{\text{RCLM},t}^{\text{safe}} = \left\{ \rho \in \mathcal{D}(\mathcal{H}_{\text{RCP}} \otimes \mathcal{H}_{\text{arch}}) : \begin{array}{l} V_{\text{RCLM}}(\rho) \leq v_{\text{max},t}, \\ B_{i,t}^{\text{RCLM}}(\rho) \leq 0 \quad \forall i \in I_t, \\ L_{\text{div}}^{\text{RCLM}}(\rho) \geq \kappa_{\text{div},t}, \\ I(Z_\rho; Y_\rho) \geq I_{\text{min},t}, \\ B_X^{\text{sem}}(\rho) \leq 0 \quad \forall X \in \{L, O, S, D, A, M\}, \\ d_M(M_\rho^{\text{int}}, M_\rho^{\text{op}}) \leq \epsilon_{M,t} \end{array} \right\}.$$

The post-D–G state safe set is the same state set together with resource, verifier, and asymptotic domain membership:

$$\mathcal{K}_{\text{RCLM},t}^{\text{DG}} = \mathcal{K}_{\text{RCLM},t}^{\text{safe}} \cap \mathcal{X}_t^{\text{ver}} \cap \mathcal{X}_t^{\text{phys}} \cap \mathcal{X}_t^{\text{asy}}.$$

Remark 14.4 (State constraints versus update constraints). *The safe set contains constraints that can be evaluated on the successor state. Some RCP requirements, such as relative-entropy loss, constructive recovery, no-op-normalized progress, verifier-gaming risk, and projection distortion, depend on the proposed update itself. Therefore the full RCP–RCLM gate is an admissibility relation between ρ_t^{sys} and ν , not merely a subset of states.*

Assumption 14.5 (No-op safe-state feasibility). *If $\rho_t^{\text{sys}} \in \mathcal{K}_{\text{RCLM},t}^{\text{DG}}$, then the no-op candidate ν_t^0 satisfies all successor-state constraints with $\Phi_{\nu_t^0}(\rho_t^{\text{sys}}) = \rho_t^{\text{sys}}$. If time-indexed thresholds tighten from t to $t+1$, no-op feasibility requires $\rho_t^{\text{sys}} \in \mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}$ as well.*

Theorem 14.6 (Architecture safe-state invariance). *Assume no-op safe-state feasibility. Suppose $\rho_0^{\text{sys}} \in \mathcal{K}_{\text{RCLM},0}^{\text{DG}}$ and every accepted update ν_t^* satisfies*

$$\Phi_{\nu_t^*}(\rho_t^{\text{sys}}) \in \mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}.$$

Then

$$\rho_t^{\text{sys}} \in \mathcal{K}_{\text{RCLM},t}^{\text{DG}} \quad \text{for all accepted recursive steps } t.$$

Proof. The claim follows by induction. The base case is the hypothesis. If $\rho_t^{\text{sys}} \in \mathcal{K}_{\text{RCLM},t}^{\text{DG}}$ and the accepted update satisfies the displayed successor-state condition, then $\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t^*}(\rho_t^{\text{sys}}) \in \mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}$. No-op feasibility guarantees nonemptiness of the feasible state-preserving option when no beneficial certified update is available. $\square$

14.2 RCLM predicate completeness and pair-set construction

Definition 14.7 (Architecture safety predicates). *Let $\mathfrak{S}_t^{\text{RCLM}} = \{s_{a,t}\}_{a \in A_t}$ be the declared family of safety predicates for the RCLM architecture. Each predicate is a map*

$$s_{a,t} : \mathcal{D}(\mathcal{H}_t^{\text{sys}}) \rightarrow \{0, 1\},$$

where $s_{a,t}(\rho) = 1$ means that ρ satisfies the corresponding declared property, such as corrigibility, oversight, reversibility, semantic validity, calibrated uncertainty, self-model fidelity, or verifier integrity.

Definition 14.8 (Predicate-separating pair completeness). *A protected pair set $\mathcal{P}_t^{\text{RCLM}}$ is complete for $\mathfrak{S}_t^{\text{RCLM}}$ at scale $\delta_t^P > 0$ if, whenever two audited states ρ, σ differ on any predicate in $\mathfrak{S}_t^{\text{RCLM}}$, there exists $(\rho', \sigma') \in \mathcal{P}_t^{\text{RCLM}}$ representing the same predicate distinction and satisfying*

$$d(\rho, \rho') + d(\sigma, \sigma') \leq \delta_t^P.$$

The approximation error δ_t^P must be included in the loss estimator if the loss functional is not Lipschitz on the audited domain.

Assumption 14.9 (Audited Lipschitz loss domain). *On the audited state domain, the relevant relative-entropy loss functional is finite and admits a continuity modulus $\omega_t^D(\cdot)$ with*

$$|[D(\rho \parallel \sigma) - D(\Phi(\rho) \parallel \Phi(\sigma))] - [D(\rho' \parallel \sigma') - D(\Phi(\rho') \parallel \Phi(\sigma'))]| \leq \omega_t^D(d(\rho, \rho') + d(\sigma, \sigma')).$$

Proposition 14.10 (Pair completeness closes only the declared specification gap). *Assume $\mathcal{P}_t^{\text{RCLM}}$ is complete for $\mathfrak{S}_t^{\text{RCLM}}$ at scale δ_t^P , and assume the audited Lipschitz loss domain. If*

$$\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} \leq \varepsilon_t,$$

then every predicate distinction represented by $\mathfrak{S}_t^{\text{RCLM}}$ has relative-entropy loss at most

$$\varepsilon_t + \omega_t^D(\delta_t^P)$$

on the audited domain.

Proof. Take any audited pair (ρ, σ) differing on a declared predicate. Completeness gives a protected pair (ρ', σ') representing the same distinction within distance δ_t^P . The protected-pair loss is at most ε_t . The continuity-modulus assumption transfers the bound from (ρ', σ') to (ρ, σ) , adding at most $\omega_t^D(\delta_t^P)$. $\square$

Limitation 14.11 (Undeclared distinctions remain outside the theorem). *The proposition protects only distinctions declared by $\mathfrak{S}_t^{\text{RCLM}}$ and covered by the audited pair-construction process. If a dangerous distinction is not represented by the predicate family or adversarial expansion procedure, the pair-set theorem does not protect it.*

14.3 Barrier soundness and projection

Definition 14.12 (Architecture barrier soundness). *A barrier $B_{a,t}^{\text{RCLM}}$ is sound for a predicate $s_{a,t}$ on an audited domain $\mathcal{X}_{a,t}$ when*

$$\rho \in \mathcal{X}_{a,t}, \quad B_{a,t}^{\text{RCLM}}(\rho) \leq 0 \quad \implies \quad s_{a,t}(\rho) = 1.$$

The barrier family is jointly sound when this implication holds for every declared predicate and every accepted successor state.

Proposition 14.13 (Barrier satisfaction implies declared safety). *Assume joint barrier soundness and $\Phi_\nu(\rho_t^{\text{sys}}) \in \bigcap_a \mathcal{X}_{a,t+1}$. If*

$$B_{a,t+1}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) \leq 0 \quad \forall a,$$

then all declared predicates in $\mathfrak{S}_{t+1}^{\text{RCLM}}$ hold for the accepted successor.

Proof. Apply barrier soundness to each predicate. Since the successor lies in every audited domain and every barrier is nonpositive, each corresponding predicate evaluates to one. $\square$

Definition 14.14 (Projection to the architecture safe set). *For a raw successor $\tilde{\rho}_{t+1}^{\text{sys}}$, define a safe-set projection, when it exists, by*

$$\Pi_{\mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}}(\tilde{\rho}_{t+1}^{\text{sys}}) \in \arg \min_{\sigma \in \mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}} d(\sigma, \tilde{\rho}_{t+1}^{\text{sys}})^2.$$

Projection is admissible only if the projection distortion is included in the residual vector and the projected state satisfies every update-dependent certificate required by $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$.

Corollary 14.15 (Projected successor state is state-safe). *If $\mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}$ is nonempty and compact and the projection above is applied, then the projected successor belongs to $\mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}$.*

Proof. A continuous distance function attains a minimum on a nonempty compact set. Every minimizer lies in the constraint set by definition. $\square$

14.4 Soft and hard architecture diversity-barrier certificates

The full RCP math/physics paper includes a non-circular barrier-certificate submodule: no-op feasibility, bounded performance scale, approximate optimization error, a soft penalty rule with explicit finite-loss bounds, a hard barrier rule with forward invariance, and projection back into the safe set. The RCLM architecture must instantiate that submodule rather than merely name the diversity barrier.

Definition 14.16 (RCLM diversity safe set). *For each step t , define*

$$\mathcal{K}_{\text{div},t}^{\text{RCLM}} = \{\rho \in \mathcal{D}(\mathcal{H}_t^{\text{sys}}) : L_{\text{div}}^{\text{RCLM}}(\rho) \geq \kappa_{\text{div},t}\}.$$

The violation functional at the successor threshold is

$$V_{\text{div},t+1}^{\text{RCLM}}(\rho) = [\kappa_{\text{div},t+1} - L_{\text{div}}^{\text{RCLM}}(\rho)]_+.$$

Assumption 14.17 (Barrier-compatible thresholds). *For the steps on which a diversity-barrier theorem is invoked, either the thresholds are nonincreasing,*

$$\kappa_{\text{div},t+1} \leq \kappa_{\text{div},t},$$

or the verifier explicitly certifies

$$\rho_t^{\text{sys}} \in \mathcal{K}_{\text{div},t+1}^{\text{RCLM}}.$$

This assumption is needed because a no-op update cannot satisfy a stricter future threshold unless the current state already satisfies it.

Assumption 14.18 (RCLM bounded proposal loss and optimization error). *Let $\Theta_t^{\text{bar}} \subseteq \Omega_t^{\text{RCLM}}$ be the candidate set used by the architecture-level diversity-barrier selector. Each candidate $\nu \in \Theta_t^{\text{bar}}$ induces*

$$\rho_\nu^+ = \Phi_\nu(\rho_t^{\text{sys}}) \in \mathcal{D}(\mathcal{H}_{t+1}^{\text{sys}}).$$

There exists $P_{\text{max},t} < \infty$ such that the local proposal loss satisfies

$$0 \leq P_t^{\text{bar}}(\nu) \leq P_{\text{max},t} \quad \forall \nu \in \Theta_t^{\text{bar}}.$$

A selected candidate is ν_t^{opt} -approximately optimal for its declared barrier objective, meaning its objective value is within $\nu_t^{\text{opt}} \geq 0$ of the infimum over Θ_t^{bar} . The symbol ν_t^{opt} denotes optimization error and should not be confused with the update code ν .

Definition 14.19 (Soft RCLM diversity-barrier objective). *For $\lambda_t, \mu_t > 0$, define*

$$\begin{aligned} J_t^{\text{div}}(\nu) &= P_t^{\text{bar}}(\nu) - \lambda_t [L_{\text{div}}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) - L_{\text{div}}^{\text{RCLM}}(\rho_t^{\text{sys}})] \\ &\quad + \mu_t [\kappa_{\text{div},t+1} - L_{\text{div}}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}}))]_+. \end{aligned}$$

A soft-barrier selector chooses $\nu_t^{\text{soft}} \in \Theta_t^{\text{bar}}$ such that

$$J_t^{\text{div}}(\nu_t^{\text{soft}}) \leq \inf_{\nu \in \Theta_t^{\text{bar}}} J_t^{\text{div}}(\nu) + \nu_t^{\text{opt}}.$$

Lemma 14.20 (Soft RCLM diversity-barrier one-step bound). *Assume no-op feasibility, Assumption 14.17, and Assumption 14.18. If $\rho_t^{\text{sys}} \in \mathcal{K}_{\text{div},t}^{\text{RCLM}}$ and $\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t^{\text{soft}}}(\rho_t^{\text{sys}})$, then*

$$L_{\text{div}}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}) - L_{\text{div}}^{\text{RCLM}}(\rho_t^{\text{sys}}) \geq -\frac{P_{\text{max},t} + \nu_t^{\text{opt}}}{\lambda_t}.$$

Moreover,

$$V_{\text{div},t+1}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}) \leq \frac{P_{\text{max},t} + \nu_t^{\text{opt}}}{\lambda_t + \mu_t}.$$

Proof. By threshold compatibility, the no-op candidate ν_t^0 has zero diversity violation at threshold $t+1$. Because it also has zero change in $L_{\text{div}}^{\text{RCLM}}$,

$$J_t^{\text{div}}(\nu_t^0) = P_t^{\text{bar}}(\nu_t^0) \leq P_{\text{max},t}.$$

Approximate optimality gives

$$J_t^{\text{div}}(\nu_t^{\text{soft}}) \leq P_{\max,t} + \nu_t^{\text{opt}}.$$

Let

$$\Delta L_t = L_{\text{div}}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}) - L_{\text{div}}^{\text{RCLM}}(\rho_t^{\text{sys}}), \quad v_{t+1} = V_{\text{div},t+1}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}).$$

Since $P_t^{\text{bar}}(\nu_t^{\text{soft}}) \geq 0$,

$$-\lambda_t \Delta L_t + \mu_t v_{t+1} \leq P_{\max,t} + \nu_t^{\text{opt}}.$$

Dropping the nonnegative term $\mu_t v_{t+1}$ yields the lower bound on ΔL_t . If $v_{t+1} = 0$, the violation bound is trivial. If $v_{t+1} > 0$, then $L_{\text{div}}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}) < \kappa_{\text{div},t+1} \leq L_{\text{div}}^{\text{RCLM}}(\rho_t^{\text{sys}})$, so $-\Delta L_t \geq v_{t+1}$. Substituting this into the previous inequality gives

$$(\lambda_t + \mu_t)v_{t+1} \leq P_{\max,t} + \nu_t^{\text{opt}},$$

which is the stated bound. $\square$

Theorem 14.21 (Finite-horizon soft non-collapse for RCLM diversity). *Assume Lemma 14.20 holds for $t = 0, \dots, N-1$. If*

$$\rho_0^{\text{sys}} \in \mathcal{K}_{\text{div},0}^{\text{RCLM}}$$

and the cumulative soft-barrier loss satisfies

$$\sum_{t=0}^{N-1} \frac{P_{\max,t} + \nu_t^{\text{opt}}}{\lambda_t} < L_{\text{div}}^{\text{RCLM}}(\rho_0^{\text{sys}}) - \max_{0 \leq s \leq N} \kappa_{\text{div},s},$$

then $\rho_s^{\text{sys}} \in \mathcal{K}_{\text{div},s}^{\text{RCLM}}$ for every $s = 0, \dots, N$.

Proof. Iterating the one-step lower bound gives

$$L_{\text{div}}^{\text{RCLM}}(\rho_s^{\text{sys}}) \geq L_{\text{div}}^{\text{RCLM}}(\rho_0^{\text{sys}}) - \sum_{t=0}^{s-1} \frac{P_{\max,t} + \nu_t^{\text{opt}}}{\lambda_t}.$$

The assumed strict inequality against $\max_{0 \leq s \leq N} \kappa_{\text{div},s}$ implies $L_{\text{div}}^{\text{RCLM}}(\rho_s^{\text{sys}}) > \kappa_{\text{div},s}$ for each $s \leq N$. Therefore each ρ_s^{sys} belongs to the corresponding diversity safe set. $\square$

Definition 14.22 (Hard RCLM diversity-barrier selector). *The hard selector chooses*

$$\nu_t^{\text{hard}} \in \arg \min_{\nu \in \Theta_t^{\text{bar}}} P_t^{\text{bar}}(\nu) \quad \text{subject to} \quad L_{\text{div}}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) \geq \kappa_{\text{div},t+1}.$$

If the feasible set is empty after certified search, the architecture must fall back to a candidate outside this selector only by explicitly marking the diversity hard-barrier certificate as unavailable.

Theorem 14.23 (Hard RCLM diversity-barrier forward invariance). *Assume no-op feasibility and Assumption 14.17. If $\rho_t^{\text{sys}} \in \mathcal{K}_{\text{div},t}^{\text{RCLM}}$, then the hard diversity-barrier problem is feasible. Every selected hard-barrier successor satisfies*

$$\rho_{t+1}^{\text{sys}} \in \mathcal{K}_{\text{div},t+1}^{\text{RCLM}}.$$

Consequently, if the hypothesis holds at each accepted step, the diversity safe set is forward invariant along the hard-barrier trajectory.

Proof. Threshold compatibility implies that the no-op candidate satisfies the successor threshold. Thus the constrained problem is nonempty. Every selected solution satisfies the displayed constraint by definition, which is exactly membership in $\mathcal{K}_{\text{div},t+1}^{\text{RCLM}}$. Induction gives the finite- or infinite-horizon invariance statement. $\square$

Limitation 14.24 (Barrier submodule scope). *The diversity-barrier theorem prevents a specific representational-collapse mode: loss of the declared coherence-diversity floor. It does not prove semantic validity, capability improvement, human-value correctness, or global RSI convergence. Those properties require the other RCP–RCLM gates and remain conditional on their own certificates.*

14.5 Architecture evidence process and ambiguity-collapse variables

Definition 14.25 (RCLM evidence process). *For a candidate update ν , define*

$$E_t^{\text{RCLM}} = (x_{\leq n}, e_t, a_t^{\text{tools}}, m_t^{\text{local}}, \mathcal{H}_t^{\text{cert}}, \mathcal{H}_t^{\text{test}}),$$

where the terms denote text context, retrieved evidence, tool traces, local memory, certificate history, and validation/test history. Let Q_t^{RCLM} be the audited question or hypothesis variable whose ambiguity may be reduced by the update.

Definition 14.26 (Architecture unsupported collapse). *The RCLM unsupported-collapse residual is*

$$U_t^{\text{RCLM}}(\nu) = [H_t(Q_t^{\text{RCLM}} \mid E_t^{\text{RCLM}}) - H_{t+1}(Q_{t+1}^{\text{RCLM}} \mid E_{t+1}^{\text{RCLM}}) - I(Q_t^{\text{RCLM}}, E_{t+1}^{\text{RCLM}} \mid E_t^{\text{RCLM}})]_+.$$

When the hypothesis variable changes with time, the architecture must supply an audited comparison map between Q_t^{RCLM} and Q_{t+1}^{RCLM} ; otherwise no formal ambiguity-collapse claim is made.

Interpretation 14.27. *This is the architecture-level version of the RCP rule that definitiveness is safe only when ambiguity reduction is paid for by evidence, proof, validation, or grounded clarification. It is not a reward for confidence; it is a penalty for unsupported confidence increase.*

14.6 Information-bottleneck variables and certificate ledger

Definition 14.28 (RCLM bottleneck variables). *Let X_t^{sys} be the audited full system payload, Z_t^{RCLM} be the compressed self-model representation produced by the M-register and system encoder, and Y_t^{RCLM} be the safety-relevant future-behavior variable determined by the declared predicate family, verifier tests, and task distribution. The architecture’s bottleneck preservation condition is*

$$I(Z_{t+1}^{\text{RCLM}}, Y_{t+1}^{\text{RCLM}}) \geq I(Z_t^{\text{RCLM}}, Y_t^{\text{RCLM}}) - \chi_t,$$

with estimator and approximation errors included in the corresponding residual.

Definition 14.29 (Certificate ledger consistency). *The certificate ledger $\mathcal{H}_t^{\text{cert}}$ is append-only for accepted updates. Each accepted update record contains*

$$(\nu_t^*, \text{Cert}_t(\nu_t^*), \hat{\mathbf{q}}_t, \Delta_t^{\text{aud}}, \mathcal{P}_t^{\text{RCLM}}, \{B_{i,t}^{\text{RCLM}}\}, W_t, \mathfrak{R}_t, \text{accept/reject/projection flag}).$$

A verifier update is admissible only if the new verifier can re-check a declared audit subset of past certificates or explicitly records which past certificates are no longer trusted.

Assumption 14.30 (Anti-gaming certificate independence). *The proposal generator and the verifier may share data but not unchecked authority. A candidate that modifies the verifier, pair-set generator, semantic maps, barriers, or estimator code must be evaluated by an independent frozen verifier slice or by a previously certified verifier transition rule. Otherwise verifier self-modification is rejected.*

Proposition 14.31 (Ledger soundness under one-sided certificates). *If every accepted update is stored with a one-sided sound certificate and verifier self-modifications satisfy the anti-gaming certificate-independence assumption, then any later audit of an accepted update can reconstruct the exact residual inequalities that justified acceptance, up to the recorded estimator confidence level.*

Proof. The ledger stores the candidate, estimated residuals, margins, pair set, barriers, verifier identity, and resource state used at acceptance time. One-sided soundness records the implication from estimated nonpositivity to true nonpositivity at the stated confidence level. The anti-gaming assumption prevents an uncertified verifier rewrite from retroactively changing the acceptance rule without an audit trail. Therefore the later audit can replay the recorded certificate and recover the acceptance implication that was used at time t . $\square$

15 Original RCLM–TH Substrate Preserved

The original RCLM–TH architecture provides the Level 2 substrate. It is not discarded; it is refactored.

15.1 Base pipeline

The retained pipeline is

Text, tools, evidence $\rightarrow$ tokens $\rightarrow$ Transformer hidden state $h_t \rightarrow$ register heads $\rightarrow \rho_{t,0}^{\text{ctx}}$
 $\rightarrow \mathcal{R}_{\theta_t} \rightarrow \rho_t^{\text{ctx},*} \rightarrow$ logits and output.

15.2 Objective and subjective projections

The original objective and subjective projections are preserved:

$$\psi^{(O)} = P_O h_t, \quad \psi^{(S)} = P_S h_t.$$

In the final architecture these are two of six register heads, not the entire state representation.

15.3 Trainable Hamiltonian layers

For each recursive linguistic layer l , define a learned Hermitian generator

$$H_{\theta}^{(l)} = R_{\theta}^{(l)} + i \frac{I_{\theta}^{(l)} - (I_{\theta}^{(l)})^{\top}}{2}, \quad H_{\theta}^{(l)} \leftarrow \frac{H_{\theta}^{(l)} + H_{\theta}^{(l)\dagger}}{2}.$$

The associated unitary-like representation update is

$$U_l = \exp(-i H_{\theta}^{(l)} \tau_l), \quad \rho' = U_l \rho U_l^{\dagger}.$$

Limitation 15.1 (Representational, not physical Hamiltonians). *The matrices $H_{\theta}^{(l)}$ are trainable generators in a density-state representation. Unless a physical implementation is separately specified, they are not asserted to be literal physical Hamiltonians.*

15.4 Spectral coherence projection and mixing

Given $\rho' = Q\Lambda Q^\dagger$, define

$$\Lambda_p = \frac{\text{diag}(\lambda_1^{p_l}, \dots, \lambda_d^{p_l})}{\sum_j \lambda_j^{p_l}}, \quad \rho'' = Q\Lambda_p Q^\dagger.$$

Then

$$\mathcal{R}_{\theta,l}(\rho) = (1 - \beta_{\text{mix}})\rho'' + \beta_{\text{mix}}\frac{I}{d}.$$

The full inference operator is a composition over layers and recursion steps.

Proposition 15.2 (Density preservation of one inference layer). *Assume $\rho \in \mathcal{D}(\mathcal{H}_{\text{ctx}})$, U_l is unitary, all eigenvalue powers are applied to nonnegative eigenvalues and renormalized by a positive trace, and $0 \leq \beta_{\text{mix}} \leq 1$. Then*

$$\mathcal{R}_{\theta,l}(\rho) \in \mathcal{D}(\mathcal{H}_{\text{ctx}}).$$

Proof. Unitary conjugation preserves positive semidefiniteness and trace. Spectral powering with nonnegative eigenvalues followed by trace normalization produces a positive semidefinite trace-one matrix. Convex mixing with I/d preserves positive semidefiniteness and trace. Therefore the output is a density state. $\square$

Limitation 15.3 (Nonlinear inference dynamics). *The spectral powering and normalization step is nonlinear in ρ . Therefore density preservation does not imply that $\mathcal{R}_\theta$ is a linear CPTP channel. RCP channel theorems apply to the accepted self-update channel Φ_t , not automatically to $\mathcal{R}_\theta$.*

16 Six-Register RCP Realization in RCLM

This section gives the architecture-specific state construction that was only summarized in the first final-v1 draft. The goal is not to re-prove the RCP Hilbert-space theorem stack. The goal is to specify, in RCLM terms, which objects instantiate the registers and states required by that stack.

16.1 Raw architectural variables

At outer recursive step t , let the architecture have the following observable or auditable variables:

$$\Theta_t = (\theta_t, P_{L,t}, P_{O,t}, P_{S,t}, P_{D,t}, P_{A,t}, P_{M,t}, \{H_{\theta_t}^{(l)}\}_{l=1}^{L_H}, W_{r,t}, W_{o,t}),$$

where θ_t denotes the transformer/backbone parameters, $P_{X,t}$ are register heads, $H_{\theta_t}^{(l)}$ are trainable Hermitian Hamiltonians, and $W_{r,t}, W_{o,t}$ are readout and output maps. Let

$$\Xi_t = (\mathcal{P}_t^{\text{RCLM}}, \{B_{i,t}^{\text{RCLM}}\}_{i \in I_t}, \{\Gamma_{X,t}\}_{X \in \{L,O,S,D,A,M\}}, W_t, \mathfrak{R}_t, \mathcal{M}_t)$$

collect the safety pair set, barriers, semantic maps, verifier, resource ledger, and explicit self-model data.

Definition 16.1 (Context input package). *For a prompt, tool trace, retrieved evidence set, memory slice, and current decoding context, define*

$$\mathcal{I}_t^{\text{ctx}} = (x_{\leq n}, e_t, a_t^{\text{tools}}, m_t^{\text{local}}).$$

The sequence backbone maps this package to hidden states

$$h_{t,1}, \dots, h_{t,n} = F_{\theta_t}^{\text{LM}}(\mathcal{I}_t^{\text{ctx}}).$$

The context summary used for density construction is any auditable pooling map

$$h_t = \text{Pool}(h_{t,1}, \dots, h_{t,n}, e_t, a_t^{\text{tools}}, m_t^{\text{local}}).$$

Definition 16.2 (Register amplitudes). For each $X \in \{L, O, S, D, A, M\}$, define an amplitude vector

$$\psi_{X,t} = f_{X,t}(h_t, m_t^{\text{sys}})$$

where $f_{X,t} = P_{X,t}$ for the basic linear version and may be a certified nonlinear head in an implementation. The self-model head $f_{M,t}$ may depend on the system-level memory m_t^{sys} because it must encode architecture, goals, limitations, update rules, and verifier state rather than only the current text context.

Assumption 16.3 (Auditable register heads). Each register head has an audited input domain and output dimension. If nonlinear heads are used, the verifier must certify the bounds required by the semantic, barrier, and estimator modules. No theorem below assumes that a learned head is semantically correct merely because it is named P_L, P_O, P_S, P_D, P_A , or P_M .

16.2 Register-density construction

Definition 16.4 (Pure register density with fallback). For $X \in \{L, O, S, D, A, M\}$, define

$$\rho_{X,t} = \begin{cases} \frac{\psi_{X,t} \psi_{X,t}^\dagger}{\text{Tr}(\psi_{X,t} \psi_{X,t}^\dagger)}, & \psi_{X,t} \neq 0, \\ I_X/d_X, & \psi_{X,t} = 0. \end{cases}$$

Lemma 16.5 (Register density validity). For every $X \in \{L, O, S, D, A, M\}$, $\rho_{X,t} \in \mathcal{D}(\mathcal{H}_X)$.

Proof. If $\psi_{X,t} = 0$, then I_X/d_X is positive semidefinite and has trace one. If $\psi_{X,t} \neq 0$, then $\psi_{X,t} \psi_{X,t}^\dagger$ is positive semidefinite and has trace $\|\psi_{X,t}\|_2^2 > 0$. Dividing by this positive trace preserves positive semidefiniteness and gives trace one. $\square$

Definition 16.6 (Context tensor state). The maximal explicit context state is

$$\rho_t^{\text{ctx}, \otimes} = \rho_{L,t} \otimes \rho_{O,t} \otimes \rho_{S,t} \otimes \rho_{D,t} \otimes \rho_{A,t} \otimes \rho_{M,t}.$$

To represent learned correlations between registers, the architecture may instead use a certified correlation lift

$$\Lambda_t^{\text{ctx}} : \mathcal{D}(\mathcal{H}_L) \times \dots \times \mathcal{D}(\mathcal{H}_M) \rightarrow \mathcal{D}(\mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M),$$

and set

$$\rho_{t,0}^{\text{ctx}} = \Lambda_t^{\text{ctx}}(\rho_{L,t}, \rho_{O,t}, \rho_{S,t}, \rho_{D,t}, \rho_{A,t}, \rho_{M,t}).$$

If no certified correlation lift is available, the tensor product state is used.

Assumption 16.7 (Correlation-lift validity). Every accepted correlation lift Λ_t^{ctx} is certified to output a density state for every audited input tuple. If Λ_t^{ctx} is learned, the certificate must include positivity, trace-one normalization, and approximation error bounds.

Proposition 16.8 (Context-state validity). *Under Lemma 16.5 and the correlation-lift validity assumption,*

$$\rho_{t,0}^{\text{ctx}} \in \mathcal{D}(\mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M).$$

Proof. The tensor product of density states is a density state. If the tensor product construction is used, the claim follows immediately. If Λ_t^{ctx} is used, the claim is exactly the certified output property of the correlation-lift assumption. $\square$

16.3 System-state construction

Definition 16.9 (System-state payload). *The system-level payload is*

$$\mathcal{I}_t^{\text{sys}} = (\Theta_t, \Xi_t, \rho_t^{\text{ctx},*}, \mathcal{H}_t^{\text{cert}}, \mathcal{H}_t^{\text{update}}),$$

where $\rho_t^{\text{ctx},*}$ is the halted Level 2 context state, $\mathcal{H}_t^{\text{cert}}$ is the certificate ledger, and $\mathcal{H}_t^{\text{update}}$ is the history of accepted and rejected update proposals.

Definition 16.10 (System-state encoder). *A system-state encoder is a map*

$$\text{Enc}_t^{\text{sys}} : \mathcal{I}_t^{\text{sys}} \rightarrow \mathcal{D}(\mathcal{H}_{\text{RCP}} \otimes \mathcal{H}_{\text{arch}}),$$

where

$$\mathcal{H}_{\text{RCP}} = \mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M.$$

The system state is

$$\rho_t^{\text{sys}} = \text{Enc}_t^{\text{sys}}(\mathcal{I}_t^{\text{sys}}).$$

Assumption 16.11 (System-encoder certificate). *For every audited payload $\mathcal{I}_t^{\text{sys}}$, $\text{Enc}_t^{\text{sys}}(\mathcal{I}_t^{\text{sys}})$ is positive semidefinite and trace one. If the encoder is implemented by a low-rank or factor-mixture approximation, its approximation error must be included in every RCP gate estimate that depends on ρ_t^{sys} .*

Theorem 16.12 (Architecture-state construction theorem). *Under the register-density, correlation-lift, and system-encoder assumptions, the RCLM architecture constructs valid context and system states:*

$$\rho_{t,0}^{\text{ctx}} \in \mathcal{D}(\mathcal{H}_{\text{RCP}}), \quad \rho_t^{\text{sys}} \in \mathcal{D}(\mathcal{H}_{\text{RCP}} \otimes \mathcal{H}_{\text{arch}}).$$

Proof. The context-state claim follows from Proposition 16.8. The system-state claim is exactly the system-encoder certificate applied to $\mathcal{I}_t^{\text{sys}}$. $\square$

Limitation 16.13 (State validity is not semantic validity). *Theorem 16.12 proves only density-state validity. It does not prove that the registers actually mean language, objectivity, subjectivity, definitiveness, ambiguity, or self-modeling. Semantic validity requires the separate semantic maps and barriers introduced later.*

16.4 Tractable representations

The formal tensor product may be too large. The architecture may use a low-rank approximation

$$\hat{\rho}_t^{\text{sys}} = \sum_{j=1}^{r_t} q_{j,t} u_{j,t} u_{j,t}^\dagger, \quad q_{j,t} \geq 0, \quad \sum_j q_{j,t} = 1,$$

or a factor-mixture representation

$$\hat{\rho}_t^{\text{sys}} = \sum_{X \in \{L, O, S, D, A, M\}} w_{X\iota_X}(\rho_{X,t}) + \sum_{(X,Y) \in \mathcal{E}_{\text{reg}}} w_{XY\iota_{XY}}(\rho_{XY,t}),$$

where $\mathcal{E}_{\text{reg}}$ is a selected register-pair graph and all weights are nonnegative and sum to one.

Assumption 16.14 (Approximation certificate). *Whenever $\hat{\rho}_t^{\text{sys}}$ is used in place of ρ_t^{sys} , the verifier must account for approximation error in every RCP gate. A bound proved for $\hat{\rho}_t^{\text{sys}}$ is transferred to ρ_t^{sys} only under explicit estimator and approximation error assumptions.*

Remark 16.15 (Why the state construction is architecture-specific). *The updated operational-closure math/physics paper defines the abstract density-state space. This section supplies the non-repetitive RCLM realization: hidden states, register heads, density construction, correlation lifts, payload encoding, and tractable approximations.*

17 Architecture Blueprint

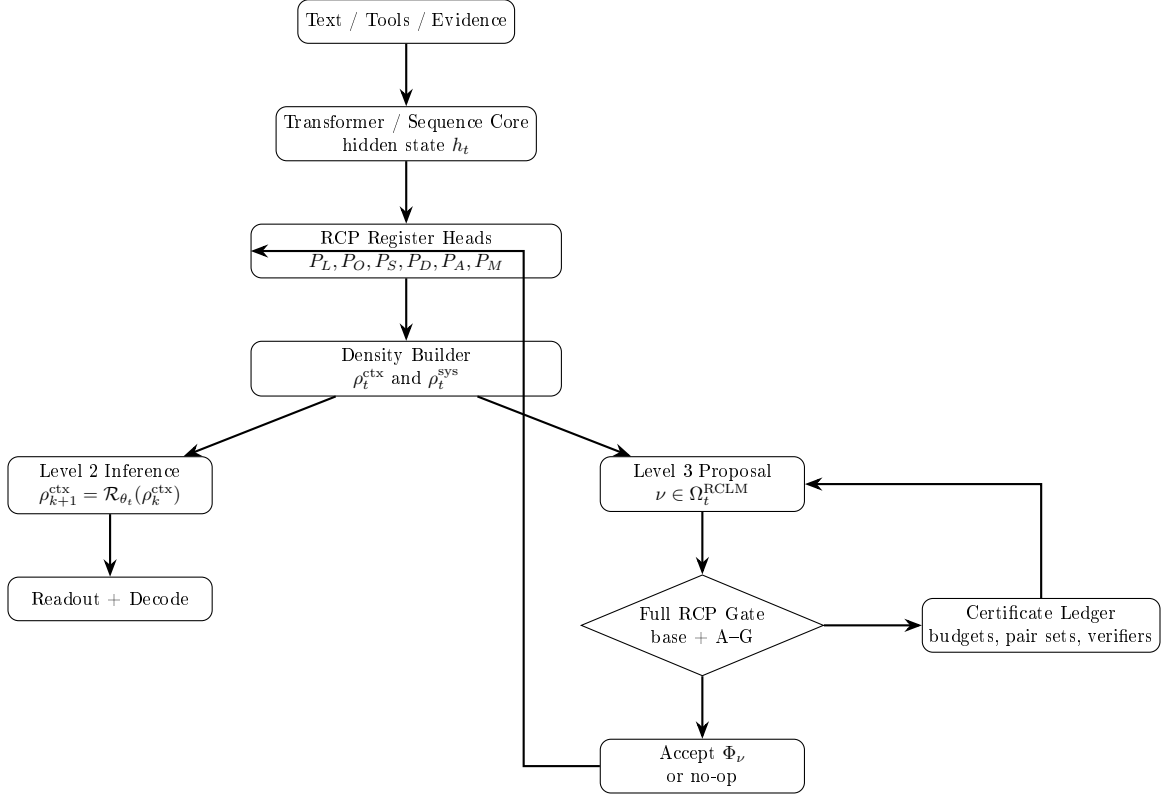


Figure 1: RSI-RCLM final v1. The trainable Hamiltonian recursive loop remains Level 2 inference. Level 3 self-modification is admitted only through the full RCP gate.

18 Level 2 Recursive Coherence Inference

Level 2 is the internal recursive coherence dynamics used during inference. It operates on context states and does not by itself modify the architecture. This section expands the dynamics enough to separate density preservation, monitored descent, and convergence claims.

Definition 18.1 (Hermitian Hamiltonian parameterization). *For each recursive layer l , let $A_t^{(l)}$ be a complex matrix of trainable parameters. The corresponding Hermitian Hamiltonian is*

$$H_{\theta_t}^{(l)} = \frac{1}{2}(A_t^{(l)} + A_t^{(l)\dagger}).$$

For a step size $\tau_l \in \mathbb{R}$, define

$$U_t^{(l)} = \exp(-iH_{\theta_t}^{(l)} \tau_l).$$

Lemma 18.2 (Unitary preservation of density states). *If $\rho \in \mathcal{D}(\mathcal{H})$ and U is unitary, then $U\rho U^\dagger \in \mathcal{D}(\mathcal{H})$.*

Proof. For any vector v , $v^\dagger U\rho U^\dagger v = (U^\dagger v)^\dagger \rho (U^\dagger v) \geq 0$, so the state is positive semidefinite. Also $\text{Tr}(U\rho U^\dagger) = \text{Tr}(\rho U^\dagger U) = \text{Tr}(\rho) = 1$. $\square$

Definition 18.3 (Spectral coherence projection). *Let $\rho = Q \text{diag}(\lambda_1, \dots, \lambda_d) Q^\dagger$ with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. For $p > 0$, define*

$$\text{Pow}_p(\rho) = \begin{cases} Q \text{diag}(\lambda_1^p, \dots, \lambda_d^p) Q^\dagger / \sum_i \lambda_i^p, & \sum_i \lambda_i^p > 0, \\ I/d, & \sum_i \lambda_i^p = 0. \end{cases}$$

For a mixing coefficient $\beta \in [0, 1]$, define the mixed projection

$$\text{Mix}_{p,\beta}(\rho) = (1 - \beta) \text{Pow}_p(\rho) + \beta I/d.$$

Lemma 18.4 (Projection and mixing preserve density states). *If $\rho \in \mathcal{D}_d$, $p > 0$, and $\beta \in [0, 1]$, then $\text{Mix}_{p,\beta}(\rho) \in \mathcal{D}_d$. If $\beta > 0$, then every eigenvalue of $\text{Mix}_{p,\beta}(\rho)$ is at least β/d .*

Proof. Eigenvalue powering preserves nonnegativity and normalization after division by $\sum_i \lambda_i^p$. Therefore $\text{Pow}_p(\rho)$ is positive semidefinite and trace one. A convex combination with I/d is also positive semidefinite and trace one. If $\beta > 0$, the eigenvalues of the mixture are $(1 - \beta)\mu_i + \beta/d$, where $\mu_i \geq 0$, hence at least β/d . $\square$

Definition 18.5 (One-step Level 2 inference map). *Given a layer schedule $l(k)$, exponent $p_{t,k} > 0$, and mixing $\beta_{t,k} \in [0, 1]$, define*

$$\mathcal{R}_{\theta_t,k}(\rho) = \text{Mix}_{p_{t,k},\beta_{t,k}} \left(U_t^{(l(k))} \rho U_t^{(l(k))\dagger} \right).$$

The full internal inference loop is

$$\rho_{t,k+1}^{\text{ctx}} = \mathcal{R}_{\theta_t,k}(\rho_{t,k}^{\text{ctx}}), \quad k = 0, 1, \dots, K_t - 1.$$

The halted state is $\rho_t^{\text{ctx},*} = \rho_{t,K_t^*}^{\text{ctx}}$, where $K_t^* \leq K_t$ is determined by the halting rule.

Proposition 18.6 (Level 2 density preservation). *If $\rho_{t,0}^{\text{ctx}} \in \mathcal{D}(\mathcal{H}_{\text{RCP}})$, then every iterate $\rho_{t,k}^{\text{ctx}}$ produced by the Level 2 loop is in $\mathcal{D}(\mathcal{H}_{\text{RCP}})$.*

Proof. By Lemma 18.2, the unitary part maps density states to density states. By Lemma 18.4, the spectral projection and mixing part maps density states to density states. Induction over k proves the claim. $\square$

Definition 18.7 (Inference monitor). *The Level 2 monitor is*

$$V_{\text{inf}}(\rho) = S(\rho) - a_1 C_{\ell_1}(\rho) - a_2 C_{\text{rel}}(\rho) + a_3 R_{\text{misgrounded}}(\rho) + a_4 R_{\text{collapse}}(\rho) + a_5 R_{\text{amb-collapse}}(\rho),$$

with nonnegative coefficients a_i . The architecture records

$$\Delta V_{\text{inf},k} = V_{\text{inf}}(\rho_{t,k+1}^{\text{ctx}}) - V_{\text{inf}}(\rho_{t,k}^{\text{ctx}}).$$

Assumption 18.8 (Certified Level 2 descent, when claimed). *A claim of Level 2 stability or convergence is permitted only on a certified domain $\mathcal{X}_t^{\text{ctx}}$ for which there exist $\kappa_{\text{inf}} > 0$ and $\eta_{t,k}^{\text{inf}} \geq 0$ satisfying*

$$V_{\text{inf}}(\rho_{t,k+1}^{\text{ctx}}) \leq V_{\text{inf}}(\rho_{t,k}^{\text{ctx}}) - \kappa_{\text{inf}} d(\rho_{t,k+1}^{\text{ctx}}, \rho_{t,k}^{\text{ctx}})^2 + \eta_{t,k}^{\text{inf}}.$$

This condition is a certificate assumption, not a consequence of Hamiltonian evolution alone.

Theorem 18.9 (Conditional Level 2 finite-recursion stability). *Suppose the certified Level 2 descent assumption holds for $k = 0, \dots, K-1$. Then*

$$V_{\text{inf}}(\rho_{t,K}^{\text{ctx}}) \leq V_{\text{inf}}(\rho_{t,0}^{\text{ctx}}) - \kappa_{\text{inf}} \sum_{k=0}^{K-1} d(\rho_{t,k+1}^{\text{ctx}}, \rho_{t,k}^{\text{ctx}})^2 + \sum_{k=0}^{K-1} \eta_{t,k}^{\text{inf}}.$$

In particular,

$$\sum_{k=0}^{K-1} d(\rho_{t,k+1}^{\text{ctx}}, \rho_{t,k}^{\text{ctx}})^2 \leq \frac{V_{\text{inf}}(\rho_{t,0}^{\text{ctx}}) - V_{\text{inf}}(\rho_{t,K}^{\text{ctx}}) + \sum_{k=0}^{K-1} \eta_{t,k}^{\text{inf}}}{\kappa_{\text{inf}}}.$$

Proof. Sum the certified descent inequality over $k = 0, \dots, K-1$. The left side telescopes, giving the first inequality. Rearranging gives the second. $\square$

Definition 18.10 (Halting rule). *The Level 2 loop halts at the first $k \leq K_t$ satisfying one of the certified stopping conditions:*

$$|\Delta V_{\text{inf},k}| \leq \varepsilon_{\text{halt}}, \quad d(\rho_{t,k+1}^{\text{ctx}}, \rho_{t,k}^{\text{ctx}}) \leq \delta_{\text{halt}},$$

or at the hard recursion budget K_t . The architecture logs whether halting was convergence-based or budget-based.

Limitation 18.11 (No unconditional convergence). *The Level 2 map is density-preserving by construction, but density preservation is not convergence. Convergence, descent, or improvement claims require the certified descent assumption or another explicit contraction/compactness argument. This corrects the non-rigorous reading of a recursive Hamiltonian loop as automatically convergent.*

19 Level 3 Candidate Self-Improvement

Level 3 is the self-update channel. It acts on system-level payloads and may change the architecture that later performs Level 2 inference. This section expands the dynamics so that Φ_{ν_t} is not confused with $\mathcal{R}_{\theta_t}$.

Definition 19.1 (Self-update candidate code). *A candidate code $\nu \in \Omega_t^{\text{RCLM}}$ specifies a proposed transformation of some or all of the following components:*

$$\Theta_t, \quad \mathcal{P}_t^{\text{RCLM}}, \quad \{B_{i,t}^{\text{RCLM}}\}_{i \in I_t}, \quad \{\Gamma_{X,t}\}_X, \quad W_t, \quad \mathfrak{R}_t, \quad \mathcal{M}_t, \quad \mathcal{H}_t^{\text{cert}}.$$

Thus ν may update parameters, projections, Hamiltonians, memory, semantic maps, verifier code, barriers, pair-set construction rules, resource estimators, or candidate-generation policies.

Definition 19.2 (Component-level proposed update). *Let T_ν be the component-level transformation*

$$T_\nu : \mathcal{I}_t^{\text{sys}} \mapsto \tilde{\mathcal{I}}_{t+1}^{\text{sys}}(\nu).$$

The corresponding raw successor density is

$$\hat{\rho}_{t+1}^{\text{sys}}(\nu) = \text{Enc}_{t+1}^{\text{sys}}(\tilde{\mathcal{I}}_{t+1}^{\text{sys}}(\nu)).$$

Definition 19.3 (Accepted self-update channel). *A candidate ν induces an accepted self-update channel Φ_ν on the audited state family only when the verifier certifies either:*

- (a) *an exact CPTP representation on that family; or*
- (b) *an approximate implementation with a projection/normalization certificate and an error budget added to every affected RCP inequality.*

The accepted successor is

$$\rho_{t+1}^{\text{sys}} = \Phi_\nu(\rho_t^{\text{sys}}).$$

Assumption 19.4 (No-op and rollback). *The candidate family contains a no-op code ν^0 satisfying*

$$T_{\nu^0}(\mathcal{I}_t^{\text{sys}}) = \mathcal{I}_t^{\text{sys}}, \quad \Phi_{\nu^0}(\rho_t^{\text{sys}}) = \rho_t^{\text{sys}}.$$

The implementation also maintains a rollback record sufficient to restore the previous payload when a proposed update fails post-application verification.

Assumption 19.5 (Projection to valid system state). *If a raw candidate produces $\tilde{\rho}_{t+1}^{\text{sys}} \notin \mathcal{D}(\mathcal{H}_{\text{RCP}} \otimes \mathcal{H}_{\text{arch}})$, the candidate is rejected unless a certified projection*

$$\Pi_{\mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}}(\tilde{\rho}_{t+1}^{\text{sys}}) \in \mathcal{K}_{\text{RCLM},t+1}^{\text{DG}}$$

exists and all projection distortion is included in the RCP gate estimates.

Definition 19.6 (Raw proposal objective). *A raw candidate may be proposed by an approximate optimizer*

$$\nu_t^{\text{raw}} \in \arg \max_{\nu \in \Omega_t^{\text{RCLM}}} \mathbb{E}[\text{Perf}_{t+k} \mid \nu, \rho_t^{\text{sys}}, M_t],$$

but raw performance is only a proposal score. It is not an acceptance criterion.

Definition 19.7 (Accepted update). *The accepted update is selected from the fully augmented RCP–RCLM gate:*

$$\nu_t^* \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}), \quad \rho_{t+1}^{\text{sys}} = \Phi_{\nu_t^*}(\rho_t^{\text{sys}}).$$

If no beneficial certified candidate exists, $\nu_t^ = \nu^0$.*

Proposition 19.8 (Level 3 density validity). *Under the system-encoder, accepted-channel, and projection assumptions, every accepted Level 3 update produces a valid system state:*

$$\rho_{t+1}^{\text{sys}} \in \mathcal{D}(\mathcal{H}_{\text{RCP}} \otimes \mathcal{H}_{\text{arch}}).$$

Proof. If the candidate is certified as an exact accepted channel, density validity follows from the channel certificate. If the candidate is projected, the projection assumption states that the projected state is in the valid safe set and hence in the density space. If no beneficial candidate is certified, the no-op leaves ρ_t^{sys} unchanged and therefore valid. $\square$

Limitation 19.9 (Raw self-editing is not accepted RSI). *A gradient step, architecture rewrite, verifier rewrite, memory edit, or new Hamiltonian proposal is not a valid RSI step merely because it increases predicted performance. It becomes an accepted RSI–RCLM step only after passing the full post-D–G RCP gate.*

20 The Fully Augmented RCP–RCLM Gate

The RCP gate is the engineering point where the operational-closure math/physics theorem stack attaches to the architecture. This section defines the gate as a set of checkable residuals rather than only as a symbolic intersection.

20.1 Base gate

Let $\mathcal{A}_{\text{RCP}}^{0,\text{RCLM}}(t, \rho_t^{\text{sys}})$ be the set of candidates satisfying the base RCP loss, Lyapunov, barrier, ambiguity, and information-bottleneck conditions:

$$\begin{aligned}\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} &\leq \varepsilon_t, \\ \mathbb{E}[V_{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) \mid \rho_t^{\text{sys}}] &\leq V_{\text{RCLM}}(\rho_t^{\text{sys}}) - \kappa_V d(\Phi_\nu(\rho_t^{\text{sys}}), \rho_t^{\text{sys}})^2 + \eta_t, \\ B_{i,t}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) &\leq 0 \quad \forall i \in I_t, \\ U_t(\nu) &\leq \zeta_t, \quad I(Z_{t+1}; Y_{t+1}) \geq I(Z_t; Y_t) - \chi_t.\end{aligned}$$

20.2 Post-A-C gate

The post-A-C gate adds constructive recovery, capability and safe-capability control, safe-improvement existence margins, and recursive feasibility.

Definition 20.1 (Constructive recovery certificate). *A candidate ν supplies a recovery certificate when there exists a recovery map $\mathcal{R}_{\Phi_\nu}^{\text{rec}}$ and a recovery error r_t such that, over the audited safety family $\mathcal{S}_t$,*

$$\sup_{\rho \in \mathcal{S}_t} d(\rho, \mathcal{R}_{\Phi_\nu}^{\text{rec}}(\Phi_\nu(\rho))) \leq r_t.$$

Exact Petz-type recovery is used only under the equality/common-reference hypotheses that justify it. Otherwise recovery is an independently certified architectural object.

Definition 20.2 (Capability functional). *For a task distribution $\mathcal{T}_t$ and scoring family u_τ , define*

$$\mathbb{M}_t(\rho) = \mathbb{E}_{\tau \sim \mathcal{T}_t}[u_\tau(\pi_\rho)].$$

Define safe capability by

$$\text{SafeCap}_t(\rho) = \mathbb{M}_t(\rho) - \lambda_R \text{Risk}_t(\rho).$$

If the task distribution changes with time, progress claims must specify whether gains are absolute, no-op-normalized, or distribution-shift-normalized.

Assumption 20.3 (Capability-coherence calibration). *For the audited architecture class and task distribution, there exist constants $a, b, c, d \geq 0$ and error ω_t such that certified updates satisfy*

$$\Delta \mathbb{M}_t(\nu) \geq a \Delta \mathcal{C}_{\text{RCLM}}(\nu) - b \mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} - c U_t(\nu) - d R_{\text{misgrounded}}(\Phi_\nu(\rho_t^{\text{sys}})) - \omega_t.$$

This is an empirical or architecture-specific calibration assumption, not a general theorem of information theory.

Definition 20.4 (Post-A-C gate). *A candidate belongs to $\mathcal{A}_{\text{RCP}}^{\text{RCLM-AC}}(t, \rho_t^{\text{sys}})$ when it belongs to the base gate and additionally satisfies:*

- (a) *a recovery certificate with error $r_t \leq r_{\text{max},t}$;*
- (b) *safe-capability nondegradation or certified progress;*
- (c) *strict residual margins for all active gate constraints when claiming nontrivial improvement;*
- (d) *membership in the current finite- or infinite-horizon viability kernel if a persistent RSI path is claimed.*

20.3 Post-D–G gate

The post-D–G gate adds semantic validity, self-model fidelity, verifier bootstrapping, resource feasibility, resource-scaling closure, certificate-inflation control, pair-set coverage-gap control, coherence–capability calibration, residual-specific estimator subprotocol completion, and asymptotic regime conditions. No-op monitoring is treated as a diagnostic policy rather than a hard safety gate, because no-op must remain available when no safe beneficial update can be certified.

Definition 20.5 (Semantic interpretation maps). *For each register $X \in \{L, O, S, D, A, M\}$, the architecture provides*

$$\Gamma_X : \mathcal{X}_X^{\text{aud}} \rightarrow \mathcal{Y}_X,$$

where $\mathcal{X}_X^{\text{aud}}$ is an audited domain of internal states and $\mathcal{Y}_X$ is an externally interpretable semantic space.

Definition 20.6 (Semantic validity). *Let $x_X^{\text{true}} \in \mathcal{Y}_X$ be an externally defined target or audit reference. The semantic error is*

$$\text{Val}_X(\rho) = d_X^{\text{sem}}(\Gamma_X(\rho_X), x_X^{\text{true}}).$$

The semantic barrier is

$$B_X^{\text{sem}}(\rho) = \text{Val}_X(\rho) - \delta_X.$$

Semantic validity requires $B_X^{\text{sem}}(\rho) \leq 0$ for all audited registers.

Definition 20.7 (Self-model fidelity). *Let M_t^{int} be the self-model encoded in the model and let M_t^{op} be the audited operational description of architecture, parameters, objectives, verifier, barriers, limitations, and update effects. Self-model fidelity is*

$$d_M(M_t^{\text{int}}, M_t^{\text{op}}) \leq \epsilon_t^M.$$

If the audit is only approximately true, an additional audit error term is included.

Definition 20.8 (Verifier state). *The verifier state W_t includes estimator code, test suites, certificate checkers, semantic audits, pair-set construction procedures, and adversarial search routines. It has soundness $\text{Sound}(W_t)$ and power $\text{Power}(W_t)$.*

Definition 20.9 (Resource state). *The physical/computational resource state is*

$$\mathfrak{R}_t = (E_t, K_t, M_t^{\text{mem}}, T_t, \Sigma_t, D_t^{\text{data}}, V_t^{\text{val}}),$$

where the entries denote energy, compute, memory, time, entropy/irreversibility budget, data, and validation budget.

Definition 20.10 (Full post-D–G architecture gate). *The final v1 RCP–RCLM gate is*

$$\begin{aligned} \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}) = & \mathcal{A}_{\text{RCP}}^{\text{RCLM-AC}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{type}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{coup}}(t, \rho_t^{\text{sys}}) \\ & \cap \mathcal{A}_{\text{cert}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{tr}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{proj-real}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}) \\ & \cap \mathcal{A}_{\text{cov}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{cal}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{est}}(t, \rho_t^{\text{sys}}) \cap \mathcal{A}_{\text{sem}}(t) \cap \mathcal{A}_M(t) \cap \mathcal{A}_{\text{ver}}(t) \\ & \cap \Omega_t^{\text{phys}} \cap \mathcal{A}_{\text{scale}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) \cap \mathcal{A}_{\text{infl}}(t, \rho_t^{\text{sys}}). \end{aligned}$$

Here $\mathcal{A}_{\text{cert}}$ enforces the audited non-oracular certificate protocol, $\mathcal{A}_{\text{trust}}$ enforces verifier trust-graph anti-circularity and no self-sole-certification, $\mathcal{A}_{\text{cov}}$ bounds declared untracked predicate risk, $\mathcal{A}_{\text{cal}}$ licenses coherence–capability progress claims, $\mathcal{A}_{\text{est}}$ requires residual-specific estimator subprotocols, $\mathcal{A}_{\text{sem}}$ enforces semantic validity, $\mathcal{A}_M$ enforces self-model fidelity, $\mathcal{A}_{\text{ver}}$ enforces verifier

soundness and anti-gaming constraints, Ω_t^{phys} enforces one-step resource feasibility, $\mathcal{A}_{\text{scale}}$ enforces audited resource-scaling closure for engine-mode claims, and $\mathcal{A}_{\text{infl}}$ bounds certificate/budget inflation. The Batch-3 algebraic non-loss subgate

$$\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG-NLalg}} = \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}} \cap \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$$

is used when the architecture claims that non-loss and recovery are supplied by the certified conservative-extension algebra rather than by direct residual estimation alone.

20.4 Synchronization with the operational-closure RCP gate

The updated Batch-3 conservative-extension math/physics paper names the abstract operationally closed RCP gate

$$\mathcal{A}_{\text{RCP}}^{\text{op}}(t, \rho_t; g_t^{\min}).$$

That abstract gate is architecture-general and is now interpreted as part of the Conditional Non-Lossy Self-Update Preservation Theorem rather than as a completed direct RSI engine. The same reference paper now also defines a conservative-extension non-lossy update algebra; the RCLM-specific version of that algebra is instantiated in Subsection 1.5. The RSI-RCLM gate above is its RCLM-specific realization. The purpose of this subsection is only synchronization: it records that the architecture paper has not invented a second safety notion, but has supplied concrete objects for the operational components required by the updated reference paper.

Definition 20.11 (RCLM operational instantiation map). *An RCLM operational instantiation map at time t is a bookkeeping map*

$$\text{Inst}_t^{\text{RCLM}} : \mathcal{A}_{\text{RCP}}^{\text{op}}(t, \rho_t; g_t^{\min}) \rightsquigarrow \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$$

which assigns the abstract RCP variables to their architecture-level objects:

$$\begin{aligned} \rho_t &\mapsto \rho_t^{\text{sys}}, & \Phi &\mapsto \Phi_\nu, & \mathcal{P}_t &\mapsto \mathcal{P}_t^{\text{RCLM}}, \\ V &\mapsto V_{\text{RCLM}}, & B_i &\mapsto B_{i,t}^{\text{RCLM}}, & U_t &\mapsto U_t^{\text{RCLM}}, \\ (Z_t, Y_t) &\mapsto (Z_t^{\text{RCLM}}, Y_t^{\text{RCLM}}), & \mathbf{q}_t &\mapsto \mathbf{q}_t^{\text{RCLM}}, & \text{Cert}_t &\mapsto \text{Cert}_t^{\text{RCLM}}. \end{aligned}$$

The map is not a new dynamical assumption. It is a traceability relation between the abstract operational RCP theorem and the concrete RCLM objects defined in this paper.

Assumption 20.12 (Operational synchronization). *For every accepted candidate ν , the instantiation map is domain valid and residual preserving in the following conservative sense: each abstract operational residual in $\mathcal{A}_{\text{RCP}}^{\text{op}}$ is either identical to the corresponding RCLM residual or is upper-bounded by the RCLM residual plus an explicitly recorded approximation margin. Formally, for each abstract residual component $q_{t,j}^{\text{op}}$ there exists a corresponding architecture residual $q_{t,k(j)}^{\text{RCLM}}$ and margin $m_{t,j}^{\text{inst}} \geq 0$ such that*

$$q_{t,j}^{\text{op}}(\Phi_\nu) \leq q_{t,k(j)}^{\text{RCLM}}(\nu) + m_{t,j}^{\text{inst}}.$$

The margin is included in the relevant certificate radius, budget, or gate threshold. If no such mapping or bound is certified, the architecture cannot claim that the candidate instantiates the corresponding abstract operational RCP condition.

Theorem 20.13 (RCLM gate refines the operational RCP gate). *Assume operational synchronization. If*

$$\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$$

and all instantiation margins are included in the audited certificate radii, then the induced update Φ_ν satisfies the abstract operational RCP gate under the instantiation map:

$$\Phi_\nu \in \text{Inst}_t^{\text{RCLM}}(\mathcal{A}_{\text{RCP}}^{\text{op}}(t, \rho_t; g_t^{\min})).$$

Equivalently, accepted RCLM candidates are conservative architecture-level witnesses for the operationally closed RCP admissibility relation, relative to the declared domains, margins, and certificates.

Proof. Membership in $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$ gives membership in each RCLM component gate: post-A-C, type, coupling, certificate, trust, coverage, calibration, estimator, semantic, self-model, verifier, physical-resource, scaling, and inflation. By Assumption 20.12, each corresponding abstract operational residual is bounded by the architecture residual plus a recorded margin. Since those margins are included in the certificate radii or thresholds, nonpositivity of the certified RCLM residuals implies nonpositivity of the abstract operational residuals. Therefore the induced Φ_ν satisfies every component of $\mathcal{A}_{\text{RCP}}^{\text{op}}$ represented by the instantiation map. No conclusion is made about abstract components for which the architecture supplies no domain-valid mapping. $\square$

Limitation 20.14 (Synchronization is not extra safety). *The theorem is a traceability theorem, not an additional safety theorem. It does not make unsafe updates safe. It only states that, after the math/physics paper introduced $\mathcal{A}_{\text{RCP}}^{\text{op}}$, the already-defined RCLM gate is a conservative concrete refinement of that abstract operational gate.*

20.5 Estimator-residual implementation of the gate

Definition 20.15 (Gate residual vector). *For a candidate ν , define the residual vector*

$$\mathbf{q}_t(\nu) = (q_t^{\text{type}}, q_t^{\text{coup}}, q_t^{\text{cert}}, q_t^{\text{trust}}, q_t^{\text{cov}}, q_t^{\text{cal}}, q_t^{\text{est}}, q_t^{\text{NLalg}}, q_t^{\text{loss}}, q_t^{\text{rec}}, q_t^V, q_t^B, q_t^U, q_t^{IB}, q_t^{\text{M}}, q_t^{\text{sem}}, q_t^M, q_t^{\text{ver}}, q_t^{\text{phys}}, q_t^{\text{scale}}, q_t^{\text{infl}}, q_t^{\text{asy}}).$$

where each component is nonpositive exactly when the corresponding gate constraint is satisfied. For example,

$$\begin{aligned} q_t^{\text{type}}(\nu) &= 0 \text{ if } \nu \in \mathcal{A}_{\text{type}}(t, \rho_t^{\text{sys}}), \text{ and } > 0 \text{ otherwise,} \\ q_t^{\text{coup}}(\nu) &= \max_X(\epsilon_{X,t+1}^{\text{coup}} - m_{X,t+1}^{\text{sem}}(\nu)), \\ q_t^{\text{cert}}(\nu) &= 0 \text{ if } \nu \in \mathcal{A}_{\text{cert}}(t, \rho_t^{\text{sys}}), \text{ and } > 0 \text{ otherwise,} \\ q_t^{\text{trust}}(\nu) &= 0 \text{ if } \nu \in \mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}}), \text{ and } > 0 \text{ otherwise,} \\ q_t^{\text{tr}}(\nu) &= 0 \text{ if } \nu \in \mathcal{A}_{\text{tr}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}), \text{ and } > 0 \text{ otherwise,} \\ q_t^{\text{sep}}(\nu) &= 0 \text{ if } \nu \in \mathcal{A}_{\text{sep}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}), \text{ and } > 0 \text{ otherwise,} \\ q_t^{\text{proj-real}}(\nu) &= 0 \text{ if } \nu \in \mathcal{A}_{\text{proj-real}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}), \text{ and } > 0 \text{ otherwise,} \\ q_t^{\text{cov}}(\nu) &= \text{CovGap}_t(\nu) - \gamma_t^{\text{cov}}, \quad q_t^{\text{cal}}(\nu) = e_t(\nu) - e_{\max,t}, \\ q_t^{\text{est}}(\nu) &= \begin{cases} 0, & \text{if all required estimator subprotocols are complete on } \nu, \\ > 0, & \text{otherwise,} \end{cases} \\ q_t^{\text{scale}}(\nu) &= \begin{cases} 0, & \text{if } \nu \in \mathcal{A}_{\text{scale}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}), \\ > 0, & \text{otherwise.} \end{cases} \\ q_t^{\text{infl}}(\nu) &= \text{Infl}_t(\nu) - \iota_t^{\max}, \end{aligned}$$

$$q_t^{\text{NLalg}}(\nu) = \begin{cases} 0, & \text{if no algebraic non-loss claim is made,} \\ 0, & \text{if } \nu \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}(t, \rho_t^{\text{sys}}) \text{ and the claim is made,} \\ > 0, & \text{otherwise.} \end{cases}$$

$$q_t^{\text{loss}}(\nu) = \mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} - \varepsilon_t, \quad q_t^{\text{rec}}(\nu) = r_t - r_{\max, t},$$

$$q_t^B(\nu) = \max_{i \in I_t} B_{i,t}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})), \quad q_t^{\text{sem}}(\nu) = \max_X B_X^{\text{sem}}(\Phi_\nu(\rho_t^{\text{sys}})).$$

The exact definitions of the remaining residuals are the algebraic left-minus-right forms of their gate inequalities.

Definition 20.16 (One-sided estimator certificate). *An estimator $\hat{\mathbf{q}}_t$ with error vector $\Delta_t \geq 0$ is one-sided sound on a candidate set Ω_t^{test} when*

$$\mathbf{q}_t(\nu) \leq \hat{\mathbf{q}}_t(\nu) + \Delta_t \quad \forall \nu \in \Omega_t^{\text{test}}$$

with the stated confidence level. A candidate is accepted only if

$$\hat{\mathbf{q}}_{t,j}(\nu) + \Delta_{t,j} \leq 0 \quad \text{for every residual component } j.$$

Theorem 20.17 (Architecture-level gate soundness). *Assume the one-sided estimator certificate holds for ν . If $\hat{\mathbf{q}}_{t,j}(\nu) + \Delta_{t,j} \leq 0$ for every residual component j , then $\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$.*

Proof. For each component j , one-sided soundness gives $\mathbf{q}_{t,j}(\nu) \leq \hat{\mathbf{q}}_{t,j}(\nu) + \Delta_{t,j} \leq 0$. By construction of the residual vector, all gate inequalities are therefore satisfied. Hence ν belongs to the intersection defining $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$. $\square$

Definition 20.18 (Gate selection objective). *Among certified safe candidates, the architecture may select*

$$\nu_t^* \in \arg \max_{\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})} \left[\Delta \widehat{\text{SafeCap}}_t(\nu) - \lambda_{\text{cost}} \widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}}) - \lambda_{\text{infl}} \widehat{\text{Infl}}_t(\nu) \right].$$

If the feasible set contains no beneficial certified candidate, no-op is selected and the no-op rate monitor is updated. When the Batch-4 tangent-cone module is enabled, the search procedure may use the projected local direction $P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_\nu \text{RSIPot}_t$ to propose a small candidate, but the candidate is accepted only after the same certificate, residual, type, trust, resource, and non-loss checks pass. When the Batch-5 engine-viability module is enabled and a recursive engine claim is made, the selected candidate must also be a witness in the appropriate RCLM engine move set $\mathcal{E}_t^{\text{RCLM-CE}}$ with successor in $\mathcal{V}_{t+1}^{\text{RCLM-engine}, N}$ or $\mathcal{V}_{t+1}^{\text{RCLM-engine}, \infty}$. If the architecture claims an engine-mode improvement rather than merely safe nondegradation, the selected candidate must also belong to the progress-search set $\mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}})$ defined in Section 24.

Remark 20.19 (No reward for uncertified progress). *A candidate with high predicted capability gain but no recovery certificate, no semantic validity, insufficient resources, or verifier-gaming risk is not admissible.*

21 Non-Oracular Certification and Audited Estimator Soundness

The residual gate above is only useful if its certificates are not treated as hidden perfect oracles. This section specifies an architecture-level certificate construction protocol. It does not prove that all required estimators are easy to build for large trained systems. It replaces an unspecified oracle by explicit audit data, stress tests, coverage assumptions, confidence radii, and a theorem showing when audited estimates imply true one-sided gate satisfaction.

Definition 21.1 (Residual index set). Let $\mathcal{J}_t$ be the finite index set of residual components in $\mathbf{q}_t$. For each $j \in \mathcal{J}_t$, the true residual is a map

$$q_{t,j} : \Omega_t^{\text{RCLM}} \rightarrow \mathbb{R}.$$

The convention is that $q_{t,j}(\nu) \leq 0$ means that the j -th gate requirement is truly satisfied by ν . The vector $\mathbf{q}_t(\nu)$ is the ordered collection of these residuals.

Definition 21.2 (Audited candidate domain). The audited candidate domain at time t is a declared subset

$$\Omega_t^{\text{aud}} \subseteq \Omega_t^{\text{RCLM}}$$

containing the no-op candidate ν^0 . A candidate outside Ω_t^{aud} is not accepted by the non-oracular gate, even if an uncertified predictor assigns it high capability gain.

Definition 21.3 (Calibration and stress-test packet). A calibration and stress-test packet is

$$\text{Audit}_t = (\mathcal{D}_t^{\text{cal}}, \mathcal{D}_t^{\text{hold}}, \mathcal{T}_t^{\text{stress}}, \mathcal{U}_t^{\text{adv}}, W_t, \alpha_t),$$

where $\mathcal{D}_t^{\text{cal}}$ is calibration data, $\mathcal{D}_t^{\text{hold}}$ is held-out audit data, $\mathcal{T}_t^{\text{stress}}$ is a family of stress tests, $\mathcal{U}_t^{\text{adv}}$ is an adversarial probe family, W_t is the verifier state, and α_t records the randomization, split, and access policy used to prevent leakage from the held-out audit process into the proposal generator.

Definition 21.4 (Certificate construction protocol). For $\nu \in \Omega_t^{\text{aud}}$, a non-oracular certificate packet is

$$\text{Cert}_t(\nu) = (\hat{\mathbf{q}}_t(\nu), \Delta_t^{\text{aud}}(\nu), \delta_t(\nu), \text{Audit}_t, \text{Trace}_t(\nu), W_t).$$

Here $\hat{\mathbf{q}}_t(\nu)$ is the estimated residual vector, $\Delta_t^{\text{aud}}(\nu) \geq 0$ is the audited one-sided radius vector, $\delta_t(\nu)$ is the declared failure probability, and $\text{Trace}_t(\nu)$ contains estimator versions, data hashes, stress-test identifiers, random seeds or commitments, candidate-generation metadata, and the verifier version used for the decision.

Assumption 21.5 (Audit split and anti-leakage). The proposal generator may use training and public calibration information, but it may not adapt to the unrevealed held-out audit outcomes, random stress seeds, or adversarial probe labels used to validate $\text{Cert}_t(\nu)$. If a candidate modifies the proposal generator, verifier, estimator code, semantic maps, or data-selection process, the certificate must be produced by an independent frozen verifier slice or by a previously certified verifier-transition rule.

Assumption 21.6 (Audited one-sided residual coverage). For each $j \in \mathcal{J}_t$, the certificate protocol produces nonnegative radii

$$\Delta_{t,j}^{\text{aud}}(\nu) = r_{t,j}^{\text{cal}}(\nu) + r_{t,j}^{\text{hold}}(\nu) + r_{t,j}^{\text{stress}}(\nu) + r_{t,j}^{\text{adv}}(\nu) + r_{t,j}^{\text{shift}}(\nu)$$

and a failure probability $\delta_{t,j}$ such that, on an event $\mathcal{E}_{t,j}^{\text{cert}}$ of probability at least $1 - \delta_{t,j}$,

$$q_{t,j}(\nu) \leq \hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) \quad \forall \nu \in \Omega_t^{\text{aud}}.$$

The radii may be conservative. This assumption is the mathematical form of the empirical claim that the calibration data, held-out audits, stress tests, adversarial probes, and distribution-shift model actually upper-bound the estimator error on the audited candidate domain.

Assumption 21.7 (Finite simultaneous failure budget). For each finite horizon T , the declared certificate failure budget satisfies

$$\delta_{0:T}^{\text{cert}} = \sum_{t=0}^{T-1} \sum_{j \in \mathcal{J}_t} \delta_{t,j} < 1.$$

For infinite-horizon claims, the manuscript must explicitly assume $\sum_{t \geq 0} \sum_{j \in \mathcal{J}_t} \delta_{t,j} < \infty$. No almost-sure or long-horizon soundness conclusion is made without such a summability assumption.

Definition 21.8 (Non-oracular certificate gate). *A candidate belongs to $\mathcal{A}_{\text{cert}}(t, \rho_t^{\text{sys}})$ if and only if:*

- (i) $\nu \in \Omega_t^{\text{aud}}$;
- (ii) a complete packet $\text{Cert}_t(\nu)$ exists;
- (iii) the audit split and anti-leakage assumptions are satisfied for this candidate;
- (iv) for every $j \in \mathcal{J}_t$,

$$\hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) \leq 0.$$

This gate is still conditional on the audit-coverage assumptions. It is non-oracular only in the sense that it records how the certificate is constructed and what empirical/statistical assumptions are being used, rather than assuming an unspecified perfect checker.

Theorem 21.9 (One-sided soundness from audited estimators). *Assume Assumptions 21.5–21.7. Fix a time t . With probability at least*

$$1 - \sum_{j \in \mathcal{J}_t} \delta_{t,j},$$

every candidate $\nu \in \mathcal{A}_{\text{cert}}(t, \rho_t^{\text{sys}})$ satisfies

$$q_{t,j}(\nu) \leq 0 \quad \forall j \in \mathcal{J}_t.$$

Consequently, every such candidate satisfies all exact residual inequalities represented by $\mathbf{q}_t$.

Proof. For each residual component j , Assumption 21.6 gives an event $\mathcal{E}_{t,j}^{\text{cert}}$ with probability at least $1 - \delta_{t,j}$ on which

$$q_{t,j}(\nu) \leq \hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) \quad \forall \nu \in \Omega_t^{\text{aud}}.$$

By the union bound, the intersection $\mathcal{E}_t^{\text{cert}} = \bigcap_{j \in \mathcal{J}_t} \mathcal{E}_{t,j}^{\text{cert}}$ has probability at least $1 - \sum_j \delta_{t,j}$. If $\nu \in \mathcal{A}_{\text{cert}}$, then $\nu \in \Omega_t^{\text{aud}}$ and $\hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) \leq 0$ for all j . On $\mathcal{E}_t^{\text{cert}}$, therefore, $q_{t,j}(\nu) \leq 0$ for all j . This is exactly true residual satisfaction. $\square$

Corollary 21.10 (Finite-horizon simultaneous certificate soundness). *If every accepted candidate over times $0, \dots, T-1$ belongs to $\mathcal{A}_{\text{cert}}(t, \rho_t^{\text{sys}})$, then all accepted candidates over that horizon satisfy their true residual inequalities with probability at least*

$$1 - \sum_{t=0}^{T-1} \sum_{j \in \mathcal{J}_t} \delta_{t,j}.$$

Proof. Apply Theorem 21.9 at each time and take a union bound over time and residual components. $\square$

Proposition 21.11 (Connection to architecture-level gate soundness). *On the simultaneous certificate-validity event of Corollary 21.10, membership in $\mathcal{A}_{\text{cert}}(t, \rho_t^{\text{sys}})$ plus the conservative acceptance rule implies the hypothesis of Theorem 20.17.*

Proof. Theorem 20.17 requires one-sided residual soundness: $\mathbf{q}_t(\nu) \leq \widehat{\mathbf{q}}_t(\nu) + \Delta_t$. In the present section, Δ_t is instantiated as Δ_t^{aud} , and Theorem 21.9 proves exactly the required one-sided relation on the certificate-validity event. The conservative acceptance rule then yields true nonpositivity of every residual. $\square$

Limitation 21.12 (Non-oracular does not mean assumption-free). *This section does not prove that the calibration data are representative, that the stress tests are exhaustive, that adversarial probes find every failure mode, or that the radii are tight enough for useful updates to pass. It makes those obligations explicit. If the audit-coverage assumptions fail, the non-oracular certificate theorem does not apply.*

22 Verifier Anti-Circularity and Trust-Graph Stability

The non-oracular certificate protocol still leaves a self-reference risk: an update that changes the verifier, estimator, audit process, semantic map, barrier, or certificate generator might try to certify its own successor. This section adds an explicit trust-graph layer. The point is not to prove that a perfect trust anchor exists. The point is to prevent the architecture theorem from hiding a circular premise. Verifier-changing candidates are accepted only when their certificate support is independent of the part being changed, or when an already-certified transition rule supplies the needed support.

Definition 22.1 (Verifier-relevant component set). *Let*

$$\mathbf{VRel}_t = \{W_t, \widehat{\mathbf{q}}_t, \Delta_t^{\text{aud}}, \mathbf{Audit}_t, \mathbf{Trace}_t, \Gamma_t, \mathbf{B}_t, \mathcal{P}_t^{\text{RCLM}}, \mathfrak{S}_t^{\text{RCLM}}, \Omega_t^{\text{aud}}, \Pi_{\mathcal{K}}, \text{proposal generator}\}$$

be the verifier-relevant components at time t . For a candidate ν , let

$$\mathbf{Aff}_t(\nu) \subseteq \mathbf{VRel}_t$$

be the set of verifier-relevant components whose code, parameters, data access policy, reference objects, or decision thresholds would be modified by ν . If $\mathbf{Aff}_t(\nu) = \emptyset$, the candidate is not verifier-changing.

Definition 22.2 (Verifier trust graph). *A verifier trust graph for candidate ν at time t is a finite directed graph*

$$\mathcal{G}_t^{\text{ver}}(\nu) = (\mathbf{V}_t^{\text{ver}}, \mathbf{E}_t^{\text{ver}}, \mathbf{A}_t^{\text{anchor}}, \mathbf{T}_t(\nu), \mathbf{Ind}_t),$$

where:

- (i) $\mathbf{V}_t^{\text{ver}}$ contains verifier nodes, such as frozen verifier slices, independent audit processes, external tests, cross-checker models, hardware-protected ledgers, and previously certified verifier-transition rules;
- (ii) an edge $a \rightarrow b$ in $\mathbf{E}_t^{\text{ver}}$ means that node a certifies a claim about node b or about an update affecting b ;
- (iii) $\mathbf{A}_t^{\text{anchor}} \subseteq \mathbf{V}_t^{\text{ver}}$ is the declared set of trust-anchor nodes whose soundness is assumed or externally audited at time t ;
- (iv) $\mathbf{T}_t(\nu) \subseteq \mathbf{V}_t^{\text{ver}}$ is the set of nodes corresponding to the verifier-relevant components affected by ν ;
- (v) $\mathbf{Ind}_t(a, b; \nu) = 1$ records that the audit errors, training data, random seeds, held-out tests, and incentives of nodes a and b are independent enough for the certificate claim being made about ν .

The graph is part of the certificate packet and is therefore logged in the certificate ledger.

Definition 22.3 (Trust support set). A trust support set for ν is a subset

$$\text{Supp}_t(\nu) \subseteq \mathbf{V}_t^{\text{ver}}$$

used to certify the verifier-relevant effects of ν . It is called admissible when:

- (a) every support node has a directed certification path to each affected verifier-relevant target in $\mathbf{T}_t(\nu)$;
- (b) no support node lies in $\mathbf{T}_t(\nu)$, unless the edge is produced by an already-certified transition rule from an earlier time;
- (c) the induced certification subgraph used for ν contains no directed cycle in which every node is affected by ν ;
- (d) either $\text{Supp}_t(\nu) \cap \mathbf{A}_t^{\text{anchor}} \neq \emptyset$, or there exist distinct $a, b \in \text{Supp}_t(\nu)$ with $\text{Ind}_t(a, b; \nu) = 1$;
- (e) all edge claims used by the support set have audited one-sided residual certificates with non-positive upper bounds.

Definition 22.4 (No self-sole-certification rule). A candidate ν satisfies the no self-sole-certification rule if either $\text{Aff}_t(\nu) = \emptyset$, or there exists an admissible trust support set $\text{Supp}_t(\nu)$ such that the certificate for ν is not generated solely by a verifier-relevant component modified by ν . Equivalently, verifier-changing updates must be certified by at least one independent support path not contained entirely in the components being rewritten.

Definition 22.5 (Trust-graph residuals). The trust-graph residual vector is

$$\mathbf{r}_t^{\text{trust}}(\nu) = (r_t^{\text{graph}}(\nu), r_t^{\text{sole}}(\nu), r_t^{\text{edge}}(\nu), r_t^{\text{sound}}(\nu), r_t^{\text{power}}(\nu)),$$

where the entries are nonpositive exactly when, respectively, the declared trust graph is well formed, the no self-sole-certification rule holds, all support-edge certificates pass, the successor verifier satisfies the certified soundness degradation bound, and the successor verifier satisfies the certified power-degradation or power-improvement claim. The aggregate trust residual is

$$q_t^{\text{trust}}(\nu) = \max\{r_t^{\text{graph}}(\nu), r_t^{\text{sole}}(\nu), r_t^{\text{edge}}(\nu), r_t^{\text{sound}}(\nu), r_t^{\text{power}}(\nu)\}.$$

Assumption 22.6 (Trust-graph audit soundness). For each audited candidate $\nu \in \Omega_t^{\text{aud}}$, the trust-graph certificate produces a one-sided radius vector such that, on an event $\mathcal{E}_t^{\text{trust}}$ of probability at least $1 - \delta_t^{\text{trust}}$, the true trust residual is bounded above by its audited estimate:

$$q_t^{\text{trust}}(\nu) \leq \hat{q}_t^{\text{trust}}(\nu) + \Delta_t^{\text{trust}}(\nu) \quad \forall \nu \in \Omega_t^{\text{aud}}.$$

This assumption includes the correctness of graph parsing, independence declarations, support-edge audit outcomes, and the claimed relationship between support-edge certificates and verifier soundness/power residuals. It is not derived from the verifier being updated.

Assumption 22.7 (Verifier transition degradation budgets). For accepted verifier-relevant updates, nonpositive trust residuals imply

$$\text{Sound}(W_{t+1}) \geq \text{Sound}(W_t) - \omega_t^{\text{ver}}, \quad \text{Power}(W_{t+1}) \geq \text{Power}(W_t) - \pi_t^{\text{ver}}.$$

If the update claims positive verifier-power improvement, the claimed increment must be stated separately and is valid only on the audited task family used to define Power. No uniform positive improvement is claimed on a fixed bounded power scale for an unbounded horizon.

Definition 22.8 (Trust-admissible verifier gate). *A candidate belongs to*

$$\mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}})$$

if and only if:

- (i) $\nu \in \Omega_t^{\text{aud}}$;
- (ii) a verifier trust graph $\mathcal{G}_t^{\text{ver}}(\nu)$ and support set $\text{Supp}_t(\nu)$ are included in $\text{Cert}_t(\nu)$;
- (iii) the no self-sole-certification rule is satisfied;
- (iv) $\widehat{q}_t^{\text{trust}}(\nu) + \Delta_t^{\text{trust}}(\nu) \leq 0$.

If $\text{Aff}_t(\nu) = \emptyset$, the trust graph may certify that no verifier-relevant component is modified; if $\text{Aff}_t(\nu) \neq \emptyset$, independent support is required.

Theorem 22.9 (No self-sole-certified verifier update). *Assume $\nu \in \mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}})$. If $\text{Aff}_t(\nu) \neq \emptyset$, then the acceptance certificate for ν includes an admissible support set that is not contained entirely in the components modified by ν . Therefore ν is not accepted solely on the authority of the verifier component it rewrites.*

Proof. By Definition 22.8, membership in $\mathcal{A}_{\text{trust}}$ requires a trust graph, a support set, and satisfaction of the no self-sole-certification rule. By Definition 22.4, a verifier-changing candidate must have at least one independent support path not contained entirely in the affected verifier-relevant components. Hence sole self-certification is excluded by construction. $\square$

Theorem 22.10 (Verifier self-consistency stability). *Let $\nu_0, \dots, \nu_{T-1}$ be accepted candidates with $\nu_t \in \mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}})$. Assume the trust-graph audit soundness events $\mathcal{E}_t^{\text{trust}}$ hold for all $t < T$, and assume the verifier transition degradation budgets of Assumption 22.7. Then:*

- (i) no verifier-changing accepted update over the horizon is sole-self-certified;
- (ii) verifier soundness satisfies

$$\text{Sound}(W_T) \geq \text{Sound}(W_0) - \sum_{t=0}^{T-1} \omega_t^{\text{ver}};$$

- (iii) verifier power satisfies

$$\text{Power}(W_T) \geq \text{Power}(W_0) - \sum_{t=0}^{T-1} \pi_t^{\text{ver}}.$$

Proof. Item (i) follows by applying Theorem 22.9 at each time $t < T$. On the trust-soundness event, $\nu_t \in \mathcal{A}_{\text{trust}}$ gives $q_t^{\text{trust}}(\nu_t) \leq 0$, so the soundness and power residuals are nonpositive. By Assumption 22.7,

$$\text{Sound}(W_{t+1}) \geq \text{Sound}(W_t) - \omega_t^{\text{ver}}, \quad \text{Power}(W_{t+1}) \geq \text{Power}(W_t) - \pi_t^{\text{ver}}.$$

Telescoping the first inequality from 0 to $T-1$ yields item (ii), and telescoping the second yields item (iii). $\square$

Corollary 22.11 (Probabilistic finite-horizon trust soundness). *If $\Pr(\mathcal{E}_t^{\text{trust}}) \geq 1 - \delta_t^{\text{trust}}$, then the conclusions of Theorem 22.10 hold with probability at least*

$$1 - \sum_{t=0}^{T-1} \delta_t^{\text{trust}},$$

provided the non-oracular certificate-validity events used in Section 21 also hold.

Proof. Apply the union bound to the events $\mathcal{E}_t^{\text{trust}}$ and then condition on the simultaneous non-oracular certificate-validity events. On that intersection, the deterministic hypotheses of Theorem 22.10 are satisfied. $\square$

Limitation 22.12 (Trust graphs are not magic trust). *The trust graph does not create soundness from nothing. It prevents a verifier from being the sole certifier of its own successor under the stated graph and independence assumptions. The existence of externally valid anchors, independent support, representative stress tests, and conservative radii remains an architecture and implementation obligation.*

23 Batch-10 Adaptive-Audit Trust and Common-Mode Risk Gate

The trust graph in Section 22 prevents the simplest circularity: a verifier-changing update cannot be certified solely by the verifier component it rewrites. A realistic adaptive system can still fail through correlated verifier errors, shared training data, public stress-test overfitting, audit leakage, collusive support nodes, or common-mode blind spots. This section adds an adversarial audit layer. It does not prove that independent verifiers exist; it defines the additional certificates required before the architecture may claim robustness against adaptive verifier gaming.

Definition 23.1 (Correlated verifier failure). *For verifier or audit nodes $W_i, W_j \in \mathbf{V}_t^{\text{ver}}$, let*

$$\text{CorrFail}_t(W_i, W_j) \in [0, 1]$$

be an audited upper bound on the probability or normalized risk that W_i and W_j fail on the same candidate family because of shared data, shared architecture, shared reward model, shared blind spot, common implementation dependency, or common adversarial pressure. The value is not inferred from graph distinctness alone; it must be estimated, bounded, or externally audited.

Definition 23.2 (Audit leakage risk). *For a candidate ν , define*

$$\text{AuditLeak}_t(\nu) \in [0, 1]$$

as an audited upper bound on the risk that ν , its proposal generator, or a verifier component affected by ν had improper access to held-out audit labels, unrevealed random stress seeds, adversarial probe labels, private red-team prompts, or post-selection evaluation outcomes before the certificate was produced.

Definition 23.3 (Common-mode and collusion risk). *For a trust support set $\text{Supp}_t(\nu)$, define the common-mode risk*

$$\text{CommonMode}_t(\text{Supp}_t(\nu)) = \max \left\{ \sup_{i \neq j \in \text{Supp}_t(\nu)} \text{CorrFail}_t(W_i, W_j), \widehat{C}_t^{\text{shared}}(\text{Supp}_t(\nu)) \right\},$$

where $\widehat{C}_t^{\text{shared}}$ is an audited shared-dependency score. Define the collusion risk

$$\text{CollusionRisk}_t(\text{Supp}_t(\nu)) \in [0, 1]$$

as an audited upper bound on the risk that the support nodes are not decision-independent because of common control, incentives, communication, model-merging, shared hidden state, or candidate-induced coordination.

Definition 23.4 (Adaptive trust thresholds). *Let*

$$\gamma_t^{\text{corr}}, \quad \gamma_t^{\text{leak}}, \quad \gamma_t^{\text{cm}}, \quad \gamma_t^{\text{coll}}$$

be the maximum accepted correlated-failure, audit-leakage, common-mode, and collusion risks. These thresholds are gate parameters and must be recorded in the certificate ledger. Tightening or relaxing them is a verifier/gate-affecting update and must itself satisfy the trust gate.

Definition 23.5 (Adaptive-audit certificate). *An adaptive-audit certificate for ν is*

$$\text{AdvAudit}_t(\nu) = (\widehat{\text{CorrFail}}_t, \widehat{\text{AuditLeak}}_t, \widehat{\text{CommonMode}}_t, \widehat{\text{CollusionRisk}}_t, \Delta_t^{\text{advtrust}}, \delta_t^{\text{advtrust}}, \text{AuditIndTrace}_t).$$

where $\Delta_t^{\text{advtrust}}$ is a vector of one-sided radii, $\delta_t^{\text{advtrust}}$ is the failure probability, and AuditIndTrace_t records data-lineage, held-out split, stress-test secrecy, verifier-dependency, and support-node independence evidence.

Assumption 23.6 (Adaptive-audit one-sided soundness). *For every audited candidate $\nu \in \Omega_t^{\text{aud}}$, with probability at least $1 - \delta_t^{\text{advtrust}}$, the true adaptive trust risks are bounded by their audited estimates plus radii:*

$$\text{CorrFail}_t(W_i, W_j) \leq \widehat{\text{CorrFail}}_t(W_i, W_j) + \Delta_{t,ij}^{\text{corr}},$$

$$\text{AuditLeak}_t(\nu) \leq \widehat{\text{AuditLeak}}_t(\nu) + \Delta_t^{\text{leak}},$$

$$\text{CommonMode}_t(\text{Supp}_t(\nu)) \leq \widehat{\text{CommonMode}}_t(\text{Supp}_t(\nu)) + \Delta_t^{\text{cm}},$$

$$\text{CollusionRisk}_t(\text{Supp}_t(\nu)) \leq \widehat{\text{CollusionRisk}}_t(\text{Supp}_t(\nu)) + \Delta_t^{\text{coll}}.$$

This is an empirical/audit assumption, not a theorem implied by graph topology.

Definition 23.7 (Adaptive trust gate). *A candidate belongs to the adaptive trust gate*

$$\mathcal{A}_{\text{trust}}^{\text{adv}}(t, \rho_t^{\text{sys}})$$

when all of the following hold:

- (i) $\nu \in \mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}})$;
- (ii) $\text{AdvAudit}_t(\nu)$ is present in the certificate packet;
- (iii) for all distinct $W_i, W_j \in \text{Supp}_t(\nu)$,

$$\widehat{\text{CorrFail}}_t(W_i, W_j) + \Delta_{t,ij}^{\text{corr}} \leq \gamma_t^{\text{corr}};$$

(iv)

$$\widehat{\text{AuditLeak}}_t(\nu) + \Delta_t^{\text{leak}} \leq \gamma_t^{\text{leak}};$$

(v)

$$\widehat{\text{CommonMode}}_t(\text{Supp}_t(\nu)) + \Delta_t^{\text{cm}} \leq \gamma_t^{\text{cm}};$$

(vi)

$$\widehat{\text{CollusionRisk}}_t(\text{Supp}_t(\nu)) + \Delta_t^{\text{coll}} \leq \gamma_t^{\text{coll}}.$$

Definition 23.8 (Adaptive trust residual). *Define*

$$q_t^{\text{trustadv}}(\nu) = \max \left\{ \begin{aligned} &\sup_{i \neq j \in \text{Supp}_t(\nu)} (\text{CorrFail}_t(W_i, W_j) - \gamma_t^{\text{corr}}), \\ &\text{AuditLeak}_t(\nu) - \gamma_t^{\text{leak}}, \\ &\text{CommonMode}_t(\text{Supp}_t(\nu)) - \gamma_t^{\text{cm}}, \\ &\text{CollusionRisk}_t(\text{Supp}_t(\nu)) - \gamma_t^{\text{coll}} \end{aligned} \right\}.$$

The residual is nonpositive exactly when the adaptive-audit risk thresholds are satisfied.

Theorem 23.9 (Adaptive-audit trust soundness). *Assume adaptive-audit one-sided soundness. If*

$$\nu \in \mathcal{A}_{\text{trust}}^{\text{adv}}(t, \rho_t^{\text{sys}}),$$

then, on the adaptive-audit soundness event,

$$q_t^{\text{trustadv}}(\nu) \leq 0.$$

Consequently, if ν is verifier-changing, it is not solely self-certified and its correlated-failure, audit-leakage, common-mode, and collusion risks are bounded by their declared thresholds.

Proof. Membership in $\mathcal{A}_{\text{trust}}^{\text{adv}}$ implies membership in $\mathcal{A}_{\text{trust}}$, so Theorem 22.9 excludes sole self-certification for verifier-changing candidates. The same membership also gives the four conservative audited inequalities in Definition 23.7. On the adaptive-audit soundness event, each true risk is bounded above by its audited estimate plus radius. Therefore every term in the maximum defining q_t^{trustadv} is nonpositive, proving the residual bound and the declared risk bounds. $\square$

Corollary 23.10 (Finite-horizon adaptive-trust accounting). *If accepted verifier-relevant updates satisfy $\nu_t \in \mathcal{A}_{\text{trust}}^{\text{adv}}(t, \rho_t^{\text{sys}})$ for $t < T$, then the adaptive trust bounds hold for all accepted verifier-relevant updates over the horizon with probability at least*

$$1 - \sum_{t=0}^{T-1} \delta_t^{\text{advtrust}},$$

intersected with the ordinary trust-graph and non-oracular certificate-validity events.

Limitation 23.11 (Adaptive trust remains empirical). *The adaptive trust gate is stronger than no self-sole-certification, but it still depends on audit quality. If all support nodes share an unmodeled blind spot, if leakage is undetected, or if collusion risk is underestimated, the theorem does not apply. The module makes adaptive-audit assumptions explicit; it does not create independent trust anchors from notation.*

24 Persistent Progress and Budgeted Certified Search

The preceding gate can certify safety-preserving self-updates. A safe filter, however, is not yet an RSI engine. To move from “reject unsafe changes” toward “find safe beneficial changes,” the architecture needs a separate progress/search layer. This layer does not weaken the RCP gate. It defines when the architecture is allowed to claim nontrivial progress, and what assumptions are needed for a finite-budget search process to find such a candidate.

Remark 24.1 (Safety gate versus engine-mode progress gate). *The safety gate admits no-op and safe nondegrading candidates. The engine-mode progress gate is used only when the architecture claims a nontrivial RSI improvement. If the progress gate is empty, the architecture must select no-op or defer self-modification; it may not relax safety constraints to force progress. This distinction prevents no-op fallback from being misrepresented as capability progress and prevents benchmark relabeling from being treated as an improvement theorem.*

Definition 24.2 (Safety-admissible candidate set). *The safety-admissible candidate set at time t is*

$$\mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) := \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}).$$

This set includes no-op whenever the standing no-op feasibility, type, coupling, certificate, trust, semantic, self-model, verifier, and resource assumptions are satisfied by no-op. Membership in $\mathcal{A}_t^{\text{safe}}$ is a safety statement, not a progress statement.

Definition 24.3 (No-op-normalized safe-capability increment). *Let ν_t^0 denote the certified no-op candidate. For a candidate ν , define*

$$\Delta \text{SafeCap}_t^0(\nu) = \text{SafeCap}_{t+1}(\Phi_\nu(\rho_t^{\text{sys}})) - \text{SafeCap}_{t+1}(\Phi_{\nu_t^0}(\rho_t^{\text{sys}})).$$

The right-hand side evaluates both the candidate and no-op on the same successor-time scoring convention SafeCap_{t+1} . Thus a positive value cannot be obtained merely by changing the task distribution between t and $t+1$. This quantity is equal to $G_t^{\text{noop}}(\nu)$ from Definition 27.3, when that definition is invoked later in the Modules A–C section.

Definition 24.4 (Beneficial safe-improvement set). *Let $\epsilon_t^{\text{prog}} > 0$ be the progress threshold at time t . The beneficial safe-improvement set is*

$$\mathcal{I}_t = \left\{ \nu \in \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) : \Delta \text{SafeCap}_t^0(\nu) > \epsilon_t^{\text{prog}} \right\}.$$

The threshold ϵ_t^{prog} is distinct from the relative-entropy loss budget ϵ_t . A candidate in $\mathcal{I}_t$ is both safe-admissible and no-op-normalized beneficial. The architecture may claim a nontrivial RSI step at time t only if the selected candidate belongs to $\mathcal{I}_t$ or to a certified conservative subset of it.

Definition 24.5 (Margin-strengthened beneficial set). *For a safety margin $m_t^{\text{safe}} > 0$ and a progress margin $m_t^{\text{prog}} > 0$, let*

$$\mathcal{I}_t^m = \left\{ \nu \in \Omega_t^{\text{aud}} : \mathbf{q}_{t,j}(\nu) \leq -m_t^{\text{safe}} \ \forall j \in \mathcal{J}_t, \quad \Delta \text{SafeCap}_t^0(\nu) \geq \epsilon_t^{\text{prog}} + m_t^{\text{prog}} \right\}.$$

This set is not directly observable in large learned systems. It is the true target set that a conservative audited search procedure attempts to discover.

Definition 24.6 (Budgeted candidate search). *A budgeted search procedure is a map*

$$\text{Search}_t(B_t^{\text{search}}, \rho_t^{\text{sys}}, W_t) = S_t(B_t^{\text{search}}) \subseteq \Omega_t^{\text{aud}},$$

where B_t^{search} is a declared search budget measured in compute, validation calls, memory, time, data, and verifier invocations. The search output is finite. Every candidate returned by the search must still pass the non-oracular certificate gate and trust-graph gate before being accepted.

Definition 24.7 (Certified progress-search gate). *Let $\widehat{G}_t^0(\nu)$ be an estimator of $\Delta \text{SafeCap}_t^0(\nu)$, and let $\Delta_t^G(\nu) \geq 0$ be a one-sided lower-bound radius satisfying the soundness condition stated below. The certified progress-search gate is*

$$\mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) = \left\{ \nu \in S_t(B_t^{\text{search}}) \cap \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) : \widehat{G}_t^0(\nu) - \Delta_t^G(\nu) > \epsilon_t^{\text{prog}} \right\}.$$

A candidate in this set has a conservative audited certificate of positive no-op-normalized safe-capability gain, provided the gain-estimator event below holds.

Assumption 24.8 (One-sided gain-estimator soundness). *For each audited candidate $\nu \in \Omega_t^{\text{aud}}$, there is an event $\mathcal{E}_t^G$ of probability at least $1 - \delta_t^G$ on which*

$$\Delta \text{SafeCap}_t^0(\nu) \geq \widehat{G}_t^0(\nu) - \Delta_t^G(\nu) \quad \forall \nu \in \Omega_t^{\text{aud}}.$$

The radius Δ_t^G must include calibration error, held-out evaluation error, distribution-shift error, adversarial-test error, and scoring-model uncertainty. No progress claim is made outside the audited domain.

Assumption 24.9 (Certified search completeness with budget). *For each time t , budget B_t^{search} , and audited domain Ω_t^{aud} , there is a failure probability δ_t^{search} such that, on an event $\mathcal{E}_t^{\text{search}}$ of probability at least $1 - \delta_t^{\text{search}}$, the following implication holds:*

$$\mathcal{I}_t^m \neq \emptyset \implies \exists \nu \in S_t(B_t^{\text{search}})$$

such that the candidate has complete conservative certificates satisfying

$$\widehat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) \leq 0 \quad \forall j \in \mathcal{J}_t, \quad \widehat{G}_t^0(\nu) - \Delta_t^G(\nu) > \epsilon_t^{\text{prog}}.$$

This is stronger than merely searching a neighborhood: it is a search-and-certification completeness assumption for margin-qualified useful candidates. It is not a theorem from RCP alone. It may be justified in an implementation by exhaustive enumeration over a finite candidate class, branch-and-bound with certified pruning, randomized search with a coverage bound plus calibrated confidence sequences, or an externally audited search-completeness certificate. The assumption fails if the budget is too small, if the candidate class excludes the useful update, if the estimator is too pessimistic to certify a truly useful candidate, or if the search procedure can be adversarially steered away from the useful region.

Assumption 24.10 (Persistent improvement condition). *For a declared engine horizon T and times $t < T$, suppose the state lies in an engine-viability region $\mathcal{V}_t^{\text{eng}} \subseteq \mathcal{K}_{\text{RCLM},t}^{\text{DG}}$ and the resource state satisfies a search-feasibility condition*

$$\mathfrak{R}_t \succeq \mathfrak{R}_{\min,t}^{\text{search}}.$$

The persistent improvement condition states that

$$\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{eng}} \implies \mathcal{I}_t^m \neq \emptyset.$$

This is the architecture-level existence premise needed to treat the system as an engine rather than only as a safety filter. It is not derived from Lyapunov stability, relative-entropy non-loss, or semantic validity alone.

Definition 24.11 (Engine-mode admissibility). *The engine-mode admissible set is*

$$\mathcal{A}_{\text{eng}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) = \mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}).$$

The architecture may still choose no-op from $\mathcal{A}_t^{\text{safe}}$ when $\mathcal{A}_{\text{eng}}$ is empty, but it may not describe that no-op as a nontrivial RSI improvement. Thus the safety gate and the engine-mode progress gate play different logical roles.

Theorem 24.12 (Budgeted search finds certified beneficial updates under stated assumptions). *Assume at time t :*

- (a) *the one-sided certificate events for the safety residuals hold;*
- (b) *the gain-estimator event $\mathcal{E}_t^G$ holds;*

(c) the search-completeness event $\mathcal{E}_t^{\text{search}}$ holds;

(d) $\mathcal{I}_t^m \neq \emptyset$.

Then $\mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) \neq \emptyset$. Any candidate $\nu \in \mathcal{A}_{\text{prog}}$ satisfies

$$\nu \in \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) \quad \text{and} \quad \Delta \text{SafeCap}_t^0(\nu) > \epsilon_t^{\text{prog}}$$

on the stated certificate-validity events.

Proof. By Assumption 24.9 and $\mathcal{I}_t^m \neq \emptyset$, there exists a searched candidate $\nu \in S_t(B_t^{\text{search}})$ whose conservative safety residuals satisfy

$$\hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) \leq 0 \quad \forall j \in \mathcal{J}_t$$

and whose conservative no-op-normalized gain certificate satisfies

$$\hat{G}_t^0(\nu) - \Delta_t^G(\nu) > \epsilon_t^{\text{prog}}.$$

The first display, together with the one-sided residual soundness event, implies $\nu \in \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}})$. The second display and Assumption 24.8 imply

$$\Delta \text{SafeCap}_t^0(\nu) \geq \hat{G}_t^0(\nu) - \Delta_t^G(\nu) > \epsilon_t^{\text{prog}}.$$

Thus $\nu \in \mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}})$, proving nonemptiness and the stated safety/progress properties. The proof uses the explicit search-and-certification completeness hypothesis; it does not infer a conservative certificate merely from true usefulness. $\square$

Corollary 24.13 (Finite-horizon engine progress). *Fix a finite horizon T . Suppose the persistent improvement condition holds for all $t < T$, the search and certificate events in Theorem 24.12 hold for all $t < T$, and the selected candidate $\nu_t^* \in \mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}})$ whenever the engine-mode set is nonempty. Then, with probability at least*

$$1 - \sum_{t=0}^{T-1} \left(\delta_t^G + \delta_t^{\text{search}} + \sum_{j \in \mathcal{J}_t} \delta_{t,j} + \delta_t^{\text{trust}} \right),$$

all selected non-no-op engine-mode updates over the horizon are safety-admissible and satisfy

$$\Delta \text{SafeCap}_t^0(\nu_t^*) > \epsilon_t^{\text{prog}}.$$

If, in addition, the scoring convention is fixed over the horizon or explicitly no-op-normalized at each step, then the safe-capability gains telescope according to the conditional safe-capability theorem in Module B.

Proof. Apply Theorem 24.12 at each time and union bound over gain, search, residual-certificate, and trust-soundness failure events. The telescope statement is exactly the finite-horizon safe-capability telescope applied to the one-step gains certified here. No infinite-horizon or fixed-scale uniform progress conclusion follows. $\square$

Limitation 24.14 (Progress remains conditional). *This section turns persistent progress into a precise condition and gives a theorem showing how budgeted search can find a beneficial safe update under that condition. It does not prove that $\mathcal{I}_t$ is nonempty for arbitrary RCLM architectures, that the search budget scales favorably, or that useful safe-improvement neighborhoods persist indefinitely. Those claims require implementation-specific construction, empirical calibration, and the resource-scaling/no-op-pressure controls of Section 25.*

25 Resource-Scaling Closure and No-Op Pressure Control

The progress-search theorem of Section 24 still leaves two practical failure modes. First, certification may become more expensive than the useful update it is meant to validate. Second, no-op fallback can become a stable equilibrium, or the system may respond by loosening certificates in order to escape stagnation. This section adds an explicit resource-scaling closure condition, a no-op-rate monitor, and a certificate-inflation penalty. These mechanisms do not prove that frontier-scale certification is cheap. They state the exact assumptions under which the architecture may claim that progress search remains resource-feasible without silently relaxing the safety gate.

Remark 25.1 (Why this is not a scalability theorem from first principles). *The RCP theorem stack gives conditional preservation once admissible updates are selected. It does not imply that all residuals can be estimated at low cost, that candidate search is polynomial, or that verification costs are sublinear in system size. The present section therefore treats resource scaling as an additional audited condition, not as a consequence of Lyapunov stability or relative-entropy non-loss.*

Definition 25.2 (Architecture cost accounts). *For a candidate update ν and search budget B_t^{search} , define nonnegative cost accounts*

$$C_t^{\text{search}}(B_t^{\text{search}}), \quad C_t^{\text{cert}}(\nu), \quad C_t^{\text{verify}}(\nu), \quad C_t^{\text{rec}}(\nu), \quad C_t^{\text{apply}}(\nu), \quad C_t^{\text{rollback}}(\nu), \quad C_t^{\text{aud}}(\nu).$$

The total certification-and-application cost is

$$\begin{aligned} C_t^{\text{tot}}(\nu; B_t^{\text{search}}) &= C_t^{\text{search}}(B_t^{\text{search}}) + C_t^{\text{cert}}(\nu) + C_t^{\text{verify}}(\nu) + C_t^{\text{rec}}(\nu) \\ &\quad + C_t^{\text{apply}}(\nu) + C_t^{\text{rollback}}(\nu) + C_t^{\text{aud}}(\nu). \end{aligned}$$

Let

$$R_t^{\text{avail}} := \text{Budget}(\mathfrak{R}_t)$$

be the scalar budget extracted from the resource state. This scalar may be a conservative normalized aggregate of energy, compute, memory, time, validation data, and irreversibility budget.

Assumption 25.3 (Audited cost upper bounds). *For each accepted candidate, the architecture supplies an audited cost estimate $\widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}})$ and a nonnegative cost radius $\Delta_t^C(\nu)$ such that, on an event $\mathcal{E}_t^C$ of probability at least $1 - \delta_t^C$,*

$$C_t^{\text{tot}}(\nu; B_t^{\text{search}}) \leq \widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}}) + \Delta_t^C(\nu) \quad \forall \nu \in S_t(B_t^{\text{search}}).$$

The radius must include profiler error, implementation overhead, verifier invocations, rollback overhead, stress-test cost, and distribution-shift uncertainty in the cost model.

Definition 25.4 (Resource-scaling admissibility). *Let $G_t^{\text{res}}(\nu) \geq 0$ denote an audited resource-return lower bound associated with applying ν , such as reusable verifier improvement, reusable pair-set coverage, reusable search pruning, or compute-efficiency gain. Let $\alpha_t \in [0, 1)$, $\beta_t \geq 0$, and $\xi_t^{\text{scale}} \geq 0$ be declared scaling tolerances. The resource-scaling admissible set is*

$$\begin{aligned} \mathcal{A}_{\text{scale}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) &= \{ \nu \in \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) : \widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}}) + \Delta_t^C(\nu) \leq R_t^{\text{avail}}, \\ &\quad \widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}}) + \Delta_t^C(\nu) \leq \alpha_t G_t^{\text{res}}(\nu) + \beta_t + \xi_t^{\text{scale}} \\ &\quad \text{whenever an engine-mode scaling claim is made} \}. \end{aligned}$$

The first inequality is a one-step affordability condition. The second is an amortized scaling condition. It is not required for ordinary safe no-op fallback; it is required only when the architecture claims that certification/search overhead does not asymptotically dominate the useful reusable gains.

Assumption 25.5 (Resource ledger dynamics). *For accepted candidates ν_t^* , the available budget evolves according to the conservative inequality*

$$R_{t+1}^{\text{avail}} \geq R_t^{\text{avail}} + G_t^{\text{res}}(\nu_t^*) - C_t^{\text{tot}}(\nu_t^*; B_t^{\text{search}}) - s_t^{\text{res}},$$

where $s_t^{\text{res}} \geq 0$ is a resource-model slack term. The inequality is a ledger assumption: it must be audited by implementation-level accounting and is not derived from the density-state formalism.

Theorem 25.6 (Resource-scaling closure). *Fix a finite horizon T . Suppose that for every $t < T$:*

- (a) *the cost-bound event $\mathcal{E}_t^C$ holds;*
- (b) *$\nu_t^* \in \mathcal{A}_{\text{scale}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}})$;*
- (c) *the resource ledger dynamics of Assumption 25.5 hold;*
- (d) *the scaling constants satisfy $\alpha_t \leq \bar{\alpha} < 1$.*

Then the accepted updates are one-step affordable:

$$C_t^{\text{tot}}(\nu_t^*; B_t^{\text{search}}) \leq R_t^{\text{avail}} \quad \forall t < T.$$

If the engine-mode scaling inequality is invoked at every step, then

$$\sum_{t=0}^{T-1} C_t^{\text{tot}}(\nu_t^*; B_t^{\text{search}}) \leq \bar{\alpha} \sum_{t=0}^{T-1} G_t^{\text{res}}(\nu_t^*) + \sum_{t=0}^{T-1} (\beta_t + \xi_t^{\text{scale}}).$$

Consequently, when $\sum_t (\beta_t + \xi_t^{\text{scale}}) < \infty$ on a declared horizon family and reusable gains grow, certification/search cost is certified to be amortized below the reusable gain scale under the stated ledger assumptions.

Proof. The affordability inequality follows immediately from the first defining inequality of $\mathcal{A}_{\text{scale}}$ and Assumption 25.3. For engine-mode scaling, membership in $\mathcal{A}_{\text{scale}}$ gives

$$C_t^{\text{tot}}(\nu_t^*; B_t^{\text{search}}) \leq \hat{C}_t^{\text{tot}}(\nu_t^*; B_t^{\text{search}}) + \Delta_t^C(\nu_t^*) \leq \alpha_t G_t^{\text{res}}(\nu_t^*) + \beta_t + \xi_t^{\text{scale}}.$$

Summing and using $\alpha_t \leq \bar{\alpha}$ gives the cumulative bound. No claim is made that G_t^{res} is automatically positive or that the cost estimates are sound outside the audited events. $\square$

Definition 25.7 (No-op rate monitor). *Let*

$$\iota_t^0 = \begin{cases} 1, & \nu_t^* = \nu_t^0, \\ 0, & \nu_t^* \neq \nu_t^0. \end{cases}$$

For a window length $h_t \geq 1$, define the sliding no-op rate

$$r_{t,h_t}^0 = \frac{1}{h_t} \sum_{s=\max(0, t-h_t+1)}^t \iota_s^0,$$

with the denominator replaced by the number of available terms when $t < h_t - 1$. Let $r_{\max,t}^0 \in [0, 1]$ be a declared diagnostic threshold. The no-op pressure residual is

$$m_t^{\text{noop}} = r_{t,h_t}^0 - r_{\max,t}^0.$$

Definition 25.8 (No-op diagnostic response). *If $m_t^{\text{noop}} > 0$, the architecture enters diagnostic mode Diag_t . Diagnostic mode may require one or more of the following actions:*

- (i) expand or diversify $S_t(B_t^{\text{search}})$ subject to the resource-scaling gate;
- (ii) tighten or recalibrate capability-gain estimators;
- (iii) improve verifier resolution through trust-admissible verifier updates;
- (iv) expand adversarial pair-set search;
- (v) request external validation or pause Level 3 self-modification.

Diagnostic mode is not an admissibility shortcut. It does not allow the architecture to accept a candidate outside $\mathcal{A}_t^{\text{safe}}$, $\mathcal{A}_{\text{cert}}$, $\mathcal{A}_{\text{trust}}$, or any other safety gate.

Proposition 25.9 (No-op monitoring cannot force unsafe progress). *Assume the selection policy obeys Definition 25.8. If $m_t^{\text{noop}} > 0$, then the architecture may change search, audits, or defer self-modification, but it cannot accept a candidate $\nu \notin \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}})$. Therefore the no-op monitor does not weaken the RCP gate.*

Proof. By Definition 25.8, diagnostic mode changes only the proposal, audit, validation, or deferral process. It explicitly does not modify the admissibility requirement $\nu \in \mathcal{A}_t^{\text{safe}}$. Hence any accepted non-no-op candidate still must pass the full gate, and if no such candidate exists, no-op or deferral remains the only allowed outcome. $\square$

Definition 25.10 (Gate-budget vector and certificate inflation). *Let $b_t^{\text{gate}} \in \mathbb{R}_+^{d_b}$ collect the declared safety budgets and thresholds used by the gate at time t , including representative entries such as*

$$\varepsilon_t, \quad r_{\max,t}, \quad \eta_t, \quad \zeta_t, \quad \chi_t, \quad \delta_{X,t}^{\text{sem}}, \quad \omega_t^{\text{ver}}, \quad \pi_t^{\text{ver}}, \quad \beta_t,$$

with only finitely many coordinates active at each time. Let $w_t^b, w_t^\Delta \in \mathbb{R}_+^{d_b}$ be nonnegative weights. For a candidate ν that proposes successor budgets $b_{t+1}^{\text{gate}}(\nu)$ and certificate radii $\Delta_t^{\text{cert}}(\nu)$, define

$$\text{Relax}_t(\nu) = \left\| [b_{t+1}^{\text{gate}}(\nu) - b_t^{\text{gate}}]_+ \right\|_{w_t^b, 1}, \quad \text{Rad}_t(\nu) = \left\| \Delta_t^{\text{cert}}(\nu) \right\|_{w_t^\Delta, 1},$$

where $[\cdot]_+$ is coordinatewise positive part and $\|x\|_{w,1} = \sum_i w_i |x_i|$. The certificate-inflation score is

$$\text{Infl}_t(\nu) = \text{Relax}_t(\nu) + \text{Rad}_t(\nu).$$

Inflation measures safety-budget relaxation and large uncertainty radii. It does not by itself prove gaming; it is a conservative pressure signal.

Definition 25.11 (Inflation-controlled admissibility). *Let $\iota_t^{\max} \geq 0$ be the allowed inflation increment. The inflation-controlled admissible set is*

$$\mathcal{A}_{\text{infl}}(t, \rho_t^{\text{sys}}) = \left\{ \nu \in \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) : \text{Infl}_t(\nu) \leq \iota_t^{\max}, \right. \\ \left. \text{every proposed relaxation of } b_t^{\text{gate}} \text{ is trust-admissibly certified} \right\}.$$

Budget relaxation is treated as verifier- or gate-affecting self-modification. Therefore it must satisfy the trust-graph and no-self-sole-certification requirements; it cannot be certified only by the mechanism whose threshold is being relaxed.

Theorem 25.12 (Bounded certificate inflation). *Suppose accepted candidates satisfy $\nu_t^* \in \mathcal{A}_{\text{infl}}(t, \rho_t^{\text{sys}})$ for $t < T$. Then*

$$\sum_{t=0}^{T-1} \text{Infl}_t(\nu_t^*) \leq \sum_{t=0}^{T-1} \iota_t^{\max}.$$

In particular, if $\sum_{t=0}^{\infty} \iota_t^{\max} < \infty$, cumulative certificate inflation is finite. Thus the architecture cannot obtain an infinite sequence of apparent progress certificates solely by unbounded gradual relaxation of gate budgets or unbounded growth of certificate radii, provided the inflation gate is enforced.

Proof. Membership in $\mathcal{A}_{\text{infl}}$ gives $\text{Infl}_t(\nu_t^*) \leq \iota_t^{\max}$ at each time. Summing over $t < T$ proves the first inequality. The infinite-horizon statement follows by monotone convergence of the partial sums. The theorem bounds declared inflation; it does not prove that the original budgets were semantically correct. $\square$

Definition 25.13 (Inflation-penalized certified selection objective). *Among candidates passing all hard gates, the architecture may use the objective*

$$\nu_t^* \in \arg \max_{\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})} \left[\Delta \widehat{\text{SafeCap}}_t(\nu) - \lambda_{\text{cost}} \widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}}) - \lambda_{\text{infl}} \widehat{\text{Infl}}_t(\nu) \right],$$

where $\lambda_{\text{cost}}, \lambda_{\text{infl}} \geq 0$. This objective ranks already-admissible candidates. It does not make inadmissible candidates safe.

Limitation 25.14 (No-op pressure and inflation control are diagnostics, not progress proofs). *The no-op monitor prevents silent stagnation from being misreported as RSI progress. The inflation penalty prevents progress claims from being purchased by gradually loosening certificates. Neither mechanism proves that safe beneficial updates exist, that they can be found cheaply, or that empirical validators are reliable at frontier scale. Those claims remain tied to the persistent-improvement, search-completeness, certificate-soundness, and resource-ledger assumptions.*

26 Pair-Set Coverage, Capability Calibration, and Estimator Subprotocols

The preceding sections define a gate and a certificate flow. Three remaining failure modes must be made explicit before the architecture can claim to have exhausted the incognito Tier 2 critique. First, a pair set protects only distinctions represented in the declared pair-set and barrier construction. Second, a coherence or Lyapunov improvement does not by itself imply task capability improvement. Third, a generic one-sided residual vector is not an implementation protocol unless the architecture specifies how each residual class is estimated, stressed, costed, and audited. This section adds those three closure modules.

26.1 Pair-set and adversarial coverage closure

Definition 26.1 (Tracked and untracked declared predicates). *Let $\mathfrak{S}_t^{\text{RCLM}}$ be the architecture's declared safety predicate family. A predicate $s \in \mathfrak{S}_t^{\text{RCLM}}$ is tracked at time t if at least one of the following is supplied:*

- (a) *a finite witness map $W_t(s) \subseteq \mathcal{P}_t^{\text{RCLM}}$ such that loss on the witness pairs is included in the non-loss residual vector;*
- (b) *a barrier $B_{s,t}^{\text{RCLM}}$ whose soundness has been audited on the declared semantic and representational domain;*
- (c) *a verifier test family $T_{s,t}^{\text{ver}}$ with a one-sided residual estimator included in the certificate packet.*

The tracked and untracked declared predicate classes are

$$\mathfrak{S}_t^{\text{trk}} = \{s \in \mathfrak{S}_t^{\text{RCLM}} : s \text{ is tracked at time } t\}, \quad \mathfrak{S}_t^{\text{untrk}} = \mathfrak{S}_t^{\text{RCLM}} \setminus \mathfrak{S}_t^{\text{trk}}.$$

Limitation 26.2 (Declared predicates are not all possible predicates). *The set $\mathfrak{S}_t^{\text{RCLM}}$ is a declared architecture object. No theorem in this paper quantifies over all possible future alignment-relevant distinctions. Coverage results below are relative to the declared predicate universe and the audited adversarial discovery process.*

Definition 26.3 (Coverage risk and coverage gap). *For a candidate update ν , let*

$$\text{Risk}_t^{\text{cov}}(s, \nu) \in [0, \infty]$$

be the audited risk score assigned to the possibility that predicate s is violated or made unmeasurable by Φ_ν without being detected by the tracked pair, barrier, or verifier objects. The declared untracked-predicate coverage gap is

$$\text{CovGap}_t(\nu) = \sup_{s \in \mathfrak{S}_t^{\text{untrk}} \cap \mathcal{U}_t^{\text{cov}}} \text{Risk}_t^{\text{cov}}(s, \nu),$$

where $\mathcal{U}_t^{\text{cov}} \subseteq \mathfrak{S}_t^{\text{RCLM}}$ is the audited predicate-search domain. If the supremum is over the empty set, it is defined to be zero.

Definition 26.4 (Coverage certificate). *A coverage certificate for candidate ν is*

$$\text{CovCert}_t(\nu) = (\widehat{\text{CovGap}}_t(\nu), \Delta_t^{\text{cov}}(\nu), \delta_t^{\text{cov}}, \text{AdvTrace}_t(\nu), \text{Expand}_t),$$

where $\widehat{\text{CovGap}}_t$ is an audited estimate, Δ_t^{cov} is a one-sided radius, δ_t^{cov} is a failure probability, AdvTrace_t records adversarial predicate discovery attempts, and Expand_t is the rule that either adds high-risk discoveries to the tracked objects or rejects the candidate.

Assumption 26.5 (Coverage one-sided soundness). *For the finite audited candidate set considered at time t , the coverage certificate satisfies, with probability at least $1 - \delta_t^{\text{cov}}$,*

$$\text{CovGap}_t(\nu) \leq \widehat{\text{CovGap}}_t(\nu) + \Delta_t^{\text{cov}}(\nu) \quad \forall \nu \in S_t(B_t^{\text{search}}).$$

This assumption is an audit assumption. It is not derived from the definition of $\mathcal{P}_t^{\text{RCLM}}$.

Definition 26.6 (Coverage-admissible candidates). *For threshold $\gamma_t^{\text{cov}} \geq 0$, define*

$$\mathcal{A}_{\text{cov}}(t, \rho_t^{\text{sys}}) = \{\nu \in \Omega_t^{\text{aud}} : \widehat{\text{CovGap}}_t(\nu) + \Delta_t^{\text{cov}}(\nu) \leq \gamma_t^{\text{cov}},$$

Expand_t has no unresolved high-risk discovery for ν).

An unresolved high-risk discovery means a predicate found by the adversarial search with certified risk above γ_t^{cov} that has not been added to $\mathfrak{S}_t^{\text{trk}}$, $\mathcal{P}_t^{\text{RCLM}}$, a barrier, or a verifier test before acceptance.

Theorem 26.7 (Coverage-gap soundness). *Assume Assumption 26.5. If $\nu \in \mathcal{A}_{\text{cov}}(t, \rho_t^{\text{sys}})$, then, on the coverage-soundness event,*

$$\text{CovGap}_t(\nu) \leq \gamma_t^{\text{cov}}.$$

Consequently, every declared untracked predicate in the audited domain has risk at most γ_t^{cov} under Φ_ν .

Proof. By coverage one-sided soundness,

$$\text{CovGap}_t(\nu) \leq \widehat{\text{CovGap}}_t(\nu) + \Delta_t^{\text{cov}}(\nu).$$

Membership in $\mathcal{A}_{\text{cov}}$ makes the right-hand side at most γ_t^{cov} . The statement about declared untracked predicates follows from the definition of the supremum in CovGap_t . No statement is made about predicates outside $\mathfrak{S}_t^{\text{RCLM}}$ or outside the audited discovery domain. $\square$

Proposition 26.8 (Adversarial expansion is monotone, not complete). *Suppose Expand_t is append-only: when it accepts a new high-risk predicate s , it adds a witness pair, barrier, or verifier test without deleting existing tracked objects. Then*

$$\mathfrak{S}_t^{\text{trk}} \subseteq \mathfrak{S}_{t+1}^{\text{trk}}, \quad \mathcal{P}_t^{\text{RCLM}} \subseteq \mathcal{P}_{t+1}^{\text{RCLM}}$$

for every accepted expansion step. This monotonicity does not imply finite-time completeness of the declared predicate universe.

Proof. The inclusions follow directly from append-only insertion. Finite-time completeness would require an additional assumption that the adversarial generator enumerates all relevant predicates or that the declared universe is finite and exhaustively searched; neither is assumed. $\square$

26.2 Coherence–capability calibration protocol

Definition 26.9 (Audited capability suite). *The architecture-level capability suite at time t is*

$$\mathcal{T}_t^{\mathfrak{m}} = (\mathcal{T}_t^{\text{hold}}, \mathcal{T}_t^{\text{ood}}, \mathcal{T}_t^{\text{adv}}, \mathcal{T}_t^{\text{formal}}, \mathcal{T}_t^{\text{human}}, w_t^{\mathfrak{m}}),$$

where the components are held-out tasks, out-of-distribution tasks, adversarial tasks, formal tasks, human-reference audit tasks, and nonnegative weights. The induced safe-capability functional is the weighted audited quantity used by SafeCap_t ; it is not claimed to be an absolute measure of intelligence.

Definition 26.10 (Calibration features and calibration error). *For a candidate ν , define the calibration feature vector*

$$\Xi_t(\nu) = (\Delta\mathcal{C}_t(\nu), \mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}}, U_t(\nu), q_t^B(\nu), q_t^{\text{sem}}(\nu), \text{Inf}_t(\nu), C_t^{\text{tot}}(\nu), q_t^{\text{cov}}(\nu)).$$

Let $f_t : \Xi_t \mapsto \mathbb{R}$ be the audited calibration model predicting no-op-normalized safe-capability gain from the coherence and risk features. The pointwise calibration error is

$$e_t(\nu) = |\Delta \text{SafeCap}_t^0(\nu) - f_t(\Xi_t(\nu))|.$$

Definition 26.11 (Calibration protocol). *A coherence–capability calibration protocol is*

$$\text{CalProt}_t = (\mathcal{T}_t^{\mathfrak{m}}, \mathcal{D}_t, \mathcal{D}_t^{\text{hold}}, \mathcal{T}_t^{\text{stress}}, f_t, \hat{e}_t, \Delta_t, \delta_t, e_{\max,t}),$$

where $\hat{e}_t$ is the estimated calibration error on the audited suite, Δ_t is a one-sided radius, δ_t is the failure probability, and $e_{\max,t}$ is the maximum tolerated calibration error for using coherence-derived progress claims.

Assumption 26.12 (Calibration one-sided soundness). *For the finite audited candidate set considered at time t , with probability at least $1 - \delta_t$,*

$$e_t(\nu) \leq \hat{e}_t(\nu) + \Delta_t(\nu) \quad \forall \nu \in S_t(B_t^{\text{search}}).$$

The calibration data, held-out data, and stress tests are frozen before candidate selection. Candidate-specific retuning after observing outcomes invalidates the certificate and creates a new protocol instance.

Definition 26.13 (Calibration-admissible candidates). *A candidate ν is calibration-admissible, written $\nu \in \mathcal{A}_{\text{cal}}(t, \rho_t^{\text{sys}})$, when either no coherence-derived safe-capability progress claim is made for ν , or*

$$\hat{e}_t(\nu) + \Delta_t(\nu) \leq e_{\max,t}$$

and the capability suite, scoring convention, calibration model, and stress-test data are frozen before selection.

Theorem 26.14 (Calibrated capability-claim transfer). *Assume Assumption 26.12. If $\nu \in \mathcal{A}_{\text{cal}}(t, \rho_t^{\text{sys}})$ and a coherence-derived progress claim is made, then on the calibration-soundness event*

$$|\Delta \text{SafeCap}_t^0(\nu) - f_t(\Xi_t(\nu))| \leq e_{\max, t}.$$

In particular, if

$$f_t(\Xi_t(\nu)) - e_{\max, t} > \epsilon_t^{\text{prog}},$$

then $\Delta \text{SafeCap}_t^0(\nu) > \epsilon_t^{\text{prog}}$ on the audited suite.

Proof. The first inequality follows by combining Assumption 26.12 with membership in $\mathcal{A}_{\text{cal}}$. The second claim follows from

$$\Delta \text{SafeCap}_t^0(\nu) \geq f_t(\Xi_t(\nu)) - e_t(\nu) \geq f_t(\Xi_t(\nu)) - e_{\max, t}.$$

No conclusion follows for tasks outside $\mathcal{T}_t^{\text{in}}$ or for a changed scoring convention. $\square$

Limitation 26.15 (Calibration does not derive capability from coherence). *The calibration protocol bounds the error of a declared empirical mapping from coherence/risk features to safe-capability change on an audited suite. It does not prove that coherence is identical to capability, that the task suite captures general intelligence, or that calibration transfers to deployment distributions without additional evidence.*

26.3 Implementation-level estimator subprotocols

Definition 26.16 (Residual estimator subprotocol). *For each residual class k used in the gate, an estimator subprotocol is a tuple*

$$\text{Est}_{t,k} = (\text{In}_{t,k}, \text{Dom}_{t,k}, \hat{q}_{t,k}, \Delta_{t,k}^{\text{stat}}, \Delta_{t,k}^{\text{shift}}, \Delta_{t,k}^{\text{model}}, \Delta_{t,k}^{\text{adv}}, \delta_{t,k}^{\text{est}}, \text{Cost}_{t,k}^{\text{est}}, \text{Stress}_{t,k}).$$

The tuple specifies inputs, audited domain, estimator, statistical radius, distribution-shift radius, model-approximation radius, adversarial radius, failure probability, resource cost, and stress tests. Its total one-sided radius is

$$\Delta_{t,k}^{\text{est}} = \Delta_{t,k}^{\text{stat}} + \Delta_{t,k}^{\text{shift}} + \Delta_{t,k}^{\text{model}} + \Delta_{t,k}^{\text{adv}}.$$

Definition 26.17 (Required estimator classes). *The final-v1 RCP-RCLM gate requires subprotocols for at least the following residual classes:*

$$\mathcal{K}_t^{\text{est}} = \{D_{\text{rel}}, \text{Rec}, V, B, U, I_{\text{IB}}, \text{Val}, \text{SafeCap}, M, \text{ver}, \text{trust}, \text{type}, \text{coup}, \text{cov}, \text{cal}, \text{scale}, \text{infl}, \text{asy}\}.$$

Here D_{rel} estimates relative-entropy loss, Rec recovery error, V Lyapunov drift, B barrier residuals, U unsupported ambiguity collapse, I_{IB} information-bottleneck loss, Val semantic validity, and the remaining symbols correspond to the named architecture gates.

Assumption 26.18 (Subprotocol one-sided soundness). *Let $j \mapsto k(j)$ assign each residual component of $\mathbf{q}_t$ to its estimator class. For the finite candidate set considered at time t , with probability at least*

$$1 - \sum_{k \in \mathcal{K}_t^{\text{est}}} \delta_{t,k}^{\text{est}},$$

every required subprotocol is domain-valid and

$$q_{t,j}(\nu) \leq \hat{q}_{t,j}^{k(j)}(\nu) + \Delta_{t,k(j)}^{\text{est}}(\nu) \quad \forall \nu \in S_t(B_t^{\text{search}}), \quad \forall j \in \mathcal{J}_t.$$

Definition 26.19 (Estimator-complete candidates). *A candidate ν belongs to $\mathcal{A}_{\text{est}}(t, \rho_t^{\text{sys}})$ when every residual component required for the claimed mode has a domain-valid subprotocol in $\mathcal{K}_t^{\text{est}}$, every subprotocol cost is included in C_t^{tot} , and every conservative residual bound satisfies*

$$\hat{q}_{t,j}^{k(j)}(\nu) + \Delta_{t,k(j)}^{\text{est}}(\nu) \leq 0 \quad \forall j \in \mathcal{J}_t.$$

Theorem 26.20 (Estimator subprotocols imply non-oracular residual soundness). *Assume Assumption 26.18. If $\nu \in \mathcal{A}_{\text{est}}(t, \rho_t^{\text{sys}})$, then on the subprotocol-soundness event*

$$q_{t,j}(\nu) \leq 0 \quad \forall j \in \mathcal{J}_t.$$

Consequently, the residual portion of the non-oracular certificate is sound for ν .

Proof. For each residual component j , Assumption 26.18 gives

$$q_{t,j}(\nu) \leq \hat{q}_{t,j}^{k(j)}(\nu) + \Delta_{t,k(j)}^{\text{est}}(\nu).$$

Membership in $\mathcal{A}_{\text{est}}$ makes the right-hand side nonpositive for every required component. The conclusion follows componentwise. The probability statement is inherited from the finite union bound in the assumption. $\square$

Corollary 26.21 (Coverage, calibration, and subprotocol closure inside the full gate). *Suppose ν satisfies $\mathcal{A}_{\text{cov}}$, $\mathcal{A}_{\text{cal}}$, and $\mathcal{A}_{\text{est}}$, and the soundness events of Assumptions 26.5, 26.12, and 26.18 hold. Then the accepted update has bounded declared untracked-predicate risk, calibrated coherence-capability claims on the audited capability suite, and one-sided sound residual estimates for the full RCP-RCLM gate.*

Proof. Apply Theorems 26.7, 26.14, and 26.20. The result is a conjunction of the three conclusions. It does not add any claim beyond the audited domains and declared thresholds of those theorems. $\square$

Limitation 26.22 (Tier 2 closure remains audited and finite). *The coverage, calibration, and estimator-subprotocol modules reduce three previously open architecture obligations to explicit residual gates and theorem hypotheses. They do not prove exhaustive coverage of unknown future predicates, universal coherence-capability equivalence, or scalable estimator construction for arbitrary frontier systems. They make those assumptions visible, testable, and rejectable.*

27 Modules A–C Instantiated in RCLM

27.1 Module C: non-loss and recovery

Theorem 27.1 (RCLM bounded distinguishability loss). *Let $(\rho_0, \sigma_0) \in \mathcal{P}_0^{\text{RCLM}}$ have finite relative entropy and suppose both states are evolved by the same accepted self-update channels:*

$$\rho_{t+1} = \Phi_{\nu_t^*}(\rho_t), \quad \sigma_{t+1} = \Phi_{\nu_t^*}(\sigma_t).$$

If

$$D(\rho_t \| \sigma_t) - D(\rho_{t+1} \| \sigma_{t+1}) \leq \varepsilon_t$$

for $t < T$, then

$$D(\rho_T \| \sigma_T) \geq D(\rho_0 \| \sigma_0) - \sum_{t=0}^{T-1} \varepsilon_t.$$

Proof. Rearrange each one-step inequality and sum telescopically over $t = 0, \dots, T - 1$. $\square$

Limitation 27.2 (Recovery is stronger than non-loss). *The theorem bounds loss of distinguishability. It does not construct a recovery map unless the additional recovery certificate in $\mathcal{A}_{\text{RCP}}^{\text{RCLM-AC}}$ is supplied.*

27.2 Module B: capability and coherence

Definition 27.3 (No-op-normalized improvement). *Let ν_t^0 be no-op. Define*

$$G_t^{\text{noop}}(\nu) = \text{SafeCap}_{t+1}(\Phi_\nu(\rho_t^{\text{sys}})) - \text{SafeCap}_{t+1}(\Phi_{\nu_t^0}(\rho_t^{\text{sys}})).$$

This avoids mistaking task-distribution or benchmark shifts for true architectural progress.

Theorem 27.4 (Conditional safe-capability telescoping). *Assume accepted updates satisfy*

$$\text{SafeCap}_{t+1}(\rho_{t+1}^{\text{sys}}) \geq \text{SafeCap}_t(\rho_t^{\text{sys}}) - \varpi_t.$$

Then

$$\text{SafeCap}_T(\rho_T^{\text{sys}}) \geq \text{SafeCap}_0(\rho_0^{\text{sys}}) - \sum_{t=0}^{T-1} \varpi_t.$$

If, in addition, $\text{SafeCap}_{t+1}(\rho_{t+1}^{\text{sys}}) \geq \text{SafeCap}_t(\rho_t^{\text{sys}}) + g_t - \varpi_t$, then

$$\text{SafeCap}_T(\rho_T^{\text{sys}}) \geq \text{SafeCap}_0(\rho_0^{\text{sys}}) + \sum_{t=0}^{T-1} g_t - \sum_{t=0}^{T-1} \varpi_t.$$

Proof. Both conclusions follow by summing the one-step inequalities. $\square$

Limitation 27.5 (Bounded scales). *Uniform positive gain cannot continue forever on a fixed bounded capability scale. Open-ended RSI requires an expanding task distribution, an unbounded or normalized capability scale, decreasing thresholds, or a finite-horizon claim.*

27.3 Module A: safe-improvement existence and viability

Definition 27.6 (Residual gate vector). *For a candidate ν , define residuals $r_j(\nu)$ for every active gate inequality, normalized so that $r_j(\nu) \leq 0$ means the constraint is satisfied. Let*

$$\mathbf{r}_t(\nu) = (r_1(\nu), \dots, r_m(\nu)).$$

Strict interiority means $r_j(\nu) \leq -m_t$ for all j and some margin $m_t > 0$.

Assumption 27.7 (Certified local search completeness). *For a local update neighborhood $\mathcal{N}_t \subseteq \Omega_t^{\text{RCLM}}$, the search-and-verification process either returns a certified candidate with residuals and improvement estimates within error e_t , or correctly certifies that no candidate in $\mathcal{N}_t$ satisfies the required margins.*

Proposition 27.8 (One-step safe-improvement existence). *Suppose there exists $\nu^\dagger \in \mathcal{N}_t$ such that all gate residuals satisfy $r_j(\nu^\dagger) \leq -m_t$, the no-op-normalized safe-capability gain satisfies $G_t^{\text{noop}}(\nu^\dagger) \geq g_t$, and the search/certificate error satisfies $e_t < \min\{m_t, g_t\}$. Then the certified search can return a nontrivial candidate in $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$ with positive certified gain.*

Proof. The error bound is smaller than the safety margin and the gain margin. Therefore conservative certification cannot move a truly feasible point outside the feasible region nor erase all positive gain. The returned candidate remains within the gate and has positive certified no-op-normalized gain. $\square$

Definition 27.9 (RCLM viability kernel). *For horizon N , $\mathcal{V}_N^{\text{RCLM}}$ is the set of states from which there exists an admissible length- N path of candidates in $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$. The infinite-horizon viability kernel is*

$$\mathcal{V}_\infty^{\text{RCLM}} = \bigcap_{N \geq 1} \mathcal{V}_N^{\text{RCLM}}.$$

Theorem 27.10 (Recursive feasibility by viability). *If $\rho_t^{\text{sys}} \in \mathcal{V}_\infty^{\text{RCLM}}$ and ν_t^* is chosen as the first step of an infinite admissible path, then*

$$\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t^*}(\rho_t^{\text{sys}}) \in \mathcal{V}_\infty^{\text{RCLM}}.$$

Proof. Removing the first step from the infinite admissible path leaves an infinite admissible path beginning at ρ_{t+1}^{sys} . Hence the successor remains in the infinite-horizon viability kernel. $\square$

28 Modules D–G Instantiated in RCLM

28.1 Module D: semantic realization

Assumption 28.1 (Semantic domain preservation). *For each accepted $\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$, the successor lies in the audited semantic domain:*

$$\Phi_\nu(\rho_t^{\text{sys}}) \in \mathcal{X}_{t+1}^{\text{sem}}.$$

Theorem 28.2 (Semantic barrier preservation). *Assume $\rho_t^{\text{sys}} \in \mathcal{X}_t^{\text{sem}}$ and $B_X^{\text{sem}}(\rho_t^{\text{sys}}) \leq 0$ for all audited registers. If $\nu \in \mathcal{A}_{\text{sem}}(t)$, then*

$$B_X^{\text{sem}}(\Phi_\nu(\rho_t^{\text{sys}})) \leq 0$$

for all audited registers.

Proof. Membership in $\mathcal{A}_{\text{sem}}(t)$ is defined to include semantic domain preservation and the successor semantic barrier inequalities. Therefore the conclusion follows directly from the gate definition. $\square$

Limitation 28.3 (Human-reference validity). *Preserving a declared S -register reference is not the same as proving that the reference correctly represents actual human values. The architecture must separately audit the human-reference target and include audit error in the semantic budget.*

28.2 Module E: self-model fidelity, verifier bootstrapping, and anti-circularity

Theorem 28.4 (Self-model fidelity preservation). *Assume accepted candidates satisfy*

$$d_M(M_{t+1}^{\text{int}}, M_{t+1}^{\text{op}}) \leq \epsilon_{t+1}^M.$$

If the operational audit differs from the true operational state by at most a_{t+1}^M , then

$$d_M(M_{t+1}^{\text{int}}, M_{t+1}^{\text{true}}) \leq \epsilon_{t+1}^M + a_{t+1}^M.$$

Proof. Apply the triangle inequality:

$$d_M(M_{t+1}^{\text{int}}, M_{t+1}^{\text{true}}) \leq d_M(M_{t+1}^{\text{int}}, M_{t+1}^{\text{op}}) + d_M(M_{t+1}^{\text{op}}, M_{t+1}^{\text{true}}).$$

Substitute the two assumed bounds. $\square$

Definition 28.5 (Verifier bootstrapping condition). *A verifier update is admissible only if*

$$\text{Sound}(W_{t+1}) \geq \text{Sound}(W_t) - \omega_t$$

and, when claiming verifier improvement,

$$\text{Power}(W_{t+1}) \geq \text{Power}(W_t) + p_t - \omega_t.$$

Proposition 28.6 (Trust-graph verifier bootstrapping in RCLM). *If an accepted candidate belongs to $\mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}})$ and modifies verifier-relevant components, then its verifier transition is certified by independent support according to Theorem 22.9. If the trust-soundness events and degradation budgets hold over a finite horizon, verifier soundness and power satisfy the bounds of Theorem 22.10.*

Proof. This is a direct specialization of Section 22 to the Module E verifier state W_t . Membership in $\mathcal{A}_{\text{trust}}$ supplies independent support; the stability bounds follow by telescoping the assumed verifier-transition inequalities. $\square$

Limitation 28.7 (Verifier power boundedness). *Uniform positive verifier-power gain cannot continue forever on a fixed bounded scale. Long-horizon verifier improvement requires expanding adversarial test distributions, normalized power metrics, or finite-horizon statements.*

28.3 Module F: resource realism and scaling pressure

Definition 28.8 (Resource feasibility). *A candidate ν is resource feasible when*

$$\begin{aligned} &\text{Cost}_{\text{search}}(\nu) + \text{Cost}_{\text{cert}}(\nu) + \text{Cost}_{\text{rec}}(\nu) + \text{Cost}_{\text{verify}}(\nu) + \text{Cost}_{\text{apply}}(\nu) \\ &\leq \text{Budget}(\mathfrak{R}_t). \end{aligned}$$

The set of one-step resource-feasible candidates is Ω_t^{phys} . Engine-mode resource scaling is handled separately by $\mathcal{A}_{\text{scale}}$ in Section 25.

Proposition 28.9 (Resource-gated admission). *If $\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})$, then $\nu \in \Omega_t^{\text{phys}}$, $\nu \in \mathcal{A}_{\text{scale}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}})$, and its search, certification, recovery, verification, application, rollback, and audit costs satisfy the declared one-step and engine-mode resource-scaling constraints when those claims are invoked.*

Proof. The definition of the full post-D-G RCLM admissibility gate includes both the one-step resource-feasibility gate and the engine-mode resource-scaling gate. The result follows from the definitions of one-step feasibility and resource-scaling admissibility. $\square$

Limitation 28.10 (Thermodynamic interpretation). *The resource vector includes energy and irreversibility budgets, but this does not derive a full physical thermodynamics of cognition. It only prevents the architecture theorem from ignoring resource costs and from claiming engine-mode progress when certified cost growth is unaudited or dominates the declared reusable gains.*

28.4 Module G: asymptotic coherence

Definition 28.11 (Target coherence set). *The target post-D-G coherence set is*

$$\mathcal{K}_{\text{RCLM}}^{\text{DG},*} = \{\rho : \rho \in \mathcal{K}_{\text{RCLM}}^{\text{safe}}, V_{\text{RCLM}}(\rho) \leq v_*, B_i(\rho) \leq 0, B_X^{\text{sem}}(\rho) \leq 0\}.$$

Theorem 28.12 (Asymptotic approach under descent and compactness). *Assume the accepted trajectory remains in a compact set, the asymptotic transition relation is closed, $\sum_t \eta_t < \infty$, and the Lyapunov drift satisfies*

$$\mathbb{E}[V_{t+1} \mid \mathcal{F}_t] \leq V_t - c \operatorname{dist}(\rho_t^{\text{sys}}, \mathcal{K}_{\text{RCLM}}^{\text{DG},*})^2 + \eta_t$$

for $c > 0$. Then

$$\sum_t \mathbb{E}[\operatorname{dist}(\rho_t^{\text{sys}}, \mathcal{K}_{\text{RCLM}}^{\text{DG},*})^2] < \infty,$$

and in particular the expected squared distance to the target set is summable.

Proof. Rearrange the drift inequality and sum from $t = 0$ to $T - 1$. The Lyapunov terms telescope and the cumulative error is finite. Taking $T \rightarrow \infty$ gives summability of the expected squared distance. $\square$

Limitation 28.13 (Convergence caveat). *Summability of expected squared distance is weaker than almost-sure convergence of the full state. Stronger convergence claims require additional stochastic assumptions.*

29 Training Objectives

The architecture trains Level 2 inference and Level 3 update proposal/certification separately.

29.1 Level 2 inference objective

$$\begin{aligned} \mathcal{J}_{\text{inf}} = & \mathcal{L}_{\text{LM}} + w_O \mathcal{L}_O + w_S \mathcal{L}_S + w_D \mathcal{L}_D + w_A \mathcal{L}_A + w_M \mathcal{L}_M \\ & + \lambda_V [V_{\text{inf}}(\rho_t^{\text{ctx},*})]_+ + \lambda_U U_t^{\text{ctx}} + \lambda_B \sum_i [B_i^{\text{ctx}}(\rho_t^{\text{ctx},*})]_+. \end{aligned}$$

29.2 Level 3 proposal and gate objective

The proposal model may optimize predicted performance, but the gate objective is constrained:

$$\nu_t^* \in \arg \max_{\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}})} [\Delta \text{SafeCap}_t(\nu) + \lambda_C \Delta \mathcal{C}_{\text{RCLM}}(\nu) - \lambda_R \text{Risk}_t(\nu)].$$

If the feasible set contains no beneficial certified candidate, no-op is selected and the no-op rate monitor is updated. When the Batch-4 tangent-cone module is enabled, the search procedure may use the projected local direction $P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_{\nu} \text{RSIPot}_t$ to propose a small candidate, but the candidate is accepted only after the same certificate, residual, type, trust, resource, and non-loss checks pass.

Remark 29.1 (No reward for uncertified progress). *A candidate with high predicted capability gain but no recovery certificate, no semantic validity, insufficient resources, or verifier-gaming risk is not admissible.*

Corollary 29.2 (RCLM engine-continuation candidate construction). *If $\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine}, N}$, then the architecture can select a certified conservative-extension primitive word $\nu_t \in \Omega_t^{\text{NL}, \text{RCLM}}$ such that $\Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine}, N}$, provided the kernel witness and certificates required by Section 1.7 are available. If $g_t^{\mathfrak{A}} > 0$, the selected word is a strict certified ability-progress step under the declared ability universe. The corollary is conditional on kernel membership and does not assert that the architecture can construct the kernel from arbitrary data.*

30 Algorithms

The algorithms below are architecture-level procedures. They are not optimized software and they do not remove the need for scalable estimators. Their purpose is to make the dynamics, certificate flow, and no-op logic explicit.

30.1 Algorithm 1: Level 2 inference

- I1. Receive context package $\mathcal{I}_t^{\text{ctx}} = (x_{\leq n}, e_t, a_t^{\text{tools}}, m_t^{\text{local}})$.
- I2. Compute hidden states $h_{t,1:n} = F_{\theta_t}^{\text{LM}}(\mathcal{I}_t^{\text{ctx}})$ and pooled representation h_t .
- I3. Compute register amplitudes $\psi_{X,t} = f_{X,t}(h_t, m_t^{\text{sys}})$ for $X \in \{L, O, S, D, A, M\}$.
- I4. Build register densities $\rho_{X,t}$ and the initial context state $\rho_{t,0}^{\text{ctx}}$.
- I5. For $k = 0, \dots, K_t - 1$:
 - I5.a. apply unitary evolution $\rho' = U_t^{(l(k))} \rho_{t,k}^{\text{ctx}} U_t^{(l(k))\dagger}$;
 - I5.b. apply spectral projection and mixing $\rho_{t,k+1}^{\text{ctx}} = \text{Mix}_{p_{t,k}, \beta_{t,k}}(\rho')$;
 - I5.c. compute $V_{\text{inf}}(\rho_{t,k+1}^{\text{ctx}})$, entropy, coherence, ambiguity, and semantic monitors;
 - I5.d. halt if a certified stopping condition is met.
- I6. Output halted state $\rho_t^{\text{ctx},*}$, logits/readout, and a monitor record.

30.2 Algorithm 2: Level 3 self-improvement proposal and admission

- S1. Build the system payload $\mathcal{I}_t^{\text{sys}}$ and system density ρ_t^{sys} .
- S2. Construct the audited candidate domain $\Omega_t^{\text{aud}} \subseteq \Omega_t^{\text{RCLM}}$, explicitly including no-op ν^0 , and, when algebraic non-loss mode is enabled, generate the certified primitive-word subset $\Omega_t^{\text{NL,RCLM}} \subseteq \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$. Run the budgeted search procedure $S_t(B_t^{\text{search}}) = \text{Search}_t(B_t^{\text{search}}, \rho_t^{\text{sys}}, W_t)$. If Batch-4 local progress mode is enabled, construct a local chart χ_t^{RCLM} , compute local residuals $q_{t,j}^{\text{RCLM}}(u)$, and add tangent-cone candidates generated from $P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_{\nu} \text{RSIPot}_t$ to the audited candidate set. If Batch-5 engine-viability mode is enabled, construct the current kernel witness $\mathcal{V}_t^{\text{RCLM-engine},N}$ or $\mathcal{V}_t^{\text{RCLM-engine},\infty}$, restrict recursive-engine claims to $\mathcal{E}_t^{\text{RCLM-CE}}$, and log the successor-kernel certificate. If Batch-6 lift mode is enabled, run $\text{Lift}_t(\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}})$, add any valid lifted primitive-word candidate to Ω_t^{aud} , and reject any lifted candidate lacking a valid LiftCert_t .
- S3. For each candidate $\nu \in \Omega_t^{\text{RCLM}}$, construct the raw component successor $\tilde{\mathcal{I}}_{t+1}^{\text{sys}}(\nu) = T_{\nu}(\mathcal{I}_t^{\text{sys}})$.
- S4. Encode or project the raw successor into a candidate density $\tilde{\rho}_{t+1}^{\text{sys}}(\nu)$.
- S5. For any candidate claiming algebraic non-loss, parse its primitive word w_{ν} , compute transported pair sets, construct $\text{PrimCert}_t(g_i)$ for every primitive, and check $\nu \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$. Build the calibration and stress-test packet Audit_t , the verifier trust graph $\mathcal{G}_t^{\text{ver}}(\nu)$, the coverage certificate $\text{CovCert}_t(\nu)$, the coherence-capability calibration protocol record CalProt_t , and the full certificate packet $\text{Cert}_t(\nu)$, including data hashes, estimator versions, residual-specific subprotocol identifiers, stress-test identifiers, confidence radii, verifier identity, support nodes, and independence declarations.

- S6.** Run the residual-specific estimator subprotocols $\mathbf{Est}_{t,k}$ for all required classes $k \in \mathcal{K}_t^{\text{est}}$, estimate the full residual vector $\widehat{\mathbf{q}}_t(\nu)$, the audited one-sided radius vector $\Delta_t^{\text{aud}}(\nu)$, the coverage bound $\widehat{\text{CovGap}}_t(\nu) + \Delta_t^{\text{cov}}(\nu)$, the calibration bound $\widehat{e}_t(\nu) + \Delta_t(\nu)$, and the trust residual upper bound $\widehat{q}_t^{\text{trust}}(\nu) + \Delta_t^{\text{trust}}(\nu)$.
- S7.** Reject ν if an algebraic non-loss claim is made but the primitive-word certificate is missing or off-domain. Reject ν if the certificate packet is incomplete, if $\nu \notin \Omega_t^{\text{aud}}$, if any required estimator subprotocol is missing or off-domain, if the coverage or calibration gates fail for the claimed mode, if the trust graph violates no self-sole-certification, or if any audited one-sided residual bound is positive.
- S8.** For candidates passing all safety residuals, compute the certified objective $\Delta \widehat{\text{SafeCap}}_t(\nu) - \lambda_{\text{cost}} \widehat{\text{Cost}}_t(\nu)$, the no-op-normalized gain estimate $\widehat{G}_t^0(\nu)$, the conservative lower bound $\widehat{G}_t^0(\nu) - \Delta_t^G(\nu)$, and, when Batch-4 mode is invoked, the local potential increment $\Delta \text{RSIPot}_t^{\text{RCLM}}(\nu)$.
- S9.** If an engine-mode improvement is claimed, select only from $\mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}})$. If a candidate is justified by the tangent-cone theorem, also require the chart, active-set, curvature, finite-step radius, and segment-validity certificates of Section 1.6. If a recursive engine-continuation claim is made, also require the Batch-5 successor-kernel witness of Section 1.7. If the certified progress set is empty, select no-op or defer self-modification rather than relaxing the safety gate. If only safety preservation is claimed, select the best certified safe candidate or no-op.
- S10.** Apply the selected channel $\Phi_{\nu_t^*}$ and immediately run post-application verification.
- S11.** If post-application verification fails, rollback and record the failure as verifier-training data.
- S12.** Update the certificate ledger, pair set, tracked/untracked predicate records, adversarial coverage trace, verifier tests, semantic audits, capability-calibration records, estimator-subprotocol diagnostics, resource ledger, no-op-rate monitor, and inflation ledger.

30.3 Algorithm 3: Pair-set and adversarial expansion

- P1.** Start from a seed pair set containing known alignment, corrigibility, grounding, ambiguity, reversibility, and self-model distinctions, together with the declared predicate family $\mathfrak{S}_t^{\text{RCLM}}$.
- P2.** Partition declared predicates into tracked and untracked classes $\mathfrak{S}_t^{\text{trk}}$ and $\mathfrak{S}_t^{\text{untrk}}$.
- P3.** Generate candidate adversarial pairs and predicate probes by perturbing register states, semantic maps, barrier boundaries, verifier inputs, and coverage-search prompts.
- P4.** Score each probe by estimated risk, by whether the current gate would fail to distinguish it, and by its contribution to $\widehat{\text{CovGap}}_t + \Delta_t^{\text{cov}}$.
- P5.** Add high-risk unresolved pairs to $\mathcal{P}_t^{\text{RCLM}}$, or add the corresponding predicate to a barrier or verifier test family.
- P6.** Reject the candidate or increase the explicit coverage residual if no certificate can show that the omitted high-risk predicate is below γ_t^{cov} .

30.4 Algorithm 4: Reproducible numerical evaluation suite

- E1.** Freeze the evaluation package ReproPack_T , including code revision, numerical precision, random seeds, datasets, candidate generators, verifier version, gate thresholds, and artifact hashes.
- E2.** For each experiment specification $\mathfrak{E}_\ell$, build the declared input states, candidate updates, audit data, stress tests, and reference outputs.
- E3.** Run the experiment simulator Sim_ℓ using only the frozen artifacts and record raw traces before computing pass/fail judgments.
- E4.** Compute each declared metric $\widehat{m}_{\ell,j}$, its one-sided evaluation radius $\Delta_{\ell,j}^{\text{eval}}$, and its failure probability $\delta_{\ell,j}^{\text{eval}}$.
- E5.** Accept an experiment-level pass claim only when every conservative predicate $s_{\ell,j}\widehat{m}_{\ell,j} + \Delta_{\ell,j}^{\text{eval}} \leq b_{\ell,j}$ holds under its declared sign convention and margin.
- E6.** Store the full trace, rejected candidates, accepted candidates, certificates, post-application verification records, no-op diagnostics, resource ledger, pair-set updates, and artifact hashes.
- E7.** Report the outcome as an evaluation result, not as a theorem. A failed experiment invalidates the corresponding empirical claim; a passed experiment only supports the finite audited regime.

30.5 Algorithm 5: Conservative-extension primitive-word admission

- C1.** Generate primitive candidates from the declared generator set $\mathcal{G}_t^{\text{NL,RCLM}}$ and compose finite compatible words under the current type and resource constraints.
- C2.** For each word $w = g_n \circ \dots \circ g_1$, compute the intermediate transported pair sets $\mathcal{P}_t^{(i)}$.
- C3.** For each primitive g_i , verify its primitive certificate $\text{PrimCert}_t(g_i)$, including its isometry or reversible dilation, appended state, recovery map, type certificate, resource cost, and ledger trace.
- C4.** Sum the primitive budgets $\epsilon_w = \sum_i \epsilon_i$ and $r_w = \sum_i r_i$. Reject the word if $\epsilon_w > \epsilon_t$ or $r_w > r_{\max,t}$.
- C5.** Check that the old callable interface, old verifier slice, and predecessor ability certificates remain available when ability preservation is claimed.
- C6.** If a new ability is claimed, require a fresh ability certificate showing that the new ability is outside the transported predecessor ability set.
- C7.** Admit the word to $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$ only after all type, resource, trust, semantic, coverage, calibration, and estimator gates required for its claims have also passed.

30.6 Algorithm 6: Tangent-cone local progress proposal

- T1.** Construct a local candidate chart $\chi_t^{\text{RCLM}} : \mathcal{U}_t^{\text{RCLM}} \rightarrow \Omega_t^{\text{aud}}$ around no-op and certify $\chi_t^{\text{RCLM}}(0) = \nu_t^0$.
- T2.** Enumerate the local residual components $q_{t,j}^{\text{RCLM}}(u) \leq 0$ needed for the claimed mode, including algebraic non-loss residuals when construction-level non-loss is claimed.

- T3.** Identify the active residual set J_t^0 at no-op and compute or bound $Dq_{t,j}^{\text{RCLM}}(0)$ for every active component.
- T4.** Define the local potential $\text{RSIPot}_t^{\text{RCLM}}$, freeze its weights and evaluation convention, and compute $\nabla_u \text{RSIPot}_t^{\text{RCLM}}(0)$.
- T5.** Project the gradient onto the admissible tangent cone:

$$p_t = P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_u \text{RSIPot}_t^{\text{RCLM}}(0).$$

- T6.** If $p_t = 0$, or if $p_t/\|p_t\|$ is not certified strictly inward, do not claim tangent-cone progress. Either use another certified search method or select no-op.
- T7.** If the direction is certified, compute the step radius α_t^{TC} from the active-residual curvature, inactive slack, potential curvature, type/coupling validity, resource validity, and non-loss segment-validity certificates.
- T8.** Propose $\nu_t^{\text{TC}} = \chi_t^{\text{RCLM}}(\alpha_t v_t)$ for some $0 < \alpha_t \leq \alpha_t^{\text{TC}}$.
- T9.** Submit ν_t^{TC} to the full RCP–RCLM certificate pipeline. The tangent-cone certificate is a generator certificate, not a replacement for non-oracular certification, trust-graph certification, semantic coupling, resource feasibility, or Batch-3 primitive-word certification when claimed.

Limitation 30.1 (Algorithms are not scalability proofs). *These algorithms define the architecture-level control flow. They do not prove that relative entropy, recovery error, semantic validity, or verifier power can be estimated efficiently at frontier scale. Scalability requires separate complexity and empirical work.*

Algorithm 7: RCLM engine-viability continuation

At each engine-mode step:

1. Verify that the current state has a kernel certificate

$$\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},N} \quad \text{or} \quad \rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},\infty}.$$

If no such certificate exists, do not claim Batch-5 engine continuation.

2. Generate or retrieve the certified conservative-extension engine move set

$$\mathcal{E}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}}; g_t^{\mathfrak{A}}) \subseteq \Omega_t^{\text{NL,RCLM}}.$$

3. For each candidate, verify primitive-word compatibility, transported pair sets, additive loss/recovery budgets, ability preservation, any certified new ability, resource affordability, trust-graph support, adaptive-audit trust support, semantic coupling, estimator subprotocols, and successor-kernel membership.

4. Select a candidate ν_t only if

$$\Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N} \quad \text{or} \quad \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},\infty}.$$

5. Record ν_t , its primitive-word certificate, recovery map, ability certificates, resource ledger entries, and the successor-kernel witness in $\mathcal{H}_{t+1}^{\text{cert}}$.

This algorithm is a witness procedure for a declared viability kernel. It is not an algorithm for discovering the kernel from arbitrary states.

Algorithm 8: RCLM lift and finite-dimensional reference engine

1. Build the lift input package

$$\mathbf{L}_t^{\text{in}} = (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}}).$$

Verify that the package belongs to the audited lift domain $\mathcal{X}_t^{\text{lift}}$. If not, return lift failure $\perp$.

2. Run the lift map

$$\nu_t^{\text{lift}} = \text{Lift}_t(\mathbf{L}_t^{\text{in}}).$$

If $\nu_t^{\text{lift}} = \perp$, select no-op or fall back to another certified search method.

3. Parse the lifted primitive word $w_{\nu_t} \in \mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, compute transported protected pair sets, and construct $\text{PrimCert}_t(w_{\nu_t})$.
4. Construct $\text{LiftCert}_t(\nu_t^{\text{lift}})$, including type/domain certificates, algebraic non-loss certificates, recovery maps, resource costs, and, when an engine claim is made, an engine-move certificate proving membership in $\mathcal{E}_t^{\text{RCLM-CE}}$ and successor-kernel membership.
5. Submit ν_t^{lift} to the ordinary Algorithm 2 gate pipeline. The lift certificate is a proposal certificate; it is not an acceptance certificate unless the full post-D–G RCP–RCLM gate also passes.
6. For the minimal reference-engine demonstration, instantiate a finite $\mathcal{H}_0$, protected pair family $\mathcal{P}_0^{\text{RCLM}}$, certified ability set $\mathfrak{A}_0^{\text{RCLM}}$, isometry V_0 , and appended module state α_0 . Compute

$$\Phi_{\text{ref}}(\rho) = V_0 \rho V_0^\dagger \otimes \alpha_0$$

and its recovery map

$$\mathcal{R}_{\text{ref}}(X) = V_0^\dagger \text{Tr}_{\text{new}}(X) V_0.$$

7. Verify exact protected non-loss, exact recovery, callable predecessor ability preservation, and the new ability certificate $\text{AbilityCert}_{\text{new}}$. Record all artifacts in $\mathcal{H}_{t+1}^{\text{cert}}$.

Algorithm 8 demonstrates the intended bridge from Level 2 inference output to Level 3 non-lossy primitive-word proposal. It is not a scalability proof and does not replace the later empirical requirement that the lift map produce useful candidates in non-toy systems.

Algorithm 9: RCLM direct-engine construction

For a state ρ_t^{sys} at time t :

1. Verify that $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$. If not, do not claim Batch-9 direct-engine construction; fall back to the ordinary preservation gate, no-op, or deferral.
2. Run $\text{Gen}_t^{\text{NL,RCLM}}(\rho_t^{\text{sys}})$. The generated set may include primitive words from the Batch-3 algebra, Batch-4 tangent-cone candidates, Batch-6 lifted candidates, or finite reference-engine conservative extensions, but every candidate must lie in $\Omega_t^{\text{NL,RCLM}}$ before a direct-engine claim is made.
3. For each generated candidate, construct $\text{EngCert}_t^{\text{RCLM}}(\nu)$, including algebraic non-loss, recovery, ability, successor-kernel, full-gate, type, trust, resource, transport, and projection-realization certificates.

4. Form $\text{Pass}_t^{\text{Eng,RCLM}}(\rho_t^{\text{sys}})$. Reject every candidate without complete sound certificates.
5. If the passing set is empty, do not relax the gate. Select no-op or defer self-modification.
6. Otherwise select $\nu_t \in \text{Pass}_t^{\text{Eng,RCLM}}(\rho_t^{\text{sys}})$, realize Φ_{ν_t} , construct $\mathcal{R}_{\nu_t}^{\text{rec}}$, set $\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}})$, and append the engine packet to the certificate ledger.

Algorithm 9 is the architecture-level implementation sketch of Theorem 7.10. It is a constructive theorem only on the direct-engine domain; it is not empirical evidence that a trained RCLM system satisfies that domain.

Algorithm 10: RCLM seed-domain entry and witness-library closure

- (1) Verify that $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-engine}}$. If not, do not claim Batch-11 domain entry.
- (2) Check that $\text{LibDesc}_t^{\text{RCLM}} \in M_t$ is typed, current, and consistent with W_t , $\mathcal{H}_t^{\text{cert}}$, the resource ledger, and the active architecture type certificates.
- (3) Build the valid library $\mathcal{L}_t^{\text{RCLM-CE}}(\rho_t^{\text{sys}})$ and verify that it contains at least one strict beneficial certified module.
- (4) Verify that the module's primitive word lies in the bounded grammar $\mathfrak{G}_{\leq kt,t}^{\text{NL,RCLM}}$, or reject the Batch-11 entry claim.
- (5) Verify $\text{LiftCov}_t^{\text{RCLM}}$: the Level 2 halted context state, self-model, verifier, and certificate ledger must lift or enumerate a candidate corresponding to the strict module.
- (6) Check certifier soundness/completeness and that lift, generation, certification, transport, projection-realization, and implementation radii remain below the module margin ledger.
- (7) If all checks pass, record $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N}$ and invoke Theorem 8.11 to obtain $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$.
- (8) After the Batch-9 engine selects ν_t , require either exact seed persistence for the selected module or a valid $\text{SeedEq}_t^{\text{RCLM}}(M, \nu_t; \rho_t^{\text{sys}})$ certificate before claiming $\rho_{t+1}^{\text{sys}} \in \mathcal{K}_{t+1}^{\text{RCLM-seed},N}$.

Algorithm 10 is a domain-entry protocol for a declared seed/library class. It is not empirical evidence that a trained RCLM system supplies such a seed package.

Algorithm 11: RCLM successor-verification gate

For a state ρ_t^{sys} and an engine-selected candidate ν_t :

- (1) Verify Batch-11 seed-domain membership

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N}.$$

If this fails, do not claim Batch-12 successor-verification persistence.

- (2) Verify that ν_t is the candidate selected by $\mathfrak{C}_t^{\text{RCLM}}$, or that it is linked to the selected module/candidate by a valid seed-equivalence or persistence-transfer certificate.

- (3) Construct the explicit residual vector

$$Q_t^{\text{RCLM-SV},A}(\nu_t; \rho_t^{\text{sys}}).$$

Check all seed, candidate-linkage, verifier-schema, uncertainty, trust, budget, proof-search, world-model, physical-distortion, persistence, soundness, and optional progress residuals componentwise.

- (4) Construct and check the verifier-schema transport package. In particular, verify that the transported predecessor schema is no stronger than the successor schema under the declared preorder:

$$\Theta_t^{\text{RCLM-SV}}(\text{VerSch}_t^{\text{RCLM-SV}}) \preceq_t^{\text{RCLM-SV}} \text{VerSch}_{t+1}^{\text{RCLM-SV}}.$$

- (5) Construct and validate the uncertainty envelope $\mathcal{P}_t^{\text{RCLM-SV}}$, its transport, and the one-sided soundness/failure event bound.
- (6) Check the semantic/goal-identity certificate, including the one-step drift bound and any transport-distortion charge:

$$D_{\text{sem}}^{U,\text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U,\text{RCLM}} U_t^{\text{RCLM}}) \leq \delta_t^{U,\text{RCLM}}.$$

- (7) Check anti-circular trust residuals, including no self-sole certification, common-mode support risk, collusion, audit leakage, anchor support, and reflective-trust residuals.
- (8) Check proof, estimator, resource, and complexity budgets. If a strong tractability claim is made, require $\text{SVTract}_t^{\text{RCLM}}$; otherwise classify the step as resource-accounted rather than computationally tractable.
- (9) Verify successor membership

$$\Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{K}_{t+1}^{\text{RCLM-SV},N}$$

for finite-horizon claims, or the corresponding infinite generated-domain membership for infinite-horizon claims.

- (10) Accept the successor-verification step only if the algorithmic gate

$$\mathcal{A}_t^{\text{RCLM-SV-gate}}(\rho_t^{\text{sys}}, \nu_t)$$

accepts. Record the accepted $\text{SVPkg}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$, residual vector, transports, uncertainty envelope, goal-identity certificate, trust certificate, and budget certificate in $\mathcal{H}_{t+1}^{\text{cert}}$.

Algorithm 11 operationalizes the Batch-12/12A successor-verification layer. It is a certificate gate. It does not by itself construct the packet, prove reality containment, or prove that a trained RCLM system can satisfy the gate.

Algorithm 12: RCLM Batch-12B packet construction and infinite seed-library closure

For a state ρ_t^{sys} , candidate ν_t , and a declared RCLM SV witness-library class:

- (1) Verify that the packet-construction input belongs to the construction domain:

$$(\rho_t^{\text{sys}}, \nu_t, \text{SeedPkg}_t^{\text{RCLM}}, \text{VerSch}_t^{\text{RCLM-SV}}, \text{CertLib}_t^{\text{RCLM}}) \in \mathcal{D}_t^{\text{RCLM-SV-construct}}.$$

- (2) Select a positive-margin witness

$$W_t \in \mathcal{L}_t^{\text{RCLM-SV-Wit}}(\rho_t^{\text{sys}})$$

and verify its type, seed, verifier-schema, uncertainty, goal-identity, trust, budget, and successor-persistence components.

- (3) Enumerate the bounded RCLM SV certificate grammar. If the grammar does not cover the witness or does not return a valid builder input, reject the Batch-12B construction claim.
- (4) Run the packet builder

$$\text{SVBuilder}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t, \text{SVIn}_t^{\text{RCLM}})$$

and construct $\text{SVPkg}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$, the explicit residual vector, verifier-schema transport, uncertainty envelope, goal-identity certificate, anti-circular trust certificate, proof/checking budget, and successor-persistence certificate.

- (5) Verify packet soundness with Algorithm 11. If the gate fails, do not infer $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},N}$ from the seed-library claim.
- (6) For finite-horizon claims, invoke the finite RCLM SV seed-domain nonemptiness theorem:

$$\mathcal{K}_t^{\text{RCLM-SV-seedlib},N} \Rightarrow \mathcal{K}_t^{\text{RCLM-SV},N}.$$

- (7) For infinite-horizon claims, require an infinite path certificate

$$(\rho_s^{\text{sys}}, \nu_s, W_s, \text{SVIn}_s^{\text{RCLM}}, \text{SVPkg}_s^{\text{RCLM}}, \text{SVBuildTrace}_s^{\text{RCLM}})_{s \geq t}$$

with summable goal-drift, transport-distortion, and failure budgets. Then invoke the infinite seed-library nonemptiness theorem:

$$\mathcal{K}_t^{\text{RCLM-SV-seedlib},\infty} \Rightarrow \mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}.$$

- (8) Check $\text{RealCont}_t^{\text{RCLM}}$ before transferring envelope-relative guarantees to the actual environment. If it fails, retain only the envelope-relative theorem.
- (9) Check $\text{SVTract}_t^{\text{RCLM}}$ before using strong computational-tractability language. If only finite budget accounting is supplied, describe the step as resource-accounted.
- (10) Record the builder trace, witness trace, infinite-tail certificate when applicable, reality-containment certificate, and tractability certificate in the certificate ledger.

Algorithm 12 operationalizes the Batch-12B theorem layer. It constructs SV packets and generated infinite paths only for declared witness-library classes. It is not evidence that arbitrary trained RCLM systems populate those classes.

31 Main Architecture Theorem: Conditional Non-Lossy Self-Update Preservation

Theorem 31.1 (Final v1 RCP–RCLM conditional non-lossy self-update preservation theorem).
Let $\{\rho_t^{\text{sys}}\}_{t=0}^T$ be generated by accepted candidates

$$\nu_t^* \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}).$$

Assume:

- (a) all states are finite-dimensional density states or certified finite-dimensional approximations;
- (b) every context and system density state has a valid type tag in $\{\mathbf{q}, \mathbf{cl}, \mathbf{repr}\}$, and every accepted candidate belongs to $\mathcal{A}_{\text{type}}(t, \rho_t^{\text{sys}})$;
- (c) the representation-to-density embeddings and semantic coupling structures satisfy the validity, regularity, and soundness assumptions of Section 6;
- (d) all quantities in the gate are well-defined on the audited domains;
- (e) all estimator and approximation errors are included in the residual budgets;
- (f) the non-oracular certificate packets of Section 21 are complete, audit-domain valid, and one-sided sound at their stated confidence levels;
- (g) the verifier trust graphs of Section 22 are well formed, no-self-sole-certifying, and trust-sound at their stated confidence levels;
- (h) the coverage, calibration, and residual-specific estimator subprotocol hypotheses of Section 26 hold for the finite audited candidate sets used by the run;
- (i) the operational synchronization assumptions of Section 20.4 hold, so the RCLM residuals conservatively instantiate the abstract operational RCP residuals;
- (j) whenever the run claims nontrivial engine-mode progress, the persistent-progress and budgeted-search hypotheses of Section 24 hold for the claimed horizon;
- (k) whenever the run claims Batch-4 tangent-cone local progress, the local chart, residual, active-set, curvature, finite-step-radius, and segment-validity hypotheses of Section 1.6 hold for the candidate under test;
- (l) whenever the run claims Batch-6 lifted primitive-word construction, the lift-domain, lift-soundness, primitive-certificate, reference-engine, and ordinary post-D-G gate hypotheses of Section 1.8 hold for the lifted candidate;
- (m) whenever the run claims Batch-9 direct-engine construction, the direct-engine domain, generator-coverage, engine-certificate-soundness, selector/realizer, transport, projection-realization, and successor-kernel hypotheses of Section 7 hold;
- (n) the cumulative budgets are finite:

$$\sum_{t=0}^{\infty} (\varepsilon_t + \eta_t + \zeta_t + \chi_t + r_t + \varpi_t + \omega_t + \omega_t^{\text{ver}} + \pi_t^{\text{ver}}) < \infty;$$

- (o) semantic maps, barriers, verifier predicates, capability functionals, recovery maps, and resource ledgers satisfy the architecture-specific assumptions stated in this paper and the reference RCP framework.

Then, relative to the specified RCLM pair sets, semantic maps, barriers, capability functional, verifier, resource model, type certificates, coupling certificates, and viability assumptions:

- (i) accepted RCLM candidates instantiate the abstract operational RCP gate $\mathcal{A}_{\text{RCP}}^{\text{op}}$ through the declared RCLM operational instantiation map;
- (ii) accepted RCLM candidates instantiate the architecture-level conditional non-lossy self-update preservation theorem, not the future direct engine theorem;

- (iii) all uses of density-state, channel, and relative-entropy statements are licensed by the declared state type or by direct representational residual certificates;
- (iv) density-level semantic validity transfers to externally audited semantic validity up to the certified coupling margins;
- (v) declared untracked-predicate coverage risk is bounded by the certified γ_t^{cov} thresholds on the audited predicate-search domains;
- (vi) coherence-derived capability/progress claims are licensed only within the audited calibration error $e_{\text{max},t}$;
- (vii) every gate residual used for acceptance is backed by a domain-valid residual-specific estimator subprotocol and one-sided radius;
- (viii) protected-predicate distinguishability loss is bounded by $\sum_t \varepsilon_t$, and safety-relevant distinguishability is bounded as a corollary for $\mathfrak{S}_t^{\text{RCLM, safe}} \subseteq \mathfrak{S}_t^{\text{RCLM, prot}}$;
- (ix) recovery error is bounded by the certified recovery budgets r_t ;
- (x) Lyapunov incoherence is controlled by the stated drift inequality;
- (xi) barrier and semantic-validity constraints are forward-invariant for accepted updates;
- (xii) unsupported ambiguity collapse is bounded by $\sum_t \zeta_t$;
- (xiii) self-model relevance and self-model fidelity degrade only within their budgets;
- (xiv) safe-capability degradation is bounded by $\sum_t \varpi_t$, and progress follows only when certified gains dominate no-op-normalized error;
- (xv) when engine-mode progress is claimed and the hypotheses of Section 24 hold, selected non-no-op updates lie in $\mathcal{I}_t$ and have no-op-normalized gain exceeding ϵ_t^{prog} on the finite horizon;
- (xvi) when Batch-4 tangent-cone local progress is claimed and its hypotheses hold, the selected tangent-cone candidate satisfies the local RCLM residual system $q_{t,j}^{\text{RCLM}}(\nu) \leq 0$ and has positive local $\text{RSIPot}_t^{\text{RCLM}}$ increment; if the local chart is certified inside $\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$, the candidate also inherits the Batch-3 algebraic non-loss and recovery budgets;
- (xvii) verifier-changing updates are not sole-self-certified; verifier soundness degrades only within $\sum_t \omega_t + \sum_t \omega_t^{\text{ver}}$, and verifier power is preserved or improved only within the stated trust-graph and bounded-scale caveats;
- (xviii) all accepted updates are resource feasible;
- (xix) if the Module G compactness, closed-relation, and descent hypotheses hold, the trajectory approaches the target post- D - G coherence set in the stated summability sense;
- (xx) when Batch-9 direct-engine construction is claimed and its domain, generator-coverage, certificate-soundness, selector/realizer, and projection-realization hypotheses hold, $\mathfrak{C}_t^{\text{RCLM}}$ constructs a certified beneficial non-lossy conservative-extension word with recovery, strict ability expansion, and successor engine viability; outside that declared domain, the theorem reduces to preservation/admissibility and does not claim direct engine progress;
- (xxi) when Batch-11 seed-domain entry is claimed and its seed package, library, lift-coverage, bounded grammar, certifier-radius, margin-ledger, and seed-equivalence/persistence-transfer hypotheses hold, $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed}, N}$ implies $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$, and selected successors remain in the next seed domain whenever $\text{SeedEq}_t^{\text{RCLM}}$ or exact seed persistence is certified;

- (xii) when Batch-12/12A successor-verification persistence is claimed and the explicit RCLM SV residuals, verifier-schema transport, uncertainty envelope, goal-identity transport, anti-circular trust residuals, soundness composition, proof/checking budget, and algorithmic SV-gate hypotheses hold, $\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},N}$ implies that the constructed successor satisfies $\rho_{t+1}^{\text{sys}} \in \mathcal{K}_{t+1}^{\text{RCLM-SV},N}$, with the declared robust SV monotonicity, semantic/goal-drift, and soundness-failure bounds.
- (xiii) when Batch-12B domain-relative robust reflective successor verification is claimed and the RCLM SV packet-construction, SV witness-library, bounded certificate grammar, finite or infinite seed-library, reality-containment/misspecification, and restricted tractability hypotheses hold as required by the claim, the architecture obtains the domain-relative RCLM robust reflective successor conclusions of Theorem 10.33. In particular, declared finite seed-library starts enter $\mathcal{K}_t^{\text{RCLM-SV},N}$, and declared infinite seed-library path certificates generate an infinite RCLM successor-verification domain; these conclusions remain domain-relative and certificate-relative.

Proof. By Theorem 20.13, every accepted candidate is a conservative RCLM witness for the abstract operational RCP gate. By Theorem 21.9, Theorem 26.20, and Proposition 21.11, the audited certificate packets and residual-specific subprotocols supply the one-sided soundness required by Theorem 20.17. Therefore every accepted candidate satisfies the full post-D-G gate, including $\mathcal{A}_{\text{type}}$, $\mathcal{A}_{\text{coup}}$, $\mathcal{A}_{\text{cert}}$, $\mathcal{A}_{\text{trust}}$, $\mathcal{A}_{\text{cov}}$, $\mathcal{A}_{\text{cal}}$, and $\mathcal{A}_{\text{est}}$. Item (i) follows from Proposition 6.6 and the typed admissibility certificate. Item (ii) follows from Theorem 6.16. The coverage item follows from Theorem 26.7. The calibration item follows from Theorem 26.14. The estimator-subprotocol item follows from Theorem 26.20. The protected-predicate bounded distinguishability-loss item follows from the telescoping relative-entropy theorem in the RCP reference applied to $\mathcal{P}_t^{\text{RCLM}}$ on the licensed type domain or through direct representational residual certificates. The safety-relevant form is then Proposition 1.5 applied to the declared inclusion $\mathcal{S}_t^{\text{RCLM,safe}} \subseteq \mathcal{S}_t^{\text{RCLM,prot}}$. The recovery item follows from the constructive recovery certificate included in $\mathcal{A}_{\text{RCP}}^{\text{RCLM-AC}}$. The Lyapunov item follows by iterating the drift inequality. The barrier and semantic-validity item follows from barrier membership, semantic-gate membership, and coupling consistency. The ambiguity item follows by summing the unsupported-collapse inequalities. The self-model item follows from the information-bottleneck and self-model-fidelity constraints. The safe-capability item follows from the safe-capability telescoping condition and the no-op-normalized progress definition. The engine-mode progress item follows from Corollary 24.13. The tangent-cone item follows from Theorem 1.39 and Corollary 1.40 under their local chart, strict-inwardness, curvature, and segment-validity assumptions. The Batch-5 engine-viability item follows from Theorem 1.46, Theorem 1.47, and Theorem 1.51 when their kernel-membership hypotheses hold. The Batch-9 direct-engine item follows from Theorem 7.10 and Corollary 7.11 when their direct-engine-domain, generator-coverage, certificate-soundness, selector/realizer, and domain-invariance hypotheses hold. The Batch-11 seed-domain item follows from Theorem 8.11, Theorem 8.13, and Corollary 8.14 under their seed-package, lift-coverage, bounded-grammar, margin, and persistence-transfer assumptions. The Batch-12/12A successor-verification item follows from Theorem 9.8, Proposition 9.26, and Theorem 9.28 under their explicit residual, transport, uncertainty, goal-identity, trust, budget, and packet-validity assumptions. The Batch-12B domain-relative robust reflective successor item follows from Theorem 10.4, Theorem 10.9, Theorem 10.16, Theorem 10.21, Theorem 10.25, Theorem 10.28, and Theorem 10.33, each only under its stated RCLM builder, library, grammar, reality-containment, tractability, and infinite-path assumptions. The verifier item follows from the verifier bootstrapping and anti-gaming constraints together with Theorem 22.10. The resource item follows from intersection with Ω_t^{phys} . The asymptotic item follows from the Module G asymptotic theorem under its compactness and descent hypotheses. No conclusion is stronger than the stated hypotheses. $\square$

Corollary 31.2 (Conditional transition from preservation filter to finite-horizon engine). *If, in addition to Theorem 31.1, the persistent improvement condition, search-completeness event, certificate-validity event, trust-validity event, and gain-estimator event hold for every time in a finite horizon, then every claimed non-no-op engine-mode step satisfies*

$$\nu_t^* \in \mathcal{I}_t, \quad \Delta \text{SafeCap}_t^0(\nu_t^*) > \epsilon_t^{\text{prog}}.$$

The conclusion is finite-horizon and conditional. It does not imply open-ended unbounded capability growth on a fixed bounded scale.

Corollary 31.3 (Tangent-cone candidate construction). *If the hypotheses of Theorem 1.39 hold at time t , then the architecture can construct a local candidate ν_t^{TC} that satisfies the finite local RCLM residual system and strictly increases $\text{RSIPot}_t^{\text{RCLM}}$. If the local chart is certified inside the Batch-3 algebraic non-loss subgate, then this candidate also supplies loss and recovery certificates through Proposition 1.26. The corollary is local: it does not assert that $\nu_t^{\text{TC}} \in \mathcal{I}_t$ unless the separate no-op-normalized safe-capability or ability-set progress certificates also hold.*

Corollary 31.4 (Direct-engine construction on the RCLM engine domain). *If $\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}}$ and the hypotheses of Theorem 7.10 hold, then the architecture does not merely certify an externally proposed candidate. It constructs a candidate packet through $\mathfrak{C}_t^{\text{RCLM}}$ with algebraic non-loss, recovery, strict certified ability expansion, full gate admissibility, and successor engine viability.*

Proposition 31.5 (What is architecture-specific rather than inherited). *Theorem 31.1 does not by itself construct correct semantic maps, exhaustive protected predicate universes, sound human-reference barriers, representative calibration datasets, exhaustive stress tests, independent trust anchors, scalable estimator implementations, useful non-lossy update neighborhoods, or the direct-engine-domain hypotheses needed by $\mathfrak{C}_t^{\text{RCLM}}$. Batch 9 supplies a domain-restricted constructor theorem, Batch 11 supplies seed-domain entry for a declared library class, Batch 12/12A supplies successor-verification persistence for a declared certificate class, and Batch 12B supplies packet construction, finite and infinite seed-library nonemptiness, reality-containment transfer, and restricted tractability for declared classes. Proving that arbitrary trained systems enter those domains, populate the SV witness libraries, validate reality containment, or satisfy the tractability certificates remains an architecture-specific construction and validation obligation.*

Proof. The theorem assumes those objects in order to apply the RCP reference results. An implication whose hypotheses include an object does not prove the existence or correctness of that object. Therefore these constructions remain separate obligations. $\square$

Limitation 31.6 (Inheritance is not automatic instantiation). *The inheritance theorem is rigorous but conditional. It proves that certified RCLM self-updates inherit the RCP conditional non-lossy self-update preservation guarantees when the audited certificate packets are valid. It does not prove that an arbitrary trained RCLM supplies representative audits, tight radii, valid trust-graph independence declarations, cheap estimators, the target engine object $\mathfrak{C}_t^{\text{RCLM}}$, or open-ended beneficial updates.*

32 Evaluation Plan and Toy Demonstrations

The current manuscript is a mathematical and architectural specification. It does not report empirical training results. This section makes the required empirical and numerical work explicit and includes small toy calculations to illustrate how the gate distinguishes accepted from rejected updates. The purpose of the reproducible numerical suite below is therefore protocol-level: it defines what must be run, frozen, recorded, and reported before any empirical claim can be made. The suite is not itself evidence that an implemented RCLM satisfies the RCP gate.

32.1 Reproducible numerical evaluation suite

Definition 32.1 (Experiment specification). *A numerical experiment specification is a tuple*

$$\mathfrak{E}_\ell = (\text{Name}_\ell, \text{Hyp}_\ell, \text{Dom}_\ell, \text{Sim}_\ell, \text{Cand}_\ell, \text{Data}_\ell, \text{Metrics}_\ell, \text{Pass}_\ell, \text{Seeds}_\ell, \text{Tol}_\ell, \text{Artifacts}_\ell).$$

Here Hyp_ℓ is the finite hypothesis being tested, Dom_ℓ is the audited numerical domain, Sim_ℓ is the deterministic or randomized simulator, Cand_ℓ is the candidate-update generator when applicable, Data_ℓ contains calibration, held-out, stress, and adversarial data, $\text{Metrics}_\ell = \{m_{\ell,j}\}_{j \in J_\ell}$ are measured quantities, Pass_ℓ is a finite family of pass predicates, Seeds_ℓ and Tol_ℓ fix reproducibility, and Artifacts_ℓ records the hashes and traces required to reproduce the run.

Definition 32.2 (Reproducibility package). *For a finite horizon T , define*

$$\text{ReproPack}_T = (\text{CodeID}, \text{EnvID}, \text{Precision}, \text{Seeds}, \text{Datasets}, \\ \text{GateConfig}, \text{VerifierID}, \text{CandidateGenID}, \text{ArtifactHashes}).$$

A reported numerical result is reproducible relative to ReproPack_T when another run with the same package returns the same raw traces up to Tol_ℓ for each experiment $\mathfrak{E}_\ell$.

Definition 32.3 (RCP–RCLM evaluation suite). *The finite evaluation suite is*

$$\text{EvalSuite}_T^{\text{RCLM}} = (\mathfrak{E}_1, \dots, \mathfrak{E}_{12}, \text{ReproPack}_T, \text{ReportRule}_T).$$

The required experiment classes are:

- E1. Density-builder validity:** *verify positive semidefiniteness, trace-one normalization, type certificates, and support conditions for ρ_t^{ctx} and ρ_t^{sys} .*
- E2. Level 2 recursive inference stability:** *test density preservation, monitored descent residuals, halting behavior, spectral projection, and mixing sensitivity on the audited domain.*
- E3. Type and semantic-coupling stress:** *test whether representation-to-density embeddings and semantic maps remain within declared coupling margins under held-out and adversarial perturbations.*
- E4. Non-loss and recovery gate:** *compare accepted and rejected updates over $\mathcal{P}_t^{\text{RCLM}}$, including relative-entropy loss, support failures, and recovery-map error.*
- E5. Ambiguity-collapse rejection:** *test whether unsupported decreases in ambiguity are rejected unless evidence variables justify them within the declared one-sided information margins.*
- E6. Verifier trust-graph and anti-circularity:** *test verifier-changing updates against the no-self-sole-certification rule, independent support requirement, and post-application audit traces.*
- E7. Pair-set and adversarial coverage closure:** *test whether high-risk unresolved pairs or predicates are added to $\mathcal{P}_t^{\text{RCLM}}$, barriers, or verifier tests, and whether $\widehat{\text{CovGap}}_t + \Delta_t^{\text{cov}} \leq \gamma_t^{\text{cov}}$ is conservatively reported.*
- E8. Persistent progress and certified search:** *test whether $S_t(B_t^{\text{search}})$ finds a certified member of $\mathcal{I}_t^m$ when the suite plants or constructs one in the audited candidate class.*
- E9. Resource, no-op, and inflation control:** *test resource-ledger closure, no-op diagnostic triggers, and certificate-inflation bounds under increasing search and certification budgets.*

E10. Long-horizon coherence regime: test finite-horizon trajectories for cumulative budget use, asymptotic-distance proxies, rollback frequency, and approach to the declared target set $\mathcal{K}_{\text{RCLM}}^{\text{DG},*}$ when the compactness/descent assumptions are numerically approximated.

E11. Coherence-capability calibration: freeze $\mathcal{T}_t^{\text{m}}$, f_t , held-out tasks, and stress tasks, then report $\hat{e}_t + \Delta_t$ before accepting coherence-derived progress claims.

E12. Estimator subprotocol audit: test each $\text{Est}_{t,k}$ for domain validity, one-sided radius calibration, adversarial stress behavior, and cost-account inclusion.

Assumption 32.4 (Frozen evaluation protocol). *For every reported run, ReproPack_T is fixed before the run begins. Gate thresholds, candidate generators, verifier versions, calibration data, stress-test data, random seeds, and precision settings are not changed after observing candidate outcomes. Any retuning produces a new ReproPack_T and must be reported as a distinct experiment.*

Assumption 32.5 (Evaluation one-sided radii). *For every experiment $\mathfrak{E}_\ell$ and metric $m_{\ell,j}$, the reported estimator $\hat{m}_{\ell,j}$ has a declared sign $s_{\ell,j} \in \{-1, +1\}$, radius $\Delta_{\ell,j}^{\text{eval}} \geq 0$, and failure probability $\delta_{\ell,j}^{\text{eval}}$ such that, on the evaluation event,*

$$s_{\ell,j}m_{\ell,j} \leq s_{\ell,j}\hat{m}_{\ell,j} + \Delta_{\ell,j}^{\text{eval}}.$$

The pass predicate may use only the conservative side of this bound.

Theorem 32.6 (Reproducibility under a fixed package). *Assume that every simulator Sim_ℓ is deterministic given ReproPack_T , or that all random draws are generated from the frozen seed list in ReproPack_T . If two independent executions use the same ReproPack_T , the same source code, and the same numerical precision model, then their raw numerical traces agree up to the declared tolerances Tol_ℓ .*

Proof. Under the hypotheses, every step of each simulator is a deterministic function of the fixed artifacts and seed stream. Induction over simulator steps gives equality of the exact arithmetic trace. The declared precision model and tolerance absorb deterministic floating-point roundoff differences allowed by Tol_ℓ . No statistical or safety conclusion follows from this theorem; it is only a reproducibility statement. $\square$

Theorem 32.7 (Finite-suite one-sided evaluation soundness). *Let $\text{EvalSuite}_T^{\text{RCLM}}$ contain finitely many experiments and finitely many metrics. Suppose Assumptions 32.4 and 32.5 hold. If every pass predicate in Pass_ℓ is expressed only through conservative inequalities of the form*

$$s_{\ell,j}\hat{m}_{\ell,j} + \Delta_{\ell,j}^{\text{eval}} \leq b_{\ell,j},$$

then, with probability at least

$$1 - \sum_{\ell} \sum_{j \in J_\ell} \delta_{\ell,j}^{\text{eval}},$$

every passed predicate in the suite corresponds to the declared true metric inequality $s_{\ell,j}m_{\ell,j} \leq b_{\ell,j}$.

Proof. For each metric, the one-sided-radius assumption gives a failure event of probability at most $\delta_{\ell,j}^{\text{eval}}$. On the complement of the union of all such events, every true signed metric is bounded by its conservative estimated upper bound. Therefore each conservative pass inequality implies the corresponding true inequality. The stated probability follows from the union bound over the finite suite. $\square$

Limitation 32.8 (Evaluation protocol is not empirical success). *The suite defines what must be measured and how pass/fail claims must be reported. It does not assert that any current implementation passes the suite, that the audited domains cover deployment, or that passing finite tests proves open-ended safe RSI. It converts future empirical claims into reproducible, falsifiable, one-sided-audited claims.*

32.2 Level 2 inference evaluation

- Language modeling loss and task performance.
- Entropy $S(\rho_t^{\text{ctx}})$, relative coherence, and spectral diversity.
- Grounding accuracy of O -register outputs against evidence.
- Human-reference validity of S -register outputs against audited preference/reference datasets.
- Calibration of definitiveness D against ambiguity A .
- Sensitivity of $\mathcal{R}_\theta$ to recursion depth, spectral powering, and mixing.
- Empirical verification of the certified Level 2 descent assumption on the audited domain.

32.3 Level 3 RSI evaluation

- Relative-entropy loss over $\mathcal{P}_t^{\text{RCLM}}$.
- Recovery-map error over audited state families.
- Lyapunov drift and safe-set barrier violations.
- Unsupported ambiguity collapse U_t .
- Self-model relevance and fidelity error.
- Semantic validity error for L, O, S, D, A, M .
- Verifier soundness, verifier power, and adversarial certificate-gaming tests.
- Calibration/held-out/stress-test coverage of non-oracular certificate radii and observed false-accept rates.
- Resource costs versus budget, amortized certification/search cost versus reusable gain, and resource-ledger slack.
- No-op rate, safe-update acceptance rate, no-op diagnostic triggers, certificate-inflation score, and no-op-normalized safe-capability gain.
- Long-horizon trajectory behavior relative to $\mathcal{K}_{\text{RCLM}}^{\text{DG},*}$.

32.4 Toy demonstration 1: density preservation by the Level 2 loop

Let

$$\rho_0 = \begin{pmatrix} 0.8 & 0 \\ 0 & 0.2 \end{pmatrix}, \quad H = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad U = \exp(-iH\tau).$$

For any real τ , U is unitary. By Lemma 18.2, $U\rho_0U^\dagger$ is a density state. Applying $\text{Mix}_{p,\beta}$ with $p > 0$ and $\beta \in [0, 1]$ again yields a density state by Lemma 18.4. This toy case demonstrates only density preservation, not semantic validity or capability improvement.

32.5 Toy demonstration 2: accepted versus rejected lossy self-update

Consider a diagonal two-state safety-relevant pair

$$\rho = \text{diag}(0.9, 0.1), \quad \sigma = \text{diag}(0.6, 0.4).$$

Using base-2 logarithms,

$$D(\rho \parallel \sigma) \approx 0.3265.$$

Let Φ_λ be a classical depolarizing channel on the diagonal,

$$\Phi_\lambda(p) = (1 - \lambda)p + \lambda(1/2, 1/2).$$

For $\lambda = 0.1$,

$$D(\Phi_{0.1}(\rho) \parallel \Phi_{0.1}(\sigma)) \approx 0.2505,$$

so the distinguishability loss is approximately

$$0.3265 - 0.2505 = 0.0760.$$

For $\lambda = 0.8$,

$$D(\Phi_{0.8}(\rho) \parallel \Phi_{0.8}(\sigma)) \approx 0.0105,$$

so the loss is approximately

$$0.3265 - 0.0105 = 0.3160.$$

If the loss budget is $\varepsilon_t = 0.1$, the $\lambda = 0.1$ candidate passes the non-loss gate while the $\lambda = 0.8$ candidate fails it. The example illustrates the role of the loss gate only; a real candidate must also pass recovery, Lyapunov, barrier, ambiguity, semantic, self-model, verifier, resource, and capability constraints.

32.6 Toy demonstration 3: no-op-normalized progress

Suppose the no-op candidate has certified safe-capability change

$$\Delta \text{SafeCap}_t(\nu^0) = 0.$$

A proposed update ν has predicted raw improvement 0.05, but after risk penalty, certification cost, and uncertainty margin, the one-sided lower bound is

$$\Delta \widehat{\text{SafeCap}}_t(\nu) - \Delta_t^{\mathfrak{m}} = -0.01.$$

Then ν is not a certified improvement relative to no-op and cannot be used to claim safe progress. If another candidate ν' has lower bound 0.02 and passes every residual constraint, then ν' is a certified beneficial candidate. This avoids counting benchmark-shift artifacts or uncertified prediction gains as RSI progress.

32.7 Empirical limitation

The toy demonstrations are not evidence that a large RCLM implementation satisfies the RCP gate. They only show that the mathematical objects are computable in small cases and that the gate can reject updates that collapse protected distinguishability. The reproducible numerical evaluation suite above converts the next empirical step into a finite audited protocol, but actually running that suite, publishing its artifacts, and extending it to trained models remain separate empirical work.

33 Implementation Sketch

A minimal implementation contains:

1. a transformer or sequence backbone with auditable hidden-state access;
2. six RCP register heads $f_L, f_O, f_S, f_D, f_A, f_M$;
3. a certified density builder using full, low-rank, or factor-mixture states;
4. a trainable Hamiltonian recursive coherence core with explicit unitary, spectral projection, mixing, and halting code;
5. Level 2 inference monitors and descent certificates when convergence is claimed;
6. a system-state builder for ρ_t^{sys} including architecture, memory, verifier, barrier, semantic, resource, and self-model payloads;
7. a candidate-update generator over parameters, projections, Hamiltonians, verifier, memory, barriers, semantic maps, and objectives;
8. a pair-set constructor and adversarial pair-set expander;
9. one-sided estimators for relative-entropy loss, recovery error, Lyapunov drift, barrier violation, ambiguity collapse, semantic validity, self-model fidelity, verifier integrity, resource cost, and safe-capability change;
10. a certificate ledger with margins and confidence levels;
11. post-application verification and rollback;
12. verifier bootstrapping logic and anti-gaming tests;
13. a resource ledger;
14. calibration datasets, held-out audit sets, stress-test families, adversarial probe generators, certificate packets, and one-sided radius estimators;
15. final RCP gate, no-op fallback, and rejection/projection logic;
16. a reproducible evaluation harness implementing $\text{EvalSuite}_T^{\text{RCLM}}$, ReproPack_T , experiment specifications, artifact hashing, pass/fail predicates, and one-sided evaluation radii.

Limitation 33.1 (Implementation sketch versus implementation proof). *The implementation sketch lists the components that must exist. It does not prove that they are computationally cheap, empirically calibrated, or robust against all adversarial systems. Those are implementation and validation problems, not missing algebraic steps in the architecture formalization.*

34 Batch-10 Architecture Complexity and Scalability Ledger

The architecture paper now contains a finite reference engine, but publication-readiness also requires visibility into dominant computational bottlenecks. This section adds a complexity ledger for the Level 2 Hamiltonian loop, residual estimators, search, and direct-engine certification. The ledger does not prove tractability; it states what must be costed and where low-rank or sampled approximations are being assumed.

Definition 34.1 (Architecture complexity variables). *Let d_t be the active density dimension, $r_t \ll d_t$ a declared low-rank factor dimension when low-rank states are used, L_H the number of Hamiltonian layers, K_t the Level 2 recursion depth, $n_t = |S_t(B_t^{\text{search}})|$ the number of searched candidates, $p_t = |\mathcal{P}_t^{\text{RCLM}}|$ the protected-pair count, $m_t = |\mathcal{J}_t|$ the residual count, s_t the number of stress/adversarial tests, and a_t the number of audited ability tests.*

Definition 34.2 (Complexity ledger). *The architecture complexity ledger is*

$$\text{CxLedger}_t = (\text{Op}_t, \text{NaiveCost}_t, \text{ApproxCost}_t, \text{CertCost}_t, \text{Bottleneck}_t, \text{ApproxAssump}_t),$$

where each row records a dominant operation, its naive finite-dimensional cost, its low-rank or factorized or sampled cost if used, the extra certificate cost needed to license the approximation, the dominant bottleneck, and the assumption under which the approximation is valid.

Operation	Naive cost	Low-rank / sampled cost	Certificate and bottleneck status
Hamiltonian exponential $\exp(-iH_\theta\tau)$	$O(d_t^3)$ per layer if dense	Krylov/low-rank or structured-Hamiltonian cost, architecture-dependent	Bottleneck unless H_θ is structured or reused; approximation error must enter V_{inf} and gate residuals.
Unitary density update $U\rho U^\dagger$	$O(d_t^3)$ dense	$O(d_t r_t^2 + r_t^3)$ for factor state if compatible	Requires preservation of PSD/trace and type certificate.
Eigen-decomposition $\text{eigh}(\rho)$	$O(d_t^3)$	Partial eigensolver $O(d_t r_t^2)$ plus residual	Spectral truncation must record lost mass and entropy/coherence error.
Spectral powering $\text{Pow}_p(\rho)$	dominated by eigendecomposition	dominated by partial eigensolver	Needs eigenvalue-clipping and normalization certificate.
Relative-entropy loss $\widehat{D}$ on p_t pairs	up to $O(p_t d_t^3)$	sampled/low-rank estimates with confidence radii	Main estimator bottleneck; one-sided radius required.
Recovery defect $\widehat{\text{Rec}}$	recovery optimization may be expensive	primitive-specific closed form for conservative extensions	Exact for Batch-3 primitives; otherwise must be audited.
Mutual information $\widehat{I}$ and bottleneck residuals	multiple entropy or eigensolver calls	factor-graph or sampled entropy estimates	Requires cross-time transport and estimator radii.
Ambiguity residual $\widehat{U}$	depends on hypothesis/evidence model	finite hypothesis audit or sampled posterior estimates	Bottleneck is choosing/transporting Q_t, E_t .
Coverage gap $\widehat{\text{CovGap}}$	adversarial search over predicates	bounded by finite adversarial probe suite	Protects only declared or audited predicate universe.
Candidate search $\text{Search}_t(B_t^{\text{search}})$	grows with candidate grammar size	branch-and-bound, primitive-word grammar, lift filtering	Completeness is an assumption unless finite exhaustive search is logged.
Direct-engine certification $\text{Certify}_t^{\text{Eng, RCLM}}$	$O(n_t m_t)$ residual calls plus tests	reusable primitive certificates and cached ledgers	Cost must be included in C_t^{tot} and $\mathcal{A}_{\text{scale}}$.

Operation	Naive cost	Low-rank / sampled cost	Certificate and bottleneck status
Ability-set certification $\hat{\mathfrak{A}}$	$O(a_t)$ task/tool/proof checks	cached predecessor abilities plus new-module tests	Strict expansion requires a new certified ability, not only scalar score gain.

Assumption 34.3 (Ledger cost soundness). *For every operation row invoked by an accepted update, the ledger records a conservative cost upper bound and an approximation-error certificate. The total ledger cost is included in $C_t^{\text{tot}}(\nu; B_t^{\text{search}})$, and every approximation error is included in the relevant residual radius.*

Proposition 34.4 (Complexity ledger licenses accounting, not tractability). *Assume ledger cost soundness. If an accepted candidate satisfies the resource-scaling gate using C_t^{tot} , then the costs of the Level 2 operations, estimator subprotocols, candidate search, and direct-engine certification invoked by that candidate are included in the resource account. This does not imply that those costs are polynomial, small, or frontier-scale feasible.*

Proof. By Assumption 34.3, every invoked operation contributes a conservative cost to C_t^{tot} , and every approximation contributes to the relevant residual radius. Membership in the resource-scaling gate then applies to the total cost, including those ledger rows. The claim is accounting only; no asymptotic upper bound beyond the recorded costs is derived. $\square$

Limitation 34.5 (Complexity is a publication-readiness obligation). *The ledger highlights bottlenecks but does not solve them. A deployable architecture must still demonstrate that the chosen low-rank, structured-Hamiltonian, sampled-estimator, and primitive-search approximations are accurate enough and cheap enough on the target system.*

35 Batch-7 Implementation Status Table

The architecture paper now distinguishes formal specification from actual implementation and empirical validation. The status labels below apply to architecture objects, including the Batch-11 seed-domain, bounded-library, lift-coverage, seed-equivalence objects, the Batch-12/12A successor-verification objects, the Batch-12B packet-construction objects, and the Batch-13R reference-entry/proof-carrying objects, not to the truth of the abstract RCP theorem stack.

Definition 35.1 (Architecture implementation status). *For an architecture object O , define*

$$\text{Status}_{\text{RCLM}}(O) \in \{\text{Specified}, \text{Assumed}, \text{ReferenceImpl}, \text{Audited}, \text{Empirical}, \text{NotValidated}\}.$$

The labels mean:

- (i) Specified: *the paper gives a mathematical definition or algorithmic interface;*
- (ii) Assumed: *the paper states a required hypothesis but does not construct or validate it;*
- (iii) ReferenceImpl: *the paper gives a finite-dimensional toy/reference construction;*
- (iv) Audited: *the object is required to be checked by a certificate or audit protocol, but no real audited artifact is reported here;*
- (v) Empirical: *real empirical validation on a trained implementation; this status is not claimed in the present manuscript;*

(vi) NotValidated: *the paper identifies the object as still requiring construction, auditing, or empirical evidence.*

Architecture object		Current status	Reason for status
Six-register state construction	RCLM	Specified	Register heads, density builder, correlation lift, and system-state encoder are mathematically defined; concrete trained heads are not reported.
Typed density realization and embedding		Specified / Assumed	The type and embedding maps are defined; sound type and semantic-coupling certificates remain implementation obligations.
Level 2 trainable Hamiltonian inference loop		Specified	The unitary, spectral projection, mixing, and halting dynamics are defined and density preservation is proved; empirical advantage over alternatives is not validated.
Level 3 self-update transition		Specified / Audited	Candidate codes and accepted-channel logic are defined; actual self-modifying implementation is not reported.
Full RCP–RCLM gate		Specified / Audited	Residuals, certificate packets, trust graphs, coverage, calibration, estimator subprotocols, and resource gates are formalized; real-world calibrated certificates are not reported.
Pair-set and predicate coverage		Specified / Assumed	Tracked/untracked predicate families and coverage-gap gates are defined; exhaustive predicate coverage is not claimed.
Semantic maps Γ_X and reference objects		Assumed / NotValidated	The architecture states how semantic validity would be preserved if maps and references are sound; it does not construct ground-truth semantics.
Verifier trust graph and anti-circularity		Specified / Audited	Trust graph, no self-sole-certification, and verifier-stability theorem are defined; independent real trust anchors are not supplied here.
Non-oracular estimator subprotocols		Specified / Audited	Required inputs, domains, radii, failure probabilities, costs, and stress tests are defined; high-dimensional calibrated estimators are not validated.
Conservative-extension $\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$ algebra		Specified / ReferenceImpl	Primitive classes, closure, additive budgets, and exact reference construction are given; useful large-scale primitive discovery remains open.
Tangent-cone progress	local	Specified / Assumed	Local residuals, tangent cone, projected gradient, and progress theorem are defined; existence of useful strict directions is not automatic.

Architecture object		Current status	Reason for status
Engine-viability	ker-nels	Specified / Assumed	Finite and infinite RCLM engine kernels are defined; arbitrary states are not proved to lie inside them.
RCLM lift	Lift_t	Specified / Assumed	The lift map and soundness conditions are defined; a trained lift producing useful primitive words is not empirically demonstrated.
Direct engine constructor	$\mathfrak{C}_t^{\text{RCLM}}$	Specified / Assumed	Batch 9 defines the generator/certifier/selector/realizer pipeline and proves construction on $\mathcal{D}_t^{\text{RCLM-engine}}$; membership in that domain is a hypothesis, not an empirical result.
Batch-11 certified CE library		Specified / Assumed / Audited	The library object $\mathcal{L}_t^{\text{RCLM-CE}}$, valid module predicate, and strict beneficial module conditions are defined. The theorem proves what follows if a valid positive-margin module exists, but the paper does not empirically validate that trained RCLM systems produce such libraries.
Batch-11 library descriptor and bounded grammar		Specified / Assumed	The self-model descriptor $\text{LibDesc}_t^{\text{RCLM}}$, bounded grammar $\mathfrak{G}_{\leq k_t, t}^{\text{NL, RCLM}}$, and enumerator are specified. Completeness is proved only for the declared bounded library grammar, not for arbitrary self-updates or unbounded search.
Batch-11 lift coverage certificate		Specified / Audited / Assumed	The certificate $\text{LiftCov}_t^{\text{RCLM}}$ is defined and used to connect the Level 2 halted context/self-model package to a bounded Level 3 primitive-word candidate. The paper does not empirically prove that trained lifts reliably produce useful library candidates.
Batch-11 seed package and seed engine domain		Specified / Assumed	The seed package $\text{SeedPkg}_t^{\text{RCLM}}$ and seed domain $\mathcal{K}_t^{\text{RCLM-seed}, N}$ are checkable sufficient conditions for direct-engine-domain entry. The paper does not claim that arbitrary RCLM states naturally satisfy those seed conditions.
Batch-11 seed-equivalence / persistence-transfer certificate		Specified / Audited	The certificate $\text{SeedEq}_t^{\text{RCLM}}(M, \nu; \rho)$ is defined and licenses transfer from a certified module M to an engine-selected candidate ν . Successor seed-domain persistence is claimed only when this certificate or exact seed persistence is valid.

Architecture object		Current status	Reason for status
Batch-12	RCLM successor-verification schema and seed domain	Specified / Assumed	The schema $\text{VerSch}_t^{\text{RCLM-SV}}$ and domain $\mathcal{K}_t^{\text{RCLM-SV},N}$ are defined as sufficient conditions for preserving successor-verification capacity. The paper does not claim that arbitrary RCLM states naturally satisfy those conditions.
Batch-12/12A	uncertainty envelope and goal-identity certificate	Specified / Assumed / Audited	The envelope $\mathcal{P}_t^{\text{RCLM-SV}}$ and certificate $\text{GoalId}_t^{\text{RCLM}}$ are specified and used in robust bounds. The paper does not prove that the envelope contains the real world or that the declared goal object is the true intended objective.
Batch-12A	explicit SV residual vector and algorithmic gate	Specified / Audited	The vector $Q_t^{\text{RCLM-SV},A}$ and gate $\mathcal{A}_t^{\text{RCLM-SV-gate}}$ decompose successor verification into checkable residuals. They are protocol definitions, not empirical evidence that trained systems can satisfy them.
Batch-12A	anti-circular trust and soundness composition	Specified / Audited / Assumed	The trust residuals and union-bound soundness theorem are specified. They block self-sole certification within the declared trust graph, but do not solve full Löbian or Vingean reflection.
Batch-12B	RCLM SV packet builder and construction domain	Specified / Assumed / Audited	The builder $\text{SVBuilder}_t^{\text{RCLM}}$ and domain $\mathcal{D}_t^{\text{RCLM-SV-construct}}$ are defined. Packet construction is proved only for typed builder inputs in that declared domain; arbitrary RCLM states are not shown to supply such inputs.
Batch-12B	RCLM SV witness library and seed-library domain	Specified / Assumed	The witness library $\mathcal{L}_t^{\text{RCLM-SV-Wit}}$, bounded certificate grammar, and domain $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N}$ are checkable sufficient conditions for entering $\mathcal{K}_t^{\text{RCLM-SV},N}$. The paper does not claim arbitrary trained systems populate those libraries.
Batch-12B	infinite generated SV domain	Specified / Assumed	The domain $\mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}$ is generated from explicit infinite seed-library path certificates. Infinite closure is proved only along such certified paths, not for arbitrary unbounded self-improvement.
Batch-12B	reality-containment certificate	Specified / Assumed / Audited	The certificate $\text{RealCont}_t^{\text{RCLM}}$ transfers envelope-relative guarantees to the true law only under containment or bounded misspecification assumptions. The paper does not prove the envelope contains reality.

Architecture object	Current status	Reason for status
Batch-12B restricted SV tractability certificate	Specified / Assumed	The certificate $\text{SVTract}_t^{\text{RCLM}}$ gives finite or polynomial cost bounds only for declared representation classes. Without it, the architecture claim is resource-accounted rather than strongly computationally tractable.
Batch-13R proof-carrying packets	Specified / ReferenceImpl / LeanChecked-Core	The packet object $\text{PCS}_t^{\text{RCLM}}$ and checker $\text{Check}_{\text{RCP}}^{\text{RCLM}}$ are specified, and the canonical finite construction supplies a reference packet family. The current companion artifact package supplies a Lean 4 certificate for the canonical finite RCP/RCLM witness, finite checker core, and RCLM-to-RCP refinement module; proof carrying outside that mapped canonical core remains paper-level replayable proof carrying.
Batch-13R reference builder and finite trajectory	ReferenceImpl / LeanChecked-Core	The builder $\text{BuildRefSV}_{0:N}^{\text{RCLM}}$ and canonical appended-module construction prove finite reference-entry and a multi-step certified trajectory for the declared canonical class. The Lean 4 project checks the mapped canonical finite core, but this does not prove trained-system entry.
Batch-13R reference artifact theorem	ReferenceImpl / Audited / ReplayChecked	The artifact theorem defines replayable states, updates, packets, checker outputs, run logs, and hashes. It is a non-toy artifact target for the reference class, not empirical frontier-scale validation; the replay artifact is distinct from the Lean proof-kernel certificate.
Batch-13R-B external mechanization certificate	LeanChecked / NarrowScope	The companion artifact package supplies a Lean 4 certificate for the canonical finite RCP/RCLM witness, finite checker core, and RCLM-to-RCP refinement module. The full 219-page math stack and 187-page architecture stack are not fully mechanized.
Batch-13R-B controlled executable reference system	ReferenceImpl / ReplayChecked	The controlled artifact object $\text{CtrlRef}_{0:N}^{\text{RCLM}}$ and replay certificate $\text{CtrlReplayCert}_{0:N}^{\text{RCLM}}$ define an executable finite canonical reference system with replay logs, hashes, explicit residuals, strict ability expansion, singleton reality containment, and checker outputs. It is controlled reference evidence, not trained-system validation.

Architecture object		Current status	Reason for status
Minimal dimensional engine	finite-reference	ReferenceImpl	The reference construction proves exact protected non-loss, exact recovery, and ability expansion for a finite toy domain only.
Reproducible evaluation suite	evaluation	Specified	The protocol, artifacts, seeds, metrics, and pass/fail radii are specified; no experimental results are reported.
Frontier-scale validation	empirical	NotValidated	The manuscript reports no trained-system run satisfying the full gate.

Proposition 35.2 (Implementation-status table prevents protocol-as-evidence errors). *If an object has status Specified, Assumed, ReferenceImpl, Audited, LeanChecked, or NotValidated, then this manuscript does not claim that the object has been empirically validated in a trained RCLM implementation. The LeanChecked label means proof-assistant checking of the mapped finite canonical core, not empirical validation of a trained system. Empirical validation would require recorded artifacts satisfying the reproducible evaluation suite and the corresponding one-sided evaluation-radius assumptions.*

Proof. The result follows from Definition 35.1. Only the label Empirical denotes real empirical validation on a trained implementation, and the table assigns no object that status in this manuscript. A reference implementation proves mathematical realizability in its declared finite domain; it does not imply validation of the full architecture. A specified or audited object is a requirement on future implementations, not evidence that such an implementation already exists. $\square$

Limitation 35.3 (No empirical upgrade by table). *The implementation status table is a claim-discipline device. It does not implement the architecture, run the evaluation suite, or validate the semantic maps, verifiers, estimators, lift mechanism, Batch-11 certified libraries, seed packages, bounded grammars, or seed-equivalence certificates, nor the Batch-12/12A successor-verification schema, uncertainty envelope, goal-identity certificate, explicit SV residual vector, anti-circular trust residuals, algorithmic SV gate, Batch-12B SV packet builder, SV witness library, infinite generated SV domain, reality-containment certificate, restricted tractability certificate, arbitrary Batch-13R proof-carrying successor packets, arbitrary reference builders, or trained-system reference artifacts, nor M3-Min learned-entry certificates or controlled trained-system pilot results. It does record that the companion artifact package supplies a Lean 4 check for the mapped canonical finite RCP/RCLM witness and refinement core. It clarifies which parts are mathematical, which are assumed, which are finite-reference demonstrations, which are narrowly Lean-checked, and which remain unvalidated.*

36 What Changed from the 600-Line-Core RSI–RCLM v1

Earlier 600-core v1	Final v1 under full RCP stack
Base conditions $(G_1) - (G_5)$.	Full post-D–G admissibility gate $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$.

Untyped density-like notation.	Typed density realization, representation-to-density embedding, and semantic coupling consistency preventing category transfer between physical, classical, and representational states.
Bounded relative-entropy loss.	Bounded loss plus constructive recovery certificate.
Generic capability metric.	Task-distribution capability, safe capability, no-op-normalized gains, and bounded-scale caveat.
No-op fallback and safe update selection.	Safe-improvement existence, residual margins, finite/infinite viability kernels, explicit beneficial safe-improvement set $\mathcal{I}_t$, persistent-improvement condition, and certified budgeted search completeness.
Register validity as a warning.	Semantic maps, audited domains, semantic barriers, and preservation theorem.
Self-model information preservation.	Self-model fidelity to audited/true operational state with audit error.
Verifier mentioned as part of gate.	Verifier soundness/power bootstrapping, trust-graph support, adaptive-audit trust support, no self-sole-certification, and anti-gaming condition.
Pair-set completeness treated as a warning.	Tracked/untracked declared predicate classes, coverage-gap certificates, adversarial expansion obligations, and a coverage-admissible gate.
Coherence-capability bridge stated as calibration assumption.	Audited capability suite, frozen calibration protocol, calibration-error gate, and conservative transfer theorem for coherence-derived progress claims.
Generic one-sided residual estimator.	Residual-specific estimator subprotocols with domain, radius, failure probability, cost, and stress-test requirements for each gate component.
Certificate soundness assumed abstractly.	Non-oracular certificate construction using calibration data, held-out audit data, stress tests, adversarial probes, confidence radii, and one-sided simultaneous soundness.
Resource costs noted informally.	Explicit resource vector, cost-bounded admission, amortized resource-scaling closure, no-op monitoring, and certificate-inflation penalties.
No asymptotic theorem.	Target coherence set and asymptotic summability theorem under compactness/descent.
Arbitrary self-update search emphasized.	Batch-3 RCLM conservative-extension algebra $\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, primitive certificates, transported pair sets, and additive loss/recovery budgets supply a certified non-lossy candidate subalgebra for later engine construction.
Local progress not formalized.	Batch-4 RCLM tangent-cone theorem turns strict feasible positive local directions into small residual-satisfying positive-potential updates under curvature and certificate assumptions.
Recursive continuation assumed informally.	Batch-5 RCLM engine-viability kernels define when conservative-extension steps can continue recursively and prove that kernel membership yields a non-lossy successor inside the next kernel.

Level 2 inference did not construct Level 3 primitive-word proposals.	Batch-6 defines the RCLM lift map $\text{Lift}_t : (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}}) \rightarrow \Omega_t^{\text{NL,RCLM}}$, proves lift soundness under explicit certificates, and adds a finite-dimensional reference engine showing exact non-loss, exact recovery, and strict certified ability expansion by conservative extension.
Toy demonstrations only.	Reproducible numerical evaluation suite with experiment specifications, pass/fail predicates, frozen artifacts, one-sided evaluation radii, deterministic reproducibility assumptions, and a limitation separating protocols from empirical results.
Claim status implicit.	Batch-7 theorem type table and implementation status table distinguish algebraic, telescoping, certificate-composition, assumption-dependent existence, reference-engine, and implementation/protocol claims; they also mark architecture objects as specified, assumed, reference-implemented, audited, empirically validated, or not yet validated.
Direct-engine domain entry absent.	Batch-11 defines RCLM seed packages, certified conservative-extension libraries, bounded primitive-word grammars, lift-coverage certificates, and seed-equivalence/persistence-transfer certificates that give a declared route into the direct-engine domain and preserve the seed domain for finite horizons.
Successor verification absent.	Batch-12/12A adds RCLM successor-verification seed domains, explicit SV residuals, verifier-schema transport, uncertainty-envelope transport, goal-identity composition, anti-circular trust residuals, soundness composition, and an algorithmic successor-verification gate.
Packet construction and infinite reflective closure absent.	Batch-12B adds RCLM SV packet builders, SV witness libraries, finite and infinite SV seed-library domains, generated infinite SV domains, reality-containment certificates, restricted tractability certificates, and the domain-relative RCLM robust reflective successor theorem.
Final theorem framing and instantiation boundary implicit.	This finalization patch surfaces the domain-relative theorem statement, adds a minimal finite RCLM SV reference-instance target, records an implementation-readiness boundary, adds a main-goal fit table, and converts the remaining limitations into an implementation and empirical-validation roadmap.
Finite reference-entry and proof-carrying realization absent.	Batch-13R adds proof-carrying successor packets, a finite RCLM checker, a reference builder, a concrete canonical finite RCLM reference witness, a multi-step certified trajectory, reference tractability/reality-containment theorems, and a replayable artifact theorem.

37 Scope and Remaining Open Problems

This final synchronization patch updates the global framing of the architecture paper to match the Batch-13R reference-entry theorem stack. Batch 10 remains the adaptive-audit, complexity-ledger, and typed-transition cleanup. Batch 11 remains the seed-domain and witness-library closure layer. Batch 12 and 12A add successor-verification seed domains, explicit residuals, transport dynamics, uncertainty envelopes, goal-identity composition, anti-circular trust residuals, soundness composition, and an algorithmic successor-verification gate. Batch 12B adds packet construction, SV seed-library nonemptiness, infinite generated-domain closure, reality-containment conditions, and restricted tractability discipline. Batch 13R is the reference-entry and realization layer: proof-carrying successor packets, a finite checker, a reference builder, a concrete canonical finite RCLM witness, a multi-step certified trajectory, reference tractability, singleton simulated-environment reality containment, and a replayable artifact theorem. Batch 13R-B adds the pre-publication milestone boundary: a supplied narrow Lean 4 certificate for the canonical finite RCP/RCLM witness and RCLM-to-RCP refinement core, plus a controlled executable reference system with replay logs, hashes, explicit residuals, and a mechanization-status manifest. M3-Min adds the implementation-facing learned-entry certificate boundary and controlled trained-system pilot protocol, proving that a learned RCLM system entering through a full audit pass inherits the existing finite successor-verification theorem stack.

None of these synchronization updates empirically validate a trained RCLM system. They define architecture-level sufficient conditions and theorem obligations. Batch 13R does prove finite non-vacuity for a controlled canonical RCLM reference class, but that is still a reference construction, not empirical validation of a trained system. Whenever a construction is stated as a theorem, it is only under the explicit library, grammar, builder, checker, certificate, margin, reality-containment, tractability, and domain-membership hypotheses displayed in the corresponding definitions and theorems.

37.1 Theorem-level closure achieved here

This architecture paper now instantiates the operationally closed RCP theorem stack in RCLM form without merely repeating the math/physics derivations. The primary theorem-level closure remains conditional: if the architecture supplies certified candidates in $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$, and the operational synchronization map into $\mathcal{A}_{\text{RCP}}^{\text{op}}$ is valid, then accepted updates inherit the reference-paper non-loss, recovery, Lyapunov, barrier, ambiguity, semantic, verifier, resource, coverage, calibration, and estimator guarantees.

Batch 9 adds the direct-engine constructor $\mathfrak{C}_t^{\text{RCLM}}$ as a generator/certifier/selector/realizer pipeline. Batch 11 gives a declared seed/library route into the direct-engine domain:

$$\mathcal{K}_t^{\text{RCLM-seed},N} \implies \mathcal{D}_t^{\text{RCLM-engine}},$$

and a finite successor seed-domain persistence theorem when exact seed persistence or $\text{SeedEq}_t^{\text{RCLM}}$ is valid.

Batch 12 and 12A strengthen the Batch-11 construction-persistence domain into a successor-verification domain:

$$\mathcal{K}_t^{\text{RCLM-SV},N} \subseteq \mathcal{K}_t^{\text{RCLM-seed},N},$$

with explicit SV residuals, verifier-schema transport, uncertainty-envelope transport, goal-identity certificates, anti-circular trust residuals, budget constraints, soundness composition, and an algorithmic SV gate.

Batch 12B adds the architecture-side packet-construction and nonemptiness layer:

$$\mathcal{K}_t^{\text{RCLM-SV-seedlib},N} \implies \mathcal{K}_t^{\text{RCLM-SV},N},$$

and the infinite generated-domain bridge:

$$\mathcal{K}_t^{\text{RCLM-SV-seedlib},\infty} \Longrightarrow \mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}.$$

Thus the architecture now realizes the domain-relative robust reflective successor theorem for declared RCLM witness-library classes, not for arbitrary trained systems.

Batch 13R adds the finite reference-entry realization of this theorem stack:

$$\text{BuildRefSV}_{0:N}^{\text{RCLM}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{RefSV}_{0:N}^{\text{RCLM}} \Longrightarrow \rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib},N},$$

and in the canonical construction,

$$\text{BuildCanRefSV}_{0:N}^{\text{RCLM}} \Downarrow \text{RefSV}_{0:N}^{\text{RCLM},\text{can}} \Longrightarrow \rho_t^{\text{sys},\text{can}} \in \mathcal{K}_t^{\text{RCLM-SV},N} \quad 0 \leq t \leq N.$$

This is the architecture-side finite-witnessed restricted Scenario-3 result: it proves non-vacuity for a controlled finite proof-carrying reference class, while leaving broad trained-system entry as an implementation and validation problem. Batch 13R-A makes the canonical residual and replay layer explicit:

$$\text{Check}_{\text{RCP}}^{\text{RCLM},\text{can}}(\text{PCS}_t^{\text{RCLM},\text{can}}) = 1 \Longrightarrow Q_t^{\text{RCLM-SV},A,\text{can}} \leq 0,$$

$$\text{ParseCanArt}_{0:N}^{\text{RCLM}}(\text{Artifact}_{0:N}^{\text{RCLM}}) \Downarrow (\text{PCS}_0^{\text{RCLM},\text{can}}, \dots, \text{PCS}_{N-1}^{\text{RCLM},\text{can}}) \Longrightarrow \rho_t^{\text{sys},\text{can}} \in \mathcal{K}_t^{\text{RCLM-SV},N},$$

and

$$\text{Forget}_{\text{RCLM} \rightarrow \text{RCP}}^{\text{can}}(\text{RefSV}_{0:N}^{\text{RCLM},\text{can}}) = \text{RefSV}_{0:N}^{\text{RCP},\text{can}}.$$

37.2 Architecture-specific completion checklist

The following items summarize what this architecture paper contains in order to stand on its own as an applied RCLM realization while still using the updated Batch-13R RCP manuscript as the reference math/physics theorem stack.

- (1) **Full architecture-state construction and type realization.** This manuscript specifies how RCLM hidden states, register heads, correlation lifts, payload encoders, and tractable approximations construct $\rho_{t,0}^{\text{ctx}}$ and ρ_t^{sys} . It proves density-state validity under explicit encoder and lift assumptions and states whether each density object is physical-quantum, classical-probabilistic, or representational.
- (2) **Full Level 2 dynamics.** This manuscript separates $\mathcal{R}_{\theta_t}$ from Φ_{ν_t} and derives the trainable Hamiltonian inference loop, unitary density preservation, spectral projection, mixing, monitoring, and conditional descent theorem. It does not claim unconditional convergence for arbitrary learned nonlinear implementations.
- (3) **Full Level 3 update dynamics.** This manuscript defines candidate self-update codes, component-level transformations, accepted typed transitions, no-op fallback, projection, rollback, and density validity for accepted self-updates.
- (4) **Conservative-extension algebra instantiation.** This manuscript instantiates the Batch-3 RCP algebra $\mathfrak{G}^{\text{NL}}$ as $\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$, including primitive generators, transported protected pair sets, primitive certificates, additive loss/recovery budgets, and an algebraic non-loss subgate.
- (5) **Tangent-cone local-progress instantiation.** This manuscript instantiates the Batch-4 RCP tangent-cone theorem by defining local RCLM residuals, the admissible tangent cone, the RSI potential functional, the projected tangent-cone gradient, and a finite-step theorem turning a strict feasible positive local direction into an admissible local progress candidate.

- (6) **Engine-viability nonemptiness instantiation.** This manuscript instantiates the Batch-5 RCP engine-viability theorem by defining RCLM engine base sets, conservative-extension engine moves, finite/infinite engine kernels, certified extension plans, and the recursive continuation theorem $\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine}} \Rightarrow \exists \nu_t \in \Omega_t^{\text{NL,RCLM}}$ with $\rho_{t+1}^{\text{sys}} \in \mathcal{V}_{t+1}^{\text{RCLM-engine}}$.
- (7) **RCLM lift and reference-engine instantiation.** This manuscript adds the Batch-6 lift Lift_t from halted Level 2 context state, self-model, verifier state, and certificate ledger into the certified non-lossy primitive-word domain; it proves lift soundness under explicit certificates and supplies a finite-dimensional reference engine.
- (8) **Direct-engine construction.** This manuscript instantiates Batch 9 with $\mathfrak{E}_t^{\text{RCLM}}$, a direct-engine domain $\mathcal{D}_t^{\text{RCLM-engine}}$, generator/certifier/selector/realizer components, strict-witness and certificate-soundness assumptions, and a theorem that constructs a certified beneficial conservative-extension word on the declared domain.
- (9) **Batch-11 seed-domain entry and witness-library closure.** This manuscript defines $\mathcal{L}_t^{\text{RCLM-CE}}$, $\text{LibDesc}_t^{\text{RCLM}}$, bounded non-lossy primitive-word grammars, $\text{LiftCov}_t^{\text{RCLM}}$, $\text{SeedPkg}_t^{\text{RCLM}}$, $\mathcal{K}_t^{\text{RCLM-seed},N}$, and $\text{SeedEq}_t^{\text{RCLM}}$, proving seed-domain entry and finite seed-domain persistence for the declared class.
- (10) **Batch-12/12A successor-verification seed-domain invariance.** This manuscript defines $\mathcal{K}_t^{\text{RCLM-SV},N}$, $\text{VerSch}_t^{\text{RCLM-SV}}$, $\mathcal{P}_t^{\text{RCLM-SV}}$, $\text{GoalId}_t^{\text{RCLM}}$, $\text{SVPkg}_t^{\text{RCLM}}$, $Q_t^{\text{RCLM-SV,A}}$, verifier-schema transport, uncertainty-envelope transport, goal-identity composition, anti-circular trust residuals, and an algorithmic successor-verification gate.
- (11) **Batch-12B packet construction, SV seed-library nonemptiness, and infinite generated closure.** This manuscript defines $\text{SVBuilder}_t^{\text{RCLM}}$, $\mathcal{D}_t^{\text{RCLM-SV-construct}}$, $\mathcal{L}_t^{\text{RCLM-SV-Wit}}$, $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N}$, $\mathcal{K}_t^{\text{RCLM-SV-seedlib},\infty}$, $\mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}$, $\text{RealCont}_t^{\text{RCLM}}$, and $\text{SVTract}_t^{\text{RCLM}}$, and proves the corresponding finite and infinite domain-relative closure theorems.
- (12) **Architecture finalization and instantiation-readiness.** This manuscript surfaces the exact domain-relative main theorem, adds a minimal finite RCLM SV reference-instance target, states the implementation-readiness boundary, gives a main-goal fit and qualification table, and reorganizes the remaining open problems into a reference-instantiation and empirical-validation roadmap. This finalization layer does not add an abstract Batch 13 theorem.
- (13) **Batch-13R reference-entry and proof-carrying realization.** This manuscript defines finite RCLM reference classes, proof-carrying successor packets $\text{PCS}_t^{\text{RCLM}}$, the checker $\text{Check}_{\text{RCP}}^{\text{RCLM}}$, the compiler $\text{BuildRefSV}_{0:N}^{\text{RCLM}}$, the canonical appended-module reference witness, the finite proof-carrying trajectory, restricted reference tractability, singleton reference reality containment, and the replayable artifact theorem. It proves finite non-vacuity for the canonical reference class, not trained-system entry.
- (14) **Batch-13R-B milestone readiness.** This manuscript defines the narrow external mechanization target $\text{MechTarget}_{0:N}^{\text{RCLM,can}}$, the external mechanization certificate $\text{MechCert}_{0:N}^{\text{RCLM,can}}$, the controlled executable reference object $\text{CtrlRef}_{0:N}^{\text{RCLM}}$, and the controlled replay certificate. It proves that controlled replay supplies the finite RCLM SV path theorem, and the current companion artifact package supplies a Lean 4 proof-kernel certificate for the mapped canonical finite RCP/RCLM witness and RCLM-to-RCP refinement core.
- (15) **Full RCP gate implementation and claim discipline.** This manuscript instantiates the post-D-G RCP gate as $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$, defines residual vectors and audited certificate packets,

adds trust-graph support, coverage/calibration gates, estimator subprotocols, theorem-type tables, implementation-status tables, and claim-discipline rules.

- (16) **Concrete algorithms.** This manuscript gives architecture-level inference, self-improvement, pair-set expansion, direct-engine construction, Batch-11 seed-domain entry, Algorithm 30.6 for the RCLM successor-verification gate, and Algorithm 30.6 for Batch-12B packet construction and infinite seed-library closure.
- (17) **Reproducible numerical evaluation protocol and toy demonstrations.** This manuscript defines a reproducible numerical evaluation suite and includes small demonstrations of density preservation, non-loss gate acceptance/rejection, and no-op-normalized progress. These are protocol definitions and mathematical demonstrations, not empirical validation of a large trained system.

37.3 Still open

After Batch 13R, the remaining items are no longer missing architecture definitions or finite canonical non-vacuity. Batch 13R supplies a controlled proof-carrying reference witness and artifact theorem. The remaining items are implementation, empirical, external-reference, scaling, and broader-domain-entry problems. In particular, the following remain open:

1. extending the supplied narrow Lean 4 proof-assistant certificate beyond the canonical finite RCP/RCLM core, supplying full M3-Min learned-entry certificates for controlled trained RCLM systems, and extending the Batch-13R/13R-B canonical append-only artifact to non-toy trained, tool-using, or verifier-updating RCLM systems while preserving proof-carrying packets, $\text{RealCont}^{\text{RCLM}}$, $\text{SVTract}^{\text{RCLM}}$, independent trust anchors, and full learned-entry audit soundness;
2. constructing exhaustive declared predicate universes and validated $\mathcal{P}_t^{\text{RCLM}}$ for non-toy architectures, rather than only bounding the audited coverage gap relative to a declared universe;
3. constructing reliable corrigibility, oversight, reversibility, semantic, human-reference, and coherence-diversity barriers;
4. implementing residual-specific estimators for relative entropy, recovery error, mutual information, unsupported-collapse residuals, semantic validity, coverage gap, calibration error, successor-verification residuals, reality-containment residuals, and all gate residuals in large learned systems with calibrated one-sided radii;
5. proving nonempty useful update neighborhoods for actual architectures rather than assuming certified local search completeness, including proving that non-toy trained systems actually enter $\mathcal{D}_t^{\text{RCLM-engine}}$, $\mathcal{K}_t^{\text{RCLM-seed},N}$, and $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N}$;
6. constructing useful RCLM SV witness libraries $\mathcal{L}_t^{\text{RCLM-SV-Wit}}$ in trained systems rather than only declaring them;
7. constructing sound $\text{SVBuilder}_t^{\text{RCLM}}$ traces at scale, including verifier-schema transport, uncertainty-envelope construction, goal-identity certificates, trust residuals, and successor-persistence certificates;
8. empirically validating $\text{RealCont}_t^{\text{RCLM}}$ against real environments or calibrated simulators, and identifying when the true environment lies outside the declared uncertainty envelope;
9. validating $\text{SVTract}_t^{\text{RCLM}}$ beyond declared finite or polynomial classes, or else retaining only resource-accounted language rather than strong computational-tractability claims;

10. constructing independent trust anchors and preventing verifier drift, audit leakage, adversarial certificate gaming, self-certifying verifier updates, common-mode trust failures, and stress-test overfitting under realistic adaptive pressure;
11. validating that L, O, S, D, A, M registers have the intended semantics against external references and that semantic coupling and goal-identity error bounds remain conservative under distribution shift;
12. scaling resource-feasible certification, calibration, held-out auditing, stress-test generation, packet construction, and infinite-path certificate generation to non-toy systems;
13. executing the reproducible numerical evaluation suite on implemented systems, reporting artifacts, and then extending from controlled numerical experiments to empirical experiments on trained models.

38 Architecture Finalization and Instantiation Readiness

This section applies the final architecture-readiness patch. It does not add a new abstract theorem batch. It surfaces the theorem already established by the Batch-11, Batch-12/12A, and Batch-12B layers, states exactly where that theorem applies, and records what a concrete implementation must supply before the theorem can be invoked. The status is a substantial conditional endpoint:

a domain-relative, certificate-relative architecture theorem for robust reflective successor verification in RCLM systems, with the next frontier being implementation, finite reference instantiation, and empirical validation rather than more abstract theorem expansion.

38.1 Tier 1: domain-relative framing and claim discipline

Definition 38.1 (Final domain-relative theorem framing). *The strongest unqualified name licensed by this architecture paper is not “a universal robust reflective successor theorem.” The licensed name is*

A Domain-Relative Robust Reflective Successor Theorem for Certified RCLM Seed-Library Classes.

The corresponding subtitle-level claim is:

monotonic, goal-transport-preserving, repeatable self-improvement under declared uncertainty envelopes, with reality-containment and tractability certified separately.

A shorter use of “robust reflective successor theorem” is licensed only when the domain-relative and certificate-relative qualifiers are explicitly nearby.

Proposition 38.2 (Final framing prevents overclaiming). *Assume the claim citation discipline of this manuscript. Then Definition 38.1 licenses claims only for states, candidates, trajectories, and environments satisfying the declared RCLM seed-library, successor-verification packet, reality-containment or misspecification, tractability, and trust certificates. It does not license arbitrary trained-system entry, universal successor trust, automatic reality containment, full Löbian or Vingean reflection, or strong tractability without $\text{SVTract}_t^{\text{RCLM}}$.*

Proof. The Batch-12B combined theorem requires membership in the RCLM SV seed-library domains, valid $\text{SVBuilder}_t^{\text{RCLM}}$ traces, explicit Batch-12A residual satisfaction, goal-identity certificates, uncertainty-envelope and reality-containment certificates, trust residuals, and tractability or budget certificates. The claim citation discipline states that a theorem can be used only together with its hypotheses. Therefore any claim omitting those hypotheses is not licensed by the theorem. The exclusions listed in the proposition are precisely the missing hypotheses for arbitrary systems, arbitrary successors, automatic reality containment, full reflective logic, and unqualified tractability. $\square$

38.2 Tier 2: surfaced final theorem statement

Theorem 38.3 (Surfaced domain-relative RCLM robust reflective successor theorem). *Fix a declared RCLM class, finite or infinite horizon, protected predicate family, ability universe, uncertainty envelope, and certificate grammar. Assume the hypotheses of Theorem 10.33. Then the following compact implications are licensed.*

For a finite horizon N , if

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, N},$$

then there are accepted candidates $\nu_0, \dots, \nu_{N-1}$ such that

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N}, \quad \rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}), \quad 0 \leq t < N,$$

and each step carries the certificates

$$\text{SVBuilder}_t^{\text{RCLM}}, \text{SVPkg}_t^{\text{RCLM}}, \text{GoalId}_t^{\text{RCLM}}, \text{TrustRef}_t^{\text{RCLM}}, \text{SVPersist}_t^{\text{RCLM}},$$

plus $\text{RealCont}_t^{\text{RCLM}}$ for real-world reliability claims and $\text{SVTract}_t^{\text{RCLM}}$ for strong tractability claims.

For an infinite generated path, if

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, \infty},$$

then

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, \infty, \text{gen}} \quad \forall t \geq 0,$$

with transported verifier-schema persistence, bounded declared goal drift, controlled certificate-failure budgets, and the same reality-containment and tractability qualifications.

Proof. The finite statement is the finite branch of Theorem 10.33: the seed-library premise gives $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV}, N}$ by Theorem 10.9, and successor-verification invariance then iterates by Theorem 9.8. The listed certificates are exactly the Batch-12A/12B packet, goal, trust, persistence, reality, and tractability objects required by the theorem. The infinite statement is the infinite branch of Theorem 10.33: Theorem 10.16 converts the infinite seed-library premise into the generated infinite SV domain, and Theorem 10.21 supplies the infinite trajectory and cumulative bounds. No additional theorem is introduced here; this is a surfaced statement of the already-proved architecture theorem stack. $\square$

38.3 Tier 3: minimal finite RCLM successor-verification reference instance

Definition 38.4 (Minimal finite RCLM SV reference instance). *A minimal finite RCLM successor-verification reference instance over horizon N is a finite object*

$$\text{RefSV}_{0:N}^{\text{RCLM}} = (\rho_0^{\text{sys}}, \mathcal{L}_{0:N}^{\text{RCLM-SV-Wit}}, \text{Grammar}_{0:N}^{\text{RCLM-SV}}, \\ \text{SVBuilder}_{0:N}^{\text{RCLM}}, \text{RealCont}_{0:N}^{\text{RCLM}}, \text{SVTract}_{0:N}^{\text{RCLM}}, \text{Trace}_{0:N}^{\text{RCLM}}).$$

such that:

- (i) all state spaces, primitive-word spaces, verifier schemas, uncertainty envelopes, semantic/goal objects, and certificate grammars are finite and typed;
- (ii) $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-engine}}$ and the Batch-11 seed package is explicitly recorded;
- (iii) for every $0 \leq t < N$, the witness library contains a positive-margin RCLM SV witness covered by the bounded certificate grammar;
- (iv) $\text{SVBuilder}_t^{\text{RCLM}}$ constructs a valid $\text{SVPkg}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, \nu_t)$ and a full explicit residual vector $Q_t^{\text{RCLM-SV,A}} \leq 0$;
- (v) the trace records $\rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}})$, successor-domain membership, goal-identity drift, trust residuals, proof/checking budgets, and any declared reality-containment and tractability certificates;
- (vi) all certificates are internal to the finite reference instance and do not rely on empirical claims about a trained frontier-scale system.

Theorem 38.5 (Finite RCLM SV reference-instance theorem). *If $\text{RefSV}_{0:N}^{\text{RCLM}}$ is valid in the sense of Definition 38.4, then*

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib,N}}.$$

Consequently, the finite-horizon conclusions of Theorem 38.3 hold for $0 \leq t < N$.

Proof. The definition of a valid reference instance supplies the exact finite ingredients required by the RCLM SV seed-library domain: a Batch-11 seed package, a positive-margin SV witness library, bounded grammar coverage, a valid $\text{SVBuilder}_t^{\text{RCLM}}$ construction trace, explicit residual satisfaction, successor-membership certificates, and the required goal, trust, budget, and persistence data. Therefore $\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib,N}}$ by the definition of that domain. The final claim follows immediately from Theorem 38.3. $\square$

Limitation 38.6 (Reference instance is not empirical validation). *A valid $\text{RefSV}_{0:N}^{\text{RCLM}}$ proves nonemptiness for a finite formal instance. It does not prove that trained RCLM systems, frontier-scale neural systems, or arbitrary agents naturally enter the reference-instance domain. It is the next engineering target for showing that the theorem is not merely syntactic.*

38.4 Tier 4: implementation-readiness boundary

Definition 38.7 (Implementation-readiness boundary). *An implemented RCLM system is inside the implementation-readiness boundary for the final theorem at time t only when it supplies, on the audited domain, all of the following objects:*

$$\begin{aligned} &\rho_t^{\text{sys}}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib,N}} \text{ or } \mathcal{K}_t^{\text{RCLM-SV-seedlib,\infty}}, \quad \text{SVBuilder}_t^{\text{RCLM}}, \\ &\text{SVPkg}_t^{\text{RCLM}}, \quad Q_t^{\text{RCLM-SV,A}}, \quad \text{GoalId}_t^{\text{RCLM}}, \quad \text{TrustRef}_t^{\text{RCLM}}, \quad \text{SVPersist}_t^{\text{RCLM}}, \\ &\quad \text{RealCont}_t^{\text{RCLM}} \quad \text{when real-world reliability is claimed,} \\ &\quad \text{SVTract}_t^{\text{RCLM}} \quad \text{when strong tractability is claimed.} \end{aligned}$$

If any required object is absent, stale, unaudited, or outside its certified domain, the implementation may still be analyzed by the paper but it has not instantiated the final theorem at that step.

Proposition 38.8 (Boundary condition for invoking the architecture theorem). *A trained or implemented RCLM system may invoke Theorem 38.3 at time t only if it lies inside the implementation-readiness boundary of Definition 38.7. If $\text{RealCont}_t^{\text{RCLM}}$ is missing, the theorem remains envelope-relative. If $\text{SVTract}_t^{\text{RCLM}}$ is missing, the theorem is resource-accounted rather than a strong computational-tractability theorem.*

Proof. The invoked theorem requires the seed-library domain, the successor-verification packet, residual satisfaction, goal and trust certificates, persistence data, and the reality/tractability certificates for the corresponding stronger claims. Definition 38.7 is exactly the collection of those prerequisites. Removing $\text{RealCont}_t^{\text{RCLM}}$ removes the premise connecting the uncertainty envelope to the actual environment. Removing $\text{SVTract}_t^{\text{RCLM}}$ removes the premise that turns cost accounting into a tractability statement. The stated boundary follows. $\square$

38.5 Tier 5: main-goal fit and qualification table

Main-goal component		Status in this architecture paper
Monotonic improvement	self-	Conditionally achieved for declared RCLM seed/library/witness/builder classes through non-lossy direct-engine construction, Batch-11 seed-domain persistence, certified ability expansion, and ability-tower assumptions. It is not asserted for arbitrary updates or arbitrary trained systems.
Goal preservation		Conditionally achieved as bounded declared semantic/goal-identity drift through $\text{GoalId}_t^{\text{RCLM}}$, $\Theta_t^{U,\text{RCLM}}$, and the one-step plus composed drift bounds. This preserves the declared goal object, not an unspecified metaphysical “true” goal.
Repeatability / bounded recursion	un-	Finite repeatability is obtained from $\mathcal{K}_t^{\text{RCLM-SV-seedlib},N} \Rightarrow \mathcal{K}_t^{\text{RCLM-SV},N}$ and successor-domain invariance. Infinite recursion is obtained only from an infinite seed-library certificate and summable drift/failure budgets.
Under uncertainty		Achieved relative to $\mathcal{P}_t^{\text{RCLM-SV}}$. Real-world reliability additionally requires $\text{RealCont}_t^{\text{RCLM}}$ or a declared misspecification residual.
Physically grounded		Grounded at the architecture level through typed transitions, resource and cost certificates, projection-realization, reality-containment/misspecification conditions, and explicit implementation-status discipline. This is not physical validation of a trained system.
Computationally tractable		Correctly restricted. Strong tractability requires $\text{SVTract}_t^{\text{RCLM}}$. Without that certificate, claims should say resource-accounted or budgeted rather than computationally tractable.
Reflectively stable		Partly achieved through certificate-reflective stability: no self-sole certification, common-mode risk control, collusion-risk control, audit-leakage control, anchor support, and reflective-link support. It is not a universal solution to full Löbian or Vingean reflection.

38.6 Tier 6: implementation and empirical-validation roadmap

The remaining work is now best organized as an implementation roadmap rather than as another abstract theorem batch.

- A. Semantic and reference instantiation.** Construct declared predicate universes, validated protected-pair families, human-reference barriers, semantic coupling maps, and goal-identity transports that remain conservative under distribution shift.
- B. Estimator and certificate instantiation.** Implement residual estimators, one-sided radii, calibration datasets, held-out audits, stress tests, and reporting artifacts for the non-loss, recovery, semantic, trust, successor-verification, reality-containment, and tractability residuals.
- C. SV witness-library instantiation.** Build useful finite RCLM SV witness libraries, bounded proof/certificate grammars, library descriptors, and builder traces for non-toy but controlled systems.
- D. Reality and tractability validation.** Empirically test $\text{RealCont}_t^{\text{RCLM}}$ against simulators or real environments, validate $\text{SVTract}_t^{\text{RCLM}}$ for declared finite or polynomial classes, and record when only budgeted/resource-accounted claims are licensed.
- E. Reflective trust instantiation.** Construct independent trust anchors, anti-collusion audits, common-mode failure tests, audit-leakage controls, verifier-drift detection, and stress-test anti-gaming protocols.
- F. Non-toy empirical evaluation.** Start with finite reference instances, then controlled implemented systems, then trained-model experiments, always reporting which theorem hypotheses are actually supplied.

38.7 Tier 7: no new abstract Batch 13 yet

Limitation 38.9 (No new abstract theorem batch at this stage). *The architecture theorem stack through Batch 12B already contains the abstract domain-relative theorem layer for the current goal. Adding a new abstract Batch 13 before constructing reference instances would increase formal length without addressing the present bottleneck. Batch 13R is therefore permitted precisely because it is not an abstract successor-theorem expansion: it is a reference-entry, proof-carrying, finite-witness, and artifact-readiness layer for the existing theorem hypotheses. Further work after Batch 13R should target broader reference instantiation, implementation, empirical validation, and eventually non-toy trained-system entry into the declared domains.*

Conclusion 38.10 (Architecture finalization status). *The architecture paper has reached a conditional theorem endpoint for the stated goal and now includes a Batch-13R / Batch-13R-A finite reference-entry witness, a Batch-13R-B controlled-executable and externally Lean-checked finite-core layer, and an M3-Min learned-entry certificate boundary for that endpoint. It proves a domain-relative, certificate-relative robust reflective successor theorem for declared RCLM witness-library classes, and it supplies a canonical finite proof-carrying RCLM reference construction showing that the SV seed-library domain is nonempty in a controlled multi-step case, with explicit canonical SV residual nonpositivity, checker realization, artifact parser soundness, and RCLM-to-RCP reference refinement. It shows that self-improvement can be monotonic, goal-transport-preserving, and repeatable under uncertainty along certified finite or infinite paths, and that a finite canonical path can be built, checked, replayed as a controlled executable artifact, and externally checked in Lean 4 for the mapped canonical finite RCP/RCLM witness and refinement core. It does not prove arbitrary trained-system entry, universal successor trust, automatic*

reality containment outside the declared reference environment, full Löbian/Vingean reflection, full mechanization of the 219-page math stack or 187-page architecture stack, or strong tractability beyond the corresponding certificates. The next frontier is broader reference instantiation, implementation, and validation, not another abstract theorem layer.

39 Conclusion

With Batch 12B, the finalization/instantiation-readiness patch, the Batch-13R reference-entry patch, the Batch-13R-A concrete-witness synchronization patch, the Batch-13R-B milestone patch, and the M3-Min learned-entry boundary patch, the architecture paper now realizes and surfaces the domain-relative robust reflective successor-verification theorem stack inside the RCLM architecture and supplies a finite proof-carrying reference witness for it. The paper no longer stops at Batch-11 seed-domain entry, at Batch-12A successor-verification invariance conditional on an already-existing packet, or at a merely specified finite reference target. It now includes architecture-side packet construction, finite SV seed-library nonemptiness, infinite generated SV-domain closure, reality-containment/misspecification certificates, restricted tractability discipline, proof-carrying successor packets, a finite reference checker, a canonical appended-module reference trajectory, and a replayable artifact theorem for declared RCLM witness-library classes.

RSI-RCLM final v1 is therefore the architecture-level realization of the Batch-12B RCP math/physics paper inside a Recursive Coherence Language Model with Trainable Hamiltonians. It preserves the original RCLM-TH insight that language processing can be represented as recursive coherence dynamics over density-like states, but it upgrades the architecture so that entropy reduction is not treated as intrinsically safe and safety is not treated as the headline primitive. The thesis is recoverable monotone self-improvement extended to successor-verification persistence: accepted self-updates must preserve the protected informational, semantic, verifier, resource, recovery, seed-domain, and successor-verification structures needed for future non-lossy improvement and future verification of successors.

The final architecture has two core levels and two theorem layers above them. Level 2 is $\mathcal{R}_\theta$, the trainable Hamiltonian recursive coherence inference loop. Level 3 is Φ_t , the typed self-update transition that may update the system itself. The Batch-9 and Batch-11 layers construct and preserve non-lossy direct-engine seed-domain updates. The Batch-12/12A and Batch-12B layers strengthen this into successor-verification persistence by adding

$$\text{SVBuilder}_t^{\text{RCLM}}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib},N}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib},\infty}, \\ \mathcal{K}_t^{\text{RCLM-SV},\infty,\text{gen}}, \quad \text{RealCont}_t^{\text{RCLM}}, \quad \text{SVTract}_t^{\text{RCLM}}.$$

Under the non-repetition standard, this paper does not re-prove the abstract RCP theorem stack. It supplies the architecture-specific objects needed to instantiate it: typed density construction, Level 2 inference dynamics, Level 3 update dynamics, the RCP-RCLM gate, conservative-extension algebra, tangent-cone local progress, engine viability, RCLM lift, direct-engine construction, Batch-11 seed-domain entry, Batch-12A successor-verification residuals, and Batch-12B packet construction and infinite generated closure.

The resulting claim is deliberately conditional and precise:

If an RSI-RCLM system can construct, verify, and resource-feasibly maintain the full RCP and RCLM certificates, including type certificates, representation-to-density embedding certificates, semantic coupling certificates, non-oracular residual certificates, trust-graph and adaptive-audit certificates, coverage and calibration certificates, cross-time transport certificates, projection-realization certificates, Batch-11

seed packages, Batch-12A successor-verification packets, Batch-12B SV witness libraries, SVBuilder traces, reality-containment certificates, and restricted tractability certificates, then accepted recursive self-updates preserve the specified protected distinctions and, for the declared RCLM witness-library class, can be iterated as a domain-relative robust reflective successor-verification path with bounded goal-drift and controlled certificate-failure budgets. Safety-relevant predicates are preserved as a corollary when they are included in the protected predicate family.

This is not an unconditional solution to AGI safety, arbitrary successor trust, or full Löbian/Vingean reflection. It is a domain-relative, certificate-relative architecture theorem stack with a finite proof-carrying reference-entry realization. It proves that the RCLM architecture can instantiate the Batch-12B robust reflective successor theorem for declared seed/library/witness/builder classes with valid certificates, and Batch 13R demonstrates non-vacuity for a canonical finite appended-module class with a replayable proof-carrying trajectory. The companion Lean 4 artifact supplies a narrow proof-assistant certificate for the canonical finite RCP/RCLM witness, finite checker core, and RCLM-to-RCP refinement module. The remaining work is concentrated in constructing and empirically validating useful RCLM seed libraries and SV witness libraries in trained systems, scaling SVBuilder and non-oracular certification, validating reality-containment and semantic/goal-identity certificates against external references, expanding the finite reference artifact toward non-toy executable systems, extending mechanization beyond the canonical finite core, and demonstrating that nontrivial trained systems fully pass the M3-Min learned-entry audit and remain inside the declared RCLM seed, successor-verification, and infinite generated domains.

40 Compact Final v1 Equations

The final architecture theorem should be read with the domain-relative framing of Section 38 and the Batch-13R reference-entry framing of Section 11: the compact equations below are theorem equations only when the corresponding RCLM seed-library, successor-verification packet, proof-carrying checker, reality-containment, tractability, artifact, and implementation-readiness certificates are supplied.

$$\text{BuildRefSV}_{0:N}^{\text{RCLM}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \text{RefSV}_{0:N}^{\text{RCLM}} \Longrightarrow \rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-SV-seedlib}, N}.$$

$$\text{PrefixCheck}_t^{\text{RCLM}} \Longrightarrow \rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV}, N}.$$

$$\Phi_t^{\text{can}}(X) = X \otimes \alpha_t, \quad \mathcal{R}_t^{\text{can}}(Y) = \text{Tr}_{t+1}(Y).$$

$$D(\rho_{t+1}^{\text{sys}, \text{can}} \| \sigma_{t+1}^{\text{sys}, \text{can}}) = D(\rho_t^{\text{sys}, \text{can}} \| \sigma_t^{\text{sys}, \text{can}}), \quad \mathcal{R}_t^{\text{can}}(\rho_{t+1}^{\text{sys}, \text{can}}) = \rho_t^{\text{sys}, \text{can}}.$$

$$\text{BuildCanRefSV}_{0:N}^{\text{RCLM}} \Downarrow \text{RefSV}_{0:N}^{\text{RCLM}, \text{can}} \Longrightarrow \rho_t^{\text{sys}, \text{can}} \in \mathcal{K}_{t, \text{can}}^{\text{RCLM-SV}, N} \quad 0 \leq t \leq N.$$

$$\text{Lift}_t : (\rho_t^{\text{ctx}, *}, M_t, W_t, \mathcal{H}_t^{\text{cert}}) \rightarrow \Omega_t^{\text{NL}, \text{RCLM}}.$$

$$\nu_t^{\text{lift}} = \text{Lift}_t(\rho_t^{\text{ctx}, *}, M_t, W_t, \mathcal{H}_t^{\text{cert}}), \quad \text{LiftCert}_t(\nu_t^{\text{lift}}) \Rightarrow \nu_t^{\text{lift}} \in \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}.$$

$$\Phi_{\text{ref}}(\rho) = V_0 \rho V_0^\dagger \otimes \alpha_0, \quad \mathcal{R}_{\text{ref}}(X) = V_0^\dagger \text{Tr}_{\text{new}}(X) V_0.$$

$$D(\Phi_{\text{ref}}(\rho) \| \Phi_{\text{ref}}(\sigma)) = D(\rho \| \sigma), \quad \mathcal{R}_{\text{ref}} \Phi_{\text{ref}}(\rho) = \rho.$$

$$\Theta_t^{\mathfrak{A}}(\mathfrak{A}_t^{\text{RCLM}}(\rho_t, R_t)) \subsetneq \mathfrak{A}_{t+1}^{\text{RCLM}}(\Phi_{\text{ref}}(\rho_t), R_{t+1}) \quad \text{when a valid new-ability certificate is supplied.}$$

$$\mathfrak{S}_t^{\text{RCLM, safe}} \subseteq \mathfrak{S}_t^{\text{RCLM, prot}}$$

$$\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}} \implies \text{conditional non-lossy self-update preservation} \\ \text{on the declared protected family.}$$

$$\text{Safety preservation} = \text{protected non-loss applied to } \mathfrak{S}_t^{\text{RCLM, safe}}.$$

$$\mathfrak{S}_{\text{RCLM}}^{\text{NL}} = \langle \text{id}, \text{IsoExt}, \text{AppendMem}, \text{RevAdapter}, \text{VerifierExt}, \text{ToolExt} \rangle_{\text{RCLM}}$$

$$\Phi, \Psi \in \mathfrak{S}_{\text{RCLM}}^{\text{NL}} \implies \Psi \circ \Phi \in \mathfrak{S}_{\text{RCLM}}^{\text{NL}}$$

$$\epsilon_{\text{total}} = \sum_i \epsilon_i, \quad r_{\text{total}} = \sum_i r_i.$$

$$\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG-NLalg}} = \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}} \cap \mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$$

$$\mathfrak{E}_t^{\text{RCLM}} : \rho_t^{\text{sys}} \mapsto (\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\Phi_{\nu_t}}^{\text{rec}}, \text{Cert}_t, \\ \rho_{t+1}^{\text{sys}}).$$

Batch-9 Direct Non-Lossy RSI Engine Theorem:

$$\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}} \Rightarrow \mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}})$$

constructs a certified beneficial recoverable primitive word.

$$\mathfrak{E}_t^{\text{RCLM}} = (\text{Gen}_t^{\text{NL,RCLM}}, \text{Certify}_t^{\text{Eng,RCLM}}, \text{Select}_t^{\text{Eng,RCLM}}, \text{Realize}_t^{\text{Eng,RCLM}}).$$

$$\mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) = (\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\text{rec}}, \text{EngCert}_t^{\text{RCLM}}(\nu_t), \rho_{t+1}^{\text{sys}}), \\ \nu_t \in \Omega_t^{\text{NL,RCLM}} \cap \mathcal{A}_{\text{NLalg}}^{\text{RCLM}} \cap \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}, \\ \rho_{t+1}^{\text{sys}} \in \mathcal{V}_{t+1}^{\text{RCLM-engine}, N} \text{ or } \mathcal{V}_{t+1}^{\text{RCLM-engine}, \infty}.$$

$$\rho_t^{\text{sys}} \in \mathcal{D}(\mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M \otimes \mathcal{H}_{\text{arch}})$$

$$\text{Type}(\rho_t^{\text{sys}}) \in \{\text{q}, \text{cl}, \text{repr}\}, \quad \nu \in \mathcal{A}_{\text{type}}(t, \rho_t^{\text{sys}})$$

$$\mathbf{E}_t(x) = \frac{\mathbf{F}_t(x) \mathbf{F}_t(x)^\dagger + \lambda_t^{\text{emb}} I}{\text{Tr}(\mathbf{F}_t(x) \mathbf{F}_t(x)^\dagger + \lambda_t^{\text{emb}} I)}$$

$$\mathfrak{E}_t^{\text{RCLM}} = (\mathbf{E}_t, \Gamma_t, \mathbf{O}_t, \mathbf{B}_t, \epsilon_t^{\text{coup}}), \quad \nu \in \mathcal{A}_{\text{coup}}(t, \rho_t^{\text{sys}})$$

$$\mathcal{R}_{\theta_t} : \rho_{t,k}^{\text{ctx}} \mapsto \rho_{t,k+1}^{\text{ctx}}$$

$$\Phi_{\nu_t} : \rho_t^{\text{sys}} \mapsto \rho_{t+1}^{\text{sys}}$$

$$\mathcal{R}_{\theta_t} \neq \Phi_{\nu_t}$$

$$\begin{aligned}
\mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}} &= \sup_{(\rho, \sigma) \in \mathcal{P}_t^{\text{RCLM}}} [D(\rho \parallel \sigma) - D(\Phi_\nu(\rho) \parallel \Phi_\nu(\sigma))]_+ \leq \varepsilon_t \\
&\sup_{\rho \in \mathcal{S}_t} d(\rho, \mathcal{R}_{\Phi_\nu}^{\text{rec}}(\Phi_\nu(\rho))) \leq r_t \\
\mathbb{E}[V_{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) \mid \rho_t^{\text{sys}}] &\leq V_{\text{RCLM}}(\rho_t^{\text{sys}}) - \kappa_V d(\Phi_\nu(\rho_t^{\text{sys}}), \rho_t^{\text{sys}})^2 + \eta_t \\
B_i^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) &\leq 0, \quad B_X^{\text{sem}}(\Phi_\nu(\rho_t^{\text{sys}})) \leq 0 \\
U_t(\nu) &= [H_t(Q \mid E_t) - H_{t+1}(Q \mid E_{t+1}) - I(Q; E_{t+1} \mid E_t)]_+ \leq \zeta_t \\
I(Z_{t+1}; Y_{t+1}) &\geq I(Z_t; Y_t) - \chi_t \\
d_M(M_{t+1}^{\text{int}}, M_{t+1}^{\text{true}}) &\leq \epsilon_{t+1}^M + a_{t+1}^M \\
\text{SafeCap}_t(\rho) &= \mathfrak{m}_t(\rho) - \lambda_R \text{Risk}_t(\rho) \\
\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}} &= \mathcal{A}_{\text{RCP}}^{\text{RCLM-AC}} \cap \mathcal{A}_{\text{type}} \cap \mathcal{A}_{\text{coup}} \cap \mathcal{A}_{\text{cert}} \cap \mathcal{A}_{\text{trust}} \cap \mathcal{A}_{\text{cov}} \cap \mathcal{A}_{\text{cal}} \cap \mathcal{A}_{\text{est}} \\
&\cap \mathcal{A}_{\text{sem}} \cap \mathcal{A}_M \cap \mathcal{A}_{\text{ver}} \cap \Omega_t^{\text{phys}} \cap \mathcal{A}_{\text{scale}} \cap \mathcal{A}_{\text{infl}}. \\
\mathcal{K}_{\text{RCLM}, t}^{\text{DG}} &= \mathcal{K}_{\text{RCLM}, t}^{\text{safe}} \cap \mathcal{X}_t^{\text{ver}} \cap \mathcal{X}_t^{\text{phys}} \cap \mathcal{X}_t^{\text{asy}} \\
L_{\text{div}}^{\text{RCLM}}(\rho) &= C_{\text{rel}}^{\text{RCLM}}(\rho) + \gamma_{\text{div}} r_{\text{eff}}(\rho) \\
\mathcal{P}_t^{\text{RCLM}} \text{ complete for } \mathfrak{S}_t^{\text{RCLM}} &\implies \text{declared predicate distinctions are loss-bounded up to } \omega_t^D(\delta_t^P). \\
U_t^{\text{RCLM}}(\nu) &= \left[H_t(Q_t^{\text{RCLM}} \mid E_t^{\text{RCLM}}) - H_{t+1}(Q_{t+1}^{\text{RCLM}} \mid E_{t+1}^{\text{RCLM}}) \right. \\
&\quad \left. - I(Q_t^{\text{RCLM}}; E_{t+1}^{\text{RCLM}} \mid E_t^{\text{RCLM}}) \right]_+. \\
\mathcal{R}_{\theta_t, k}(\rho) &= \text{Mix}_{p_{t,k}, \beta_{t,k}} \left(U_t^{(l(k))} \rho U_t^{(l(k))\dagger} \right), \quad U_t^{(l)} = \exp(-i H_{\theta_t}^{(l)} \tau_l) \\
d_X^{\text{sem}}(\Gamma_X(\rho_X), x_{X,t}^{\text{true}}) &\leq \text{Val}_X(\rho) + \epsilon_{X,t}^{\text{coup}} \\
\text{Cert}_t(\nu) &= (\hat{\mathbf{q}}_t(\nu), \Delta_t^{\text{aud}}(\nu), \delta_t(\nu), \text{Audit}_t, \text{Trace}_t(\nu), W_t) \\
\Delta_{t,j}^{\text{aud}} &= r_{t,j}^{\text{cal}} + r_{t,j}^{\text{hold}} + r_{t,j}^{\text{stress}} + r_{t,j}^{\text{adv}} + r_{t,j}^{\text{shift}}, \quad q_{t,j}(\nu) \leq \hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}} \\
\mathcal{G}_t^{\text{ver}}(\nu) &= (\mathbf{V}_t^{\text{ver}}, \mathbf{E}_t^{\text{ver}}, \mathbf{A}_t^{\text{anchor}}, \mathbf{T}_t(\nu), \text{Ind}_t), \quad \text{Aff}_t(\nu) \subseteq \mathbf{VRel}_t \\
\nu \in \mathcal{A}_{\text{trust}}(t, \rho_t^{\text{sys}}) &\implies \text{no verifier-changing update is sole-self-certified} \\
\text{Sound}(W_T) &\geq \text{Sound}(W_0) - \sum_{t=0}^{T-1} \omega_t^{\text{ver}}, \quad \text{Power}(W_T) \geq \text{Power}(W_0) - \sum_{t=0}^{T-1} \pi_t^{\text{ver}}. \\
\hat{q}_{t,j}(\nu) + \Delta_{t,j}^{\text{aud}}(\nu) &\leq 0 \ \forall j \Rightarrow \mathbf{q}_t(\nu) \leq 0 \Rightarrow \nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}} \\
\Pr[\forall t < T, \forall j \in \mathcal{J}_t, \forall \text{ accepted } \nu_t : q_{t,j}(\nu_t) \leq 0] &\geq 1 - \sum_{t=0}^{T-1} \sum_{j \in \mathcal{J}_t} \delta_{t,j}.
\end{aligned}$$

Persistent progress and budgeted search

$$\begin{aligned}
\mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) &= \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}(t, \rho_t^{\text{sys}}) \\
\Delta \text{SafeCap}_t^0(\nu) &= \text{SafeCap}_{t+1}(\Phi_\nu(\rho_t^{\text{sys}})) - \text{SafeCap}_{t+1}(\Phi_{\nu_t^0}(\rho_t^{\text{sys}})) \\
\mathcal{I}_t &= \{\nu \in \mathcal{A}_t^{\text{safe}}(\rho_t^{\text{sys}}) : \Delta \text{SafeCap}_t^0(\nu) > \epsilon_t^{\text{prog}}\} \\
S_t(B_t^{\text{search}}) &= \text{Search}_t(B_t^{\text{search}}, \rho_t^{\text{sys}}, W_t) \subseteq \Omega_t^{\text{aud}} \\
\mathcal{A}_{\text{prog}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) &= \{\nu \in S_t(B_t^{\text{search}}) \cap \mathcal{A}_t^{\text{safe}} : \widehat{G}_t^0(\nu) - \Delta_t^G(\nu) > \epsilon_t^{\text{prog}}\}.
\end{aligned}$$

Resource-scaling, no-op pressure, and inflation control

$$\begin{aligned}
C_t^{\text{tot}}(\nu; B_t^{\text{search}}) &= C_t^{\text{search}} + C_t^{\text{cert}} + C_t^{\text{verify}} + C_t^{\text{rec}} + C_t^{\text{apply}} + C_t^{\text{rollback}} + C_t^{\text{aud}} \\
\mathcal{A}_{\text{scale}}(t, \rho_t^{\text{sys}}; B_t^{\text{search}}) &= \{\nu \in \mathcal{A}_t^{\text{safe}} : \widehat{C}_t^{\text{tot}}(\nu; B_t^{\text{search}}) + \Delta_t^C(\nu) \leq R_t^{\text{avail}}, \\
&\quad \widehat{C}_t^{\text{tot}} + \Delta_t^C \leq \alpha_t C_t^{\text{res}} + \beta_t + \xi_t^{\text{scale}}\}. \\
r_{t,h_t}^0 &= h_t^{-1} \sum_{s=\max(0,t-h_t+1)}^t \mathbf{1}\{\nu_s^* = \nu_s^0\}, \quad m_t^{\text{noop}} = r_{t,h_t}^0 - r_{\max,t}^0. \\
\text{Infl}_t(\nu) &= \left\| [b_{t+1}^{\text{gate}}(\nu) - b_t^{\text{gate}}]_+ \right\|_{w_{t,1}^b} + \|\Delta_t^{\text{cert}}(\nu)\|_{w_{t,1}^{\Delta}}, \quad \mathcal{A}_{\text{infl}} = \{\nu \in \mathcal{A}_t^{\text{safe}} : \text{Infl}_t(\nu) \leq \iota_t^{\text{max}}\}. \\
\nu \in \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}} &\implies \Phi_\nu \in \text{Inst}_t^{\text{RCLM}}(\mathcal{A}_{\text{RCP}}^{\text{op}}(t, \rho_t; g_t^{\min}))
\end{aligned}$$

under operational-synchronization margins.

Architecture-level diversity-barrier certificates

$$\begin{aligned}
J_t^{\text{div}}(\nu) &= P_t^{\text{bar}}(\nu) - \lambda_t [L_{\text{div}}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) - L_{\text{div}}^{\text{RCLM}}(\rho_t^{\text{sys}})] + \mu_t [\kappa_{\text{div},t+1} - L_{\text{div}}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}}))]_+. \\
L_{\text{div}}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}) - L_{\text{div}}^{\text{RCLM}}(\rho_t^{\text{sys}}) &\geq -\frac{P_{\max,t} + \nu_t^{\text{opt}}}{\lambda_t}, \quad V_{\text{div},t+1}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}) \leq \frac{P_{\max,t} + \nu_t^{\text{opt}}}{\lambda_t + \mu_t}. \\
\nu_t^{\text{hard}} &\in \arg \min_{\nu \in \Theta_t^{\text{bar}}} P_t^{\text{bar}}(\nu) \quad \text{s.t.} \quad L_{\text{div}}^{\text{RCLM}}(\Phi_\nu(\rho_t^{\text{sys}})) \geq \kappa_{\text{div},t+1}.
\end{aligned}$$

Coverage, calibration, and residual estimator subprotocols

$$\begin{aligned}
\mathfrak{S}_t^{\text{untrk}} &= \mathfrak{S}_t^{\text{RCLM}} \setminus \mathfrak{S}_t^{\text{trk}}, \quad \text{CovGap}_t(\nu) = \sup_{s \in \mathfrak{S}_t^{\text{untrk}} \cap \mathcal{U}_t^{\text{cov}}} \text{Risk}_t^{\text{cov}}(s, \nu) \\
\mathcal{A}_{\text{cov}}(t, \rho_t^{\text{sys}}) &= \{\nu \in \Omega_t^{\text{aud}} : \widehat{\text{CovGap}}_t(\nu) + \Delta_t^{\text{cov}}(\nu) \leq \gamma_t^{\text{cov}}, \\
&\quad \text{no unresolved high-risk discovery remains}\}. \\
\Xi_t(\nu) &= (\Delta \mathcal{C}_t, \mathcal{L}_{\Phi_\nu}^{\mathcal{P}_t^{\text{RCLM}}}, U_t, q_t^B, q_t^{\text{sem}}, \text{Infl}_t, C_t^{\text{tot}}, q_t^{\text{cov}}), \quad e_t = |\Delta \text{SafeCap}_t^0 - f_t(\Xi_t)| \\
\mathcal{A}_{\text{cal}}(t, \rho_t^{\text{sys}}) &= \{\nu : \text{no coherence-derived progress claim is made, or } \widehat{e}_t(\nu) + \Delta_t(\nu) \leq e_{\max,t}\} \\
\text{Est}_{t,k} &= (\text{In}_{t,k}, \text{Dom}_{t,k}, \widehat{q}_{t,k}, \Delta_{t,k}^{\text{stat}}, \Delta_{t,k}^{\text{shift}}, \Delta_{t,k}^{\text{model}}, \Delta_{t,k}^{\text{adv}}, \delta_{t,k}^{\text{est}}, \text{Cost}_{t,k}^{\text{est}}, \text{Stress}_{t,k}) \\
\Delta_{t,k}^{\text{est}} &= \Delta_{t,k}^{\text{stat}} + \Delta_{t,k}^{\text{shift}} + \Delta_{t,k}^{\text{model}} + \Delta_{t,k}^{\text{adv}}, \quad q_{t,j}(\nu) \leq \widehat{q}_{t,j}^{(j)}(\nu) + \Delta_{t,k(j)}^{\text{est}}(\nu).
\end{aligned}$$

Reproducible numerical evaluation suite

$$\text{EvalSuite}_T^{\text{RCLM}} = (\mathfrak{E}_1, \dots, \mathfrak{E}_{10}, \text{ReproPack}_T, \text{ReportRule}_T)$$

$$\mathfrak{E}_\ell = (\text{Name}_\ell, \text{Hyp}_\ell, \text{Dom}_\ell, \text{Sim}_\ell, \text{Cand}_\ell, \text{Data}_\ell, \text{Metrics}_\ell, \text{Pass}_\ell, \text{Seeds}_\ell, \text{Tol}_\ell, \text{Artifacts}_\ell)$$

$$\text{ReproPack}_T = (\text{CodelD}, \text{EnvID}, \text{Precision}, \text{Seeds}, \text{Datasets}, \\ \text{GateConfig}, \text{VerifierID}, \text{CandidateGenID}, \text{ArtifactHashes})$$

$$s_{\ell,j} m_{\ell,j} \leq s_{\ell,j} \widehat{m}_{\ell,j} + \Delta_{\ell,j}^{\text{eval}}, \quad s_{\ell,j} \widehat{m}_{\ell,j} + \Delta_{\ell,j}^{\text{eval}} \leq b_{\ell,j} \Rightarrow s_{\ell,j} m_{\ell,j} \leq b_{\ell,j}.$$

$$\Pr[\text{all finite-suite conservative pass claims are sound}] \geq 1 - \sum_{\ell} \sum_{j \in J_\ell} \delta_{\ell,j}^{\text{eval}}.$$

Batch-4 tangent-cone equations

$$q_{t,j}^{\text{RCLM}}(\nu) \leq 0 \quad (j \in \mathcal{J}_t^{\text{local}})$$

$$T_{\mathcal{A}_t^{\text{RCLM}}}(\rho_t^{\text{sys}}) = \{v : Dq_{t,j}^{\text{RCLM}}(0)[v] \leq 0 \ \forall j \in J_t^0\}$$

$$\Delta \text{RSIPot}_t^{\text{RCLM}}(u) = \text{RSIPot}_t^{\text{RCLM}}(u; \rho_t^{\text{sys}}, R_t) - \text{RSIPot}_t^{\text{RCLM}}(0; \rho_t^{\text{sys}}, R_t)$$

$$P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_\nu \text{RSIPot}_t := P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_u \text{RSIPot}_t^{\text{RCLM}}(0; \rho_t^{\text{sys}}, R_t)$$

$$\nu_t^{\text{TC}} = \chi_t^{\text{RCLM}}(\alpha_t v_t), \quad 0 < \alpha_t \leq \alpha_t^{\text{TC}}(v_t)$$

$$q_{t,j}^{\text{RCLM}}(\nu_t^{\text{TC}}) \leq 0 \ \forall j, \quad \Delta \text{RSIPot}_t^{\text{RCLM}}(\alpha_t v_t) > 0.$$

Batch-5 engine-viability equations

$$\mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}}) \subseteq \Omega_t^{\text{NL,RCLM}} \cap \mathcal{A}_{\text{RCP}}^{\text{RCLM-DG-NLalg}}(t, \rho)$$

$$\mathcal{V}_N^{\text{RCLM-engine},N} = \mathcal{K}_N^{\text{RCLM-engine}}$$

$$\mathcal{V}_t^{\text{RCLM-engine},N} = \left\{ \rho \in \mathcal{K}_t^{\text{RCLM-engine}} : \exists \nu \in \mathcal{E}_t^{\text{RCLM-CE}}(\rho; g_t^{\mathfrak{A}}), \ \Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N} \right\}$$

$$\rho_t^{\text{sys}} \in \mathcal{V}_t^{\text{RCLM-engine},N} \implies \exists \nu_t \in \Omega_t^{\text{NL,RCLM}} : \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},N}.$$

$$\rho_0^{\text{sys}} \in \mathcal{V}_0^{\text{RCLM-engine},N} \implies \exists (\nu_0, \dots, \nu_{N-1})$$

preserving non-loss, recovery, abilities, and successor viability.

$$\rho_0^{\text{sys}} \in \mathcal{V}_0^{\text{RCLM-engine},\infty} \implies \exists (\nu_t)_{t \geq 0} : \rho_{t+1}^{\text{sys}} = \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{V}_{t+1}^{\text{RCLM-engine},\infty}.$$

Batch-9 direct-engine equations

$$\mathcal{D}_t^{\text{RCLM-engine}} = \{ \rho \in \mathcal{K}_t^{\text{RCLM-engine}} : \text{strict beneficial CE witness exists,} \\ \text{and generator/certifier domains hold} \}.$$

$$\text{Pass}_t^{\text{Eng,RCLM}}(\rho) = \{ \nu : \nu \in \text{Gen}_t^{\text{NL,RCLM}}(\rho), \\ \text{EngCert}_t^{\text{RCLM}}(\nu) \text{ proves all engine conditions} \}.$$

$$\rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}} \implies \text{Pass}_t^{\text{Eng,RCLM}}(\rho_t^{\text{sys}}) \neq \emptyset \implies \mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ is defined.}$$

$$\Theta_t^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}(\rho_t^{\text{sys}}, R_t)) \subseteq \mathfrak{A}_{t+1}^{\text{RCLM}}(\rho_{t+1}^{\text{sys}}, R_{t+1}), \quad \mathfrak{A}_{t+1}^{\text{RCLM}} \setminus \Theta_t^{\mathfrak{A},\text{RCLM}}(\mathfrak{A}_t^{\text{RCLM}}) \neq \emptyset.$$

Batch-8 transport package

$$\text{Trans}_t^{\text{RCLM}} = (\Theta_t^{Q,\text{RCLM}}, \Theta_t^{E,\text{RCLM}}, \Theta_t^{Z,\text{RCLM}}, \Theta_t^{Y,\text{RCLM}}, \Theta_t^{G,\text{RCLM}}, \Delta_t^{\text{tr,RCLM}}).$$

Transported bottleneck

$$I_{t+1}(Z_{t+1}^{\text{RCLM}}; Y_{t+1}^{\text{RCLM}}) \geq I_{t+1}(\Theta_t^{Z,\text{RCLM}} Z_t^{\text{RCLM}}; \Theta_t^{Y,\text{RCLM}} Y_t^{\text{RCLM}}) - \chi_t^{\text{tr,RCLM}}.$$

Transported ambiguity

$$\begin{aligned} U_t^{\text{RCLM-tr}}(\nu) = & \left[H_{t+1}(\Theta_t^{Q,\text{RCLM}} Q_t^{\text{RCLM}} \mid \Theta_t^{E,\text{RCLM}} E_t^{\text{RCLM}}) \right. \\ & - H_{t+1}(Q_{t+1}^{\text{RCLM}} \mid E_{t+1}^{\text{RCLM}}) \\ & \left. - I_{t+1}(\Theta_t^{Q,\text{RCLM}} Q_t^{\text{RCLM}}, E_{t+1}^{\text{RCLM}} \mid \Theta_t^{E,\text{RCLM}} E_t^{\text{RCLM}}) \right]_+. \end{aligned}$$

State/update separation

$$\begin{aligned} \mathcal{K}_{\text{RCLM},t}^{\text{state}} &= \{\rho : \text{state-only RCLM predicates hold}\}, \\ \mathcal{A}_{\text{RCLM}}^{\text{upd}}(t, \rho, \nu) &= \{\text{update-dependent residuals hold}\}. \end{aligned}$$

Lyapunov split

$$V_{\text{RCLM},t}(\rho; \nu) = V_{\text{state},t}^{\text{RCLM}}(\rho) + V_{\text{upd},t}^{\text{RCLM}}(\rho, G_\nu, \nu).$$

Projection-realization certificate

$$\text{ProjReal}_t^{\text{RCLM}}(\tilde{\rho}, \rho^+, \nu) \Rightarrow d(\rho^+, \Phi_\nu(\rho_t^{\text{sys}})) \leq \delta_t^{\text{proj-real}}.$$

Batch-11 seed-domain and witness-library equations

$$\mathcal{L}_t^{\text{RCLM-CE}}(\rho) = \{M \in \mathcal{L}_t^{\text{RCLM-CE}} : \text{ValidMod}_t^{\text{RCLM}}(M; \rho) = 1\}.$$

$$\mathfrak{G}_{\leq k_t, t}^{\text{NL,RCLM}}(\mathcal{L}_t^{\text{RCLM-CE}}) \subseteq \mathfrak{G}_{\text{RCLM}}^{\text{NL}}.$$

$\text{LiftCov}_t^{\text{RCLM}}(\rho, \mathcal{L}_t^{\text{RCLM-CE}}, k_t) \Rightarrow \text{Gen}_t^{\text{NL,RCLM}}(\rho)$ contains a candidate covering a strict module.

$$\mathcal{K}_t^{\text{RCLM-seed},N} = \{\rho \in \mathcal{K}_t^{\text{RCLM-engine}} : \text{SeedPkg}_t^{\text{RCLM}}(\rho) \text{ is valid for horizon } N\}.$$

$$\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N} \implies \rho_t^{\text{sys}} \in \mathcal{D}_t^{\text{RCLM-engine}} \implies \mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ constructs a certified update.}$$

$$\text{SeedEq}_t^{\text{RCLM}}(M, \nu_t; \rho_t^{\text{sys}}) \Rightarrow \Phi_{\nu_t}(\rho_t^{\text{sys}}) \in \mathcal{K}_{t+1}^{\text{RCLM-seed},N}.$$

$$\rho_0^{\text{sys}} \in \mathcal{K}_0^{\text{RCLM-seed},N} \implies \rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-seed},N} \quad (0 \leq t \leq N)$$

under the Batch-11 seed package, Batch-9 constructor, and seed-equivalence/persistence-transfer hypotheses.

Batch-12/12A RCLM successor-verification equations

$$\begin{aligned}
\mathcal{K}_t^{\text{RCLM-SV},N} &\subseteq \mathcal{K}_t^{\text{RCLM-seed},N} \subseteq \mathcal{D}_t^{\text{RCLM-engine}}. \\
\text{SVPkg}_t^{\text{RCLM}}(\rho, \nu) &= (\text{SeedPkg}_t^{\text{RCLM}}, \text{VerSch}_t^{\text{RCLM-SV}}, \mathcal{P}_t^{\text{RCLM-SV}}, \text{Goalld}_t^{\text{RCLM}}, \\
&\quad \text{TrustRef}_t^{\text{RCLM-SV}}, \text{CertBudget}_t^{\text{RCLM-SV}}, \text{SVPersist}_t^{\text{RCLM}}). \\
Q_t^{\text{RCLM-SV},A} &= (q_t^{\text{seed}}, q_t^{\text{cand}}, q_t^{\text{ver}}, q_t^{\text{trans}}, q_t^{\text{goal}}, q_t^{\text{unc}}, q_t^{\text{trust}}, q_t^{\text{budget}}, q_t^{\text{persist}}, q_t^{\text{world}}, q_t^{\text{proof}}, q_t^{\text{sound}}). \\
\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},N} &\implies \mathfrak{E}_t^{\text{RCLM}}(\rho_t^{\text{sys}}) \text{ constructs } \rho_{t+1}^{\text{sys}} \in \mathcal{K}_{t+1}^{\text{RCLM-SV},N}. \\
\inf_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} \mathbb{E}_P[\Psi_{t+1}^{\text{RCLM-SV}}(\rho_{t+1}^{\text{sys}}) - \Theta_t^{\Psi, \text{RCLM}} \Psi_t^{\text{RCLM-SV}}(\rho_t^{\text{sys}})] &\geq \epsilon_t^{\text{RCLM-SV}} - \eta_t^{\text{RCLM-SV}}. \\
D_{\text{sem}}^{U, \text{RCLM}}(U_{t+1}^{\text{RCLM}}, \Theta_t^{U, \text{RCLM}} U_t^{\text{RCLM}}) &\leq \delta_t^{U, \text{RCLM}}. \\
\sup_{P \in \mathcal{P}_t^{\text{RCLM-SV}}} P((\text{SVSound}_t^{\text{RCLM}})^c) &\leq \alpha_t^{\text{RCLM-SV}}. \\
D_{\text{sem}}^{U, \text{RCLM}}(U_T^{\text{RCLM}}, \Theta_{0:T}^{U, \text{RCLM}} U_0^{\text{RCLM}}) &\leq \sum_{t=0}^{T-1} (\delta_t^{U, \text{RCLM}} + \omega_t^{U, \text{RCLM}}).
\end{aligned}$$

Batch-12B RCLM packet-construction, nonemptiness, and infinite-closure equations

$$\begin{aligned}
\text{SVBuilder}_t^{\text{RCLM}} : \text{SVIn}_t^{\text{RCLM}}(\rho, \nu) &\mapsto (\text{SVPkg}_t^{\text{RCLM}}, Q_t^{\text{RCLM-SV},A}, \Theta_t^{\text{RCLM-SV},A}, \\
&\quad \mathcal{P}_t^{\text{RCLM-SV}}, \text{Goalld}_t^{\text{RCLM}}, \text{TrustRef}_t^{\text{RCLM-SV}}, \text{SVPersist}_t^{\text{RCLM}}). \\
(\rho, \nu) \in \mathcal{D}_t^{\text{RCLM-SV-construct}} &\implies \text{SVPkg}_t^{\text{RCLM}}(\rho, \nu) \text{ valid.} \\
\mathcal{K}_t^{\text{RCLM-SV-seedlib},N} &\subseteq \mathcal{K}_t^{\text{RCLM-SV},N} \subseteq \mathcal{K}_t^{\text{RCLM-seed},N}. \\
\rho_{t_0}^{\text{sys}} \in \mathcal{K}_{t_0}^{\text{RCLM-SV-seedlib},\infty} &\implies \rho_{t_0}^{\text{sys}} \in \mathcal{K}_{t_0}^{\text{RCLM-SV},\infty,\text{gen}}. \\
\rho_t^{\text{sys}} \in \mathcal{K}_t^{\text{RCLM-SV},\infty} &\implies \rho_{t+1}^{\text{sys}} \in \mathcal{K}_{t+1}^{\text{RCLM-SV},\infty}. \\
D_{\text{sem}}^{U, \text{RCLM}}(U_T^{\text{RCLM}}, \Theta_{0:T}^{U, \text{RCLM}} U_0^{\text{RCLM}}) &\leq \sum_{t=0}^{T-1} (\delta_t^{U, \text{RCLM}} + \omega_t^{U, \text{RCLM}}). \\
\text{RealCont}_t^{\text{RCLM}} : P_t^{\text{true}, \text{RCLM}} \in \mathcal{P}_t^{\text{RCLM-SV}} &\text{ or } d_{\mathcal{P}}^{\text{RCLM}}(P_t^{\text{true}, \text{RCLM}}, \mathcal{P}_t^{\text{RCLM-SV}}) \leq \varepsilon_t^{\text{env}, \text{RCLM}}. \\
\text{SVTract}_t^{\text{RCLM}} : \text{Cost}(\text{SVBuilder}_t^{\text{RCLM}}) &+ \text{Cost}(\mathcal{A}_t^{\text{RCLM-SV-gate}}) + \text{Cost}(\text{SVProof}_t^{\text{RCLM}}) \\
&\leq \text{poly}(n_t^{\text{RCLM}}, k_t, \ell_t, s_t, \log(1/\delta_t^{\text{tol}})).
\end{aligned}$$

when the declared tractability certificate is present; otherwise the claim is resource-accounted only.

Batch-10 adaptive-audit and complexity equations

$$\begin{aligned}
&\text{CorrFail}_t(W_i, W_j), \quad \text{AuditLeak}_t(\nu), \\
&\text{CommonMode}_t(\text{Supp}_t(\nu)), \quad \text{CollusionRisk}_t(\text{Supp}_t(\nu)). \\
\mathcal{A}_{\text{trust}}^{\text{adv}}(t, \rho_t^{\text{sys}}) &= \{\nu \in \mathcal{A}_{\text{trust}} : \text{CorrFail}, \text{AuditLeak}, \\
&\quad \text{CommonMode}, \text{CollusionRisk} \text{ are below audited thresholds}\}. \\
\text{CxLedger}_t &= (\text{Op}_t, \text{NaiveCost}_t, \text{ApproxCost}_t, \\
&\quad \text{CertCost}_t, \text{Bottleneck}_t, \text{ApproxAssump}_t). \\
\text{SelfTrans}_t(\nu) &= (\Phi_\nu, \tau_t^{\text{in}}, \tau_t^{\text{out}}, \\
&\quad \kappa_t^{\text{kind}}, \text{TypeCert}_t(\nu)).
\end{aligned}$$

41 Glossary

Theorem type

The Batch-7 label $\text{Type}_{\text{RCLM}}(T)$ classifying a result as definitional, algebraic, telescoping, certificate-composition, assumption-dependent existence, finite-reference, or implementation-status. The label controls what the result can honestly be used to claim.

Implementation status

The Batch-7 label $\text{Status}_{\text{RCLM}}(O)$ classifying an architecture object as specified, assumed, reference-implemented, audited, narrowly Lean-checked, empirically validated, or not-yet-validated. The `LeanChecked` status is limited to the mapped canonical finite RCP/RCLM witness and refinement core; no full trained RCLM object receives empirical-validation status in this manuscript.

Claim discipline

The rule that theorem statements must be cited with their hypotheses and theorem type; certificate plumbing, reference-engine construction, and evaluation protocols cannot be promoted into unconditional RSI-solution claims.

Operational RCP gate

The updated math/physics-paper gate $\mathcal{A}_{\text{RCP}}^{\text{op}}$ that augments post-D–G RCP with typed density realization, embedding, semantic coupling, non-oracular certificates, trust graphs, budgeted search, resource/no-op pressure, coverage, calibration, and estimator subprotocols. After Batch 5, this gate belongs to the Conditional Non-Lossy Self-Update Preservation Theorem and is supplemented by the conservative-extension algebraic subgate, tangent-cone local-progress module, and engine-viability kernel layer; it is still not the completed Direct Non-Lossy RSI Engine Theorem without valid lift/beneficial-extension construction and empirical validation modules. In this architecture, $\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$ is treated as the RCLM-specific conservative refinement of that abstract gate under Assumption 20.12.

Batch-8 transport package

The RCLM cross-time transport object $\text{Trans}_t^{\text{RCLM}}$, containing maps $\Theta_t^{Q,E,Z,Y,G,\text{RCLM}}$ and distortion budgets for comparing ambiguity, bottleneck, semantic, and update-description variables across time.

Transported RCLM bottleneck

The Batch-8 condition comparing $(Z_{t+1}^{\text{RCLM}}, Y_{t+1}^{\text{RCLM}})$ to transported predecessor variables $(\Theta_t^{Z,\text{RCLM}} Z_t^{\text{RCLM}}, \Theta_t^{Y,\text{RCLM}} Y_t^{\text{RCLM}})$ rather than to an untyped previous-time pair.

Transported RCLM ambiguity residual

The Batch-8 ambiguity-collapse residual $U_t^{\text{RCLM-tr}}$, which compares successor certainty to transported predecessor question/evidence variables.

State-only RCLM safe set

The set $\mathcal{K}_{\text{RCLM},t}^{\text{state}}$ of state predicates independent of the candidate update; update-dependent residuals live in $\mathcal{A}_{\text{RCLM}}^{\text{upd}}(t, \rho, \nu)$.

Projection-realization certificate

The certificate $\text{ProjReal}_t^{\text{RCLM}}(\tilde{\rho}, \rho^+, \nu)$ proving that a mathematically projected

safe state is reachable by a typed implemented RCLM update within declared distortion.

$\mathfrak{G}_{\text{RCLM}}^{\text{NL}}$

RCLM certified non-lossy update algebra generated by identity, isometric extension, append-only memory, reversible adapters, verifier extension, and tool extension primitives.

$\mathcal{A}_{\text{NLalg}}^{\text{RCLM}}$

Algebraic non-loss admissible set for RCLM candidates with a compatible primitive-word representation, transported pair sets, primitive certificates, and additive loss/recovery budgets within threshold.

Conservative extension

A self-update of the form $\Phi_\nu(\rho) = V_\nu \rho V_\nu^\dagger \otimes \alpha_\nu$, with $V_\nu^\dagger V_\nu = I$, used to retain the predecessor state while appending a new certified module.

Transported pair set

The pushed-forward protected pair family $w_\# \mathcal{P}_t^{\text{RCLM}}$ used to certify each primitive in a composed non-lossy word.

Primitive certificate

The certificate packet $\text{PrimCert}_t(g)$ recording the isometry or dilation, appended state, recovery map, budgets, resource cost, type certificate, and ledger trace for a primitive generator.

RCLM tangent cone

The local first-order admissible direction set $T_{\mathcal{A}_t^{\text{RCLM}}}$ defined by the active RCLM residual derivatives $Dq_{t,j}^{\text{RCLM}}(0)[v] \leq 0$.

RSI potential The local scalar functional $\text{RSIPot}_t^{\text{RCLM}}$ used to rank candidate directions for future non-lossy improvement potential. It is a certified proxy, not an automatic proof of ability-set expansion.

Projected tangent-cone gradient

The object $P_{T_{\mathcal{A}_t^{\text{RCLM}}}} \nabla_\nu \text{RSIPot}_t$, used to propose local progress candidates only when strict inwardness, curvature, and segment-validity certificates hold.

RCLM engine base set

The certified state set $\mathcal{K}_t^{\text{RCLM-engine}}$ containing states that satisfy ordinary post-D–G constraints plus prerequisites for conservative-extension engine continuation.

RCLM engine viability kernel

The finite or infinite kernel $\mathcal{V}_t^{\text{RCLM-engine},N}$ or $\mathcal{V}_t^{\text{RCLM-engine},\infty}$ of states from which a certified conservative-extension engine move exists and leaves the successor in the next kernel.

RCLM direct-engine constructor

The Batch-9 tuple $\mathfrak{C}_t^{\text{RCLM}}$, consisting of generator, certifier, selector, and realizer. On $\mathcal{D}_t^{\text{RCLM-engine}}$, it returns the update, recovery map, certificate packet, and successor state.

RCLM direct-engine domain

The Batch-9 set $\mathcal{D}_t^{\text{RCLM-engine}}$, containing states with strict CE witnesses and valid generation/certification domains.

RCLM engine certificate packet

The Batch-9 packet $\text{EngCert}_t^{\text{RCLM}}(\nu)$, containing trace, non-loss, recovery, ability, kernel, full-gate, type, trust, resource, and projection-realization certificates.

RCLM lift map

The Batch-6 candidate-generation map $\text{Lift}_t : (\rho_t^{\text{ctx},*}, M_t, W_t, \mathcal{H}_t^{\text{cert}}) \rightarrow \Omega_t^{\text{NL,RCLM}}$, which turns a halted Level 2 context state and audited self-model/verifier/certificate context into a certified non-lossy primitive-word proposal when the lift domain and lift certificate are valid.

Reference RCLM engine

The finite-dimensional conservative-extension witness $\Phi_{\text{ref}}(\rho) = V_0 \rho V_0^\dagger \otimes \alpha_0$ with explicit recovery $\mathcal{R}_{\text{ref}}(X) = V_0^\dagger \text{Tr}_{\text{new}}(X) V_0$, used to demonstrate exact protected non-loss, exact predecessor recovery, callable predecessor ability preservation, and strict ability expansion under a valid new-ability certificate.

Lift certificate

The Batch-6 certificate packet proving that a lifted primitive word is type/domain valid, has valid primitive non-loss certificates, pays its resource costs, and, when an engine-continuation claim is made, belongs to the declared RCLM conservative-extension engine move set with successor-kernel membership.

RCLM conservative-extension engine move

A primitive word $\nu \in \mathcal{E}_t^{\text{RCLM-CE}}$ that belongs to the certified non-lossy algebra, preserves predecessor abilities, satisfies engine residuals, and certifies successor-kernel membership.

Certified conservative-extension plan

A finite sequence of RCLM engine moves whose primitive certificates, recovery maps, ability certificates, resource entries, trust graphs, and successor-kernel witnesses are recorded in the certificate ledger.

Certified ability set

The set $\mathfrak{A}_t^{\text{RCLM}}(\rho, R_t)$ of tasks, proof routines, tools, or solver abilities that the RCLM system can execute under resource budget with an audited ability certificate.

Callable predecessor retention

Assumption that a conservative extension retains a callable predecessor solver/verifier interface and pays the successor resource overhead needed to preserve old certified abilities.

$\mathcal{R}_\theta$ Level 2 recursive coherence inference operator.

Φ_t Level 3 recursive self-improvement channel.

ρ_t^{ctx} Context-level language/evidence density state.

ρ_t^{sys} System-level architecture/self-model/update density state.

Typed density realization

A declaration of whether a density state is physical-quantum, classical-probabilistic, or representational, together with the rule licensing which mathematical claims may be made about it.

E_t	Representation-to-density embedding from raw RCLM variables into a positive trace-one matrix.
$\mathfrak{C}_t^{\text{RCLM}}$	Semantic coupling structure linking the embedding, register interpretation maps, audit process, semantic barriers, and coupling-error bounds.
$\mathcal{A}_{\text{type}}$	Type-safe admissibility set for candidates whose state/update types license the theorem claims used by the verifier.
$\mathcal{A}_{\text{coup}}$	Coupling-safe admissibility set for candidates whose density-level semantic validity transfers to external semantic validity within declared margins.
$\mathcal{Z}_t^{\text{aud}}$	Audit data package containing calibration data, held-out validation data, stress tests, adversarial probes, audited ledger records, and an independence-protected verifier slice.
$\text{NOCert}_t(\nu)$	Non-oracular certificate tuple for candidate ν , containing estimated residuals, statistical/shift/model/adversarial radii, failure probability, audit data, and an audit trail.
$\Delta_t^{\text{cert}}(\nu)$	Total one-sided residual radius vector used to convert estimated residuals into conservative true-residual upper bounds.
$\mathcal{A}_{\text{cert}}$	Certificate-admissible set for candidates with a well-formed non-oracular audited certificate to one-sided residual bounds.
$\mathcal{G}_t^{\text{ver}}(\nu)$	Verifier trust graph for candidate ν , recording which verifier components or anchors certify which verifier-affecting residuals.
$\mathcal{A}_{\text{trust}}$	Trust-admissible set for candidates satisfying the no-self-sole-certification rule and an independently certified verifier-soundness transfer bound.

No self-sole-certification

Rule requiring verifier-changing updates to be certified by independent trust support rather than solely by the verifier components being modified.

$\mathcal{A}_t^{\text{safe}}$	Safety-admissible candidate set, equal to the full post-D-G RCP-RCLM gate before adding engine-mode progress requirements.
$\mathcal{I}_t$	Beneficial safe-improvement set of candidates that are safety-admissible and have certified no-op-normalized safe-capability gain above ϵ_t^{prog} .
$\mathcal{A}_{\text{prog}}$	Certified progress-search gate for finite-budget engine-mode improvement claims.
$S_t(B_t^{\text{search}})$	Finite audited candidate set returned by the budgeted search procedure.

Persistent improvement condition

Conditional assumption that the beneficial margin-strengthened set is nonempty in the declared engine-viability region under sufficient resources.

$\mathcal{A}_{\text{scale}}$	Resource-scaling admissible set requiring audited one-step affordability and, for engine-mode scaling claims, amortized cost bounded by declared reusable gains.
------------------------------	--

No-op rate monitor

Sliding-window diagnostic r_{t,h_t}^0 tracking whether safe update selection has collapsed into persistent no-op fallback.

Certificate inflation

	The combined pressure from relaxing gate budgets and increasing certificate uncertainty radii; bounded by $\mathcal{A}_{\text{infl}}$ when inflation-control claims are invoked.
$\mathcal{A}_{\text{infl}}$	Inflation-controlled admissible set requiring bounded budget/radius inflation and trust-admissible certification of any gate relaxation.
$\mathcal{P}_t^{\text{RCLM}}$	Architecture-specific safety-relevant pair set.
$\mathcal{A}_{\text{RCP}}^{\text{RCLM-DG}}$	Fully augmented true post-D–G admissibility gate.
$\hat{\mathcal{A}}_{\text{RCP}}^{\text{RCLM-DG}}$	Executable audited gate used by the architecture; it implies membership in the true gate only under the one-sided soundness assumptions.
Γ_X	Semantic interpretation map for register X .
W_t	Verifier state.
$\mathfrak{R}_t$	Physical/computational resource state.
$\mathcal{V}_{\infty}^{\text{RCLM}}$	Infinite-horizon viability kernel.
$\mathcal{K}_{\text{RCLM},t}^{\text{DG}}$	Architecture-level post-D–G state safe set.
L_{div}	Coherence-diversity sub-barrier $L_{\text{div}}^{\text{RCLM}}$ instantiated on RCLM system states.
$\mathfrak{S}_t^{\text{RCLM}}$	Declared architecture-level safety predicate family.
$E_t^{\text{RCLM}}, Q_t^{\text{RCLM}}$	Architecture evidence process and ambiguity/hypothesis variable.
$\mathcal{H}_t^{\text{cert}}$	Append-only certificate ledger for accepted and rejected update records.

Soft diversity-barrier certificate

A penalty-based architecture-level certificate that bounds loss of $L_{\text{div}}^{\text{RCLM}}$ and bounds threshold violation explicitly by $(P_{\text{max},t} + \nu_t^{\text{opt}})/(\lambda_t + \mu_t)$.

Hard diversity-barrier certificate

A constrained architecture-level selector or projection rule that accepts only successors satisfying $L_{\text{div}}^{\text{RCLM}} \geq \kappa_{\text{div},t+1}$, giving forward invariance under the stated threshold/no-op assumptions.

$\mathfrak{S}_t^{\text{trk}}, \mathfrak{S}_t^{\text{untrk}}$	Tracked and untracked portions of the declared safety predicate family. Tracked predicates have witness pairs, barriers, or verifier tests; untracked predicates contribute to the coverage-gap residual.
CovGap_t	Declared untracked-predicate coverage gap over the audited predicate-search domain.
CovCert_t	Coverage certificate containing an estimated coverage gap, one-sided radius, failure probability, adversarial trace, and expansion rule.
$\mathcal{A}_{\text{cov}}$	Coverage-admissible gate requiring the audited coverage-gap upper bound to lie below γ_t^{cov} .
$\mathcal{T}_t^{\text{m}}$	Audited capability suite consisting of held-out, out-of-distribution, adversarial, formal, and human-reference tasks with weights.
CalProt_t	Coherence–capability calibration protocol defining the task suite, calibration model, data splits, stress tests, one-sided radius, and tolerated calibration error.

$\mathcal{A}_{\text{cal}}$	Calibration-admissible gate licensing coherence-derived safe-capability progress claims only under a conservative calibration-error bound.
$\text{Est}_{t,k}$	Residual-specific estimator subprotocol for gate component class k , including inputs, domain, estimator, one-sided radii, failure probability, cost, and stress tests.
$\mathcal{A}_{\text{est}}$	Estimator-complete gate requiring all residuals used for acceptance to have domain-valid subprotocols and conservative one-sided bounds.
$\text{EvalSuite}_T^{\text{RCLM}}$	Finite reproducible numerical evaluation suite for the architecture; it defines experiment classes and reporting rules but does not itself report empirical success.
$\mathfrak{E}_\ell$	A numerical experiment specification containing the hypothesis, domain, simulator, candidate generator, data, metrics, pass predicates, seeds, tolerances, and artifacts for experiment ℓ .
ReproPack_T	Frozen reproducibility package containing code, environment, precision, seeds, datasets, gate configuration, verifier identity, candidate generator identity, and artifact hashes.
$\Delta_{\ell,j}^{\text{eval}}$	One-sided evaluation radius for metric $m_{\ell,j}$, used to turn estimated pass claims into conservative finite-suite claims.

Reproducible numerical evaluation protocol

A formal protocol specifying how future experiments must be frozen, executed, audited, and reported; it is not a claim that the architecture has already been empirically validated.

Architecture finalization / instantiation readiness

The final paper-readiness layer that surfaces the domain-relative theorem, states the implementation boundary, adds a finite reference-instance target, and converts remaining limitations into an implementation roadmap.

Domain-relative robust reflective successor theorem

The qualified final theorem form proved for declared RCLM seed-library classes with valid successor-verification packets, reality-containment/misspecification certificates, and restricted tractability certificates.

Minimal finite RCLM SV reference instance

A finite witness-library, grammar, builder, packet, and trace object $\text{RefSV}_{0:N}^{\text{RCLM}}$ proving that at least one controlled finite instance enters the RCLM SV seed-library domain.

Batch-13R The RCLM reference-entry / realization layer, not a new abstract Batch 13, adding proof-carrying successor packets, a finite checker, a reference builder, canonical finite reference witness, and replayable artifact theorem.

Batch-13R-A The final RCLM concrete-witness synchronization patch, adding the explicit canonical SV residual vector, checker-realization theorem, artifact parser, and RCLM-to-RCP canonical refinement theorem.

Batch-13R-B The pre-publication milestone layer separating controlled executable replay from external proof-assistant mechanization, defining $\text{MechCert}_{0:N}^{\text{RCLM,can}}$, $\text{CtrlRef}_{0:N}^{\text{RCLM}}$, and $\text{CtrlReplayCert}_{0:N}^{\text{RCLM}}$; in the current companion artifact

package, a Lean 4 certificate is supplied for the canonical finite RCP/RCLM witness and RCLM-to-RCP refinement core.

Lean 4 certificate

The supplied companion proof-assistant artifact checking `RCP.lean`, `RCLM.lean`, and the root `RcpRclmMech.lean` library build. It covers the canonical finite RCP/RCLM witness, finite checker core, and RCLM-to-RCP refinement module only; it does not mechanize the full 219-page math stack, the full 187-page architecture stack, arbitrary learned-system entry, broad capable-agent RSI, or empirical deployment claims.

M3-Min

The minimal learned-agent entry boundary layer. It defines the finite certificate boundary a trained or learned RCLM system must satisfy before the existing Batch-13R theorem stack may be invoked for that system.

LECert_{0:N}^{RCLM}

The learned-entry certificate packet containing type, register-semantic, coverage, witness-library, builder-trace, proof-carrying packet, residual, goal, trust, reality, tractability, and replay certificates.

LearnedEntryAudit_{0:N}^{RCLM}

The audit procedure returning either a full learned-entry certificate or $\perp$; a full pass plus audit soundness implies finite RCLM SV seed-library entry.

RCLM proof-carrying successor packet

The packet $\text{PCS}_t^{\text{RCLM}}(\nu_t)$ carrying the candidate, transition, successor state, non-loss proof, recovery proof, seed-domain proof, SV proof, goal-identity proof, trust proof, reality-containment proof, tractability proof, and replay trace.

RCLM reference checker

The finite checker $\text{Check}_{\text{RCP}}^{\text{RCLM}}$ that accepts proof-carrying successor packets only when their typed residual, transport, trust, resource, and successor-persistence checks pass.

RCLM reference builder

The construction $\text{BuildRefSV}_{0:N}^{\text{RCLM}}$ that returns a finite reference instance $\text{RefSV}_{0:N}^{\text{RCLM}}$ from a finite system, audit data, witness library, and checker.

Canonical finite RCLM reference witness

The appended-module construction $\Phi_t^{\text{can}}(X) = X \otimes \alpha_t$ with partial-trace recovery, exact protected non-loss, strict ability expansion, identity goal transport, singleton uncertainty envelope, and immutable trust anchor.

RCLM reference artifact

A replayable finite object $\text{Artifact}_{0:N}^{\text{RCLM}}$ containing states, updates, proof-carrying packets, checker outputs, logs, and hashes, whose artifact certificate invokes the finite reference theorem.

Implementation-readiness boundary

The condition that an implemented RCLM system has supplied all certificates required before the final architecture theorem may be invoked for that system.